Cryptography & Network Security

Mid-Sem Notes: Lectures 1–14 (up to Message Integrity)

★★ = asked in *both* past papers ★ = asked once / high priority ▸ = new in this year's syllabus, likely

★ MUST KNOW **TRAP** highlight = write this in the exam

Contents

1 What to Study First **5**
- 1.1 What the paper looks like . . . 5
- 1.2 What the two past papers asked . . . 5
- 1.3 What is new this year ▸ . . . 5
- 1.4 Priority and study order . . . 6

2 Model for Network Security **6**
- 2.1 Four tasks in designing a security service . . . 6

3 Network Access Security Model **6**

4 Cryptography **7**
- 4.1 Symmetric cipher model . . . 7
- 4.2 Types of cryptography . . . 7

5 Security of Encryption Schemes **7**

6 Security Goals, Attacks and Cryptanalysis **8**
- 6.1 Security goals (CIA) . . . 8
- 6.2 Attacks (Forouzan's taxonomy) . . . 8
- 6.3 Kerckhoff's principle and cryptanalysis attacks ★ . . . 8
- 6.4 History notes from the slides . . . 8

7 Classical Encryption Techniques **9**
- 7.1 Substitution technique . . . 9
- 7.2 Transposition technique . . . 9
- 7.3 Substitution vs. transposition . . . 9

8 Caesar Cipher **10**
- 8.1 Letters as numbers (start at 0) . . . 10
- 8.2 Algorithm . . . 10
- 8.3 Worked example: encrypt "neso academy" (k = 3) . . . 10
- 8.4 Homework 1: decrypt "QHWZRUN VHFXULWB" (k = 3) . . . 10

9 Shift Cipher (the general form) **10**
- 9.1 Pros and cons of the Caesar cipher . . . 11

10 Brute-Force Attack on the Caesar/Shift Cipher **11**

11 Monoalphabetic Cipher **12**
- 11.1 Permutations . . . 12
- 11.2 The cipher . . . 12
- 11.3 Breaking it: frequency analysis . . . 12
- 11.4 Pros, cons, countermeasure . . . 13
- 11.5 Homework: encrypt with a key phrase . . . 13

12 Playfair Cipher ★★ **13**
- 12.1 Building the 5×5 key matrix . . . 13
- 12.2 Step 1: split the plaintext into digrams . . . 13

12.3 Step 2: encrypt each pair (three cases) ★ . . . 13
12.4 Worked examples (keyword MONARCHY) . . . 14
12.5 Solved question . . . 14
12.6 When I or J is in the keyword or the message ★ . . . 14
12.7 Decryption . . . 15

13 Hill Cipher **15**
13.1 Encryption example: “pay more money” . . . 16
13.2 Decryption: finding K^{-1} . . . 16
13.3 Decryption example: RRLMWBKASPDH . . . 17

14 Polyalphabetic Ciphers **17**
14.1 Vigenère cipher ★ . . . 17
14.2 Autokey system ★ . . . 18
14.3 Vernam cipher . . . 18

15 One-Time Pad ★ **19**
15.1 How it grew out of the earlier ciphers . . . 19
15.2 Example 1: letters . . . 19
15.3 Example 2: bits . . . 19
15.4 Why it can’t be broken . . . 20
15.5 The rules, and what breaks if you bend them . . . 20
15.6 Two fundamental difficulties . . . 20
15.7 Perfect secrecy . . . 20

16 Transposition Ciphers **21**
16.1 Rail fence . . . 21
16.2 Decrypting a rail fence . . . 21
16.3 Row–column transposition . . . 22
16.4 Two stages: more complex . . . 22

17 Classical Ciphers at a Glance **22**

18 Traditional Ciphers: Remaining Topics (Lecture 5) **23**
18.1 Multiplicative and affine ciphers ★ . . . 23
18.2 Cryptanalysis of Vigenère: the Kasiski test . . . 23
18.3 Cryptanalysis of Hill: known-plaintext . . . 23
18.4 Rotor ciphers and Enigma . . . 24
18.5 Transposition: keyless, keyed, combined ★ . . . 24
18.6 Stream vs. block ciphers . . . 24

19 Steganography **24**
19.1 Historical techniques . . . 24
19.2 Drawbacks . . . 25
19.3 LSB steganography . . . 25
19.4 Watermarking vs. image steganography . . . 25
19.5 OpenStego demo . . . 25

20 Solved Exam Questions **26**

21 Prime Numbers **26**
21.1 Why primes matter in cryptography . . . 26

22 Modular Arithmetic **27**
22.1 Reduce first . . . 27
22.2 Properties of $\mathbb{Z}_n$ and inverses ★ . . . 27
22.3 Linear congruences $ax \equiv b \pmod{n}$ . . . 27

23 Modular Exponentiation ★ **28**
23.1 Part 1: when the base is close to the modulus . . . 28
23.2 Part 2: repeated squaring . . . 28

24 GCD and the Euclidean Algorithm **29**
24.1 The key fact . . . 29
24.2 Method 1: the Q A B R table . . . 30
24.3 Method 2: recursion . . . 30
24.4 Relatively prime . . . 30

25 Extended Euclidean Algorithm ★★ **31**
25.1 Multiplicative inverse with the table ★★ . . . 31
25.2 Linear Diophantine equations $ax + by = c$ ★ . . . 31

26 Relatively Prime Numbers, Euler's Totient and $\mathbb{Z}_n^*$ **32**
26.1 Relatively prime (co-prime) numbers . . . 32
26.2 Euler's totient function $\phi(n)$ ★ . . . 32
26.3 Computing $\phi(n)$: the four rules ★ . . . 32
26.4 The sets $\mathbb{Z}_n$ and $\mathbb{Z}_n^*$ . . . 33
26.5 Why cryptography cares about $\phi(n)$ . . . 34

27 Fermat's and Euler's Theorems, Inverses and Primitive Roots **34**
27.1 Fermat's little theorem and exponent reduction ★ . . . 35
27.2 Euler's theorem ★ . . . 35
27.3 Multiplicative inverses via Fermat and Euler ★ . . . 36
27.4 Order of an element ▸ . . . 37
27.5 Primitive roots ▸ . . . 38

28 Primes, Primality Testing, Factorization, CRT and Discrete Logs **40**
28.1 Primes and how many there are . . . 40
28.2 Primality testing: deterministic and probabilistic . . . 41
28.3 Miller–Rabin test ▸ . . . 41
28.4 Factorization . . . 43
28.5 Chinese remainder theorem ▸ . . . 43
28.6 Fast exponentiation: square-and-multiply ★ . . . 45
28.7 Discrete logarithms ▸ . . . 46

29 Algebraic Structures: Groups, Rings, Fields ★★ **48**
29.1 Groups . . . 48
29.2 Order, subgroups and cyclic subgroups . . . 49
29.3 Rings and fields . . . 49
29.4 Finite fields: why the size must be p^n . . . 50
29.5 GF(p) arithmetic and division ★ . . . 50
29.6 Why GF(2^n) instead of $\mathbb{Z}_{2^n}$ . . . 51
29.7 Polynomial representation and addition . . . 51
29.8 Irreducible polynomials: the modulus . . . 51
29.9 Polynomial long division over GF(2) ★ . . . 51
29.10 Multiplication in GF(2^n) ★ . . . 52
29.11 Multiplicative inverse in GF(2^n): extended Euclid on polynomials ★★ . . . 53
29.12 Small fields: GF(2^2) and GF(2^3) . . . 56

30 Modern Symmetric-Key Ciphers **57**
30.1 Modern block ciphers and padding . . . 57
30.2 Components of a modern block cipher . . . 57
30.3 Product ciphers: confusion, diffusion and rounds ★ . . . 59
30.4 Feistel ciphers ★ . . . 60

30.5 Non-Feistel ciphers . . . 61
30.6 Attacks on modern block ciphers . . . 61
30.7 Modern stream ciphers . . . 62

31 Data Encryption Standard (DES) 63
31.1 DES structure ★ . . . 63
31.2 Initial and final permutations . . . 63
31.3 Rounds: mixer, swapper and the reverse cipher . . . 64
31.4 The DES function $f(R_{i-1}, K_i)$ ★ . . . 64
31.5 Key generation ★ . . . 66
31.6 DES analysis: avalanche and completeness . . . 68
31.7 Weaknesses of DES . . . 68
31.8 Multiple DES . . . 69
31.9 Security of DES in one line each . . . 69

32 Advanced Encryption Standard (AES) 70
32.1 Parameters and data units . . . 70
32.2 Round structure . . . 71
32.3 SubBytes (substitution) ★ . . . 71
32.4 ShiftRows (permutation) ★ . . . 73
32.5 MixColumns (mixing) ★ . . . 73
32.6 AddRoundKey (key adding) ★ . . . 74
32.7 Key expansion (AES-128) ★ . . . 75
32.8 One complete round: FIPS-197 example . . . 76
32.9 Decryption . . . 76
32.10 Analysis of AES . . . 76
32.11 AES versus DES . . . 76

33 Asymmetric-Key Cryptography and RSA ▶ 77
33.1 Why asymmetric-key cryptography exists ▶ . . . 77
33.2 Trapdoor one-way functions ▶ . . . 78
33.3 RSA: key generation, encryption, decryption ▶ . . . 78
33.4 Why decryption works: proof of correctness ▶ . . . 78
33.5 The slide examples ▶ . . . 79
33.6 Exam-format RSA numericals ▶ . . . 79
33.7 Choosing e; why $e = 2$ never works ▶ . . . 80
33.8 Faster decryption with the CRT ▶ . . . 80
33.9 Attacks on RSA ▶ . . . 81
33.10 Recommendations . . . 81

34 ElGamal and Elliptic Curve Cryptosystems ▶ 82
34.1 ElGamal cryptosystem ▶ . . . 82
34.2 Security of ElGamal ▶ . . . 83
34.3 Elliptic curves over the real numbers . . . 83
34.4 Elliptic curves over GF(p) ▶ . . . 84
34.5 Point addition and doubling over GF(p) ▶ . . . 85
34.6 Scalar multiplication and the ECDLP ▶ . . . 86
34.7 ECC simulating ElGamal ▶ . . . 86
34.8 Security of ECC and key size ▶ . . . 87

35 Message Integrity and Hash Functions 88
35.1 Integrity versus secrecy; message and digest ▶ . . . 88
35.2 The three security criteria ▶ . . . 88
35.3 The random oracle model ▶ . . . 89
35.4 The four birthday problems ▶ . . . 89

35.5 Attacks on an n-bit digest ▸ . . . 90
35.6 Iterated hash functions: the Merkle–Damgård scheme ▸ . . . 92
35.7 Compression functions: from scratch or from a block cipher ▸ . . . 92
35.8 SHA-512: overview and parameters ▸ . . . 93
35.9 SHA-512: padding and length field ▸ . . . 94
35.10 SHA-512: word expansion and the compression function ▸ . . . 94
35.11 Exercises on the SHA-512 operations ▸ . . . 95
35.12 Whirlpool . . . 96

36 Previous Year Papers (MNNIT, CSN15403), Fully Solved **98**
36.1 Quick-revision method boxes . . . 98
36.2 Mid-Sem 2024–25, solved . . . 98
36.3 Mid-Sem 2025–26, solved . . . 100
36.4 Mock Mid-Sem paper (same style), 90 minutes, 25 marks . . . 102
36.5 Mock paper: model answers . . . 103

37 Exam-Pattern Drill Bank **104**
37.1 Drill bank: model answers . . . 106

38 Practice Problems **109**

39 Answer Key **115**

1 What to Study First

1.1 What the paper looks like

The mid-sem (CSN15403) is 90 minutes for 25 marks, with five to seven questions and the instruction that only a "precise answer with proper justification" gets full marks. In practice this means that every numerical must show its working in the textbook layout: the extended-Euclid table with its $q, r_1, r_2, r, t_1, t_2, t$ columns, the Playfair digrams, the long divisions of polynomials, and a final check. The questions are not written from the slides. They are taken almost word for word from **Forouzan** (Eve and Alice, $\mathbb{Z}_{100}$, keyed transposition, GF(2^8)) and **Stallings** (the "sendmoremoney" one-time pad, the AES state values), so the notes teach the textbook method wherever the slides are thin.

1.2 What the two past papers asked

Both papers are solved in full in §36. The table lists every question by topic, with its marks.

	Topic	2024–25	2025–26	Where
★★	Playfair encryption	3	4	§12
★★	GF(2^8) inverse by the extended Euclidean algorithm	4	4	§29
★★	Extended Euclid: integer inverse ("value in iteration k"), Diophantine equation	5	3	§25
★	One-time-pad version of Vigenère (encrypt, then find a key)	–	8	§14, §15
★	AES: ShiftRows and AddRoundKey on a given state	–	6	§32
★	DES: Feistel structure, round function, key generation, pseudocode	5	–	§31
★	GF(p) division, Fermat exponent reduction, inverse that does not exist	3	–	§27
★	Keyed transposition and the attack type	3	–	§18, §6
★	Characters changed: multiplicative, affine, Vigenère, auto-key	2	–	§18
▸	Lectures 10–14: primality, CRT, RSA, ElGamal, ECC, hashing	–	–	§28–§35

Three patterns stand out. First, classical ciphers carried 20 of the 50 marks, and Playfair appeared in both years, so it is the safest marks in the paper. Second, the extended Euclidean algorithm appeared in both years twice over: once with integers (an inverse or a Diophantine equation) and once with polynomials (the GF(2^8) inverse). Together these were 16 of the 50 marks, all from one table method. Third, each year had one block-cipher question worth 5–6 marks, DES in one year and AES in the other.

1.3 What is new this year ▸

Neither paper asked anything from Lectures 10–14, most likely because those lectures came after the mid-sem in earlier years. This year the syllabus runs up to "Message Integrity", so you should expect at least one question from these lectures. The most likely is an RSA numerical in Forouzan's style: given p, q and e, find d with the extended-Euclid table, then encrypt and decrypt using square-and-multiply. This combines three topics you

already need anyway. The next most likely are a CRT system, a Miller–Rabin test, an ElGamal or elliptic-curve computation, and a theory question on hash functions (the three resistance properties, or the birthday attack). This is a prediction from the syllabus, not from a paper, so the chapters marked ▸teach these topics fully rather than as summaries.

1.4 Priority and study order

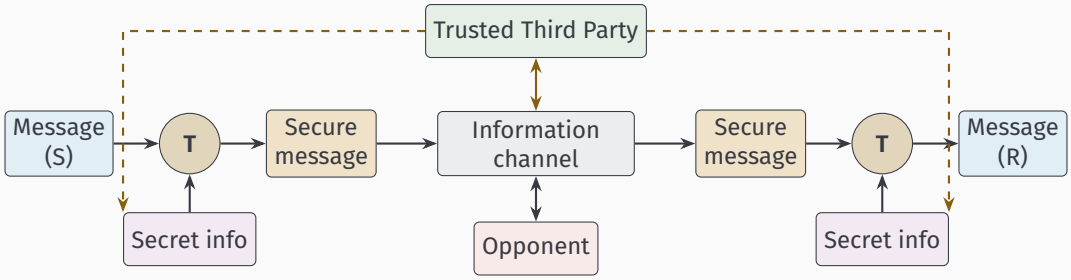
The order below puts the highest marks per hour first. Each step names the chapters to read and the exercise that proves you have learned it.

1. ★★ **Extended Euclid** (§25). Reproduce the $23^{-1} \bmod 100$ table and the $19x+13y=20$ solution without looking. Everything in steps 2, 5 and 6 reuses this table.
2. ★★ **GF(2^8) inverse** (§29). Learn polynomial long division over GF(2), then redo the x^5 and x^7 inverses from the papers.
3. ★★ **Playfair** (§12), then ★ **Vigenère, auto-key and the one-time pad** (§14, §15), then multiplicative/affine, keyed transposition and the attack types (§18, §6). These are mechanical and fast to practise.
4. ★ **AES and DES** (§30–§32). Be able to draw the DES round and key schedule, and to do ShiftRows, AddRoundKey, an S-box lookup and one MixColumns column by hand.
5. ★ **Modular exponentiation, Fermat and Euler** (§23, §26, §27). Exponent reduction, inverses, and "the inverse does not exist" traps.
6. ▸ **RSA** (§33), then **CRT, Miller–Rabin and discrete logs** (§28), then **ElGamal and ECC** (§34), then **hash functions** (§35).
7. Finally, the drill bank (§37) and the timed mock paper in §36.

Topics without a marker (the security model, Caesar and monoalphabetic ciphers, Hill, steganography, the GCD chapter) are worth one read for the theory questions, but they did not earn marks in either paper.

2 Model for Network Security

Sender *S* wants to send a message to receiver *R* over an insecure channel (the Internet). An **opponent** can sniff the channel.



- **T (security-related transformation)** takes two inputs: the message and **secret info** (a key). Output: a secure message (ciphertext). *R* applies the reverse *T* with its secret info.
- It is called *T*, not *E*, because encryption is only one option. **Steganography** (hiding data inside an image, audio, etc.) is another.
- The opponent can intercept the secure message but cannot read it.
- Security rests on the **key** (its secrecy and bit length) and algorithm strength, not on hiding the algorithm. Small key space ⇒ brute force works.
- **Trusted Third Party (TTP)**: distributes keys, issues certificates, settles disputes. Example: a Certificate Authority signing a bank's SSL certificate. (Details come later in the course.)

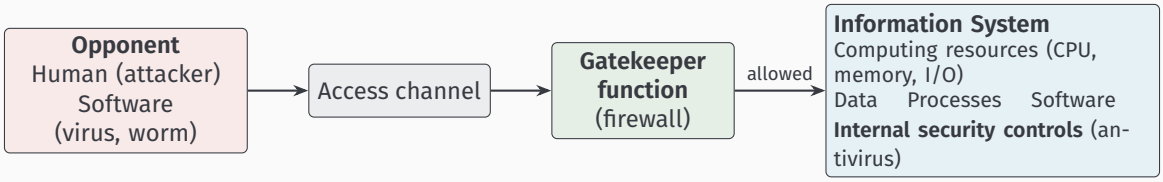
2.1 Four tasks in designing a security service

1. **Design an algorithm** for *T* (encrypt at *S*, decrypt at *R*) that the opponent cannot defeat.
2. **Generate the secret information** (large keys, so brute force is infeasible).
3. **Develop methods to distribute and share** the secret info (get keys to *S* and *R* safely).
4. **Specify a protocol**: the rules both parties follow to use the algorithm and keys (e.g., SSL/TLS, IPsec).

Memory aid: *Algorithm → Key → Distribute → Protocol*.

3 Network Access Security Model

Protects a *system* from unwanted access, rather than protecting a message in transit.



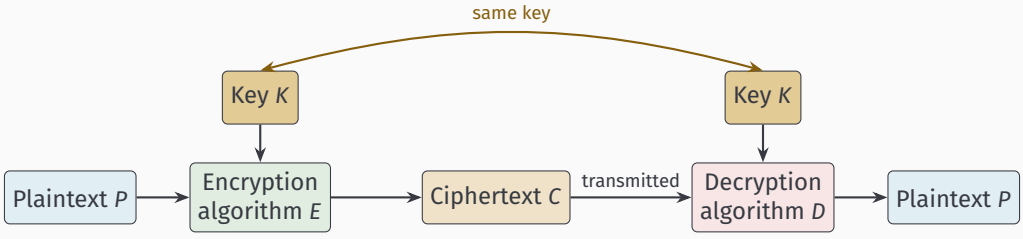
- **Opponent**: a human hacker or malicious software.
- **Gatekeeper**: first line of defence, like a guard at a gate. Filters traffic and blocks intruders (firewall, login checks).
- **Internal security controls**: second line, inside the system, catching what slips past the gate (antivirus, OS permissions).
- Insiders (e.g., employees) can also attack, but this model focuses on outside threats.

4 Cryptography

Definition

The art or science of transforming an **intelligible** message (plaintext) into an **unintelligible** one (ciphertext), and then **retransforming** it back to the original form.

4.1 Symmetric cipher model



$$C = E(K, P) \qquad P = D(K, C) = D\big(K, E(K, P)\big)$$

- Encryption needs **two** inputs (P and K); decryption needs C and K.
- E and D are **public**. Only K is secret (Kerckhoffs's principle).
- If K leaks, *all* past and future messages between S and R can be read.
- Toy example: $K = 100$ at both ends. Real keys are huge (128, 256, 2048 bits), so guessing is infeasible.

4.2 Types of cryptography

	Symmetric (private / secret key)	**Asymmetric** (public key)
Keys	One shared key	Key pair: public + private
Use	Same key encrypts and decrypts	One key encrypts, the other (mathematically related) decrypts
Secrecy	Key known only to S and R	Public key published; private key kept by owner
Toy example	Encrypt 100, decrypt 100	Encrypt 100, decrypt 500

Asymmetric works because the two keys satisfy a mathematical relationship, and the private key cannot practically be computed from the public one. (Which key does what is covered in a later lecture.)

5 Security of Encryption Schemes

Unconditionally secure

The ciphertext does **not contain enough information to uniquely determine the plaintext**, no matter how much ciphertext is available. Holds even against infinite computing power. Example: one-time pad.

Computationally secure (either condition)

1. **Cost** of breaking the cipher > **value** of the encrypted information.
2. **Time** to break the cipher > **useful lifetime** of the information.

- Cost: spending $1,000,000 to steal a $1,000 message is pointless.
- Time: a quote valid for 10 minutes that takes 50 years to crack is useless when cracked.
- Real ciphers (AES, RSA) are computationally secure; unconditional security needs a key as long as the message, which is impractical.

Exam tips

- Draw the network security model with all labels: S, R, T, secret info, secure message, opponent, TTP.
- Know the four tasks in order.
- "Why T and not E?" ⇒ steganography is also a transformation.
- Symmetric = same key; asymmetric = different but related keys.
- Quote both conditions of computational security word for word.

6 Security Goals, Attacks and Cryptanalysis

6.1 Security goals (CIA)

Confidentiality	hide information from unauthorised access, in storage *and* in transmission (military secrets, bank accounts).
Integrity	changes only by authorised entities through authorised mechanisms. Violation need not be malicious (a power surge can corrupt data).
Availability	information must be accessible to authorised entities when needed; unavailability is as harmful as a breach.

6.2 Attacks (Forouzan's taxonomy)

Attack	Threatens	Type	What happens
Snooping	confidentiality	passive	unauthorised access/interception of data. Counter: encipherment.
Traffic analysis	confidentiality	passive	learning addresses, timing, request/response pairs from traffic even when it's encrypted.
Modification	integrity	active	intercept and change (or delete, delay) a message.
Masquerading (spoofing)	integrity	active	impersonating someone (stolen card + PIN; a fake bank site).
Replaying	integrity	active	capture a message and resend it later (a second payment).
Repudiation	integrity	active	a *party to the communication* denies having sent or received a message.
Denial of service	availability	active	slow down or stop the service (bogus requests; deleting responses).

Passive attacks only obtain information: they don't change data, are hard to *detect*, and are prevented by encipherment. **Active** attacks change data or harm the system: they are easier to detect than to prevent.

Stallings' four-way picture (normal flow source → destination): **interruption** (availability), **interception** (confidentiality), **modification** (integrity), **fabrication** (authenticity / masquerade).

Trap: repudiation is not an outsider attack

Every other attack is by a third party (Eve). Repudiation is done by Alice or Bob themselves. It is countered by digital signatures (non-repudiation), not encryption.

6.3 Kerckhoff's principle and cryptanalysis attacks ★

★ **In the papers:** 2024–25 Q3(a): Eve types "abcdefghij" on Alice's computer and reads the output: which attack? (**chosen-plaintext**, since she chose the plaintext and saw its ciphertext).

Kerckhoff's principle: assume Eve knows the algorithm; security must rest *only* on the secrecy of the key.

Attack	Eve has	Note
Ciphertext-only	only ciphertext	most probable. Methods: brute force (key space must be huge), statistical (letter frequencies), pattern attacks.
Known-plaintext	earlier plaintext/ciphertext pairs (same key)	easier, less likely (key may have changed).
Chosen-plaintext	pairs for plaintexts she chose (access to Alice's machine)	much easier, much less likely.
Chosen-ciphertext	plaintexts for ciphertexts she chose (access to Bob's machine)	

6.4 History notes from the slides

Atbash (Hebrew, reversed alphabet), scytale (Greek, rod transposition), Caesar (shift 3), Vigenère (1587, broken by Kasiski 1863), Enigma (Scherbius 1918, rotor machine), Shannon (1949, proved OTP unbreakable), DES (IBM, 1970s), Diffie–Hellman (1976), RSA (1977), ECC (Miller, Koblitz 1985), MD5 (Rivest 1991), AES (2001), SHA-512 (2001).

ADFGVX (German army, WWI): a keyed 6 × 6 Polybius square (letters + digits, rows/columns labelled A D F G V X) followed by a **columnar transposition**.

	A	D	F	G	V	X
A	1	4	7	r	e	g
D	i	m	n	t	a	b
F	c	d	f	h	j	k
G	l	o	p	q	s	u
V	v	w	x	y	z	0
X	2	3	5	6	8	9

Square keyword `147regiment` (then the rest of a–z, 0–9). Encrypt `attackat1200am`: *a* → DV, *t* → DG, … giving DVDGDGDVFAFXDVDGAAXAVXVXDVDD. Write it under the transposition key `PRIVACY`, read columns in alphabetical order of the key letters (A, C, I, P, R, V, Y):
DXXV GDAD DAAX DVDX VFGV GFAD DVVD (verified).

Corrections to the notes / slides

- Slide 13's figure lists replaying under integrity (correct) but places it visually near confidentiality; use the table above.
- The ADFGVX decryption example (keys `monkeys`/`zebras`): the transposition step with key `zebras` does give the slide's intermediate text GXFGAXXFDFDAFXDDDXGAAXDFGG, but a square built from `monkeys` does not give "the way is clear"; the slide's square is a different (unshown) one. Learn the method from the encryption example.

7 Classical Encryption Techniques

Plaintext sent in the open over a channel invites attacks: **passive** (eavesdropping, reading the data) and **active** (altering or injecting data). Encryption turns plaintext into ciphertext so an interceptor learns nothing.

Classical techniques are **obsolete**: easy to break, keys recovered quickly. We study them because modern ciphers are built on the same two ideas: *substitution* and *transposition*.

7.1 Substitution technique

Letters are **replaced** by other letters or symbols. Positions stay the same.

Key (a mapping table): a→M, b→X, g→A, x→Z

Plaintext	b	a	g
	↓	↓	↓
Ciphertext	**X**	**M**	**A**

- The mapping table *is* the key.
- Sender and receiver share the same table ⇒ symmetric encryption.

7.2 Transposition technique

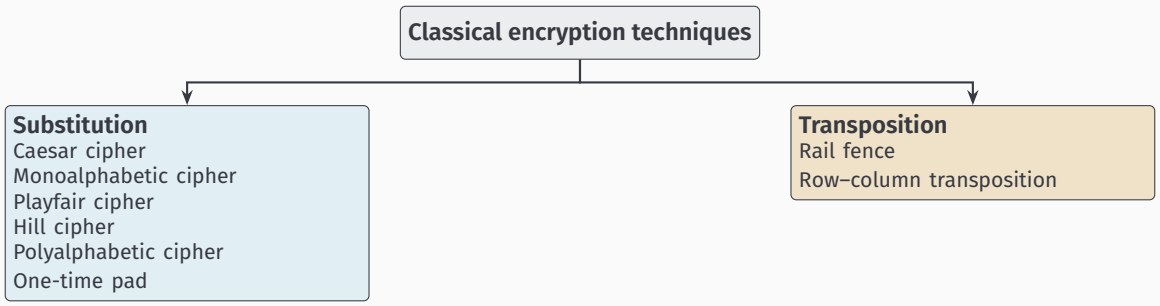
Apply a **permutation**: the same letters are **rearranged**. No new letters appear.

Plaintext NESO → ESON, SONE, ONES, ENOS, … e.g. ESON takes letters in order (2, 3, 4, 1); SONE takes (3, 4, 1, 2).

A 4-letter word has only 4! = 24 arrangements, so short messages are easy to guess. Transposition is only effective for **long** plaintexts ($n!$ grows fast).

7.3 Substitution vs. transposition

	Substitution	**Transposition**
What changes	The letters themselves	The order of the letters
Positions	Unchanged	Permuted
Letters in ciphertext	New letters / symbols	Exactly the plaintext letters
Letter frequencies	Moved onto different symbols	Identical to plaintext



8 Caesar Cipher

Earliest and simplest substitution cipher, used by Julius Caesar. Replace each letter with the letter **three places further down** the alphabet, wrapping from z back to a.

8.1 Letters as numbers (start at 0)

A	B	C	D	E	F	G	H	I	J	K	L	M
0	1	2	3	4	5	6	7	8	9	10	11	12
N	O	P	Q	R	S	T	U	V	W	X	Y	Z
13	14	15	16	17	18	19	20	21	22	23	24	25

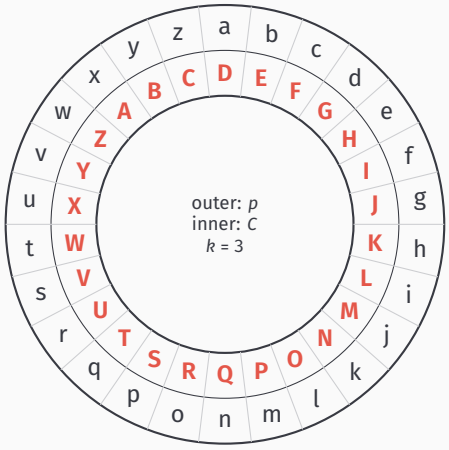
Numbering from 0 matches the remainders mod 26, which are exactly $0, 1, \ldots, 25$.

8.2 Algorithm

Encryption and decryption

$$C = E(p, k) = (p + k) \bmod 26$$
$$p = D(C, k) = (C - k) \bmod 26$$

Caesar cipher: $k = 3$.



- p = plaintext letter's number, C = ciphertext letter's number, k = key (shift).
- Convention: plaintext in lowercase, ciphertext in UPPERCASE.
- **mod gives the remainder, not the quotient.** 27 mod 26 = 1.
- Negative during decryption? Add 26. −2 mod 26 = 24.

8.3 Worked example: encrypt "neso academy" ($k = 3$)

Plaintext	n	e	s	o	a	c	a	d	e	m	y
p	13	4	18	14	0	2	0	3	4	12	24
$p + 3$	16	7	21	17	3	5	3	6	7	15	**27**
mod26	16	7	21	17	3	5	3	6	7	15	**1**
Ciphertext	**Q**	**H**	**V**	**R**	**D**	**F**	**D**	**G**	**H**	**P**	**B**

Answer: **QHVR DFDGHPB**. The only wrap-around is y: 24 + 3 = 27, and 27 = 26 × 1 + **1**, so $C = 1 =$ B. (Counting on: y → z → a → **b**.) For the other letters you can just count three letters forward.

8.4 Homework 1: decrypt "QHWZRUN VHFXULWB" ($k = 3$)

Ciphertext	Q	H	W	Z	R	U	N	V	H	F	X	U	L	W	B
C	16	7	22	25	17	20	13	21	7	5	23	20	11	22	1
$C - 3$	13	4	19	22	14	17	10	18	4	2	20	17	8	19	−2
mod26	13	4	19	22	14	17	10	18	4	2	20	17	8	19	**24**
Plaintext	n	e	t	w	o	r	k	s	e	c	u	r	i	t	y

Answer: **network security**. Note the last letter: 1 − 3 = −2, add 26 ⇒ 24 = y.

9 Shift Cipher (the general form)

Same formulas, but the key k can be any of $1, 2, \ldots, 25$. **Caesar cipher = shift cipher with $k = 3$.**

- $k = 0$ is useless: $C = (p + 0) \bmod 26 = p$, so the ciphertext equals the plaintext. (Homework: "Neso", $k = 0 \Rightarrow$ NESO.)
- $k = 26$ is the same as $k = 0$. So there are only **25 usable keys**.

Example: "Neso", $k = 4$: N 13+4=17 →**R**, e 4+4=8 →**I**, s 18+4=22 →**W**, o 14+4=18 →**S** ⇒ **RIWS**.

Homework 2: "cryptography", $k = 4$:

Plaintext	c	r	y	p	t	o	g	r	a	p	h	y
p	2	17	24	15	19	14	6	17	0	15	7	24
$p+4 \bmod 26$	6	21	**2**	19	23	18	10	21	4	19	11	**2**
Ciphertext	G	V	C	T	X	S	K	V	E	T	L	C

Answer: **GVCTXSKVETLC** (both y's wrap: $24+4=28\equiv 2$).

In code: the 0–25 numbering is a convention. With ASCII (A=65, a=97), shift to 0 first, then back: $C = ((p - 65 + k) \bmod 26) + 65$ for uppercase (use 97 for lowercase).

9.1 Pros and cons of the Caesar cipher

Pros

1. Simple.
2. Easy to implement.

Cons

1. The encryption and decryption algorithms are known.
2. Only 25 keys to try ⇒ vulnerable to **brute-force attack**.
3. The plaintext language is known and easily recognisable, so you can tell when you have hit the right key.

10 Brute-Force Attack on the Caesar/Shift Cipher

Brute force: try every possible key until readable plaintext appears. With 25 keys this takes seconds by hand.

Ciphertext: SQDYMZK. Shift every letter forward by 0, 1, 2, ..., 25 and look for a real word:

Fwd	Back	Result
0	26	SQDYMZK
1	25	TREZNAL
2	24	USFAOBM
3	23	VTGBPCN
4	22	WUHCQDO
5	21	XVIDREP
6	20	YWJESFQ
7	19	ZXKFTGR
8	18	AYLGUHS
9	17	BZMHVIT
10	16	CANIWJU
11	15	DBOJXKV*
12	14	ECPKYLW
13	13	FDQLZMX
14	**12**	**GERMANY**
15	11	HFSNBOZ
16	10	IGTOCPA
17	9	JHUPDQB
18	8	KIVQERC
19	7	LJWRFSD
20	6	MKXSGTE
21	5	NLYTHUF
22	4	OMZUIVG*
23	3	PNAVJWH
24	2	QOBWKXI
25	1	RPCXLYJ

* Recomputed; the notes had these rows slightly wrong (see corrections box).

- Forward + back = 26, because shifting forward 14 is the same as shifting back 12: $C + 14 \equiv C - 12 \pmod{26}$.
- Only one row is English: **GERMANY**. The attacker now has the plaintext *and* the key.
- **Which number is the key?** The sender encrypted with k = **12** (G 6 + 12 = 18 = S, ..., Y 24 + 12 = 36 ≡ 10 = K). The 14 is the forward shift that *undoes* it. The lecture calls 14 "the key"; be clear which one you mean in an exam.

Decryption vs. cryptanalysis

Decryption: the receiver *has* the key and reverses the encryption.
Cryptanalysis: an attacker *without* the key recovers the plaintext and/or the key (e.g. by brute force).

Exam tips

- Substitution = replace letters; transposition = rearrange letters. Know the list of ciphers under each.
- Write both formulas: $C = (p + k) \bmod 26$, $p = (C - k) \bmod 26$. Caesar is $k = 3$.
- Always show the wrap-around step (e.g. 27 mod 26 = 1) and fix negatives by adding 26.
- Key space is 25 (not 26): $k = 0$ changes nothing.
- Three cons of Caesar: algorithms known, only 25 keys, language recognisable.
- For brute force: draw the table, box the readable row, state the key.

Corrections to the notes / slides

- "neso academy" encrypts to QHVR DFDGHPB. The notes' final answer QHVRDFDGHB drops the P.
- Brute-force table: row 11 is DBOJXKV and row 22 is OMZUIVG. Row 12 (ECPKYLW) was already correct.
- The notes call con #1 a "Kerckhoffs violation". It isn't: public algorithms are normal (see the intro sheet). The problem is a public algorithm *plus* only 25 keys.
- Slide typo: "used *be* Julius Caesar" → "used *by*".

11 Monoalphabetic Cipher

Why: Caesar shifts every letter by the *same* amount, and there are only 25 keys. Find the shift for one letter and you have the whole message.

11.1 Permutations

A **permutation** of a finite set S is an ordered sequence of all the elements of S, with **each element appearing exactly once**.

$S = \{a, b, c\}$ has $3! = 6$ permutations: `abc`, `acb`, `bac`, `bca`, `cab`, `cba`. In general n elements have $n! = n(n-1)\cdots 2 \cdot 1$ permutations ($2! = 2$, $4! = 24$).

11.2 The cipher

The "cipher" line can be **any permutation of the 26 letters**. Each plaintext letter always maps to the same ciphertext letter, and no two plaintext letters share a ciphertext letter.

- Key space: $26! \approx 4 \times 10^{26}$ keys. This *seems* to rule out brute force (compare 25 for Caesar).
- It is still a **substitution** cipher. The permutation is applied to the *alphabet* (to build the key), not to the message (that would be transposition).

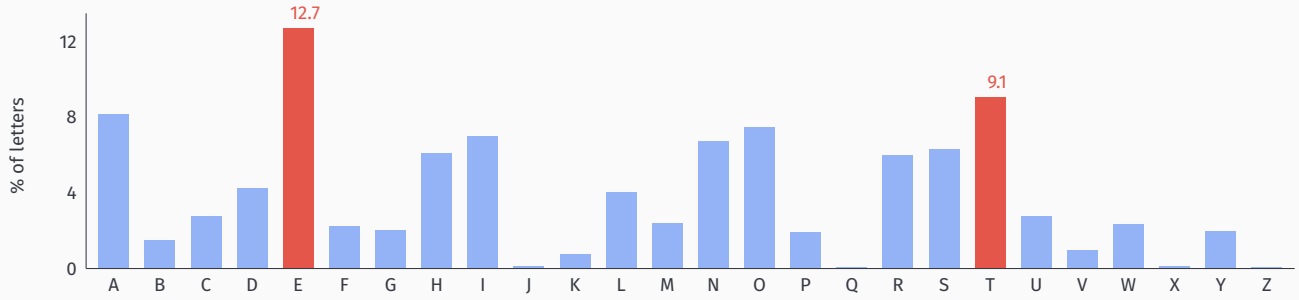
Example: "Neso" → `DULA`. The distance moved differs for every letter, so one letter's mapping tells you nothing about the others:

Plaintext	N (13)	E (4)	S (18)	O (14)
Ciphertext	**D** (3)	**U** (20)	**L** (11)	**A** (0)
Shift $(C - p) \bmod 26$	16	16	19	12

Caesar: shift is always 3.

11.3 Breaking it: frequency analysis

Letters in English are **not equally common**. A monoalphabetic cipher only renames letters, so the ciphertext keeps the same frequency pattern.



Most common: **E** (12.7%), **T** (9.1%), then A, O, I, N, S, H, R. Rarest: J, X, Q, Z.

Method: count ciphertext letters → the most frequent one is probably `e`, the next `t` → look for common words (`the`, `that`) and fill in the rest by guessing.

Lecture example (120 letters):

```
UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZ
VUEPHZHMDZSHZOWSFPAPPDTSVPQUZWYMXUZUHSX
EPYEPOPDZSZUFPOMBZWPFUPZHMDJUDTMOHMQ
```

1. Count: `P` appears 16 times (13.33%), `Z` 14 times (11.67%). Guess `P`=e, `Z`=t.
2. `ZW` shows up 3 times and `ZWP` appears once: t?e ⇒ "the", so `W`=h.
3. Keep guessing with common words until it reads as English:

it was disclosed yesterday that several informal but direct contacts have been made with political representatives of the viet cong in moscow

Recovered key (cipher → plain):

Cipher	A	B	D	E	F	G	H	I	J	M	O	**P**	Q	S	T	U	V	**W**	X	Y	**Z**
Plain	b	f	n	r	v	y	c	u	g	o	s	**e**	w	a	m	i	d	**h**	l	p	**t**

11.4 Pros, cons, countermeasure

Pros

1. Better security than Caesar cipher.

Cons

1. Easy to break because the ciphertext reflects the frequency data of the original alphabet.
2. Prone to guessing attacks using English letter frequencies.

Countermeasure: homophones. Give a common letter several possible substitutes (e.g. e gets many symbols, picked at random each time). This flattens the frequency peaks.

11.5 Homework: encrypt with a key phrase

Key: "The quick brown fox jumps over the lazy dog". Drop spaces and every repeated letter. It uses all 26 letters, so it becomes the whole cipher line:

Plain	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Cipher	T	H	E	Q	U	I	C	K	B	R	O	W	N	F	X	J	M	P	S	V	L	A	Z	Y	D	G

"Attack postponed to tomorrow and do not use our secret paper until further info" →

`TVVTEO JXSVJXFUQ VX VXNXPPXZ TFQ QX FXV LSU XLP SUEPUV JTJUP LFVBW ILPVKUP BFIX`

(Check one: a→T, t→V, t→V, a→T, c→E, k→O ⇒ TVVTEO.)

12 Playfair Cipher ★★

★ **In the papers:** 2024–25 Q2 (3 marks): key "GUIDANCE", message "The key is hidden under the door pad". 2025–26 Q3 (4 marks): given key matrix, "Insta Reels Pe Time Waste Na Kar", filler X, I/J in one cell. Both solved in §36.

- Also called the Playfair square or Wheatstone–Playfair cipher. A manual **symmetric** technique.
- **Invented** in 1854 by **Charles Wheatstone**; named after **Lord Playfair**, who **promoted** it.
- The first literal **digram** substitution cipher: it encrypts **two letters at a time**.

Single-letter vs. multiple-letter

Caesar and monoalphabetic: a letter always becomes the *same* ciphertext letter.
Playfair: a letter's ciphertext depends on its *partner* in the pair, so the same letter can become different letters. Single-letter frequencies are hidden.

12.1 Building the 5×5 key matrix

1. Write the keyword row by row, **skipping repeated letters**.
2. Fill the remaining cells with the unused letters **in alphabetical order**.
3. 26 letters, 25 cells: **I and J share one cell**. If the keyword contains I or J, that cell is already used.

Keyword **MONARCHY** (highlighted), then B, D, E, F, G, I/J, K, L, P, Q, S, T, U, V, W, X, Z.

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

12.2 Step 1: split the plaintext into digrams

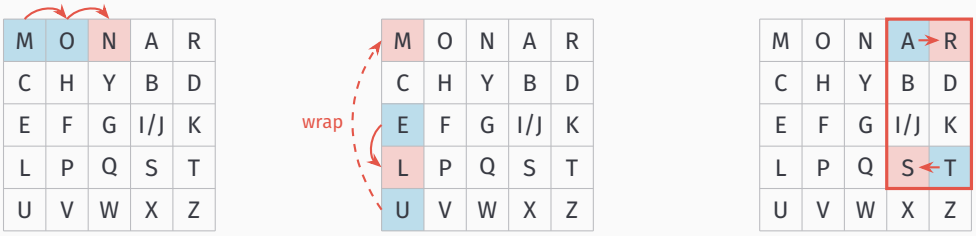
- Split into pairs. **Two identical letters in one pair** ⇒ insert filler x between them and re-pair from there.
- **Odd letter left over** at the end ⇒ add filler x.

Plaintext	Digrams	Why
attack	`at ta ck`	nothing to fix (`tt` is split across two pairs)
neso academy	`ne so ac ad em yx`	11 letters: `y` is left over, add `x`
balloon	`ba lx lo on`	`ll` in one pair: `l x`, then re-pair `l o o n`

12.3 Step 2: encrypt each pair (three cases) ★

Encryption rules

1. **Same row**: take the letter to the **right** of each (wrap to the start of the row).
2. **Same column**: take the letter **below** each (wrap to the top).
3. **Rectangle** (different row and column): each letter is replaced by the letter **in its own row**, in the **other letter's column**. Order matters: `at`→RS but `ta`→SR.



blue = plaintext pair, red = ciphertext

12.4 Worked examples (keyword MONARCHY)

"attack" → `at ta ck`
- `at`: rectangle → RS
- `ta`: rectangle → SR
- `ck`: rectangle (C row 2, K row 3) → DE

Answer: **RSSRDE**

"mosque" → `mo sq ue`
- `mo`: same row → ON
- `sq`: same row → TS
- `ue`: same column, U wraps to top → ML

Answer: **ONTSML**

Rapid fire:

Digram	oa	ab	gk	dc	rz	zr	mz	rm
Case	row	column	row	row	column	column	rectangle	row
Wrap?			K→E	D→C	Z→R	Z→R		R→M
Answer	**NR**	**BI**	**IE**	**CH**	**DR**	**RD**	**RU**	**MO**

(BI/IE: the I/J cell can be written as I or J.)

12.5 Solved question

Encrypt "Hide the gold under the carpet" with keyword "Neso Academy".

N	E	S	O	A
C	D	M	Y	B
F	G	H	I/J	K
L	P	Q	R	T
U	V	W	X	Z

Keyword letters: N E S O A C D M Y

25 letters (odd), so the last `t` gets a filler `x`. No pair has a repeated letter.

`hi de th eg ol du nd er th ec ar pe tx`

Pair	Case	Out	Pair	Case	Out
hi	row	IK	er	rect.	OP
de	column	GD	th	rect.	QK
th	rect.	QK	ec	rect.	ND
eg	column	DP	ar	rect.	OT
ol	rect.	NR	pe	column	VD
du	rect.	CV	tx	rect.	RZ
nd	rect.	EC			

Answer: **IKGDQKDPNRCVECOPQKNDOTVDRZ**

The plaintext `e` became D (in `de`, `eg`, `pe`), O (in `er`) and N (in `ec`). That is the multiple-letter property. But the same *pair* always gives the same output (`th`→QK both times). *(Not in the lecture: this is why Playfair can still be attacked using pair frequencies.)*

12.6 When I or J is in the keyword or the message ★

I/J rules

- I and J share one cell, so **replace every J with I before you do anything else**.
- In the **keyword**: a J takes the I/J cell. A later I (or J) in the keyword is then a **repeat**, so skip it.
- In the **message**: after the swap, `ij` or `ji` becomes `ii`, which is a **double letter** and needs a filler.
- The ciphertext never contains J; the cell is written as I. The receiver uses context to decide between i and j.

Example: encrypt "meet me in fiji" with keyword **JUPITER**.

I/J	U	P	T	E
R	A	B	C	D
F	G	H	K	L
M	N	O	Q	S
V	W	X	Y	Z

Keyword: J U P T E R

Matrix: J goes into the first cell as I/J. The I in "JUPITER" is a repeat of that cell, so it is skipped. Then come U P T E R and the rest of the alphabet.
Digrams: `meetmeinfiji` → swap J: `meetmeinfiii`
- `me et`: the `ee` is split across two pairs, so **no filler**.
- `fi`, then `ii` (was `ji`) is a double: `ix`, then the last `i` is alone: `ix`.

`me et me in fi ix ix`

Pair	Case		Out	Pair	Case		Out
me	rectangle		SI	fi	column	F→M, I→R	MR
et	row	E wraps to I	IE	ix	rectangle	I→P, X→V	PV
me	rectangle		SI	ix	rectangle		PV
in	rectangle	I→U, N→M	UM				

Ciphertext: **SIIESIUMMRPVPV**. Decrypting gives `me et me in fi ix ix`. Drop the fillers to get "meet me in fiii"; context says **meet me in fiji**.

12.7 Decryption

Reverse the rules

Same row ⇒ move **left**. Same column ⇒ move **up**. Rectangle ⇒ **same** rule as encryption. Then drop the filler x's.

Homework: decrypt ODZFQSEZSONTSW with keyword "Neso App".

N	E	S	O	A
P	B	C	D	F
G	H	I/J	K	L
M	Q	R	T	U
V	W	X	Y	Z

Keyword letters: N E S O A P

Pair	Case	Plain
OD	column (up; O wraps to bottom)	yo
ZF	column (up)	ua
QS	rectangle	re
EZ	rectangle	aw
SO	row (left)	es
NT	rectangle	om
SW	rectangle	ex

`yo ua re aw es om ex` → drop the filler → **you are awesome**.

Exam tips

- Monoalphabetic key space is 26!; Caesar's is 25. A bigger key space still falls to **frequency analysis**.
- Know E and T are the most frequent English letters; know what homophones are for.
- Playfair: inventor **Wheatstone** (1854), promoter **Lord Playfair**. It is a **digram** (multiple-letter) cipher.
- Matrix: keyword without repeats, then the rest of the alphabet, I/J in one cell. Always draw it.
- Fix digrams *before* encrypting: split doubled letters with x, pad an odd length with x.
- Row → right, column → down (with wrap), rectangle → own row, other's column. Decrypt: left, up, rectangle unchanged.

Corrections to the notes / slides

- **Frequency example:** Gemini's copy of the ciphertext has an extra Z. The correct text (above) has 120 letters, with P 16 times and Z 14 times (the notes said 131 letters, P 18 times). In the notes, the step "S = h, ZSZ = that" is wrong: W = h (from ZWP = the) and S = a.
- **Short frequency example** ("GZGEWVGRNCP" → "we execute plan") is left out. The letters in the notes don't produce that sentence (11 letters vs. 13, and the given mapping yields "ewexcteplan"). Check the video at 11:08 if you need it.
- Neso App homework: F→A is not a wrap (F is in row 2). Only O→Y wraps.
- Video 3 (about 08:00): the speaker says "W" / "MW" but means O / mo. The slide is right: mo→ON.
- Not covered in the video: if the filler itself would double (e.g. xx), a different filler such as q or z is normally used.

13 Hill Cipher

- A **multiletter** substitution cipher by mathematician **Lester Hill (1929)**. Better security than Playfair.

- Encrypts a **block of m letters at a time** using an $m \times m$ **key matrix** K: 2×2 for digraphs, 3×3 for trigraphs, $m \times m$ for polygraphs.
- Like Playfair, the same letter can become different ciphertext letters depending on its block.
- Maths needed: matrix multiplication mod 26 (enough for encryption); **determinant** and **multiplicative inverse** mod 26 (needed for decryption).

Hill cipher

Encrypt: $C = PK \bmod 26$ Decrypt: $P = CK^{-1} \bmod 26$

Works because $CK^{-1} = (PK)K^{-1} = P(KK^{-1}) = PI = P$.

For $m = 3$, the plaintext block is a **row vector** $(P_1\ P_2\ P_3)$:

$$(C_1\ C_2\ C_3) = (P_1\ P_2\ P_3)\begin{pmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{pmatrix} \bmod 26$$

$$\begin{aligned} C_1 &= (P_1K_{11} + P_2K_{21} + P_3K_{31}) \bmod 26 \\ C_2 &= (P_1K_{12} + P_2K_{22} + P_3K_{32}) \bmod 26 \\ C_3 &= (P_1K_{13} + P_2K_{23} + P_3K_{33}) \bmod 26 \end{aligned}$$

Sizes: $(1 \times 3)(3 \times 3) = (1 \times 3)$. Each C_j = plaintext row · column j of K.

PK **vs.** KP**:** this lecture uses $C = PK$ (plaintext as a row). Some books use a **column** vector: $C = KP$, and then decryption is $P = K^{-1}C$. Use one convention consistently.

13.1 Encryption example: "pay more money"

$$K = \begin{pmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{pmatrix}$$

Blocks of 3: `pay` | `mor` | `emo` | `ney` (pad with x if needed)

Block 1, `pay` = (15 0 24):

$$\begin{aligned} C_1 &= 15 \cdot 17 + 0 \cdot 21 + 24 \cdot 2 = 255 + 0 + 48 = 303 = 11 \cdot 26 + \mathbf{17} && \to \mathrm{R} \\ C_2 &= 15 \cdot 17 + 0 \cdot 18 + 24 \cdot 2 = 255 + 0 + 48 = 303 \equiv \mathbf{17} && \to \mathrm{R} \\ C_3 &= 15 \cdot 5 + 0 \cdot 21 + 24 \cdot 19 = 75 + 0 + 456 = 531 = 20 \cdot 26 + \mathbf{11} && \to \mathrm{L} \end{aligned}$$

Block	P	PK (before mod)	mod 26	Ciphertext
pay	(15, 0, 24)	(303, 303, 531)	(17, 17, 11)	**RRL**
mor	(12, 14, 17)	(532, 490, 677)	(12, 22, 1)	**MWB**
emo	(4, 12, 14)	(348, 312, 538)	(10, 0, 18)	**KAS**
ney	(13, 4, 24)	(353, 341, 605)	(15, 3, 7)	**PDH**

Answer: **RRLMWBKASPDH**.

13.2 Decryption: finding K^{-1}

Recipe

$K^{-1} = (\det K)^{-1} \cdot \mathrm{adj}(K) \bmod 26$

1. $\det K \bmod 26$.
2. Its multiplicative inverse mod 26.
3. $\mathrm{adj}(K) \bmod 26$.
4. Multiply (2) by (3), reduce mod 26. Check $KK^{-1} \equiv I$.

Step 1: determinant (expand along row 1, signs + − +):

$$\det K = 17\begin{vmatrix} 18 & 21 \\ 2 & 19 \end{vmatrix} - 17\begin{vmatrix} 21 & 21 \\ 2 & 19 \end{vmatrix} + 5\begin{vmatrix} 21 & 18 \\ 2 & 2 \end{vmatrix} = 17(300) - 17(357) + 5(6) = 5100 - 6069 + 30 = -939$$

$-939 = -36 \cdot 26 - 3$, so $\det K \equiv -3 \equiv -3 + 26 = \mathbf{23} \pmod{26}$.

Step 2: inverse of 23 mod 26. Find x with $23x \bmod 26 = 1$ by trying $x = 1, 2, 3, \ldots$

x	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	**17**
$23x \bmod 26$	23	20	17	14	11	8	5	2	25	22	19	16	13	10	7	4	**1**

$23 \times 17 = 391 = 15 \cdot 26 + 1$, so $\mathbf{23^{-1} \equiv 17}$. An inverse exists only if $\gcd(\det K, 26) = 1$, i.e. $\det K$ is odd and not 13. Otherwise the key can't be used, because messages could not be decrypted.

Step 3: adjoint, using the 5×5 **shortcut.**

1. Copy the first two columns to the right, then the first two rows to the bottom (5 × 5).
2. Delete the first row and first column. That leaves a 4 × 4 grid.
3. Slide a 2 × 2 window over the grid. Each window gives $ad - bc$. **Columns 1–2 of the grid give row 1 of** adj K (windows at rows 1–2, 2–3, 3–4); columns 2–3 give row 2; columns 3–4 give row 3. The signs come out automatically.

$$\begin{array}{ccccc} 17 & 17 & 5 & 17 & 17 \\ 21 & 18 & 21 & 21 & 18 \\ 2 & 2 & 19 & 2 & 2 \\ 17 & 17 & 5 & 17 & 17 \\ 21 & 18 & 21 & 21 & 18 \end{array} \Rightarrow \left[\begin{array}{cc|cc} 18 & 21 & 21 & 18 \\ 2 & 19 & 2 & 2 \\ \hline 17 & 5 & 17 & 17 \\ 18 & 21 & 21 & 18 \end{array}\right]$$

first window: $18 \cdot 19 - 21 \cdot 2 = 300$

$$\operatorname{adj} K = \begin{pmatrix} 18\cdot 19 - 21\cdot 2 & 2\cdot 5 - 19\cdot 17 & 17\cdot 21 - 5\cdot 18 \\ 21\cdot 2 - 21\cdot 19 & 19\cdot 17 - 2\cdot 5 & 5\cdot 21 - 17\cdot 21 \\ 21\cdot 2 - 18\cdot 2 & 2\cdot 17 - 2\cdot 17 & 17\cdot 18 - 17\cdot 21 \end{pmatrix} = \begin{pmatrix} 300 & -313 & 267 \\ -357 & 313 & -252 \\ 6 & 0 & -51 \end{pmatrix}$$

$$\equiv \begin{pmatrix} 14 & -1 & 7 \\ -19 & 1 & -18 \\ 6 & 0 & -25 \end{pmatrix} \equiv \begin{pmatrix} 14 & 25 & 7 \\ 7 & 1 & 8 \\ 6 & 0 & 1 \end{pmatrix} \pmod{26}$$

(Reduce mod 26, then add 26 to each negative: $-1 \to 25$, $-19 \to 7$, $-18 \to 8$, $-25 \to 1$.)

Step 4: combine.

$$K^{-1} = 17 \begin{pmatrix} 14 & 25 & 7 \\ 7 & 1 & 8 \\ 6 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 238 & 425 & 119 \\ 119 & 17 & 136 \\ 102 & 0 & 17 \end{pmatrix} \equiv \boxed{\begin{pmatrix} 4 & 9 & 15 \\ 15 & 17 & 6 \\ 24 & 0 & 17 \end{pmatrix}} \pmod{26}$$

Check: $KK^{-1} = \begin{pmatrix} 443 & 442 & 442 \\ 858 & 495 & 780 \\ 494 & 52 & 365 \end{pmatrix} \equiv \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = I$ ✓ ($443 = 17 \cdot 26 + 1$, $442 = 17 \cdot 26$, ...)

13.3 Decryption example: RRLMWBKASPDH

Block	C	CK^{-1} (before mod)	mod26	Plaintext
RRL	(17, 17, 11)	(587, 442, 544)	(15, 0, 24)	**pay**
MWB	(12, 22, 1)	(402, 482, 329)	(12, 14, 17)	**mor**
KAS	(10, 0, 18)	(472, 90, 456)	(4, 12, 14)	**emo**
PDH	(15, 3, 7)	(273, 186, 362)	(13, 4, 24)	**ney**

e.g. $P_1 = 17 \cdot 4 + 17 \cdot 15 + 11 \cdot 24 = 68 + 255 + 264 = 587 = 22 \cdot 26 + 15 \to$ p. Answer: **pay more money**.

Not in the lecture: Hill hides single-letter frequencies well, but it is linear. If an attacker knows some plaintext–ciphertext pairs, they can solve for K.

14 Polyalphabetic Ciphers

Monoalphabetic ciphers fall to frequency analysis because every letter always gets the same substitute. **Polyalphabetic** ciphers use **different** monoalphabetic rules at different positions:

1. A **set of related monoalphabetic substitution rules** is used.
2. A **key** decides which rule is used for each letter.

14.1 Vigenère cipher ★

★ **In the papers:** 2025–26 Q1 (8 marks, the biggest question so far): one-time-pad version of Vigenère, encrypt "sendmoremoney" with a key stream, then find the key that turns the ciphertext into "cashnotneeded". 2024–25 Q1(c): characters changed.

The rule set is the **26 Caesar ciphers** (shifts 0–25). Each key letter picks the shift for one plaintext letter. The key is **repeated** to the length of the message.

Vigenère (key length m)

$$C_i = (P_i + K_{i \bmod m}) \bmod 26 \qquad P_i = (C_i - K_{i \bmod m}) \bmod 26$$

Example: plaintext `wearediscoveredsaveyourself` (27 letters), key `deceptive` (9 letters, repeated 3 times).

Key	d	e	c	e	p	t	i	v	e	d	e	c	e	p
K	3	4	2	4	15	19	8	21	4	3	4	2	4	15
Plain	w	e	a	r	e	d	i	s	c	o	v	e	r	e
P	22	4	0	17	4	3	8	18	2	14	21	4	17	4
(*P*+*K*) mod 26	25	8	2	21	19	22	16	**13**	6	17	25	6	21	19
Cipher	**Z**	**I**	**C**	**V**	**T**	**W**	**Q**	**N**	**G**	**R**	**Z**	**G**	**V**	**T**

Key	t	i	v	e	d	e	c	e	p	t	i	v	e
K	19	8	21	4	3	4	2	4	15	19	8	21	4
Plain	d	s	a	v	e	y	o	u	r	s	e	l	f
P	3	18	0	21	4	24	14	20	17	18	4	11	5
(*P*+*K*) mod 26	22	**0**	21	25	7	**2**	16	24	**6**	**11**	12	**6**	9
Cipher	**W**	**A**	**V**	**Z**	**H**	**C**	**Q**	**Y**	**G**	**L**	**M**	**G**	**J**

Answer: **ZICVTWQNGRZGVTWAVZHCQYGLMGJ** (highlighted cells wrapped past 25).

Why it's stronger: the same plaintext letter gets different ciphertext letters. e at position 2 (key e=4) → 8 = I; e at position 5 (key p=15) → 19 = T. Letter frequencies get flattened.

Cryptanalysis: the weakness is that the **key repeats**. Once the attacker finds the key length *m*, they split the ciphertext into *m* groups (every *m*-th letter). Each group is just a **Caesar cipher**, which falls to frequency analysis or brute force.

14.2 Autokey system ★

★ **In the papers:** 2024–25 Q1(d): one plaintext change alters **2** ciphertext characters in auto-key (the letter itself and the one it keys).

To stop the key from repeating, Vigenère proposed the **autokey**: the keyword is followed by the **plaintext itself**, giving a **running key** as long as the message.

Key	**d**	**e**	**c**	**e**	**p**	**t**	**i**	**v**	**e**	w	e	a	r	e	d	i	s	c	o	v	e	r	e	d	s	a	v
Plain	w	e	a	r	e	d	i	s	c	o	v	e	r	e	d	s	a	v	e	y	o	u	r	s	e	l	f
Cipher	Z	I	C	V	T	W	Q	N	G	K	Z	E	I	I	G	A	S	X	S	T	S	L	V	V	W	L	A

Answer: **ZICVTWQNGKZEIIGASXSTSLVVWLA**. The first 9 letters match Vigenère (same key); after that the key is the plaintext (w e a r e …).

- No period to find, so the key-length attack above fails.
- Still breakable: the key is ordinary English too, so key and plaintext **share the same letter frequencies**, and statistics can still be applied.

14.3 Vernam cipher

- **Gilbert Vernam**, AT&T engineer, **1918**.
- Works on **binary bits**, not letters.
- Goal: a key **as long as the message** that **does not repeat**, so no statistical pattern is left.

Vernam

$C_i = P_i \oplus K_i$ $\qquad$ $P_i = C_i \oplus K_i$

$\oplus$ = XOR: output 1 if the bits differ, 0 if they are equal.

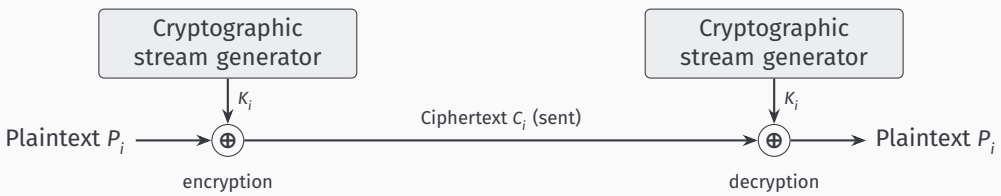
a	*b*	*a* ⊕ *b*
0	0	0
0	1	1
1	0	1
1	1	0

P	1011
K	0110
C	1101
C ⊕ *K*	1011 = *P*

(bit example is mine, not from the lecture)

Decryption works because XOR undoes itself:

$$C_i \oplus K_i = P_i \oplus K_i \oplus K_i = P_i \oplus 0 = P_i$$



Both sides need the **same, synchronised key stream**.

Weakness: Vernam's machine read the key from a **loop of paper tape**, so the key eventually **repeated**. With enough ciphertext, and some known or probable plaintext, it can be broken. **Fix:** make the key truly random, as long as the message, and never reused. That is the **one-time pad**, which gives perfect secrecy (next topic).

Exam tips

- Hill: $C = PK \bmod 26$, $P = CK^{-1} \bmod 26$. State which convention (PK or KP) you use.
- $K^{-1} = (\det K)^{-1} \operatorname{adj} K \bmod 26$. Show every step: det, inverse (table), adjoint, product, then check $KK^{-1} = I$.
- Negative numbers mod 26: add 26 until the result is in 0–25.
- The key matrix must have $\gcd(\det K, 26) = 1$, or it can't be inverted.
- Polyalphabetic = many monoalphabetic rules, and the key picks which one. Vigenère = 26 Caesar ciphers.
- Vigenère weakness: the repeating key. Autokey fixes it (keyword + plaintext), but key and plaintext share frequencies.
- Vernam: Gilbert Vernam, 1918, bits, XOR. Weakness: repeating tape. Leads to the one-time pad.

Corrections to the notes / slides

- All of Gemini's numbers for this batch are correct (Hill encryption, det, adjoint, K^{-1}, decryption, Vigenère, autokey; checked by script).
- Slide typos noted by Gemini: for `ney`, the slide shows the old vector $(348, 312, 538)$ instead of $(353, 341, 605)$. For RRL, the slide shows (17 17 14) labelled C; it should be (17 17 11) and the output is P.
- The notes put "key and plaintext share letter frequencies" under Vigenère cryptanalysis. That point is really about the autokey system, where the key *is* plaintext, so it is placed there above.

15 One-Time Pad ★

★ **In the papers:** 2025–26 Q1 (8 marks): OTP-Vigenère with key stream 9 0 1 7 23 15 21 14 11 11 2 8 9 (Stallings 2.x). Part (b), finding a key that maps the ciphertext to another plaintext, is exactly why an OTP is unbreakable: every plaintext of the same length is possible.

The last substitution technique. It is an **improvement to the Vernam cipher**:

One-time pad

- The key is **random** and **as long as the message**, so it never repeats.
- The key is used to encrypt and decrypt **a single message**, then **discarded**. Each new message needs a new key of the same length.
- Such a scheme is **unbreakable**: "the ultimate in security".

15.1 How it grew out of the earlier ciphers

Cipher	Key	What goes wrong
Vigenère	short keyword, **repeated**	repeats every m letters ⇒ split into Caesar ciphers
Autokey	keyword + plaintext	key is English ⇒ has English letter frequencies
Vernam	long key stream (paper tape)	tape loops, so the key **eventually repeats**
One-time pad	**truly random**, **same length** as message, **used once**	nothing to exploit

The OTP is a Vigenère/Vernam cipher whose key *never* repeats and has *no* pattern. The formulas don't change:

$$C_i = (P_i + K_i) \bmod 26, \quad P_i = (C_i - K_i) \bmod 26 \quad \text{(letters)} \qquad C_i = P_i \oplus K_i, \quad P_i = C_i \oplus K_i \quad \text{(bits)}$$

15.2 Example 1: letters

Plaintext `attackatdawn`, random key `qvhzmxkbplwe` (12 letters for 12 letters):

Plain	a	t	t	a	c	k	a	t	d	a	w	n
P	0	19	19	0	2	10	0	19	3	0	22	13
Key	q	v	h	z	m	x	k	b	p	l	w	e
K	16	21	7	25	12	23	10	1	15	11	22	4
$P + K$	16	**40**	**26**	25	14	**33**	10	20	18	11	**44**	17
mod26	16	14	0	25	14	7	10	20	18	11	18	17
Cipher	**Q**	**O**	**A**	**Z**	**O**	**H**	**K**	**U**	**S**	**L**	**S**	**R**

Ciphertext **QOAZOHKUSLSR**. The receiver computes $(C - K) \bmod 26$ letter by letter to get `attackatdawn` back, then destroys the key. Note the two `t`'s in `att` became O and A. The two S's in the ciphertext came from different letters (d, w).

15.3 Example 2: bits

"HI" in 8-bit ASCII, XOR a random key:

Plaintext (H, I)	01001000	01001001	
Key	10110101	01101100	
Ciphertext $P \oplus K$	11111101	00100101	
$C \oplus K$	01001000	01001001	= “HI” again

15.4 Why it can’t be broken

- It produces random output with **no statistical relationship to the plaintext**.
- The ciphertext contains **no information whatsoever** about the plaintext, so there is simply no way to break the code.
- The security comes **entirely from the randomness of the key**.

Look at one letter. The attacker sees Q. Every plaintext letter could have produced it, each with exactly one key letter:

If the plaintext was	a	b	c	d	e	…	z
then the key was	q	p	o	n	m	…	r

The key is random, so all 26 cases are **equally likely**. Seeing Q tells the attacker nothing.

Look at the whole message. The same ciphertext decrypts to anything of the same length, depending on the key:

Ciphertext	Key tried	Result
QOAZOHKUSLSR	qvhzmxkbplwe	**attack at dawn**
QOAZOHKUSLSR	zkhikhruztku	**retreat at six**
KXFUDM	kemubc	**attack**
KXFUDM	htaqqj	**defend**

Brute force produces *every* meaningful message of that length, and there is nothing to say which one is right. (Compare Caesar: only one of the 25 tries made sense, GERMANY.)

15.5 The rules, and what breaks if you bend them

Rule 1: never reuse a key (the “two-time pad”)

Encrypt a second message holdposition with the *same* key qvhzmxkbplwe: C_2 = XJSCBLCJITKR.

- Subtracting the ciphertexts cancels the key: $C_1 - C_2 = (P_1 + K) - (P_2 + K) = P_1 - P_2$ (TFIXNWILKSIA both ways). What’s left is two English texts mixed together, which has structure the attacker can attack.
- Worse: if the attacker knows or guesses P_1 (“attack at dawn” is a common phrase), then $K = C_1 - P_1$ = qvhzmxkbplwe, and $P_2 = C_2 - K$ = **hold position**.

With bits it is the same: $C_1 \oplus C_2 = P_1 \oplus P_2$.

Rule 2: the key must be truly random

If the key is text from a book (a “running key”), it has English letter frequencies, and it can be attacked like the autokey. If it comes from a formula or a short seed, it is really a stream cipher: at best *computationally* secure, not perfect.

Rule 3: the key must be at least as long as the message

A shorter key has to repeat, and then you’re back to Vigenère/Vernam and their weaknesses.

Known plaintext doesn’t hurt an OTP. If the attacker learns one plaintext and its ciphertext, they get $K = C - P$, but only for that message. That key is never used again.

15.6 Two fundamental difficulties

1. **Making large quantities of random keys.** Every message needs fresh random key material of the same size.
2. Even more daunting: **key distribution and protection**. Both sides need the same huge key, delivered and stored securely. (If you had a secure way to send that much key, you could send the message that way instead.)

So the one-time pad is of **limited use**. It suits **low-bandwidth channels that need very high security**.

15.7 Perfect secrecy

The one-time pad is the **only** cryptosystem with **perfect secrecy**: the ciphertext reveals **absolutely nothing** about the plaintext.

This is exactly the **unconditionally secure** idea from Section 5: no amount of ciphertext or computing power helps. (Formally, $P(\text{plaintext} \mid \text{ciphertext}) = P(\text{plaintext})$, a definition due to Shannon.) Everything else in this course (AES, RSA, ...) is only **computationally** secure: breaking it is possible in principle, just too slow or too costly.

16 Transposition Ciphers

Some sort of **permutation** is applied to the plaintext letters. Letters are **not replaced**; only their **positions** change. The simplest such cipher is the **rail fence**.

16.1 Rail fence

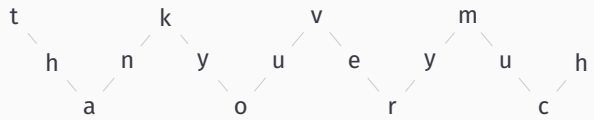
Write the plaintext **down and up along diagonals** over D rows (the **depth**), then **read off row by row**. Spaces and punctuation are dropped.

Example: "neso academy is the best", depth 2 (20 letters):



Row 1: NSAAEYSHBS Row 2: EOCDMITEET ⇒ **NSAAEYSHBSEOCDMITEET**

Depth 3, "Thank you very much" (16 letters). The path goes 1, 2, 3, 2, 1, 2, 3, ... (repeats every $2(D-1) = 4$ letters):



Rows: TKVM + HNYUEYUH + AORC ⇒ **TKVMHNYUEYUHAORC**

Homework ("neso academy is the best", depth 3): rows NAESB + EOCDMITEET + SAYHS ⇒ **NAESBEOCDMITEETSAYHS**.

16.2 Decrypting a rail fence

The receiver knows the depth D. The problem is that they don't know where one row ends and the next begins in the ciphertext. So first work out **how many letters each row holds**.

Rail fence decryption, step by step

1. Count the letters L. The zig-zag pattern of rows repeats every $2(D-1)$ letters.
2. Write the **row number of every position** (1, 2, ..., D, ..., 2, 1, 2, ...) and **count** how many times each row appears.
3. **Cut** the ciphertext into pieces of those lengths: the first piece is row 1, the next is row 2, and so on.
4. Put each piece back into its row's positions, left to right. Then **read along the zig-zag** (position 1, 2, 3, ...).

Example 1: depth 2. NSAAEYSHBSEOCDMITEET (20 letters). Pattern 1 2 1 2 ... so each row has 10 letters: row 1 = NSAAEYSHBS, row 2 = EOCDMITEET. Take letters alternately from row 1 and row 2: NESOAC... ⇒ **nesoacademyisthebest**. (Depth-2 shortcut: split in half and interleave. With an odd length, row 1 gets the extra letter.)

Example 2: depth 3. TKVMHNYUEYUHAORC (16 letters). The pattern repeats every $2(3-1) = 4$:

Position	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Row	1	2	3	2	1	2	3	2	1	2	3	2	1	2	3	2

Row 1 appears 4 times, row 2 8 times, row 3 4 times. Cut: TKVM | HNYUEYUH | AORC. Fill the row-1 positions (1, 5, 9, 13) with T K V M, the row-2 positions with H N Y U E Y U H, and the row-3 positions with A O R C. Reading positions 1 to 16 gives **thankyouverymuch**.

Example 3: depth 4, unequal rows (the tricky case). CGIRORYSYTAHFNPPU (17 letters). The pattern repeats every $2(4-1) = 6$:

Position	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Row	1	2	3	4	3	2	1	2	3	4	3	2	1	2	3	4	3

Counts: row 1 = 3, row 2 = 5, row 3 = 6, row 4 = 3 (total 17 ✓). Cut: CGI | RORYS | YTAHFN | PPU. Put the pieces back:



Read along the line: **cryptographyisfun** (“cryptography is fun”). *Trap:* don’t assume the rows are equal, or that the middle rows are equal to each other. Here rows 2 and 3 have different lengths because the message stops partway through a cycle.

Check yourself: decrypt the homework answer NAESBEOCDMITEETSAYHS (depth 3). The counts are 5, 10, 5.

16.3 Row–column transposition

A more complex scheme

Write the message into a rectangle **row by row**. **Read** it **column by column**. The **key** is the **order of the columns**. Fill leftover cells with filler letters (e.g. x y z).

Example: “Kill Corona Virus at twelve am tomorrow” (33 letters), 7 columns × 5 rows = 35 cells, so 2 fillers (y, z). Key **4 3 1 2 5 6 7**.

4	3	**1**	2	5	6	7
k	i	l	l	c	o	r
o	n	a	v	i	r	u
s	a	t	t	w	e	l
v	e	a	m	t	o	m
o	r	r	o	w	y	z

numbers above = read order

Order	Column	Letters
1	col 3	LATAR
2	col 4	LVTMO
3	col 2	INAER
4	col 1	KOSVO
5	col 5	CIWTW
6	col 6	OREOY
7	col 7	RULMZ

Answer: **LATARLVTMOINAERKOSVOCIWTWOREOYRULMZ**

16.4 Two stages: more complex

A single transposition leaves patterns that can be pieced back together. So **feed the ciphertext back in** and transpose again with the same key:

4	3	**1**	2	5	6	7
L	A	T	A	R	L	V
T	M	O	I	N	A	E
R	K	O	S	V	O	C
I	W	T	W	O	R	E
O	Y	R	U	L	M	Z

Read columns 3, 4, 2, 1, 5, 6, 7:
TOOTR AISWU AMKWY LTRIO RNVOL LAORM VECEZ

Answer: **TOOTRAISWUAMKWYLTRIORNVOLLAORMVECEZ**. The full key is now: the rectangle size, the column order and the number of stages.

Decrypting (not shown in the lecture): the receiver knows the rectangle size, so each column has 5 letters. Write the ciphertext *into columns* in key order, then read the rows. For two stages, do this twice.

17 Classical Ciphers at a Glance

Cipher	Type	Key	Main weakness
Caesar	substitution, 1 letter	shift 3 (fixed)	algorithm known, no real key
Shift	substitution, 1 letter	shift 1–25	only 25 keys: brute force
Monoalphabetic	substitution, 1 letter	permutation (26!)	letter frequencies preserved
Playfair	substitution, 2 letters	5×5 keyword matrix	same pair → same output
Hill	substitution, m letters	invertible $m \times m$ matrix	linear (known plaintext)
Vigenère	polyalphabetic	repeating keyword	key repeats: split into Caesars
Autokey	polyalphabetic	keyword + plaintext	key has English frequencies
Vernam	polyalphabetic, bits (XOR)	key stream	tape key repeated
One-time pad	substitution	random, once, message length	none (perfect secrecy); key generation and distribution
Rail fence	transposition	depth	few possibilities
Row–column	transposition	column order (+ stages)	one stage can be reconstructed

Transposition ciphers keep the letter frequencies of the plaintext unchanged (only positions move).

Exam tips

- OTP: random key, same length as message, used once then discarded ⇒ unbreakable, perfect secrecy.
- Reusing an OTP key: $C_1 - C_2 = P_1 - P_2$ (or $C_1 \oplus C_2 = P_1 \oplus P_2$), so the key cancels out. One known plaintext then reveals the other message.
- Know the two difficulties word for word: (1) making large quantities of random keys; (2) key distribution and protection.
- Substitution changes letters; transposition changes positions.
- Rail fence: draw the zig-zag, then read row by row. State the depth.
- Row–column: draw the grid with the key above it, pad the last row, and read columns in key order. Two stages = do it again on the ciphertext.

Corrections to the notes / slides

- "Thank you very much", depth 3, is TKVMHNYUEYUH**AORC**. The notes' final string ends in AORE, but their own row reading gives AORC.
- Row–column: the speaker says "R U L M C" for column 7, but the grid (and slide) give RULMZ. All other results were checked by script and are correct.
- The OTP formulas, Shannon and $P(M \mid C) = P(M)$ come from Gemini's notes, not the slides. They are standard and correct.

18 Traditional Ciphers: Remaining Topics (Lecture 5)

18.1 Multiplicative and affine ciphers ★

★ **In the papers:** 2024–25 Q1(a,b): one plaintext change alters only **1** ciphertext character in multiplicative and affine ciphers (each letter is enciphered on its own).

Cipher	Encrypt	Decrypt / key space
Additive (shift)	$C = (P + k) \bmod 26$	$P = (C - k) \bmod 26$; 26 keys
Multiplicative	$C = (P \times k) \bmod 26$	$P = (C \times k^{-1}) \bmod 26$; $k \in \mathbb{Z}_{26}^*$: **12 keys**
Affine	$C = (Pk_1 + k_2) \bmod 26$	$P = ((C - k_2)k_1^{-1}) \bmod 26$; $12 \times 26 =$ **312** keys

$\mathbb{Z}_{26}^* = \{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}$ (odd and not 13).

- Multiplicative, $k = 7$: hello $= 7, 4, 11, 11, 14 \to 49, 28, 77, 77, 98 \bmod 26 = 23, 2, 25, 25, 20 =$ XCZZU.
- Affine, $(k_1, k_2) = (7, 2)$: hello → ZEBBW. Decrypt with $7^{-1} = 15$: $(25 - 2) \cdot 15 = 345 \equiv 7 = h$, etc.

Chosen-plaintext attack on affine (slide example): Eve encrypts et on Alice's machine with two candidate algorithms and gets WC and WF.

- et→WC: $4k_1 + k_2 \equiv 22$, $19k_1 + k_2 \equiv 2$. Subtract: $15k_1 \equiv -20 \equiv 6 \Rightarrow k_1 = 16$: **rejected** ($\gcd(16, 26) \neq 1$).
- et→WF: $15k_1 \equiv 5 - 22 \equiv 9$, $15^{-1} = 7$, $k_1 = 63 \bmod 26 =$ **11**, $k_2 = 22 - 44 \equiv$ **4**. Decrypting the intercepted text gives "best time of the year is spring when flowers bloom".

18.2 Cryptanalysis of Vigenère: the Kasiski test

1. Find repeated segments (≥ 3 letters) in the ciphertext; note the distances $d_1, d_2, \ldots$
2. The key length m divides $\gcd(d_1, d_2, \ldots)$.
3. Split the ciphertext into m interleaved pieces (letters $1, m{+}1, 2m{+}1, \ldots$); each piece is a plain shift cipher, so break each by frequency analysis.

Slide example: JSU (100), SUM (48), VWV (60), MPH (8): gcd = 4 ⇒ try $m = 4$. Each piece keeps single-letter frequencies even though the whole ciphertext doesn't.

18.3 Cryptanalysis of Hill: known-plaintext

Brute force is hopeless ($26^{m \times m}$ keys) and Hill hides single-letter, digram and trigram statistics. But with m known plaintext/ciphertext blocks, stack them as rows of P and C: since $C = PK$, $\boxed{K = P^{-1}C}$ (if P is invertible mod 26; otherwise use other blocks).

$$P = \begin{pmatrix} 5 & 7 & 10 \\ 13 & 17 & 7 \\ 0 & 5 & 4 \end{pmatrix}, \quad C = \begin{pmatrix} 3 & 6 & 0 \\ 14 & 16 & 9 \\ 3 & 17 & 11 \end{pmatrix}, \quad \det P \equiv 9, \quad P^{-1} = \begin{pmatrix} 21 & 14 & 1 \\ 0 & 8 & 25 \\ 13 & 3 & 8 \end{pmatrix}, \quad K = P^{-1}C = \begin{pmatrix} 2 & 3 & 7 \\ 5 & 7 & 9 \\ 1 & 2 & 11 \end{pmatrix}$$

(All entries mod 26, verified.)

18.4 Rotor ciphers and Enigma

A **rotor cipher** is a monoalphabetic substitution whose mapping **changes after every character** (the rotor turns one position). **Enigma** (Scherbius; German army, WWII): keyboard, plugboard, three rotors (slow, middle, fast) and a reflector; the wiring changes as you type.

18.5 Transposition: keyless, keyed, combined ★

★ **In the papers:** 2024–25 Q3(b): "abcdefghij" → "CABDEHFGIJ" with a keyed transposition: the pattern repeats every 5 letters, so the permutation key has size **5**.

- **Keyless**: rail fence (`meetmeatthepark` → MEMATEAKETETHPR), or write row by row into 4 columns and read column by column (→ MMTAEEHREAEKTTP).
- **Keyed (block) transposition**: split into blocks and permute each with the key. `enemy attac kston ightz`, encryption key `3 1 4 5 2` (output position i takes input character key[i]) → EEMYN TAACT TKONS HITZG.
- **Keyed columnar (combined)**: write row by row, permute columns with the key, read column by column → ETTHEAKIMAOTYCNZNTSG.
- **Key inversion**: for each index i, `DecKey[EncKey[i]] = i`. Encryption key `3 1 4 5 2` → decryption key `2 5 1 3 4`; `2 6 3 1 4 7 5` → `4 1 3 5 7 2 6`.
- **As matrices**: the key is a permutation matrix (one 1 in each row and column); $C = PK$, and $K^{-1} = K^T$, so decryption just transposes the key matrix.
- **Cryptanalysis**: single-letter frequencies survive (statistical attack); brute force over $1! + 2! + \cdots + L!$ keys, but the column count must divide L ($L = 20 \Rightarrow 2, 4, 5, 10$); **pattern attack** on the regular position pattern (03 08 13 18 01 06 ...). **Double transposition** (repeat the whole thing, usually with the same key) destroys the regular pattern.

18.6 Stream vs. block ciphers

Stream: one symbol at a time with a key stream, $c_i = E_{k_i}(p_i)$: additive, monoalphabetic, Vigenère, OTP. **Block**: $m > 1$ symbols encrypted together with one key; each ciphertext block depends on the whole plaintext block: Playfair, Hill.

Trap: Vigenère is a stream cipher

Even though the key repeats every m letters, Vigenère encrypts one character at a time, each with its own key element: a **stream** cipher. Playfair works on pairs, Hill on m-letter blocks: **block** ciphers.

19 Steganography

Steganography conceals the existence of the message: the message is hidden. It is **not an encryption scheme**. **Cryptography** renders the message **unintelligible** to outsiders by transforming the text.

	Cryptography	Steganography
Hides	the **meaning**	the **existence**
Attacker sees	a scrambled message; knows something secret is being sent	an ordinary-looking text/image; doesn't suspect anything
Output	ciphertext	a normal-looking **cover** (text, image, audio) with the message inside

This is the other kind of "security-related transformation T" mentioned in the network security model (Section 1): T doesn't have to be encryption.

Examples from the lecture (read the first letters):

- **S**imply **e**ncrypt **c**orrect **r**eading **e**xactly **t**wice. ⇒ **Secret**
- **T**ell **h**enry **a**lice **n**ot **k**ane / **Y**esterday **o**urplan **u**ndone! ⇒ **Thank You**

19.1 Historical techniques

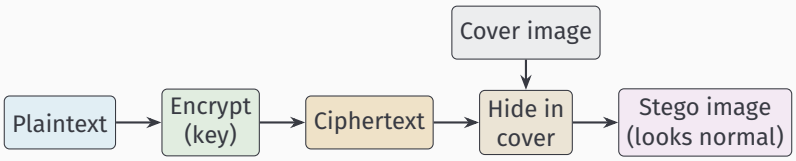
Character marking	Chosen letters of a printed text are overwritten faintly in pencil. Visible only when the paper is held at an angle to bright light.
Invisible ink	Written with a substance (e.g. lemon juice) that leaves no trace until heat or a chemical is applied.
Pin punctures	Tiny pin holes on selected letters. Invisible until the paper is held up to a light.
Typewriter correction ribbon	Text typed between the lines with a correction ribbon. Visible only under strong light.

19.2 Drawbacks

1. **Lots of overhead**: a large cover is needed to hide a few bits of message.
2. **Once the system is discovered, it becomes virtually worthless**. Unlike encryption, there is no secret key left to protect the message.

Best of both

A message can be **first encrypted and then hidden** using steganography. If the hiding is discovered, the attacker still only has ciphertext.



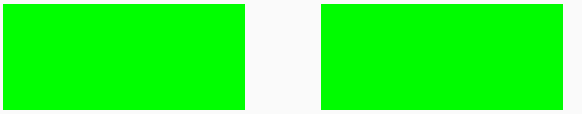
19.3 LSB steganography

LSB = least significant bit(s): the rightmost bits of a number, which change its value the least.

$$1010\,10\,\underbrace{01}_{\text{LSBs}} \qquad \text{bit weights: } 128\ 64\ 32\ 16\ 8\ 4\ \underline{2\ 1}$$

RGB model: each pixel is 24 bits, 8 each for Red, Green and Blue (0–255).

Red $(255, 0, 0)$ = $(\texttt{11111111}, \texttt{00000000}, \texttt{00000000})$ Green $(0, 255, 0)$ Blue $(0, 0, 255)$



$(0, 255, 0)$, G = `11111111` $(0, 252, 0)$, G = `11111100`

The two boxes look identical, but the last 2 bits differ. **Those LSBs can store hidden information.** Using 2 LSBs in each of R, G and B hides $2 \times 3 =$ **6 bits per pixel**, and no channel value changes by more than 3.

Worked example (mine): hide the bits `11 01 10` in the pixel $(100, 50, 200)$:

Channel	Value	Binary	Replace last 2 bits	New value
R	100	`01100100`	`01100111`	103 (+3)
G	50	`00110010`	`00110001`	49 (−1)
B	200	`11001000`	`11001010`	202 (+2)

To extract, the receiver just reads the last 2 bits of each channel. **Capacity:** an 800×600 image holds $800 \cdot 600 \cdot 6 =$ 2,880,000 bits = 360,000 bytes.

Blue orchids: the lecture shows the image without and with hidden information side by side, and you can't tell them apart.

19.4 Watermarking vs. image steganography

	Watermarking	**Image steganography**
Purpose	prove **ownership** / copyright	**secret communication**
Is the mark secret?	no; it's often visible (like the `nesoacademy.org` logo)	yes; its **existence** is hidden
Hidden data	logo, owner info	any message or file

19.5 OpenStego demo

1. **Hide Data:** message file `My Message.txt` ("Kill Corona Virus", 17 bytes) + cover file `Blue Orchids.jpg` (58.4 KB) → stego file `BO.png` (824 KB). No password. "Message embedded into 1 cover file(s)."
2. **Extract Data:** input `BO.png` → output folder. "Message file extracted successfully", and the text file comes back unchanged.

Why PNG and why so much bigger (not explained in the video): JPEG compression slightly changes pixel values, which would wipe out the LSBs. So the stego image is saved in the lossless PNG format, which is much larger. That size jump is itself a hint something is going on.

20 Solved Exam Questions

Q1. Sender and receiver use the **same key**: **(b) private key** and **(c) symmetric** cryptography are both correct (two names for the same thing).

Q2. 19 mod 3: $19 = 6 \cdot 3 + \mathbf{1}$, so the answer is **(d) 1**. (mod gives the remainder.)

Q3 (UGC NET 2016). An attacker captures information and **retransmits it later without altering it**. Options: DoS / masquerade / simple / complex. Official answer: **(a) denial of service**. Reason given: repeated retransmissions flood the receiver so it can't serve real users.

Exam note

The textbook name for "capture and retransmit later" is a **replay attack**. If *replay* is one of the options, choose it. Here it isn't, so DoS is the best available answer.

Q4 (ISRO 2020). Columnar transposition, plaintext "the tomato is a plant in the night shade family", keyword **TOMATO**. 38 letters, 6 columns ⇒ 7 rows (42 cells), 4 fillers ×. Number the keyword letters alphabetically; **for repeated letters, go left to right**:

T	O	M	A	T	O
5	3	2	**1**	6	4
t	h	e	t	o	m
a	t	o	i	s	a
p	l	a	n	t	i
n	t	h	e	n	i
g	h	t	s	h	a
d	e	f	a	m	i
l	y	*x*	*x*	*x*	*x*

Read order A, M, O, O, T, T = columns 4, 3, 2, 6, 1, 5:

TINESAX EOAHTFX HTLTHEY MAIIAIX TAPNGDL OSTNHMX

Answer: **(a)**.

(The slide gave both O's the number 3 and both T's the number 4, then read left to right. That is the same order.)

Q5 (GATE 2015). N people, every pair needs its own **symmetric** key that nobody else knows. One key per pair:

$$\binom{N}{2} = \frac{N(N-1)}{2} \qquad (N = 2 : 1,\ N = 3 : 3,\ N = 4 : 6,\ N = 5 : 10) \qquad \text{Answer: } \mathbf{(c)}$$

Common follow-up (not in the video): with **public-key** cryptography, each person needs just one key pair: N pairs, $2N$ keys in total. This is one big reason public-key cryptography exists.

21 Prime Numbers

Prime: > 1 with exactly two divisors (1 and itself). **Composite**: more than two divisors. **1 is neither**; 2 is the only even prime. Every $n > 1$ factors uniquely into primes (fundamental theorem of arithmetic), e.g. $14688 = 2^5 \cdot 3^3 \cdot 17$.

21.1 Why primes matter in cryptography

Many encryption algorithms (e.g. RSA) are based on primes. **Multiplying** two large primes is **very fast**. The **reverse** (**factoring** the product back into its primes) is **extremely slow**.

$$p, q \xrightarrow{\text{easy}} n = p \cdot q \qquad\qquad n \xrightarrow{\text{very hard}} p, q$$

This "easy one way, hard to reverse" shape is called a **one-way (trapdoor) function**.

Exam tips

- Steganography hides the *existence*; cryptography hides the *meaning*. Steganography is not encryption.
- Two drawbacks: lots of overhead; worthless once discovered. Fix: encrypt first, then hide.
- LSB: 2 LSBs × 3 channels = 6 bits per pixel; each value changes by at most 3.
- Watermarking proves ownership (not secret); steganography hides communication.
- Symmetric keys for N people: $N(N-1)/2$. Public key: N pairs.
- Columnar key with repeated letters: number them left to right.
- 1 is not prime; 2 is the only even prime. Security rests on *factoring the product* being hard.

Corrections to the notes / slides

- **Slide wording:** "Factoring very large prime numbers is very hard" should say factoring the *product* of two large primes. A prime can't be factored at all.
- The factorisation table writes $11 = 1^1 \times 11^1$ and lists 1 as a prime factor. 1 is not prime; 11 and 37 are simply prime.
- The notes call watermarks "typically visible". The lecture's example is visible, but watermarks can also be invisible. The real difference is the purpose (ownership vs. secret communication).
- Fermat's and Euler's theorems need $\gcd(a,p) = 1$ / $\gcd(a,n) = 1$; the notes omitted this condition.
- Everything else checked: TOMATO → (a) (by script), $N(N-1)/2$, all six numbers prime, factorisations correct.

22 Modular Arithmetic

Definitions

$$a \equiv b \pmod{m} \iff m \mid (a-b) \iff a = km + b \text{ for some } k \in \mathbb{Z}$$

The operator $a \bmod m$ always returns the **least non-negative residue** $0 \le r < m$. **Residue class** $[a]_m$: all integers $\equiv a \pmod{m}$. $\mathbb{Z}_n = \{0, \dots, n-1\}$; $\mathbb{Z}_n^* = \{a \in \mathbb{Z}_n : \gcd(a,n) = 1\}$; for a prime p, $\mathbb{Z}_p^* = \{1, \dots, p-1\}$.

Negative numbers: $-255 = (-23)(11) + (-2)$ has $r < 0$; fix it by $q \to q-1$, $r \to r+n$: $-255 = (-24)(11) + 9$, so $-255 \bmod 11 = 9$.

Statement	$a-b$	Divisible by m?	
$38 \equiv 14 \pmod{12}$	24	yes	**valid** (but $38 \bmod 12 = 2$, not 14)
$10 \equiv 2 \pmod{6}$	8	no	**invalid**
$13 \equiv 3 \pmod{13}$	10	no	**invalid**
$-8 \equiv 7 \pmod{5}$	−15	yes	**valid**
$-3 \equiv -8 \pmod{5}$	5	yes	**valid**

22.1 Reduce first

$$(a \pm b) \bmod n = [(a \bmod n) \pm (b \bmod n)] \bmod n \qquad (ab) \bmod n = [(a \bmod n)(b \bmod n)] \bmod n$$

There is **no division rule**: dividing by a means multiplying by a^{-1}, which exists only if $\gcd(a,n) = 1$.

E.g. $(123 \times 456) \bmod 7 = (4 \times 1) \bmod 7 = 4$; $3^{100} \bmod 10 = (3^4)^{25} \bmod 10 = 1$.

22.2 Properties of $\mathbb{Z}_n$ and inverses ★

$+$ and $\times$ are commutative, associative, distributive; identities 0 and 1. Subtraction is not commutative.

- **Additive inverse**: $b = n - a$ (every element has one). Pairs in $\mathbb{Z}_{10}$: (0,0), (1,9), (2,8), (3,7), (4,6), (5,5).
- **Multiplicative inverse**: $a \cdot a^{-1} \equiv 1 \pmod{n}$, **exists iff** $\gcd(a,n) = 1$. Found with the extended Euclidean algorithm (Section 25). 8^{-1} in $\mathbb{Z}_{10}$: none. $11^{-1} \bmod 26 = 19$; $23^{-1} \bmod 100 = 87$; $12^{-1} \bmod 26$: none ($\gcd = 2$).

a in $\mathbb{Z}_8$	0	1	2	3	4	5	6	7
$-a$	0	7	6	5	4	3	2	1
a^{-1}	–	1	–	3	–	5	–	7

Caesar decryption = adding the additive inverse of the key; Hill decryption needs $(\det K)^{-1} \bmod 26$, so $\gcd(\det K, 26) = 1$.

22.3 Linear congruences $ax \equiv b \pmod{n}$

Let $d = \gcd(a,n)$. If $d \nmid b$: **no solution**. If $d \mid b$: **exactly d solutions**. Divide through by d, solve $x_0 = (b/d)(a/d)^{-1} \bmod (n/d)$; the solutions are $x_0 + k(n/d)$, $k = 0, \dots, d-1$.

- $10x \equiv 2 \pmod{15}$: $d = 5 \nmid 2 \Rightarrow$ no solution.
- $14x \equiv 12 \pmod{18}$: $d = 2$; $7x \equiv 6 \pmod{9}$, $7^{-1} = 4$, $x_0 = 24 \bmod 9 = 6$; solutions **6, 15**.
- $3x + 4 \equiv 6 \pmod{13}$: $3x \equiv 2$, $3^{-1} = 9$, $x = 18 \bmod 13 = \mathbf{5}$.

Exam tips

- $a \equiv b \pmod{m} \iff m \mid (a-b)$: fastest validity test.
- "$\equiv \pmod{m}$" allows any representative; "mod" gives only $0..m-1$.
- $a^{-1} \bmod n$ exists only when $\gcd(a,n) = 1$; a linear congruence has $\gcd(a,n)$ solutions or none.

23 Modular Exponentiation ★

★ **In the papers:** 2024–25 Q5(ii): $100^{95} \bmod 9$, $658^{17} \bmod 17$, $145^{102} \bmod 101$. ▸ This year every RSA/ElGamal numerical also needs it (§33).

Exponentiation performed over a modulus: $a^b \bmod m$, which means $(a^b) \bmod m$. The mod applies to the **whole power**, not just the exponent. It is the core operation of **RSA** ($C = M^e \bmod n$), **Diffie–Hellman** ($g^{ab} \bmod p$), ECC and the Rabin cryptosystem.

The point is to **never compute the huge number**. 3^{100} has 48 digits, but $3^{100} \bmod 29$ can be done by hand.

The two tools

1. **Reduce the base first**: if $a \equiv r \pmod m$ then $a^b \equiv r^b \pmod m$. Pick the **smallest** r, which may be **negative**: if $r > m/2$, use $r - m$ (e.g. $23 \equiv -7 \pmod{30}$).
2. **Repeated squaring**: build $a^1, a^2, a^4, a^8, \ldots$ (each the square of the previous one, reduced mod m). Then multiply the ones whose exponents add up to b.

Never reduce the exponent mod m: only the base can be reduced. (See practice Q41.)

23.1 Part 1: when the base is close to the modulus

Problem	Base becomes	Working	Answer
$23^3 \bmod 30$	$23 \equiv -7$	$(-7)^3 = 49 \times (-7) = -133 \equiv -13 \equiv -13 + 30$	**17**
$31^{500} \bmod 30$	$31 \equiv 1$	$1^{500} = 1$, however large the exponent	**1**
$242^{329} \bmod 243$	$242 \equiv -1$	329 is odd, so $(-1)^{329} = -1 \equiv -1 + 243$	**242**
$11^7 \bmod 13$	$11 \equiv -2$	$(-2)^7 = -128 = -10 \cdot 13 + 2$	**2**

- $(-1)^n = +1$ if n is even, -1 if n is odd. So $a \equiv -1$ makes any power trivial.
- A negative result? Add m: $-133 \equiv -13 \equiv 17 \pmod{30}$. (Formally $-133 = -5 \cdot 30 + 17$.)
- Check by brute force once: $23^3 = 12167 = 405 \cdot 30 + 17$ ✓. For big exponents this is impossible, which is why the tools matter.

23.2 Part 2: repeated squaring

When the base is far from the modulus (e.g. 88 vs. 187: $88 - 187 = -99$ is no smaller), use repeated squaring:

Repeated squaring

1. Write b as a **sum of powers of 2** (its binary form): e.g. $7 = 4 + 2 + 1$, $100 = 64 + 32 + 4$.
2. Compute $a^1, a^2, a^4, \ldots$ up to the biggest power of 2 that is $\le b$. Each is the **square of the previous**, reduced mod m (use negatives to keep numbers small).
3. **Multiply** the values you need, reducing mod m after each multiplication.

Why it's fast: 3^{100} needs 6 squarings ($3^2, 3^4, \ldots, 3^{64}$) and 2 multiplications, 8 in total instead of 99. For a 2048-bit RSA exponent that's about 4000 steps instead of 2^{2048}.

Example 1: $88^7 \bmod 187$ **($7 = 4 + 2 + 1$)**

Power	Computation	mod 187
88^1		88
88^2	$88 \times 88 = 7744 = 41 \cdot 187 + 77$	77
88^4	$77 \times 77 = 5929 = 31 \cdot 187 + 132$	132

$88^7 = 88^4 \cdot 88^2 \cdot 88^1 \equiv 132 \cdot 77 \cdot 88$. The lecture multiplies everything first ($894{,}432 = 4783 \cdot 187 + 11$). It's easier to reduce after each step: $132 \cdot 77 = 10164 \equiv 66$, then $66 \cdot 88 = 5808 \equiv$ **11**.

Bookmark this (the lecturer's advice)

$88^7 \bmod 187 = 11$ appears again in the **RSA** lecture. The homework is the reverse: $11^{23} \bmod 187 = 88$ (practice Q37). Together they are an RSA encryption/decryption pair: $e = 7$, $d = 23$, $n = 187$.

Example 2: last two digits of 29^5

The last k digits of a number are the number $\bmod 10^k$ (last digit: mod 10; last two: mod 100; last three: mod 1000). So find $29^5 \bmod 100$, with $5 = 4 + 1$:

Power	Computation	mod 100	or
29^1		**29**	−71
29^2	$29 \times 29 = 841$	41	−59
29^4	$41 \times 41 = 1681$	81	**−19**

$29^5 = 29^4 \cdot 29^1 \equiv (-19)(29) = -551 \equiv -51 \equiv$ **49**. Check another way: $29^2 \cdot 29^2 \cdot 29 = 41 \cdot 41 \cdot 29 = 48749 \to 49$ ✓. Choose whichever of r and $r - m$ is smaller in size: 29 beats −71, and −19 beats 81.

Example 3: 3^{100} mod 29 **(**$100 = 64 + 32 + 4$**, binary `1100100`)**

Power	Computation	mod 29	or	Use
3^1		3	−26	3
3^2	3×3	9	−20	9
3^4	$9 \times 9 = 81$	23	−6	−6 ← needed
3^8	$(-6)^2 = 36$	7	−22	7
3^{16}	$7 \times 7 = 49$	20	−9	−9
3^{32}	$(-9)^2 = 81$	23	−6	−6 ← needed
3^{64}	$(-6)^2 = 36$	7	−22	**7** ← needed

$3^{100} = 3^{64} \cdot 3^{32} \cdot 3^4 \equiv 7 \cdot (-6) \cdot (-6) = 252 = 8 \cdot 29 + 20 \Rightarrow$ **20**. Notice the values repeat ($-6 \to 7 \to -9 \to -6 \to 7$): powers mod m always eventually cycle. *Preview (Fermat, coming up):* 29 is prime, so $3^{28} \equiv 1$ and $3^{100} = 3^{3 \cdot 28 + 16} \equiv 3^{16} \equiv 20$ ✓.

Example 4: 23^{16} mod 30 **(exponent is a power of 2)**

When $b = 2^k$, just square k times, reducing each time:

$$23^{16} = \left(\left((23^2)^2\right)^2\right)^2 : \quad 23 \equiv -7 \xrightarrow{2} 49 \equiv 19 \equiv -11 \xrightarrow{2} 121 \equiv 1 \xrightarrow{2} 1 \xrightarrow{2} \mathbf{1}$$

Once you hit 1, every further square stays 1.

Lecture homework: (1) 11^{23} mod 187; (2) the last 3 digits of 175^{209}. Full solutions are in the practice section (Q37, Q38).

Exam tips

- a^b mod $m = (a^b)$ mod m. Reduce the **base**, never the exponent (mod m).
- Base $\equiv 1$: the answer is 1. Base $\equiv -1$: the answer is 1 or $m - 1$, depending on whether the exponent is even or odd.
- Repeated squaring: write the exponent as powers of 2, build a table $a^1, a^2, a^4, \ldots$, then multiply the needed rows.
- Use negative residues ($r - m$) when they are smaller in size, and turn the final answer positive.
- "Last k digits" means mod 10^k.
- Remember 88^7 mod 187 = 11 for RSA.

Corrections to the notes / slides

- All of Gemini's numbers are correct (checked by script: 17, 1, 242, 2, 11, 49, 20, 1).
- Slide glitches noted by Gemini: in Example 2 the summary line briefly reads "88^7 mod 187 = 49" (copy–paste error; it should be 29^5 mod 100 = 49). Example 3's slide stops at "252 mod 29" (the answer, 20, is only spoken).
- −133 mod 30: the lecture writes the intermediate step as −13 (truncated division), then adds 30. Formally $-133 = -5 \cdot 30 + 17$. Same answer.
- The slide writes -7^3 for $(-7)^3$. Strictly $-7^3 = -(7^3)$; the brackets matter when the exponent is even (e.g. $(-7)^2 = 49$ but $-7^2 = -49$).

24 GCD and the Euclidean Algorithm

Divisibility: $n \mid a$ iff $a = qn$. Properties: $a \mid 1 \Rightarrow a = \pm 1$; $a \mid b$ and $b \mid a \Rightarrow a = \pm b$; $a \mid b, b \mid c \Rightarrow a \mid c$; $a \mid b, a \mid c \Rightarrow a \mid (mb + nc)$.

24.1 The key fact

$$\gcd(a, b) = \gcd(b,\ a \bmod b) \qquad \gcd(a, 0) = a$$

Why it's true: $a = qb + r$. Anything that divides a and b also divides $r = a - qb$. Anything that divides b and r also divides $a = qb + r$. So the pairs (a, b) and (b, r) have exactly the same common divisors, and therefore the same GCD. Each step replaces the numbers with smaller ones until one of them is 0.

24.2 Method 1: the Q A B R table

1. Columns Q (quotient), A, B, R (remainder). Start with A = larger number, B = smaller number.
2. Each row: $Q = \lfloor A/B \rfloor$, $R = A \bmod B$.
3. Next row: B **moves into** A and R **moves into** B (the diagonal arrows).
4. **Stop when** $B = 0$ (you can't divide by 0; write X for Q and R). **The GCD is the A in that row.**

Q	A	B	R	
2	33	12	9	$33 = 2 \cdot 12 + 9$
1	12	9	3	$12 = 1 \cdot 9 + 3$
3	9	3	0	$9 = 3 \cdot 3 + 0$ (don't stop yet!)
X	3	0	X	$B = 0$: stop. gcd(12, 33) = **3**

gcd(750, 900)

Q	A	B	R
1	900	750	150
5	750	150	0
X	**150**	0	X

gcd(252, 105)

Q	A	B	R
2	252	105	42
2	105	42	21
2	42	21	0
X	**21**	0	X

Shortcut: the GCD is the **last non-zero remainder** (9, 3, **3** then 0; or 42, **21** then 0). The extra "$B = 0$" row is just where the lecture reads it off.

24.3 Method 2: recursion

```
Prerequisite: a > b
Euclid_GCD(a, b):
   if b = 0 then
      return a;
   else
      return Euclid_GCD(b, a mod b);
```

gcd(50, 12)
= gcd(12, 50 mod 12) = gcd(12, 2)
= gcd(2, 12 mod 2) = gcd(2, 0)
= **2**

gcd(83, 19)
= gcd(19, 7) (83 = 4 · 19 + 7)
= gcd(7, 5) (19 = 2 · 7 + 5)
= gcd(5, 2) (7 = 1 · 5 + 2)
= gcd(2, 1) (5 = 2 · 2 + 1)
= gcd(1, 0) = **1** (2 = 2 · 1 + 0)

Both methods do exactly the same divisions; use whichever you like. If you're given $a < b$ (e.g. gcd(10, 20)), swap them first. (The algorithm fixes it on its own anyway: gcd(12, 33) → gcd(33, 12 mod 33) = gcd(33, 12), which wastes one step.)

24.4 Relatively prime

a and b are **relatively prime** (coprime) if $\gcd(a, b) = 1$. Example: 83 and 19. Coprime numbers need not be prime themselves: gcd(8, 9) = 1.

Why cryptography cares:

- $a^{-1} \bmod n$ exists **only if** $\gcd(a, n) = 1$. That was the Hill key condition $\gcd(\det K, 26) = 1$.
- The **extended** Euclidean algorithm (next on the roadmap) runs these same divisions backwards to actually *find* a^{-1}, instead of the trial table we used for $23^{-1} \bmod 26$.
- RSA key generation picks an e with $\gcd(e, \phi(n)) = 1$.

Not in the lecture: Euclid is fast because the remainder at least halves every two steps. The slowest inputs are consecutive Fibonacci numbers, e.g. gcd(89, 55) takes 9 divisions, all with quotient 1.

Lecture homework: gcd(1005, 105) and gcd(529, 123). Solutions are in practice Q43.

Exam tips

- GCD = HCF. $\gcd(a,b) = \gcd(b, a \bmod b)$, and $\gcd(a,0) = a$.
- Table method: stop when $B = 0$ (not when $R = 0$); the answer is that row's A, i.e. the last non-zero remainder.
- Put the larger number first.
- gcd = 1 means relatively prime (coprime), which is exactly when a multiplicative inverse exists.
- Show every division as $a = q \cdot b + r$ in exams.

Corrections to the notes / slides

- All results are correct (checked by script): 3, 25, 1, 3, 150, 21, 2, 1.
- Verbal slips only: "divisors of 13" means 33 (video 1); "a = 9" means 19 (video 2). The slides are right.
- Wording: "A mod B" gives the *remainder*; "A divided by B" gives the quotient Q. The table needs both.

25 Extended Euclidean Algorithm ★★

★ **In the papers:** 2024–25 Q7 (5 marks): pseudocode + 23^{-1} in $\mathbb{Z}_{100}$ with "value in iteration k" sub-questions. 2025–26 Q5 (3 marks): solve $19x + 13y = 20$. The same table, with polynomials, gives the GF(2^8) inverse asked in both years (§29), and d in RSA (§33).

Finds s, t with $\boxed{s \cdot a + t \cdot b = \gcd(a,b)}$ along with the gcd.

Algorithm (Forouzan table)

$r_1 = a, r_2 = b$; $s_1 = 1, s_2 = 0$; $t_1 = 0, t_2 = 1$. While $r_2 > 0$:

$q = \lfloor r_1 / r_2 \rfloor$, $r = r_1 - q r_2$, $s = s_1 - q s_2$, $t = t_1 - q t_2$; shift $(r_1, r_2) \leftarrow (r_2, r)$, $(s_1, s_2) \leftarrow (s_2, s)$, $(t_1, t_2) \leftarrow (t_2, t)$.

At the end: $\gcd = r_1$, $s = s_1$, $t = t_1$.

Example: $a = 161$, $b = 28$.

q	r_1	r_2	r	s_1	s_2	s	t_1	t_2	t
5	161	28	21	1	0	1	0	1	−5
1	28	21	7	0	1	−1	1	−5	6
3	21	7	0	1	−1	4	−5	6	−23
	7	0		**−1**	4		**6**	−23	

$\gcd(161, 28) = 7$, $s = -1$, $t = 6$: $(-1)(161) + 6(28) = 7$ ✓. Other results: $(84, 320) \to \gcd 4$, $s = -19$, $t = 5$; $(17, 0) \to 17, 1, 0$; $(0, 45) \to 45, 0, 1$.

25.1 Multiplicative inverse with the table ★★

To find $b^{-1} \bmod n$: run the table with $r_1 = n$, $r_2 = b$, **only the t columns** (start $t_1 = 0, t_2 = 1$). If the final $r_1 = 1$, then $b^{-1} = t_1 \bmod n$; if $r_1 \neq 1$, no inverse.

$11^{-1} \bmod 26$

q	r_1	r_2	r	t_1	t_2	t
2	26	11	4	0	1	−2
2	11	4	3	1	−2	5
1	4	3	1	−2	5	−7
3	3	1	0	5	−7	26
	1	0		**−7**	26	

$23^{-1} \bmod 100$

q	r_1	r_2	r	t_1	t_2	t
4	100	23	8	0	1	−4
2	23	8	7	1	−4	9
1	8	7	1	−4	9	−13
7	7	1	0	9	−13	100
	1	0		**−13**	100	

$11^{-1} \equiv -7 \equiv \mathbf{19} \pmod{26}$; $23^{-1} \equiv -13 \equiv \mathbf{87} \pmod{100}$; $12^{-1} \bmod 26$: final $r_1 = 2$, no inverse.

25.2 Linear Diophantine equations $ax + by = c$ ★

Let $d = \gcd(a,b)$. If $d \nmid c$: no solution. If $d \mid c$: infinitely many. Divide by d, find s, t with $s(a/d) + t(b/d) = 1$; then

$$\text{particular: } x_0 = (c/d)\,s, \quad y_0 = (c/d)\,t \qquad \text{general: } x = x_0 + k\,(b/d), \quad y = y_0 - k\,(a/d).$$

- $21x + 14y = 35$: $d = 7$; $3x + 2y = 5$, $s = 1$, $t = -1$; particular $(5, -5)$; general $x = 5 + 2k$, $y = -5 - 3k$: $(5, -5), (7, -8), (9, -11), \ldots$
- Cash a Rs. 100 cheque in Rs. 20 and Rs. 5 notes: $20x + 5y = 100$, $d = 5$, $4x + y = 20$, $s = 0$, $t = 1$; particular $(0, 20)$; non-negative solutions $(0, 20), (1, 16), (2, 12), (3, 8), (4, 4), (5, 0)$.

Exam tips

- Always show the full table with all columns; the exam awards marks per row.
- For an inverse, run with $r_1 = n$ (the modulus) first; a negative t_1 is fine, add n at the end.
- Check: $sa + tb = \gcd$, or $b \cdot b^{-1} \bmod n = 1$.

26 Relatively Prime Numbers, Euler's Totient and $\mathbb{Z}_n^*$

26.1 Relatively prime (co-prime) numbers

Two positive integers a and b are relatively prime (co-prime) if $\gcd(a, b) = 1$, that is, if their only common positive divisor is 1. Neither number has to be prime: the condition is about the pair, not about each number. Since $12 = 2^2 \cdot 3$ and $77 = 7 \cdot 11$ share no prime factor, they are co-prime although both are composite, while $15 = 3 \cdot 5$ and $21 = 3 \cdot 7$ are not, because 3 divides both. If p is prime, every integer from 1 to $p - 1$ is co-prime to p, since the only divisors of p are 1 and p itself.

Cryptography cares about this property for one reason: a **has a multiplicative inverse modulo** n **exactly when** $\gcd(a, n) = 1$. If $ax \equiv 1 \pmod n$, then $ax - kn = 1$ for some integer k, so any common divisor of a and n divides 1. Conversely, if $\gcd(a, n) = 1$, the extended Euclidean algorithm (§25) produces s, t with $sa + tn = 1$, and reducing modulo n gives $sa \equiv 1$. Co-primality is therefore always checked with the Euclidean algorithm, never by eye.

a, b	Divisors / Euclid steps	GCD	Co-prime?	Note
4, 13	$\{1, 2, 4\}$, $\{1, 13\}$	1	yes	composite + prime
15, 21	$\{1, 3, 5, 15\}$, $\{1, 3, 7, 21\}$	3	**no**	share the factor 3
11, 17		1	yes	both prime
11, 21		1	yes	prime + composite ($21 = 3 \cdot 7$)
12, 77		1	yes	**both composite**
83, 19	$83 = 4 \cdot 19 + 7$, $19 = 2 \cdot 7 + 5$, $7 = 5 + 2$, $5 = 2 \cdot 2 + 1$	1	yes	
790, 121	$790 = 6 \cdot 121 + 64$, $121 = 64 + 57$, $64 = 57 + 7$, $57 = 8 \cdot 7 + 1$	1	yes	lecture homework

26.2 Euler's totient function $\phi(n)$ ★

▸ **Likely this year:** $\phi(n)$ for a given n (Forouzan: "find $\phi(29)$, $\phi(32)$, $\phi(80)$, $\phi(100)$, $\phi(101)$"), and $\phi(n) = (p - 1)(q - 1)$ inside every RSA question.

Many results in public-key cryptography (Euler's theorem, RSA key generation, inverses by exponentiation) need to know *how many* residues modulo n are invertible. Since invertibility is the same as being co-prime to n, this count gets its own name. Euler's totient (phi) function $\phi(n)$ is the number of integers in $\{1, 2, \ldots, n - 1\}$ that are relatively prime to n. Zero and negative numbers are not counted. The lecture videos write it as $\Phi(n)$; Forouzan and the slides write $\phi(n)$.

By hand, $\phi(n)$ is found by computing $\gcd(k, n)$ for every k from 1 to $n - 1$ and counting the 1s. For $n = 8$:

k	1	2	3	4	5	6	7	
$\gcd(k, 8)$	1	2	1	4	1	2	1	
co-prime?	✓	×	✓	×	✓	×	✓	$\Rightarrow \phi(8) =$ **4**

In the same way $\phi(5) = 4$ and $\phi(11) = 10$, since every number below a prime is co-prime to it. This direct count needs $n - 1$ GCDs, which is useless for $n = 1000$ and hopeless for a 1024-bit RSA modulus, so in practice $\phi(n)$ is always computed from the prime factorisation of n using the rules below.

26.3 Computing $\phi(n)$: the four rules ★

The slides (following Forouzan) give four rules. Each one follows from counting which numbers *fail* to be co-prime to n, and the derivations are short enough to reproduce in an exam.

Rule 1: $\phi(1) = 1$ **by convention** (the slides write $\phi(1) = 0$; see the corrections box). It never matters in practice.

Rule 2: for a prime p**,** $\phi(p) = p - 1$. The only divisors of p are 1 and p, so none of $1, \ldots, p - 1$ shares a factor with p. For example $\phi(7) = 6$, $\phi(13) = 12$, $\phi(31) = 30$.

Rule 3: for a prime power, $\phi(p^k) = p^k - p^{k-1}$. A number shares a factor with p^k exactly when it is a multiple of p, because p is the only prime in p^k. Among $1, \ldots, p^k$ the multiples of p are $p, 2p, \ldots, p^{k-1} \cdot p$, which is p^{k-1} numbers, and removing them leaves $p^k - p^{k-1} = p^{k-1}(p - 1)$. For example $\phi(8) = 8 - 4 = 4$, $\phi(25) = 25 - 5 = 20$, and $\phi(49) = 49 - 7 = 42$.

Rule 4: if $\gcd(m, n) = 1$**, then** $\phi(mn) = \phi(m)\,\phi(n)$ (the function is *multiplicative*). By the Chinese remainder theorem (§28), each x modulo mn corresponds to exactly one pair ($x \bmod m$, $x \bmod n$), and x is co-prime to mn exactly when it is co-prime to both m and n. The co-prime x therefore correspond to pairs chosen from $\phi(m)$ values and $\phi(n)$ values, giving $\phi(m)\phi(n)$ of them. The condition $\gcd(m, n) = 1$ is essential: $\phi(4) = 2$ but $\phi(2)\phi(2) = 1$.

Two distinct primes: $\phi(pq) = (p-1)(q-1)$. This is the RSA case, and it is worth deriving directly. Among $1, \dots, pq$, the numbers sharing a factor with pq are the multiples of p (there are q of them: $p, 2p, \dots, qp$) and the multiples of q (there are p of them). The number pq itself is in both lists and has been removed twice, so

$$\phi(pq) = pq - q - p + 1 = (p-1)(q-1).$$

For example $\phi(21) = 2 \cdot 6 = 12$, $\phi(77) = 6 \cdot 10 = 60$, and $\phi(187) = \phi(11 \cdot 17) = 10 \cdot 16 = 160$.

General formula. Combining rules 3 and 4, if $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$ with distinct primes p_i, then

$$\phi(n) = \left(p_1^{e_1} - p_1^{e_1-1}\right)\left(p_2^{e_2} - p_2^{e_2-1}\right)\cdots\left(p_k^{e_k} - p_k^{e_k-1}\right) = n\left(1 - \tfrac{1}{p_1}\right)\left(1 - \tfrac{1}{p_2}\right)\cdots\left(1 - \tfrac{1}{p_k}\right).$$

The second form works for every n and contains rules 2 and 3 and the pq formula as special cases. Only the *distinct* primes appear in the product; the exponents are already accounted for by the factor n. As a consequence, computing $\phi(n)$ is exactly as hard as factoring n, which is the reason RSA can publish n but must keep $\phi(n)$ secret.

★ MUST KNOW

$$\phi(p) = p-1, \qquad \phi(p^k) = p^k - p^{k-1}, \qquad \phi(pq) = (p-1)(q-1)\ (p \neq q), \qquad \phi(n) = n \prod_{p \mid n}\left(1 - \tfrac{1}{p}\right).$$

$\phi(mn) = \phi(m)\phi(n)$ **only when** $\gcd(m, n) = 1$. Always factorise n completely first.

Example: $\phi(35)$, with $35 = 5 \times 7$: $(5-1)(7-1) = 4 \times 6 = \mathbf{24}$. The grid below checks it by crossing out the multiples of 5 and of 7 among $1, \dots, 34$; none is a multiple of both, since the first common multiple is 35.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34

red = multiple of 5 (6 numbers), blue = multiple of 7 (4 numbers); $34 - 10 = 24$ white cells

Trap: repeated prime factors

$25 = 5 \times 5$ is **not** a product of two distinct primes, so $(5-1)(5-1) = 16$ is wrong. Use rule 3: $\phi(25) = 25 - 5 = \mathbf{20}$. (Check: among $1, \dots, 24$ only 5, 10, 15, 20 share a factor with 25, leaving $24 - 4 = 20$.) The slides make the same point with $\phi(49)$: writing $\phi(49) = \phi(7)\phi(7) = 36$ is wrong because 7 and 7 are not co-prime; the correct value is $49 - 7 = 42$.

WORKED EXAMPLE : $\phi(n)$ from the factorisation (Forouzan / slide style)

Q. Find (a) $\phi(29)$ (b) $\phi(32)$ (c) $\phi(80)$ (d) $\phi(100)$ (e) $\phi(101)$ (f) $\phi(240)$ (g) $\phi(1000)$ (h) $\phi(7000)$.
A. Factorise first, then apply the rules.

n	Factorisation	Working	$\phi(n)$
29	prime	$29 - 1$	**28**
32	2^5	$2^5 - 2^4 = 32 - 16$	**16**
80	$2^4 \cdot 5$	$(2^4 - 2^3)(5-1) = 8 \cdot 4$	**32**
100	$2^2 \cdot 5^2$	$(4-2)(25-5) = 2 \cdot 20$	**40**
101	prime	$101 - 1$	**100**
240	$2^4 \cdot 3 \cdot 5$	$(16-8)(3-1)(5-1) = 8 \cdot 2 \cdot 4$	**64**
1000	$2^3 \cdot 5^3$	$1000 \cdot \frac{1}{2} \cdot \frac{4}{5}$ (or $\phi(8)\phi(125) = 4 \cdot 100$)	**400**
7000	$2^3 \cdot 5^3 \cdot 7$	$7000 \cdot \frac{1}{2} \cdot \frac{4}{5} \cdot \frac{6}{7} = 7000 \cdot \frac{24}{70}$	**2400**

Check: a direct count of the co-prime $k < n$ by script gives the same eight values. Lecture homework: $\phi(369) = \phi(3^2)\phi(41) = 6 \cdot 40 = \mathbf{240}$ and $\phi(372) = \phi(2^2)\phi(3)\phi(31) = 2 \cdot 2 \cdot 30 = \mathbf{120}$.

$\phi(n)$ **is even for every** $n > 2$. If $\gcd(k, n) = 1$ then also $\gcd(n-k, n) = 1$, so the co-prime numbers below n come in pairs $\{k, n-k\}$. The two members of a pair can only coincide if $n = 2k$, but then $\gcd(k, n) = k$, which equals 1 only for $n = 2$. Hence for $n > 2$ the co-prime numbers split into disjoint pairs and their count is even; in $\mathbb{Z}_{14}^*$ the pairs are $\{1, 13\}, \{3, 11\}, \{5, 9\}$.

26.4 The sets $\mathbb{Z}_n$ and $\mathbb{Z}_n^*$

$\mathbb{Z}_n = \{0, 1, \dots, n-1\}$ is the set of residues modulo n. Every element has an additive inverse in it ($a + (n-a) \equiv 0$), but not every element has a multiplicative inverse. Since cryptosystems such as the affine cipher, Hill cipher and RSA need to *divide*, they work in the smaller set

$$\mathbb{Z}_n^* = \{\, a \in \mathbb{Z}_n : \gcd(a, n) = 1 \,\},$$

which is exactly the set of residues that have multiplicative inverses. By definition its size is $|\mathbb{Z}_n^*| = \phi(n)$. The slide's rule of thumb: use $\mathbb{Z}_n$ when additive inverses are needed and $\mathbb{Z}_n^*$ when multiplicative inverses are needed. $\mathbb{Z}_n^*$ is closed under multiplication modulo n, because a product of two numbers co-prime to n is again co-prime to n (no prime factor of n can appear in it). Together with the inverses this makes $\langle \mathbb{Z}_n^*, \times \rangle$ a group of order $\phi(n)$ (§29), and the theorems of the next section are statements about this group. The slides single out the two sets with a *prime* modulus, $\mathbb{Z}_p$ and $\mathbb{Z}_p^*$. When $n = p$ is prime, $\mathbb{Z}_p^* = \{1, 2, \ldots, p-1\}$ contains every non-zero residue (e.g. $\mathbb{Z}_{13}^* = \{1, \ldots, 12\}$), so every non-zero element of $\mathbb{Z}_p$ can be divided by, which is what makes $\mathbb{Z}_p$ the field GF(p).

n	$\mathbb{Z}_n^*$	$\lvert\mathbb{Z}_n^*\rvert = \phi(n)$
6	$\{1, 5\}$	$\phi(2)\phi(3) = 2$
7	$\{1, 2, 3, 4, 5, 6\}$	$7 - 1 = 6$
10	$\{1, 3, 7, 9\}$	$\phi(2)\phi(5) = 4$
14	$\{1, 3, 5, 9, 11, 13\}$	$\phi(2)\phi(7) = 1 \cdot 6 = 6$ (slide Example 5)
21	$\{1, 2, 4, 5, 8, 10, 11, 13, 16, 17, 19, 20\}$	$\phi(3)\phi(7) = 2 \cdot 6 = 12$
26	the 12 odd numbers except 13	$\phi(2)\phi(13) = 12$ (valid keys/determinants mod 26)

26.5 Why cryptography cares about $\phi(n)$

Three later topics rest on $\phi(n)$. First, Euler's theorem (§27) states that $a^{\phi(n)} \equiv 1 \pmod{n}$ for every $a \in \mathbb{Z}_n^*$, which allows exponents to be reduced modulo $\phi(n)$ and inverses to be computed as $a^{\phi(n)-1}$. Second, RSA chooses $n = pq$ and computes $\phi(n) = (p-1)(q-1)$; the keys satisfy $ed \equiv 1 \pmod{\phi(n)}$, and Euler's theorem then makes decryption undo encryption. In the standard example $n = 187 = 11 \cdot 17$, $\phi(187) = 160$ and $7 \cdot 23 = 161 \equiv 1 \pmod{160}$, which is why $88^7 \bmod 187 = 11$ and $11^{23} \bmod 187 = 88$ undo each other. Third, the number of primitive roots of a prime p is $\phi(p-1)$, which matters for Diffie–Hellman and ElGamal.

Security. Knowing $\phi(n)$ for an RSA modulus is equivalent to knowing its factors. From $n = pq$ and $\phi(n) = n - (p+q) + 1$ one gets $p + q = n - \phi(n) + 1$, and p, q are then the roots of $z^2 - (p+q)z + n = 0$. For $n = 143$ and $\phi(n) = 120$: $p + q = 24$, $pq = 143$, so $p, q =$ **11, 13**. This is why $\phi(n)$ is part of the private key.

Corrections to the notes / slides

- All values recomputed by script: $\phi(5) = 4$, $\phi(8) = 4$, $\phi(11) = 10$, $\phi(35) = 24$, $\phi(25) = 20$, $\phi(49) = 42$, $\phi(240) = 64$, $\phi(1000) = 400$, $\phi(7000) = 2400$, $|\mathbb{Z}_{14}^*| = 6$, $|\mathbb{Z}_{21}^*| = 12$. The homework answers in the old notes ($\phi(369) = 240$, $\phi(372) = 120$) are correct.
- Slide L10-06 gives $\phi(1) = 0$, which follows Forouzan's "integers smaller than n" wording. The standard convention is $\phi(1) = 1$ (count $1 \le k \le n$). It only affects $n = 1$ and never matters in RSA.
- The lecture video's "case 3" ("either or both numbers composite") makes the product formula $n \prod(1 - 1/p)$ look like a last resort. It actually works for every n, and the prime and pq formulas are special cases of it.

Check your understanding

1. **Define $\phi(n)$ and explain why $\phi(p) = p - 1$ for a prime p.** $\phi(n)$ is the number of integers in $1, \ldots, n-1$ co-prime to n. A prime has no divisors other than 1 and itself, so all $p - 1$ numbers below it are co-prime to it.
2. **Why is $\phi(49) \neq \phi(7) \cdot \phi(7)$?** Multiplicativity needs co-prime factors, and $\gcd(7, 7) = 7$. Rule 3 gives $\phi(49) = 49 - 7 = 42$, not 36.
3. **Find $\phi(36)$ and $\phi(60)$.** $36 = 2^2 \cdot 3^2$, so $\phi = 2 \cdot 6 = 12$. $60 = 2^2 \cdot 3 \cdot 5$, so $\phi = 2 \cdot 2 \cdot 4 = 16$.
4. **Derive $\phi(pq) = (p-1)(q-1)$.** Of $1, \ldots, pq$, the q multiples of p and the p multiples of q are not co-prime to pq, and pq is counted in both lists, so $\phi(pq) = pq - p - q + 1 = (p-1)(q-1)$.
5. **List $\mathbb{Z}_{15}^*$ and state its size.** $\{1, 2, 4, 7, 8, 11, 13, 14\}$, which has $\phi(15) = 2 \cdot 4 = 8$ elements.
6. **Why is $\phi(n)$ even for $n > 2$?** The co-prime residues pair up as $k \leftrightarrow n - k$, and the two members of a pair are never equal when $n > 2$.
7. **An RSA modulus is $n = 143$ and someone learns $\phi(n) = 120$. What can they do?** They get $p + q = n - \phi(n) + 1 = 24$ and $pq = 143$, so $p = 11$, $q = 13$; the modulus is broken. Hence $\phi(n)$ must stay secret.
8. **What is the difference between $\mathbb{Z}_n$ and $\mathbb{Z}_n^*$?** $\mathbb{Z}_n$ contains all residues $0, \ldots, n-1$ and is used where addition and additive inverses are needed. $\mathbb{Z}_n^*$ contains only the $\phi(n)$ residues co-prime to n, which are exactly those with multiplicative inverses.

27 Fermat's and Euler's Theorems, Inverses and Primitive Roots

Public-key cryptography computes powers $a^b \bmod n$ with exponents hundreds of digits long, and it needs inverses modulo n. Square-and-multiply (§23) makes such powers feasible, but the two theorems of this section do more: they say that the powers of a modulo n repeat with a known period, so a huge exponent can be replaced by a small one, and they give a formula for the inverse. The same periodicity, studied element by element, leads to the order of an element and to primitive roots, which Diffie–Hellman and ElGamal are built on.

27.1 Fermat's little theorem and exponent reduction ★

★ **In the papers:** 2024–25 Q5(ii) (part of 3 marks): $100^{95} \bmod 9$, $658^{17} \bmod 17$, $145^{102} \bmod 101$.

First version. If p is prime and a is an integer not divisible by p, then $a^{p-1} \equiv 1 \pmod p$. Both conditions must be checked before the theorem is used: for a composite modulus or for $p \mid a$ it says nothing.

Why it is true. Multiply each of $1, 2, \dots, p-1$ by a and reduce modulo p. No product is 0, because p divides neither factor, and no two products are equal, because $ai \equiv aj$ would mean $p \mid a(i-j)$ and hence $p \mid (i-j)$, i.e. $i = j$. The $p-1$ products are therefore just $1, \dots, p-1$ in a different order; for $p = 7$, $a = 3$ they are $3, 6, 2, 5, 1, 4$. Multiplying all of them together gives $a^{p-1}(p-1)! \equiv (p-1)! \pmod p$, and since $(p-1)!$ is co-prime to p it can be cancelled, leaving $a^{p-1} \equiv 1$.

Second version. If p is prime, then $a^p \equiv a \pmod p$ for every integer a. This removes the condition on a. If $p \nmid a$, it is the first version multiplied by a; if $p \mid a$, both sides are $\equiv 0$.

p, a	Conditions	Working	Result
5, 2	prime, $5 \nmid 2$	$2^4 = 16 = 3 \cdot 5 + 1$	$\equiv 1$ ✓
13, 11	prime, $13 \nmid 11$	$11 \equiv -2$, $(-2)^{12} = \left((-2)^4\right)^3 = 16^3 \equiv 3^3 = 27 \equiv 1$	$\equiv 1$ ✓
6, 2	**6 not prime**	$2^5 = 32 = 5 \cdot 6 + 2$	$\equiv 2 \neq 1$: theorem silent
11, 5 (homework)	prime, $11 \nmid 5$	$5^2 \equiv 3$, $5^4 \equiv 9$, $5^8 \equiv 81 \equiv 4$, $5^{10} = 5^8 \cdot 5^2 \equiv 4 \cdot 3 = 12$	$\equiv 1$ ✓

Exponent reduction. Since $a^{p-1} \equiv 1$, the powers of a modulo p repeat with period dividing $p-1$. Writing the exponent as $b = q(p-1) + r$ with $r = b \bmod (p-1)$ gives

$$a^b = \left(a^{p-1}\right)^q a^r \equiv 1^q \cdot a^r = a^{\,b \bmod (p-1)} \pmod p \qquad (p \text{ prime}, \ p \nmid a).$$

The slides' examples follow directly: $6^{10} \bmod 11 = 1$ because the exponent is exactly $p-1$, and $3^{12} \bmod 11 = 3^{11} \cdot 3 \equiv 3 \cdot 3 = 9$ by the second version (equivalently $12 \bmod 10 = 2$ and $3^2 = 9$). In practice the base is reduced modulo p first, then the exponent modulo $p-1$, and the small power that remains is computed by repeated squaring.

Trap: what to reduce, and by what

The **base** is reduced modulo p; the **exponent** is reduced modulo $p-1$, never modulo p. And the modulus must be prime: in $100^{95} \bmod 9$ the modulus $9 = 3^2$ is composite, so Fermat does not apply. There the base reduction alone finishes the job ($100 \equiv 1 \pmod 9$), or Euler's theorem with $\phi(9) = 6$ can be used.

WORKED EXAMPLE : exponent reduction by Fermat (Forouzan exercise)

Q. Using Fermat's little theorem, find (a) $5^{15} \bmod 13$ (b) $15^{18} \bmod 17$ (c) $456^{17} \bmod 17$.
A.

	Conditions	Base	Exponent	Result
(a)	13 prime, $13 \nmid 5$	5	$15 = 1 \cdot 12 + 3 \to 3$	$5^3 = 125 = 9 \cdot 13 + 8 \Rightarrow$ **8**
(b)	17 prime, $17 \nmid 15$	$15 \equiv -2$	$18 = 1 \cdot 16 + 2 \to 2$	$(-2)^2 =$ **4**
(c)	17 prime	$456 = 26 \cdot 17 + 14 \to 14$	second version: $a^{17} \equiv a$	**14**

Check (by script): $5^{15} \bmod 13 = 8$, $15^{18} \bmod 17 = 4$, $456^{17} \bmod 17 = 14$.

Fermat as a primality test. Read backwards, the theorem gives a test: if $a^{n-1} \not\equiv 1 \pmod n$ for some a with $\gcd(a, n) = 1$, then n is certainly composite (e.g. $2^{14} \bmod 15 = 4$). The converse fails: $2^{340} \equiv 1 \pmod{341}$ although $341 = 11 \cdot 31$. The Fermat test, Carmichael numbers and Miller–Rabin are covered in §28.

27.2 Euler's theorem ★

▸ **Likely this year:** Exponent reduction modulo $\phi(n)$ for a composite n, and "why does RSA decryption work?" (Euler is its basis).

Fermat's theorem needs a prime modulus, but RSA works modulo $n = pq$. Euler's theorem is the generalisation to any modulus. **First version:** if $\gcd(a, n) = 1$, then $a^{\phi(n)} \equiv 1 \pmod n$. When $n = p$ is prime, $\phi(p) = p - 1$ and this is exactly Fermat's theorem.

Why it is true. The proof is the same permutation argument, run over $\mathbb{Z}_n^*$ (§26) instead of $\{1, \dots, p-1\}$. Multiplying the $\phi(n)$ elements of $\mathbb{Z}_n^*$ by a permutes them, because a is invertible; for $n = 10$, $a = 3$ the set $\{1, 3, 7, 9\}$ becomes $\{3, 9, 1, 7\}$. Taking the product of all elements on both sides gives $a^{\phi(n)} \cdot \Pi \equiv \Pi$, and the product Π is invertible, so $a^{\phi(n)} \equiv 1$.

The co-prime condition is genuinely needed. If $\gcd(a, n) = d > 1$, every power of a is a multiple of d, and so is

every number $\equiv a^k \pmod n$; such a number can never be $\equiv 1$. The lecture homework $11^{20} \bmod 33$ shows this: the powers of 11 alternate $11, 22, 11, 22, \ldots$, so $11^{20} \equiv 22 \neq 1$ although $\phi(33) = 20$.

a, n	gcd	Working	Result
3, 10	1	$\phi(10) = 4$; $3^4 = 81 = 8 \cdot 10 + 1$	$\equiv 1$ ✓
2, 10	**2**	$2^4 = 16 \equiv 6$	$\neq 1$: theorem silent
10, 11	1	$\phi(11) = 10$; $10 \equiv -1$, $(-1)^{10} = 1$	$\equiv 1$ ✓
6, 35	1	$\phi(35) = 24$; slide example $6^{24} \bmod 35$	$\equiv 1$ ✓
11, 33 (homework)	**11**	$\phi(33) = 20$; powers alternate 11, 22	$11^{20} \equiv 22 \neq 1$

Exponent reduction. Exactly as with Fermat, $\gcd(a, n) = 1$ and $b = q\phi(n) + r$ give $a^b \equiv a^{b \bmod \phi(n)} \pmod n$. The work is in computing $\phi(n)$, which needs the factorisation of n.

Second version. For $n = pq$ (and more generally any squarefree n), $a^{k\phi(n)+1} \equiv a \pmod n$ for every integer a and every $k \geq 0$. This removes the co-prime condition, which RSA needs because a message M might, by bad luck, share the factor p with n. The proof works one prime at a time. Modulo p: if $p \nmid a$, then $a^{k\phi(n)} = \left(a^{p-1}\right)^{k(q-1)} \equiv 1$ by Fermat, so $a^{k\phi(n)+1} \equiv a$; if $p \mid a$, both sides are $\equiv 0$. The same holds modulo q. Since the distinct primes p and q both divide $a^{k\phi(n)+1} - a$, so does pq. For example $11^{21} = 11^{\phi(33)+1} \equiv 11 \pmod{33}$ although $\gcd(11, 33) \neq 1$. The argument breaks when a prime appears squared: for $n = 4$, $a = 2$, $k = 1$ the claim would be $2^3 \equiv 2$, but $2^3 = 8 \equiv 0 \pmod 4$.

Why RSA decryption works. RSA picks e and d with $ed \equiv 1 \pmod{\phi(n)}$, i.e. $ed = k\phi(n) + 1$. Decrypting $C = M^e$ gives $C^d = M^{ed} = M^{k\phi(n)+1} \equiv M \pmod n$ by the second version, for every message $M < n$. In the example $n = 187$, $e = 7$, $d = 23$: $ed = 161 = 1 \cdot 160 + 1$, and indeed $88^7 \bmod 187 = 11$ and $11^{23} \bmod 187 = 88$.

★ MUST KNOW

	First version	Second version (no co-prime condition)
Fermat (p prime)	$a^{p-1} \equiv 1 \pmod p$ if $p \nmid a$	$a^p \equiv a \pmod p$
Euler	$a^{\phi(n)} \equiv 1 \pmod n$ if $\gcd(a, n) = 1$	$a^{k\phi(n)+1} \equiv a \pmod n$ ($n = pq$, squarefree)
Reduction	$a^b \equiv a^{b \bmod (p-1)} \pmod p$ and	$a^b \equiv a^{b \bmod \phi(n)} \pmod n$ (co-prime a)
Inverse	$a^{-1} \equiv a^{p-2} \pmod p$ and	$a^{-1} \equiv a^{\phi(n)-1} \pmod n$ (co-prime a)

WORKED EXAMPLE : exponent reduction by Euler

Q. Using Euler's theorem, find (a) $11^{2024} \bmod 35$ (b) the last two digits of 3^{1003} (c) $20^{62} \bmod 77$ (slide example).
A.

	gcd, $\phi(n)$	Exponent	Remaining power	Result
(a)	$\gcd(11, 35) = 1$, $\phi(35) = 4 \cdot 6 = 24$	$2024 = 84 \cdot 24 + 8$	$11^2 = 121 \equiv 16$, $11^4 \equiv 16^2 = 256 \equiv 11$, $11^8 \equiv 11^2 \equiv 16$	**16**
(b)	$\gcd(3, 100) = 1$, $\phi(100) = 40$	$1003 = 25 \cdot 40 + 3$	$3^3 = 27$	**27**
(c)	$\gcd(20, 77) = 1$, $\phi(77) = 6 \cdot 10 = 60$	$62 = 1 \cdot 60 + 2$	$20^2 = 400 = 5 \cdot 77 + 15$	**15**

For (b), "last two digits" means the residue modulo 100. **Check** (by script): $11^{2024} \bmod 35 = 16$, $3^{1003} \bmod 100 = 27$, $20^{62} \bmod 77 = 15$.

27.3 Multiplicative inverses via Fermat and Euler ★

★ **In the papers:** 2024–25 Q5(iii): $7^{-1} \bmod 17$, $15^{-1} \bmod 99$ (does not exist).

The multiplicative inverse of a modulo n is the residue a^{-1} with $a \cdot a^{-1} \equiv 1 \pmod n$; it exists if and only if $\gcd(a, n) = 1$ (derived in §26). It depends on the modulus as well as on a: 2^{-1} is 3 modulo 5 but 4 modulo 7. For small n it can be found by trial, multiplying a by $1, 2, 3, \ldots$ until the product is $\equiv 1$:

$a \bmod n$	gcd	Search	Inverse	Note
3 mod 5	1	$3 \cdot 2 = 6 \equiv 1$	**2**	so $2^{-1} \bmod 5 = 3$ as well
2 mod 11	1	$2 \cdot 6 = 12 \equiv 1$	**6**	
4 mod 5	1	$4 \cdot 4 = 16 \equiv 1$	**4**	its own inverse ($4 \equiv -1$)
2 mod 7	1	$2 \cdot 4 = 8 \equiv 1$	**4**	not 3: depends on n
5 mod 10	**5**	$5k \bmod 10 \in \{0, 5\}$ only	**none**	

Inverse by exponentiation. Fermat's theorem already contains the inverse. If p is prime and $p \nmid a$, then $a \cdot a^{p-2} = a^{p-1} \equiv 1$, so

$$a^{-1} \equiv a^{p-2} \pmod p, \qquad \text{and in the same way} \qquad a^{-1} \equiv a^{\phi(n)-1} \pmod n \text{ if } \gcd(a, n) = 1.$$

This replaces the extended Euclidean algorithm by one modular exponentiation, which is convenient when the modulus is prime or when $\phi(n)$ is known. The price is that for a composite n the factorisation is needed to get $\phi(n)$. The slides' examples (L10-15 and L10-20), all checked by script:

Fermat (n prime)		Euler (n composite)		
$8^{-1} \bmod 17 = 8^{15}$	$= 15$	$8^{-1} \bmod 77 = 8^{59}$	$= 29$	$\phi(77) = 60$
$5^{-1} \bmod 23 = 5^{21}$	$= 14$	$7^{-1} \bmod 15 = 7^{7}$	$= 13$	$\phi(15) = 8$
$60^{-1} \bmod 101 = 60^{99}$	$= 32$	$60^{-1} \bmod 187 = 60^{159}$	$= 53$	$\phi(187) = 160$
$22^{-1} \bmod 211 = 22^{209}$	$= 48$	$71^{-1} \bmod 100 = 71^{39}$	$= 31$	$\phi(100) = 40$
$2^{-1} \bmod 13 = 2^{11}$	$= 7$	$3^{-1} \bmod 20 = 3^{7}$	$= 7$	$\phi(20) = 8$

Each answer is checked by one multiplication, e.g. $8 \cdot 15 = 120 = 7 \cdot 17 + 1$ and $7 \cdot 13 = 91 = 6 \cdot 15 + 1$.

Trap: the inverse may not exist

Before using any formula, compute $\gcd(a, n)$. If it is not 1 there is no inverse, and the exam expects exactly that answer with the reason. Blindly applying Euler's formula still produces a number, which is wrong: for $15^{-1} \bmod 99$, $\phi(99) = 60$ and $15^{59} \bmod 99 = 36$, but $15 \cdot 36 = 540 \equiv 45 \not\equiv 1 \pmod{99}$.

WORKED EXAMPLE : 2024–25 Q5(ii) and (iii), as it should be written

Q. (ii) Find the values of $100^{95} \bmod 9$, $658^{17} \bmod 17$ and $145^{102} \bmod 101$. (iii) Find the values of $7^{-1} \bmod 17$ and $15^{-1} \bmod 99$.
A. (ii) $100^{95} \bmod 9$. The modulus 9 is not prime, so Fermat's theorem cannot be used; reduce the base instead. $100 = 11 \cdot 9 + 1$, so $100 \equiv 1 \pmod 9$ and $100^{95} \equiv 1^{95} = \mathbf{1}$. (Euler agrees: $\gcd(100, 9) = 1$, $\phi(9) = 6$, $95 \bmod 6 = 5$, $1^5 = 1$.)
$658^{17} \bmod 17$. 17 is prime, so by the second version of Fermat's theorem $a^{17} \equiv a \pmod{17}$. $658 = 38 \cdot 17 + 12$, hence $658^{17} \equiv 658 \equiv \mathbf{12}$.
$145^{102} \bmod 101$. 101 is prime and $101 \nmid 145$. Base: $145 \equiv 44$. Exponent: $102 = 1 \cdot 100 + 2$, and $44^{100} \equiv 1$ by Fermat. So $145^{102} \equiv 44^2 = 1936 = 19 \cdot 101 + 17$, i.e. **17**.
(iii) $7^{-1} \bmod 17$. 17 is prime and $17 \nmid 7$, so $7^{-1} \equiv 7^{17-2} = 7^{15} \pmod{17}$. With $15 = 8 + 4 + 2 + 1$:

7^1	7^2	7^4	7^8	$7^{15} = 7^8 \cdot 7^4 \cdot 7^2 \cdot 7$
7	$49 \equiv 15$	$15^2 = 225 \equiv 4$	$4^2 = 16$	$16 \cdot 4 = 64 \equiv 13$; $13 \cdot 15 = 195 \equiv 8$; $8 \cdot 7 = 56 \equiv \mathbf{5}$

So $7^{-1} \bmod 17 = \mathbf{5}$. Check: $7 \cdot 5 = 35 = 2 \cdot 17 + 1$ ✓.
$15^{-1} \bmod 99$. $\gcd(99, 15)$: $99 = 6 \cdot 15 + 9$, $15 = 1 \cdot 9 + 6$, $9 = 1 \cdot 6 + 3$, $6 = 2 \cdot 3 + 0$, so $\gcd = 3 \neq 1$. Every number $15x - 99k$ is a multiple of 3 and can never equal 1, therefore $15^{-1} \bmod 99$ **does not exist**.

Which method? All four methods give the same inverse when it exists; they differ in what they need. The summary below is the decision to make in an exam.

Method	Formula	When to use
Trial	$a \cdot 1, a \cdot 2, \ldots$ until $\equiv 1$	Small n only (mod 26 ciphers, GF(7) division).
Fermat	$a^{-1} = a^{p-2} \bmod p$	Prime modulus; compute with square-and-multiply.
Euler	$a^{-1} = a^{\phi(n)-1} \bmod n$	Composite n whose factorisation (hence $\phi(n)$) is known.
Extended Euclid	$sa + tn = 1 \Rightarrow a^{-1} = s \bmod n$	The general method: works for any n without factoring, and detects "no inverse" (final gcd $\neq 1$). The past papers ask for it explicitly (2024–25 Q7, §25); RSA uses it to find d.

27.4 Order of an element ▸

▸ **Likely this year:** "Find the order of every element of $\mathbb{Z}_7^*$ / $\mathbb{Z}_{10}^*$" (Forouzan), usually as the first step of a primitive-root question.

Euler's theorem says that $a^{\phi(n)} \equiv 1$ for every $a \in \mathbb{Z}_n^*$, but many elements return to 1 much earlier. The order of a modulo n, $\mathrm{ord}(a)$, is the smallest positive integer i with $a^i \equiv 1 \pmod n$. It exists for every $a \in \mathbb{Z}_n^*$, and Euler's theorem guarantees $\mathrm{ord}(a) \leq \phi(n)$. After $a^{\mathrm{ord}(a)} = 1$ the powers start over, so the sequence $a, a^2, a^3, \ldots$ is periodic with period exactly $\mathrm{ord}(a)$.

The key property: $a^m \equiv 1 \iff \mathrm{ord}(a) \mid m$. Write $m = q \cdot \mathrm{ord}(a) + r$ with $0 \leq r < \mathrm{ord}(a)$. Then $a^m = \left(a^{\mathrm{ord}(a)}\right)^q a^r \equiv a^r$, and by the minimality of the order $a^r \equiv 1$ only for $r = 0$. Two consequences follow. First, since $a^{\phi(n)} \equiv 1$, **the order always divides** $\phi(n)$ (this is Lagrange's theorem for the group $\mathbb{Z}_n^*$), so to find an order only the divisors of $\phi(n)$ need to be tried. Second, exponents can be reduced modulo $\mathrm{ord}(a)$, which is sharper than reducing modulo $\phi(n)$: $\mathrm{ord}(2) = 3$ modulo 7, so $2^{100} \equiv 2^{100 \bmod 3} = 2^1 = 2 \pmod 7$.

The table lists $a^i \bmod 7$ for the whole group $\mathbb{Z}_7^*$ (slide L11-30); the shaded cell is the first 1 in each row, and its column is the order. Every order is a divisor of $\phi(7) = 6$, and only $a = 3$ and $a = 5$ reach the maximum 6.

a	$i=1$	2	3	4	5	6	ord(a)
1	**1**	1	1	1	1	1	1
2	2	4	**1**	2	4	1	3
3	3	2	6	4	5	**1**	**6**
4	4	2	**1**	4	2	1	3
5	5	4	6	2	3	**1**	**6**
6	6	**1**	6	1	6	1	2

Two more slide examples (L11-27, L11-29): in $\mathbb{Z}_{10}^* = \{1,3,7,9\}$ the orders are $1,4,4,2$ ($3^2 = 9$, $3^4 = 81 \equiv 1$), and in $\mathbb{Z}_8^* = \{1,3,5,7\}$ they are $1,2,2,2$, because $3^2 = 9$, $5^2 = 25$ and $7^2 = 49$ are all $\equiv 1 \pmod 8$.

27.5 Primitive roots ▸

▸ **Likely this year:** "Find all primitive roots of 7 / 11 / 13", "is 3 a primitive root of 17?", "how many primitive roots does 17 have?" (needed for Diffie–Hellman and ElGamal).

The idea, for a prime p. Take the multiplicative group $\mathbb{Z}_p^* = \{1, 2, \dots, p-1\}$, which is $\mathbb{Z}_p$ without 0. Pick an element g and write down its powers $g^1, g^2, g^3, \dots \pmod p$. Two things can happen. Either the powers run through *every* element of $\mathbb{Z}_p^*$ before returning to 1, or they hit 1 early and then repeat a smaller cycle. In the first case g is called a primitive root of p: its powers $g^1, \dots, g^{p-1}$ produce each of $1, \dots, p-1$ exactly once. Modulo 7, for example:

	g^1	g^2	g^3	g^4	g^5	g^6	
$g=3$	3	2	6	4	5	1	all six elements: **primitive root**
$g=2$	2	4	1	2	4	1	only $\{2,4,1\}$: **not** a primitive root

Since the powers start repeating as soon as they reach 1, the number of different powers equals the order of g. Producing all $p-1$ elements therefore means exactly that $\text{ord}(g) = p-1$, the largest order possible. When such a g exists, every element of the group is a power of it, so the group is *cyclic* and g is a *generator*. Every element y is then g^x for exactly one $x \in \{1, \dots, p-1\}$, and that x is the discrete logarithm of y (§28).

Why cryptography needs them. Diffie–Hellman and ElGamal publish p and a base g, keep an exponent x secret, and publish $g^x \bmod p$. If g had a small order d, then g^x could take only d different values, and Eve could find a matching exponent by trying d values. Choosing a primitive root makes all $p-1$ values possible, so the search is as large as it can be.

The same idea for any n. Nothing changes except the group. For a general n the group is $\mathbb{Z}_n^*$, the numbers below n that are coprime to n, and it has $\phi(n)$ elements instead of $p-1$. So $g \in \mathbb{Z}_n^*$ is a primitive root of n if $\text{ord}(g) = \phi(n)$, which means $g^1, \dots, g^{\phi(n)}$ produce every element of $\mathbb{Z}_n^*$ once each. Modulo 10, $\mathbb{Z}_{10}^* = \{1,3,7,9\}$ and the powers of 3 are $3, 9, 7, 1$, so 3 is a primitive root of 10. The powers never produce 0, 2, 4, 5, 6 or 8, and they are not supposed to: a power of a number coprime to n is itself coprime to n, so only $\mathbb{Z}_n^*$ can ever be generated, never all of $\mathbb{Z}_n$. For a prime this is the same statement as before, since $\mathbb{Z}_p^*$ is $\mathbb{Z}_p$ without 0 and $\phi(p) = p-1$.

Unlike primes, many composite n have *no* primitive root at all, because every element returns to 1 too early. Modulo 8, the group $\mathbb{Z}_8^* = \{1,3,5,7\}$ has $\phi(8) = 4$ elements, but 3^2, 5^2 and 7^2 are all $\equiv 1$, so no order reaches 4 (slide L11-28). Modulo 15, every element satisfies $a^4 \equiv 1$, because $a^2 \equiv 1 \pmod 3$ and $a^4 \equiv 1 \pmod 5$, so no order reaches $\phi(15) = 8$. The complete answer is a theorem worth memorising:

★ MUST KNOW : primitive roots

g is a primitive root of $n \iff \text{ord}(g) = \phi(n) \iff$ the powers of g produce all of $\mathbb{Z}_n^*$ (for a prime: all of $1, \dots, p-1$).
Primitive roots exist **only** for $n = 2,\ 4,\ p^k,\ 2p^k$ (p an odd prime). Every prime has them; 9, 10, 18 have them; 8, 12, 15 do not.
When they exist there are exactly $\phi(\phi(n))$ of them; for a prime p that is $\phi(p-1)$.

Test 1: list the powers. Multiply by g repeatedly, reducing each time; for example $3^4 \equiv 6 \cdot 3 = 18 \equiv 4 \pmod 7$, so you never need $3^6 = 729$. If a 1 appears before the $(p-1)$-th power, stop: g is not a primitive root.

Test	Powers $g^1, g^2, \ldots \pmod p$	Verdict
2 mod 5	2, 4, 3, 1	order $4 = p-1$: **yes**
3 mod 5	3, 4, 2, 1	**yes**
4 mod 5	4, **1**	order 2: no
3 mod 7	3, 2, 6, 4, 5, 1	**yes**
2 mod 7	2, 4, **1**	order 3: no
2 mod 11 (homework)	2, 4, 8, 5, 10, 9, 7, 3, 6, 1	**yes**

The homework "primitive roots of 5" is answered by the first three rows: 1 never works ($1^k = 1$), 2 and 3 do, and 4 does not, so the primitive roots of 5 are **2 and 3**.

Test 2: the fast test. Listing takes up to $p-1$ multiplications. It is faster to use the fact that the order of g must divide $p-1$: if g is not a primitive root, its order is a proper divisor of $p-1$, and every proper divisor divides $(p-1)/q$ for some prime factor q of $p-1$. So it is enough to check those few exponents:

$$g \text{ is a primitive root of } p \iff g^{(p-1)/q} \not\equiv 1 \pmod p \text{ for every prime } q \text{ dividing } p-1.$$

For $p = 7$, $p-1 = 6 = 2 \cdot 3$, so check g^3 and g^2. For $g = 3$: $3^3 = 27 \equiv 6$ and $3^2 = 9 \equiv 2$, neither is 1, so 3 is a primitive root. For $g = 2$: $2^3 = 8 \equiv 1$, so 2 is not.

Finding all of them from one. Once one primitive root g is known, the others are the powers g^k with $\gcd(k, p-1) = 1$. The reason is that g^k has order $\frac{p-1}{\gcd(k,p-1)}$, which equals $p-1$ exactly when the gcd is 1. Counting those k gives the $\phi(p-1)$ from the box: 5 has $\phi(4) = 2$ (2, 3), 7 has $\phi(6) = 2$ (3, 5), 11 has $\phi(10) = 4$ (2, 6, 7, 8), and 13 has $\phi(12) = 4$ (2, 6, 7, 11).

WORKED EXAMPLE : all primitive roots of a prime

Q. Find all the primitive roots of 11 (i.e. of the group $G = \langle \mathbb{Z}_{11}^*, \times \rangle$).
A. $\phi(11) = 10 = 2 \cdot 5$, so a primitive root g must satisfy $g^{10/2} = g^5 \not\equiv 1$ and $g^{10/5} = g^2 \not\equiv 1$. Try $g = 2$: $2^2 = 4 \not\equiv 1$ and $2^5 = 32 = 2 \cdot 11 + 10 \equiv 10 \not\equiv 1$, so **2 is a primitive root**. (For contrast, $g = 3$ fails: $3^5 = 243 = 22 \cdot 11 + 1 \equiv 1$.)
The other primitive roots are 2^k with $\gcd(k, 10) = 1$, i.e. $k \in \{1, 3, 7, 9\}$:

k	1	3	7	9
$2^k \bmod 11$	2	8	$128 = 11 \cdot 11 + 7 \to 7$	$512 = 46 \cdot 11 + 6 \to 6$

The primitive roots of 11 are **2, 6, 7, 8**. **Check:** their number is $\phi(10) = 4$ ✓, and the full list of powers of 2 is 2, 4, 8, 5, 10, 9, 7, 3, 6, 1, all ten residues.

WORKED EXAMPLE : fast test and counting

Q. Show that 3 is a primitive root of 17 but 2 is not. How many primitive roots does 17 have? List them.
A. $\phi(17) = 16 = 2^4$, whose only prime factor is 2, so g is a primitive root iff $g^{16/2} = g^8 \not\equiv 1 \pmod{17}$. For $g = 3$: $3^2 = 9$, $3^4 = 81 \equiv 13$, $3^8 \equiv 13^2 = 169 \equiv 16 \not\equiv 1$, so **3 is a primitive root**. For $g = 2$: $2^4 = 16$, $2^8 \equiv 16^2 = 256 = 15 \cdot 17 + 1 \equiv 1$, so **2 is not** (its order is 8). The number of primitive roots is $\phi(16) = \mathbf{8}$. They are 3^k for odd k: $3^1, 3^3, \ldots, 3^{15} \equiv 3, 10, 5, 11, 14, 7, 12, 6$, i.e. **3, 5, 6, 7, 10, 11, 12, 14**.

Corrections to the notes / slides

- All values recomputed by script. The slides' inverse examples (L10-15: 15, 14, 32, 48; L10-20: 29, 13, 53, 31) and examples $6^{10} \bmod 11 = 1$, $3^{12} \bmod 11 = 9$, $6^{24} \bmod 35 = 1$, $20^{62} \bmod 77 = 15$ are correct. The video homework answers are correct: $5^{10} \equiv 1 \pmod{11}$, $11^{20} \equiv 22 \pmod{33}$, 2 is a primitive root of 11, the primitive roots of 5 are 2 and 3.
- Slide L10-16 says the second version of Euler's theorem "removes the condition that a and n should be coprime". That is true only for squarefree n (such as $n = pq$ in RSA). It fails for $n = 4$, $a = 2$: $2^{\phi(4)+1} = 2^3 = 8 \equiv 0 \not\equiv 2$.
- Slide L10-18 solves $20^{62} \bmod 77$ with the second version, but $\gcd(20, 77) = 1$, so the first version already gives $20^{62} = 20^{60} \cdot 20^2 \equiv 400 \equiv 15$.
- Video notation: "-2^{12}" and "-1^{10}" mean $(-2)^{12}$ and $(-1)^{10}$. Without brackets $-2^{12} = -(2^{12})$; write the brackets in the exam.
- The general definition of a primitive root (any n) was stated in the video but not explained: only $n = 2, 4, p^k, 2p^k$ have primitive roots, and then there are $\phi(\phi(n))$ of them.

Check your understanding

1. **State both versions of Fermat's little theorem and the difference between them.** If p is prime and $p \nmid a$, then $a^{p-1} \equiv 1 \pmod p$; for every a, $a^p \equiv a \pmod p$. The second version drops the condition $p \nmid a$, since for $p \mid a$ both sides are 0.

2. **Why is the exponent reduced modulo $p-1$ and not modulo p?** Because the powers of a repeat when the exponent grows by $p-1$ ($a^{p-1} \equiv 1$), not by p. For example $2^{2026} \bmod 11$: $2026 \bmod 10 = 6$ and $2^6 = 64 \equiv \mathbf{9}$.
3. **Differentiate Fermat's and Euler's theorems.** Fermat needs a prime modulus and gives period $p-1$; Euler allows any modulus n with $\gcd(a,n)=1$ and gives period $\phi(n)$. Fermat is the special case $n=p$ of Euler.
4. **Find $3^{-1} \bmod 20$ by Euler's theorem.** $\gcd(3,20)=1$, $\phi(20)=8$, so $3^{-1} \equiv 3^7 = 2187 \equiv \mathbf{7}$; check $3 \cdot 7 = 21 \equiv 1$.
5. **Why does $15^{-1} \bmod 99$ not exist, and what goes wrong if Euler's formula is used anyway?** $\gcd(15,99)=3$, so $15x - 99k$ is always a multiple of 3. The formula gives $15^{59} \bmod 99 = 36$, but $15 \cdot 36 \equiv 45 \neq 1$.
6. **Why does RSA decryption recover the message?** $ed = k\phi(n)+1$, so $(M^e)^d = M^{k\phi(n)+1} \equiv M \pmod n$ by the second version of Euler's theorem, which holds for $n=pq$ even if $\gcd(M,n) \neq 1$.
7. **What is the order of 2 modulo 13, and is 2 a primitive root?** The order divides 12; $2^4 \equiv 3$ and $2^6 = 64 \equiv 12$, so neither $12/3 = 4$ nor $12/2 = 6$ gives 1. Hence $\operatorname{ord}(2) = 12$ and 2 is a primitive root of 13. The others are $2^5 \equiv 6$, $2^7 \equiv 11$, $2^{11} \equiv 7$, so the primitive roots of 13 are 2, 6, 7, 11.
8. **How many primitive roots does 23 have, and is 5 one of them?** $\phi(22) = 10$. Since $22 = 2 \cdot 11$, check $5^{11} \equiv 22 \neq 1$ and $5^2 = 25 \equiv 2 \neq 1$, so 5 is a primitive root.
9. **Why does $\mathbb{Z}_8^*$ have no primitive root?** $\phi(8)=4$, but 3^2, 5^2 and 7^2 are all $\equiv 1 \pmod 8$, so no element has order 4.
10. **Why must the Diffie–Hellman base be a primitive root?** If g had order $d < p-1$, then g^x would take only d values and the secret exponent could be found among d candidates; a primitive root makes g^x range over all $p-1$ residues.

28 Primes, Primality Testing, Factorization, CRT and Discrete Logs

Asymmetric-key cryptography is built on a handful of facts about primes, and this chapter collects them. RSA needs two large random primes, so we need a fast way to decide whether a 1024-bit number is prime; this is *primality testing*. RSA is secure only because nobody can split $n=pq$ back into p and q; this is the hardness of *factorization*. RSA decryption is made about four times faster by working modulo p and q separately and gluing the results with the *Chinese remainder theorem*. Every public-key operation is a modular power, computed by *square-and-multiply*. Finally, Diffie–Hellman and ElGamal rely on the fact that undoing a modular power, the *discrete logarithm*, is infeasible.

28.1 Primes and how many there are

A positive integer is prime if it has exactly two divisors, 1 and itself; a *composite* has more than two divisors, and the number 1 (exactly one divisor) is neither. Two integers are *coprime* (relatively prime) when their gcd is 1, so a prime p is coprime to every integer from 1 to $p-1$, which is why $\mathbb{Z}_p^* = \{1, \dots, p-1\}$.

Infinitely many primes. Suppose the primes were only $p_1, \dots, p_k$, and let $N = p_1 p_2 \cdots p_k + 1$. Dividing N by any p_i leaves remainder 1, so N either is a new prime or has a prime factor outside the list; either way the list was incomplete. Note that N itself need not be prime: $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 + 1 = 30031 = 59 \times 509$, but 59 and 509 are new primes.

Counting primes. The function $\pi(n)$ counts the primes $\le n$: $\pi(1)=0$, $\pi(2)=1$, $\pi(3)=2$, $\pi(10)=4$, $\pi(20)=8$, $\pi(50)=15$, $\pi(100)=25$. For large n there is no formula, only bounds (Lagrange found the lower one, Gauss the upper):

$$\frac{n}{\ln n} < \pi(n) < \frac{n}{\ln n - 1.08366}.$$

For $n = 10^6$: $n/\ln n = 72{,}382.4$ and $n/(\ln n - 1.08366) = 78{,}543.2$, so $72{,}383 \le \pi(10^6) \le 78{,}543$; the actual count is 78,498. The practical consequence is that primes thin out only slowly: near N about one number in $\ln N$ is prime. For 1024-bit numbers $\ln 2^{1024} \approx 710$, so if we test random *odd* 1024-bit numbers, about one in 355 is prime, and generating an RSA prime takes a few hundred primality tests. This is why the test itself must be fast.

Checking a number by $\sqrt{n}$. If $n = ab$ with $1 < a \le b$, then $a^2 \le ab = n$, so $a \le \sqrt{n}$. Therefore a composite n always has a prime factor $\le \lfloor \sqrt{n} \rfloor$, and it is enough to try the primes up to $\lfloor \sqrt{n} \rfloor$.

- Is 97 prime? $\lfloor \sqrt{97} \rfloor = 9$, and none of 2, 3, 5, 7 divides 97, so **97 is prime**.
- Is 301 prime? $\lfloor \sqrt{301} \rfloor = 17$, so we would try 2, 3, 5, 7, 11, 13, 17; but already $7 \mid 301$ ($301 = 7 \times 43$), so **301 is composite**.

Mersenne and Fermat numbers. These were early attempts at formulas that produce primes. A *Mersenne number* is $M_p = 2^p - 1$. If the exponent factors as ab, then $2^a - 1$ divides $2^{ab} - 1$ (since $x-1 \mid x^b - 1$ with $x = 2^a$), so M_p can be prime only for prime p. The condition is not sufficient: $M_2, M_3, M_5, M_7, M_{13}, M_{17}, M_{19} = 3, 7, 31, 127, 8191, 131071, 524287$ are prime, but $M_{11} = 2047 = 23 \times 89$ and $M_{23} = 8388607 = 47 \times 178481$. A *Fermat number* is $F_n = 2^{2^n} + 1$. Fermat saw that $F_0, \dots, F_4 = 3, 5, 17, 257, 65537$ are prime and guessed that all are, but Euler found $F_5 = 4294967297 = 641 \times 6700417$, and no larger Fermat number is known to be prime. $F_4 = 65537 = 2^{16} + 1$ survives in practice as the standard RSA public exponent, because its binary form has only two 1-bits, which makes encryption with square-and-multiply cheap.

28.2 Primality testing: deterministic and probabilistic

The problem is to decide, for a number of several hundred digits, whether it is prime. A *deterministic* algorithm always gives the correct answer; a *probabilistic* algorithm is much faster but may declare a composite number "prime" with a small, controllable probability. Its "composite" answers are always correct, because they come with a witness.

Divisibility test (deterministic). Try every $r = 2, 3, \dots, \lfloor\sqrt{n}\rfloor$ and return "composite" as soon as $r \mid n$, otherwise "prime". It is correct by the $\sqrt{n}$ argument above, but its cost is exponential in the size of the input: an n_b-bit number has $\sqrt{n} \approx 2^{n_b/2}$ candidate divisors. For a 200-bit n this is 2^{100} bit operations; at 2^{30} operations per second that takes 2^{70} seconds, about 3.7×10^{13} years.

AKS (deterministic, polynomial). In 2002 Agrawal, Kayal and Saxena gave the first deterministic test whose time is polynomial in the number of bits, $O((\log_2 n)^{12})$ bit operations, based on the identity $(x-a)^p \equiv x^p - a \pmod p$, which holds exactly when p is prime. It settled a theoretical question, but in practice the probabilistic tests below are far faster.

Fermat test (probabilistic). Fermat's little theorem says that if n is prime then $a^{n-1} \equiv 1 \pmod n$ for every a with $n \nmid a$. Read in reverse: if $a^{n-1} \not\equiv 1 \pmod n$, then n is certainly composite; if $a^{n-1} \equiv 1$, then n is only *probably* prime, because some composites also satisfy it. For example $341 = 11 \times 31$ gives $2^{340} \equiv 1 \pmod{341}$ (since $2^{10} = 1024 = 3 \cdot 341 + 1$), so base 2 is fooled, but base 3 exposes it: $3^{340} \equiv 56 \pmod{341}$.

Carmichael numbers. Trying more bases does not always help. A *Carmichael number* is a composite n with $a^{n-1} \equiv 1 \pmod n$ for *every* a coprime to n, so the Fermat test fails for all such bases. The smallest is $561 = 3 \times 11 \times 17$ (then 1105, 1729, 2465, 2821). The reason is visible from the factors: $561 - 1 = 560$ is divisible by $3-1$, $11-1$ and $17-1$, so for each prime factor p, $a^{560} = (a^{p-1})^{560/(p-1)} \equiv 1 \pmod p$, and a number that is $\equiv 1$ modulo 3, 11 and 17 is $\equiv 1$ modulo 561 (by the CRT below). In general n is Carmichael exactly when it is squarefree and $(p-1) \mid (n-1)$ for every prime $p \mid n$ (Korselt's criterion). Only bases sharing a factor with n catch it, e.g. $3^{560} \equiv 375 \pmod{561}$, and for large n such bases are too rare to find by chance.

Square-root test. If $n = p$ is prime and $x^2 \equiv 1 \pmod p$, then $p \mid (x-1)(x+1)$, and since p is prime it divides one of the two factors, so $x \equiv \pm 1 \pmod p$. Hence modulo a prime, the only square roots of 1 are $+1$ and -1 (written $n-1$ for -1). A composite modulus usually has extra, *nontrivial* square roots of 1, and finding one proves that n is composite.

n	Square roots of 1 mod n	Comment
7 (prime)	1, 6	only ± 1
17 (prime)	1, 16	only ± 1
8 (composite)	1, 3, 5, 7	extra roots 3, 5 $\Rightarrow$ composite
15 (composite)	1, 4, 11, 14	extra roots 4, 11
22 (composite)	1, 21	only ± 1 although composite
561 (composite)	1, 67, 188, 254, 307, 373, 494, 560	$2^3 = 8$ roots (3 prime factors)

The test is one-way: $n = 22 = 2 \times 11$ has only ± 1, because modulo 2 there is a single square root of 1. For an odd n with r distinct prime factors there are 2^r square roots of 1 (a choice of ± 1 modulo each prime factor, glued by the CRT). A nontrivial root also *factors* n: since $n \mid (x-1)(x+1)$ but divides neither factor, $\gcd(x \pm 1, n)$ are proper factors; for $x = 67$, $\gcd(66, 561) = 33$ and $\gcd(68, 561) = 17$.

28.3 Miller–Rabin test ▸

▸ **Likely this year:** "Use the Miller–Rabin test with base 2 to check whether 561 (or 61, 341) is prime": write $n - 1 = m \cdot 2^k$, then the T table.

The Fermat test only looks at the final value a^{n-1}, and Carmichael numbers make that value 1 for every base. Miller–Rabin also looks at the values *on the way* to a^{n-1} and applies the square-root test to them, which catches Carmichael numbers too.

Idea. Write $n - 1 = m \cdot 2^k$ with m odd (n odd, so $k \geq 1$). Then a^{n-1} is obtained from a^m by squaring k times:

$$a^m,\ a^{2m},\ a^{4m},\ \dots,\ a^{2^k m} = a^{n-1} \pmod n,$$

each term being the square of the previous one. If n is prime, the last term is 1 by Fermat. Walking backwards from it, every term whose square is 1 must be ± 1, by the square-root test. So for a prime n the sequence either starts with $a^m \equiv 1$ (then it is all 1s), or it contains a -1 somewhere before the last position, after which it is all 1s. If the sequence reaches $+1$ *without* a -1 just before it, the previous term is a nontrivial square root of 1 and n is composite; if it never reaches ± 1, Fermat already fails.

★ MUST KNOW : Miller–Rabin (base a)

Find m odd and k with $n-1 = m \times 2^k$. Set $T \leftarrow a^m \bmod n$. If $T = \pm 1$, return **prime**.
Repeat at most $k-1$ times: $T \leftarrow T^2 \bmod n$; if $T = +1$ return **composite** (the previous T was a nontrivial square root of 1); if $T = -1$ return **prime**.
If the loop ends without a decision, return **composite**.

The loop stops after $k-1$ squarings because the k-th square is a^{n-1} itself: if the previous T is not ± 1, then either its square is 1 (a nontrivial root) or it is not 1 (Fermat fails), so n is composite in both cases and nothing is gained by computing it. "Prime" here means "probably prime": a composite n passes for at most $1/4$ of the bases (such bases are called *strong liars*), so t independent random bases leave an error probability of at most 4^{-t}, e.g. $4^{-40} \approx 10^{-24}$. The *recommended primality test* (Forouzan) is therefore trial division by small primes first, which cheaply removes most composites, followed by Miller–Rabin with several bases.

WORKED EXAMPLE : Miller–Rabin on 561 and 61 (Forouzan, base 2)

Q. Use the Miller–Rabin test with base 2 to decide whether (a) 561 and (b) 61 are prime.
(a) $n-1 = 560 = 35 \times 2^4$, so $m = 35$, $k = 4$, $a = 2$, and at most $k-1 = 3$ squarings. First $2^{35} \bmod 561$ by repeated squaring: $2^8 = 256$, $2^{16} = 256^2 \bmod 561 = 460$, $2^{32} = 460^2 \bmod 561 = 103$, so $2^{35} = 2^{32} \cdot 2^3 \equiv 103 \times 8 = 824 \equiv 263$.

Step	Computation	T	Decision
Init	$2^{35} \bmod 561$	263	not ± 1, continue
1	$263^2 = 69169 \bmod 561$	166	not ± 1, continue
2	$166^2 = 27556 \bmod 561$	67	not ± 1, continue
3	$67^2 = 4489 \bmod 561$	**1**	$+1 \Rightarrow$ **composite**

561 is composite. Justification: $67^2 \equiv 1$ with $67 \neq \pm 1$, so 67 is a nontrivial square root of 1, impossible modulo a prime. (Indeed $561 = 3 \times 11 \times 17$; and $\gcd(66, 561) = 33$, $\gcd(68, 561) = 17$.) Note that one more squaring gives $2^{560} \equiv 1$, so 561 *passes* the Fermat test; Miller–Rabin catches it only because it inspects the step from 67 to 1.

(b) $n-1 = 60 = 15 \times 2^2$, so $m = 15$, $k = 2$, at most one squaring. $2^{15} = 2^8 \cdot 2^4 \cdot 2^2 \cdot 2 \equiv 12 \times 16 \times 4 \times 2 = 1536 = 25 \cdot 61 + 11 \equiv 11 \pmod{61}$.

Step	Computation	T	Decision
Init	$2^{15} \bmod 61$	11	not ± 1, continue
1	$11^2 = 121 \bmod 61$	$60 = -1$	$-1 \Rightarrow$ **prime**

61 passes (probably prime), and 61 is indeed prime. Check: $2^{60} = (2^{30})^2 \equiv (-1)^2 = 1$, consistent with Fermat.

WORKED EXAMPLE : Fermat test vs Miller–Rabin on 341 (new)

Q. Show that 341 passes the Fermat test with base 2, then apply the Miller–Rabin test with base 2. What can you conclude?
Fermat test. $2^{10} = 1024 = 3 \times 341 + 1$, so $2^{10} \equiv 1 \pmod{341}$ and $2^{340} = (2^{10})^{34} \equiv 1$. 341 passes the Fermat test with base 2.
Miller–Rabin. $340 = 85 \times 2^2$, so $m = 85$, $k = 2$, at most one squaring. $2^{85} = (2^{10})^8 \cdot 2^5 \equiv 1 \cdot 32 = 32$.

Step	Computation	T	Decision
Init	$2^{85} \bmod 341$	32	not ± 1, continue
1	$32^2 = 1024 \bmod 341$	**1**	$+1 \Rightarrow$ **composite**

341 is composite: 32 is a nontrivial square root of 1 modulo 341. Base 2 is a Fermat liar for 341 but not a strong liar. The root even gives the factors: $\gcd(31, 341) = 31$ and $\gcd(33, 341) = 11$, so $341 = 11 \times 31$.

WORKED EXAMPLE : a prime with a long sequence (new)

Q. Apply the Miller–Rabin test with base 2 to $n = 97$.
$96 = 3 \times 2^5$, so $m = 3$, $k = 5$, at most 4 squarings.

Step	Computation	T	Decision
Init	$2^3 \bmod 97$	8	continue
1	$8^2 = 64$	64	continue
2	$64^2 = 4096 = 42 \cdot 97 + 22$	22	continue
3	$22^2 = 484 = 5 \cdot 97 - 1$	$96 = -1$	**prime**

97 passes (and is prime, as the $\sqrt{97}$ check showed earlier).

The remaining slide runs and two instructive edge cases:

n, base	$n-1 = m \cdot 2^k$	T sequence	Result
27, 2	$13 \cdot 2^1$	11 (no loop, $k-1=0$)	composite (correct)
4033, 2	$63 \cdot 2^6$	3521, **4032** ($=-1$)	"prime" (wrong: 37×109)
4033, 3	$63 \cdot 2^6$	3551, 2443, 3442, 2443, 3442, 2443	composite (correct)
2047, 2	$1023 \cdot 2^1$	**1**	"prime" (wrong: $M_{11} = 23 \times 89$)
25, 7	$3 \cdot 2^3$	18, **24** ($=-1$)	"prime" (wrong: 5^2)

Bases 2 for 4033 and 2047, and 7 for 25, are strong liars; this is exactly why the test is repeated with several bases (4033 fools base 2 but not base 3).

28.4 Factorization

Fundamental theorem of arithmetic. Every integer $n > 1$ can be written as a product of primes, $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$, and this representation is unique apart from the order of the factors. Uniqueness lets us read gcd and lcm directly from the exponents. A common divisor can use each prime at most as often as *both* numbers contain it, and a common multiple must use it at least as often as *either* does:

$$\gcd(a,b) = \prod p_i^{\min(a_i,b_i)}, \qquad \operatorname{lcm}(a,b) = \prod p_i^{\max(a_i,b_i)}, \qquad \gcd(a,b) \times \operatorname{lcm}(a,b) = a \times b,$$

the last because $\min(a_i,b_i) + \max(a_i,b_i) = a_i + b_i$ for every prime. Example: $2160 = 2^4 \cdot 3^3 \cdot 5$ and $1200 = 2^4 \cdot 3 \cdot 5^2$ give gcd $= 2^4 \cdot 3 \cdot 5 = 240$ and lcm $= 2^4 \cdot 3^3 \cdot 5^2 = 10800$; check $240 \times 10800 = 2{,}592{,}000 = 2160 \times 1200$. This method is useless for large numbers, since it needs the factorization, which is exactly the hard part; Euclid's algorithm finds the gcd without it.

Why factorization matters. Finding $\phi(n)$, and with it the RSA private key, is as hard as factoring $n = pq$. Primality testing is easy (Miller–Rabin), but finding the factors of a number already known to be composite is believed to be hard, and RSA lives in the gap between the two problems.

Trial division. Divide n by $a = 2, 3, 4, \ldots$ while $a \le \sqrt{n}$; whenever $a \mid n$, output a, replace n by n/a, and try the same a again (to catch repeated factors). If $n > 1$ when the loop ends, the remaining n is a prime factor. Examples: $1233 = 3^2 \times 137$, and $1523357784 = 2^3 \times 3^2 \times 13 \times 37 \times 43987$. Like the divisibility test, it costs up to $\sqrt{n} = 2^{n_b/2}$ divisions, so it only works when n has small factors.

Fermat's factorization method. Any odd $n = ab$ (with $a \ge b$) is a difference of two squares: with $x = (a+b)/2$ and $y = (a-b)/2$ we get $x^2 - y^2 = ab = n$. Conversely, if we find x with $w = x^2 - n$ a perfect square y^2, then $n = (x+y)(x-y)$. The method starts at $x = \lceil \sqrt{n} \rceil$ and increases x by 1 until $x^2 - n$ is a square. Since the search ends at $x = (a+b)/2$, it is fast exactly when the two factors are close to $\sqrt{n}$.

x	$w = x^2 - 5959$	Square?	
78	125	no	($\lceil \sqrt{5959} \rceil = 78$)
79	282	no	
80	441	yes, 21^2	$5959 = (80+21)(80-21) =$ **101×59**

For 8051 it succeeds immediately ($90^2 - 8051 = 49 = 7^2$, so $8051 = 97 \times 83$), while $4033 = 37 \times 109$ needs 10 steps ($x = 64, \ldots, 73$; $73^2 - 4033 = 1296 = 36^2$). The security lesson is that RSA primes p and q must not be close to each other.

Other methods (named on the slides). *Pollard $p-1$* finds a prime factor p quickly when $p-1$ has only small prime factors, by computing $\gcd(a^{B!} - 1, n)$. *Pollard rho* iterates $x \leftarrow x^2 + c \bmod n$ and finds a factor p in about $\sqrt{p}$ steps from a repetition modulo p, detected with a gcd. The *quadratic sieve* and the *number field sieve* build many congruences $x^2 \equiv y^2 \pmod{n}$ (a generalisation of Fermat's idea) and are sub-exponential; the number field sieve is the fastest known method for large RSA moduli, but it is still far too slow for 2048-bit n. (A *sieve of Eratosthenes*, by contrast, is the school method for *listing* all primes up to a bound by crossing out multiples.)

28.5 Chinese remainder theorem ▸

▸ **Likely this year:** Forouzan style "Solve $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$" with the full M, M_i, M_i^{-1} table and the inverses by extended Euclid; or a word problem / RSA speed-up.

Problem. We are given several congruences in one unknown with different moduli,

$$x \equiv a_1 \pmod{m_1}, \quad x \equiv a_2 \pmod{m_2}, \quad \ldots, \quad x \equiv a_k \pmod{m_k},$$

and want all x that satisfy them simultaneously. The Chinese remainder theorem (CRT) states that if the moduli are pairwise coprime, the system has exactly one solution modulo $M = m_1 m_2 \cdots m_k$, and it gives a formula for it. For example $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$ has the solution $x = 23$ (and $23 + 105t$).

Idea. Build x as a sum of k terms in which the i-th term is $\equiv a_i$ modulo m_i and $\equiv 0$ modulo every other modulus. Then modulo m_i all terms but the i-th vanish and $x \equiv a_i$, for each i at once. A number divisible by every modulus except m_i is $M_i = M/m_i$; multiplying it by its inverse modulo m_i turns it into 1 modulo m_i without disturbing the zeros, and multiplying by a_i gives the required value.

★ MUST KNOW : CRT in four steps

1. $M = m_1 \times m_2 \times \cdots \times m_k$ (the common modulus). 2. $M_i = M/m_i$ for each i.
3. M_i^{-1} = the inverse of M_i **modulo** m_i (reduce M_i mod m_i first, then extended Euclid).
4. $x = \left(a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1} + \cdots + a_k M_k M_k^{-1}\right) \bmod M$.

Why it works. For $j \neq i$, m_j divides M_i, so the term $a_i M_i M_i^{-1}$ is $\equiv 0 \pmod{m_j}$. Modulo m_i the term equals $a_i \cdot (M_i M_i^{-1}) \equiv a_i \cdot 1$. Therefore $x \equiv a_i \pmod{m_i}$ for every i. Step 3 needs $\gcd(M_i, m_i) = 1$, and this is exactly where pairwise coprimality is used: M_i is a product of moduli each coprime to m_i.

Why the solution is unique modulo M. If x and x' both solve the system, every m_i divides $x - x'$; since the m_i are pairwise coprime, their product M divides $x - x'$. So all solutions are $x + tM$, and there is exactly one in $\{0, \ldots, M-1\}$. Equivalently, the map $x \mapsto (x \bmod m_1, \ldots, x \bmod m_k)$ is a one-to-one correspondence between $\mathbb{Z}_M$ and $\mathbb{Z}_{m_1} \times \cdots \times \mathbb{Z}_{m_k}$, which is what the applications below exploit.

Edge case: moduli not coprime. Then the theorem does not apply, and the system may have no solution or several. For instance $x \equiv 1 \pmod 4$ and $x \equiv 2 \pmod 6$ has no solution, because the first forces x odd and the second forces x even.

WORKED EXAMPLE : the standard CRT question (Forouzan, slide example)

Q. Solve $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$ using the Chinese remainder theorem.
Step 1. The moduli 3, 5, 7 are pairwise coprime, so the CRT applies. $M = 3 \times 5 \times 7 = 105$.
Steps 2–3.

i	a_i	m_i	$M_i = M/m_i$	$M_i \bmod m_i$	$M_i^{-1} \bmod m_i$	$a_i M_i M_i^{-1}$
1	2	3	35	2	2	$2 \cdot 35 \cdot 2 = 140$
2	3	5	21	1	1	$3 \cdot 21 \cdot 1 = 63$
3	2	7	15	1	1	$2 \cdot 15 \cdot 1 = 30$

Inverses (extended Euclid with $r_1 = m_i$, $r_2 = M_i \bmod m_i$, $t_1 = 0$, $t_2 = 1$). For $35^{-1} \bmod 3 = 2^{-1} \bmod 3$:

q	r_1	r_2	r	t_1	t_2	t
1	3	2	1	0	1	−1
2	2	1	0	1	−1	3
	1	0		**−1**	3	

$r_1 = 1$, so the inverse exists and equals $t_1 = -1 \equiv \mathbf{2} \pmod 3$ (check $2 \cdot 2 = 4 \equiv 1$). Since $21 \equiv 1 \pmod 5$ and $15 \equiv 1 \pmod 7$, the other two inverses are 1.
Step 4. $x = (140 + 63 + 30) \bmod 105 = 233 \bmod 105 = \mathbf{23}$.
Check. $23 = 7 \cdot 3 + 2$, $23 = 4 \cdot 5 + 3$, $23 = 3 \cdot 7 + 2$ ✓. All solutions: $x = 23 + 105t$.

WORKED EXAMPLE : CRT with non-trivial inverses (new)

Q. Find the smallest positive x with $x \equiv 3 \pmod 7$, $x \equiv 5 \pmod{11}$, $x \equiv 7 \pmod{13}$.
Step 1. 7, 11, 13 are distinct primes, hence pairwise coprime. $M = 7 \times 11 \times 13 = 1001$.
Steps 2–3.

i	a_i	m_i	M_i	$M_i \bmod m_i$	$M_i^{-1} \bmod m_i$	$a_i M_i M_i^{-1}$
1	3	7	143	3	5	$3 \cdot 143 \cdot 5 = 2145$
2	5	11	91	3	4	$5 \cdot 91 \cdot 4 = 1820$
3	7	13	77	12	12	$7 \cdot 77 \cdot 12 = 6468$

Inverses by extended Euclid (columns $q, r_1, r_2, r, t_1, t_2, t$):

$3^{-1} \bmod 7$

q	r_1	r_2	r	t_1	t_2	t
2	7	3	1	0	1	−2
3	3	1	0	1	−2	7
	1	0		**−2**	7	

$3^{-1} \bmod 11$

q	r_1	r_2	r	t_1	t_2	t
3	11	3	2	0	1	−3
1	3	2	1	1	−3	4
2	2	1	0	−3	4	−11
	1	0		**4**	−11	

So $143^{-1} \equiv 3^{-1} \equiv -2 \equiv \mathbf{5} \pmod 7$ (check $3 \cdot 5 = 15 \equiv 1$) and $91^{-1} \equiv 3^{-1} \equiv \mathbf{4} \pmod{11}$ (check $3 \cdot 4 = 12 \equiv 1$). For the third, $77 \equiv 12 \equiv -1 \pmod{13}$, and $(-1)^{-1} = -1$, so $77^{-1} \equiv \mathbf{12} \pmod{13}$.
Step 4. $x = (2145 + 1820 + 6468) \bmod 1001 = 10433 \bmod 1001 = \mathbf{423}$.
Check. $423 = 60 \cdot 7 + 3$, $423 = 38 \cdot 11 + 5$, $423 = 32 \cdot 13 + 7$ ✓.

Trap: inverses modulo the wrong number

M_i^{-1} is the inverse modulo m_i (the small modulus), not modulo M. Reduce M_i modulo m_i before running extended Euclid, but multiply the *unreduced* M_i in step 4; using $M_i \bmod m_i$ in the final sum breaks the zeros modulo the other moduli.

More slide examples (all checked):

System	M	M_i	M_i^{-1}	x
$x \equiv 3\ (7),\ 3\ (13),\ 0\ (12)$	1092	156, 84, 91	4, 11, 7	**276**
$S \equiv 1\ (11),\ 12\ (13),\ 2\ (17)$	2431	221, 187, 143	1, 8, 5	**155**
$z \equiv 61\ (99),\ 65\ (98),\ 69\ (97)$	941094	9506, 9603, 9702	50, 97, 49	**457**

Applications.

- *Word problems.* "Find an integer that leaves remainder 3 when divided by 7 and by 13 but is divisible by 12" is the first row above: $x = 276$ ($276 = 12 \times 23$, $276 = 39 \cdot 7 + 3 = 21 \cdot 13 + 3$).
- *Secret sharing.* The secret S is handed out as residues modulo 11, 13, 17, 19: shares 1, 12, 2, 3. Any 3 shares determine S uniquely modulo the product of their moduli, which exceeds S, so CRT recovers it; every one of the four 3-subsets gives $S = 155$ (the second row uses $\{1, 12, 2\}$).
- *Arithmetic on big numbers with small registers.* To compute $z = 123 + 334$ on a machine that holds only numbers below 100, represent $123 \equiv 24, 25, 26$ and $334 \equiv 37, 40, 43$ modulo $99, 98, 97$; add componentwise to get $z \equiv 61, 65, 69$; CRT gives $z = 457$, correct because $457 < M = 941094$ and the solution is unique modulo M.
- *RSA decryption speed-up.* Since $n = pq$ with p, q coprime, the plaintext $P = C^d \bmod n$ is determined by $P \bmod p$ and $P \bmod q$. These are computed with exponents reduced by Fermat, $d_p = d \bmod (p-1)$ and $d_q = d \bmod (q-1)$, on numbers half as long, and are then combined by CRT. A modular multiplication on half-length numbers costs about a quarter as much and the exponents are half as long, so the two half-size powers together are roughly four times faster than one full-size power.

WORKED EXAMPLE : RSA decryption with CRT (new)

Q. In RSA with $p = 7$, $q = 11$, $n = 77$, $e = 13$, $d = 37$, decrypt $C = 41$ using the CRT.
Reduced exponents. $d_p = 37 \bmod 6 = 1$, $d_q = 37 \bmod 10 = 7$.
Half-size powers. $P_p = 41^1 \bmod 7 = 6$.
$P_q = 41^7 \bmod 11 = 8^7 \bmod 11$: $8^2 = 64 \equiv 9$, $8^4 \equiv 81 \equiv 4$, $8^7 = 8^4 \cdot 8^2 \cdot 8 \equiv 4 \cdot 9 \cdot 8 = 288 \equiv 2$.
CRT. Solve $P \equiv 6 \pmod 7$, $P \equiv 2 \pmod{11}$ with $M = 77$:
$M_1 = 11 \equiv 4 \pmod 7$, $4^{-1} \bmod 7 = 2$; $M_2 = 7$, $7^{-1} \bmod 11 = 8$ ($7 \cdot 8 = 56 = 5 \cdot 11 + 1$).
$P = (6 \cdot 11 \cdot 2 + 2 \cdot 7 \cdot 8) \bmod 77 = (132 + 112) \bmod 77 = 244 \bmod 77 = \mathbf{13}$.
Check. Directly, $41^{37} \bmod 77 = 13$, and re-encrypting, $13^{13} \bmod 77 = 41$ ✓.

28.6 Fast exponentiation: square-and-multiply ★

▸ **Likely this year:** Needed inside every RSA / ElGamal / Diffie–Hellman numerical: show the bit table for $a^x \bmod n$.

Problem. Public-key cryptography computes $y = a^x \bmod n$ with x and n of 1024 bits or more. Multiplying by a again and again needs $x - 1$ multiplications, about 2^{1024}, which is impossible. (The size of intermediate results is the smaller problem: reducing modulo n after every multiplication keeps them below n^2.)

Idea. Write the exponent in binary, $x = x_{n_b-1} 2^{n_b-1} + \cdots + x_1 2^1 + x_0 2^0$ with each $x_i \in \{0, 1\}$. Then

$$a^x = \prod_{i\,:\,x_i=1} a^{2^i}, \qquad \text{e.g.} \quad a^{22} = a^{16} \cdot a^4 \cdot a^2 \quad (22 = 10110_2).$$

The powers $a, a^2, a^4, a^8, \ldots$ are each the square of the previous one, so they cost one squaring each, and we multiply into the result only those whose bit is 1.

Mechanism (Forouzan, right-to-left). Start with $y = 1$. Scan the bits from x_0 (least significant) upwards: if $x_i = 1$, set $y \leftarrow a \times y \bmod n$; then square, $a \leftarrow a^2 \bmod n$ (not needed after the last bit). Return y. An n_b-bit exponent needs at most $n_b - 1$ squarings and n_b multiplications, so the cost is $O(n_b)$ modular multiplications, linear in the number of bits instead of exponential. A 1024-bit exponent needs at most about 2048 multiplications. The public exponent $e = 65537 = 10000000000000001_2$ needs 16 squarings and a single non-trivial multiplication, which is why it is the usual choice.

$17^{22} \bmod 21$ **(slide example),** $22 = 10110_2$

i	x_i	Multiplication (init. $y = 1$)	Squaring (init. $a = 17$)
0	0	–	$a = 17^2 \bmod 21 = 16$
1	1	$y = 1 \times 16 \bmod 21 = 16$	$a = 16^2 \bmod 21 = 4$
2	1	$y = 16 \times 4 \bmod 21 = 1$	$a = 4^2 \bmod 21 = 16$
3	0	–	$a = 16^2 \bmod 21 = 4$
4	1	$y = 1 \times 4 \bmod 21 =$ **4**	(not needed)

So $17^{22} \bmod 21 = 4$, using 4 squarings and 3 multiplications instead of 21 multiplications. Section 23 computes the same kind of power left-to-right (scanning from the most significant bit, square then multiply by a); both give the same answer, and either is acceptable in the exam as long as every intermediate value is shown.

WORKED EXAMPLE : RSA encryption and decryption by square-and-multiply (Forouzan numbers)

Q. In RSA, $n = 77$, $e = 13$, $d = 37$. Encrypt $M = 5$ and verify by decrypting, using the square-and-multiply method.
Encryption $C = 5^{13} \bmod 77$, $13 = 1101_2$ ($x_0 = 1, x_1 = 0, x_2 = 1, x_3 = 1$):

i	x_i	Multiplication ($y = 1$)	Squaring ($a = 5$)
0	1	$y = 1 \times 5 = 5$	$a = 5^2 = 25$
1	0	–	$a = 25^2 = 625 \bmod 77 = 9$
2	1	$y = 5 \times 9 = 45$	$a = 9^2 = 81 \bmod 77 = 4$
3	1	$y = 45 \times 4 = 180 \bmod 77 =$ **26**	(not needed)

Decryption $M = 26^{37} \bmod 77$, $37 = 100101_2$:

i	x_i	Multiplication ($y = 1$)	Squaring ($a = 26$)
0	1	$y = 26$	$a = 26^2 = 676 \bmod 77 = 60$
1	0	–	$a = 60^2 = 3600 \bmod 77 = 58$
2	1	$y = 26 \times 58 = 1508 \bmod 77 = 45$	$a = 58^2 = 3364 \bmod 77 = 53$
3	0	–	$a = 53^2 = 2809 \bmod 77 = 37$
4	0	–	$a = 37^2 = 1369 \bmod 77 = 60$
5	1	$y = 45 \times 60 = 2700 \bmod 77 =$ **5**	(not needed)

Answer. $C = 26$, and decryption returns $M = 5$ ✓.

28.7 Discrete logarithms ▸

▸ **Likely this year:** "Find x such that $3^x \equiv 7 \pmod{17}$" via a table of powers, using the log rules; or theory: why the discrete log is hard and which systems rely on it.

Problem. Square-and-multiply computes $y = a^x \bmod n$ quickly. The reverse question is: given a, y and n, find x. Over the real numbers this is $\log_a y$; modulo n it is the *discrete logarithm*, and no comparably fast method is known for it.

Primitive roots (reminder). The order of a in $\mathbb{Z}_n^*$ is the smallest $i \geq 1$ with $a^i \equiv 1$, and a is a *primitive root* when its order equals $\phi(n)$; details and examples are in §27. For a prime p, $\mathbb{Z}_p^*$ always has primitive roots, and if g is one then $g^1, g^2, \ldots, g^{p-1}$ are all different and run through every element $1, \ldots, p-1$ exactly once. Hence every $y \in \mathbb{Z}_p^*$ equals g^x for exactly one $x \in \{1, \ldots, p-1\}$; this x is the *discrete logarithm* of y to the base g, written $x = L_g\, y$ (the modulus is understood). In $\mathbb{Z}_7^*$ the primitive roots are 3 and 5, giving two logarithm tables:

y	1	2	3	4	5	6
$x = L_3\, y \pmod 7$	6	2	1	4	5	3
$x = L_5\, y \pmod 7$	6	4	5	2	1	3

With the table, $4 \equiv 3^x \pmod 7$ gives $x = L_3 4 = 4$ ($3^4 = 81 \equiv 4$), and $6 \equiv 5^x \pmod 7$ gives $x = L_5 6 = 3$ ($5^3 = 125 \equiv 6$).

Rules. Since $g^{p-1} \equiv 1$, exponents of g only matter modulo $p-1 = \phi(p)$, so discrete logs obey the usual logarithm laws with arithmetic modulo $\phi(n)$:

$$L_g 1 \equiv 0, \qquad L_g(x \cdot y) \equiv L_g x + L_g y, \qquad L_g(x^k) \equiv k \cdot L_g x \pmod{\phi(n)}.$$

The second rule follows from $g^{Lx} \cdot g^{Ly} = g^{Lx+Ly}$. (The table writes $L_g 1 = 6$ rather than 0, because it lists exponents from 1 to $p-1$; $6 \equiv 0 \pmod 6$.) Example in $\mathbb{Z}_7^*$: $L_3(4 \cdot 5) = L_3 6 = 3$, and $L_3 4 + L_3 5 = 4 + 5 = 9 \equiv 3 \pmod 6$ ✓.

Edge case: base not a primitive root. Then the powers of a do not cover the whole group, so $a^x \equiv y$ may have no solution or several. In $\mathbb{Z}_7^*$, 2 has order 3 (2^x cycles through 2, 4, 1), so $2^x \equiv 3 \pmod 7$ has no solution, while $2^x \equiv 4$ has $x = 2$ and $x = 5$.

Why it is hard. The only general method on the slides is *exhaustive search*: for $x = 1, 2, \ldots, n-1$, test whether $y = a^x \bmod n$ and return the first x that works. That is up to $p-1$ exponentiations, exponential in the number of bits, while computing a^x needs only $O(\log p)$ multiplications. Better algorithms exist (baby-step giant-step in about $\sqrt{p}$ steps; index calculus, sub-exponential), but for a well-chosen 2048-bit prime they are still infeasible, roughly as hard as factoring a 2048-bit RSA modulus. Modular exponentiation is therefore a *one-way function*, and this asymmetry is the basis of **Diffie–Hellman** (both parties publish g^a and g^b and share g^{ab}; an eavesdropper would need $a = L_g(g^a)$) and of **ElGamal** (the private key d hides behind the public $e_2 = e_1^d \bmod p$). For example, with $p = 17$, $g = 3$, $a = 4$, $b = 7$: the public values are $3^4 \equiv 13$ and $3^7 \equiv 11$, and the shared key is $11^4 \equiv 13^7 \equiv 4 \pmod{17}$.

WORKED EXAMPLE : discrete logarithms modulo 17 (new)

Q. (a) Show that 3 is a primitive root of 17 and tabulate $L_3\, y$ for $\mathbb{Z}_{17}^*$. (b) Solve $3^x \equiv 7 \pmod{17}$. (c) Using only $L_3 2$ and $L_3 7$ and the rules, find $L_3 14$. (d) Solve $3^x \equiv 8 \pmod{17}$ from $L_3 2$.
(a) Successive powers $3^x \bmod 17$, each 3× the previous:

x	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$3^x \bmod 17$	3	9	10	13	5	15	11	16	14	8	7	4	12	2	6	1

The first power equal to 1 is 3^{16}, so $\operatorname{ord}(3) = 16 = \phi(17)$ and 3 is a primitive root (all 16 values differ). Reading the table backwards:

y	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$L_3\, y$	16	14	1	12	5	15	11	10	2	3	7	13	4	9	6	8

(b) $x = L_3 7 =$ **11**. Check: $3^{11} \bmod 17 = 7$ (from the power table).
(c) $L_3 14 = L_3(2 \cdot 7) \equiv L_3 2 + L_3 7 = 14 + 11 = 25 \equiv$ **9** $\pmod{16}$. Check: $3^9 \bmod 17 = 14$ ✓.
(d) $L_3 8 = L_3(2^3) \equiv 3 \cdot L_3 2 = 42 \equiv$ **10** $\pmod{16}$. Check: $3^{10} \bmod 17 = 8$ ✓.

Corrections to the notes / slides

- Slide (Fermat test): "561 = 33 × 17" is true but is not the prime factorisation; 561 = 3 × 11 × 17, the smallest Carmichael number.
- Slide (AKS): the example computes $(\log_2 200)^{12} \approx 3.9 \times 10^{10}$ bit operations for a 200-bit n (the printed 39,547,615,483 is not even that value exactly; it is ≈ 39,787,815,242). For a 200-bit number $\log_2 n = 200$, so the bound $(\log_2 n)^{12}$ is $200^{12} \approx 4.1 \times 10^{27}$: still polynomial, but not "40 seconds".
- Slide (recommended test, 4033): the list of trial divisors "2, 3, 5, 7, 11, 17, 23" skips 13 and 19 (neither divides 4033). A full divisibility test up to $\lfloor\sqrt{4033}\rfloor = 63$ would find 37 and settle the question; the recommended test uses trial division by *small* primes only.
- Slide (Fermat test): "If x is a prime, then $a^{n-1} \equiv 1$" should read "if n is a prime".
- Slide $\phi(1) = 0$ follows Forouzan's "integers smaller than n"; the standard convention is $\phi(1) = 1$. It never matters in RSA. (Totient chapter.)
- Euler's second version $a^{k\phi(n)+1} \equiv a \pmod n$ holds for squarefree n (such as $n = pq$ in RSA), not for every n; it fails for $n = 4$, $a = 2$. (See §27.)

Check your understanding

1. **Why is it enough to test divisors up to $\sqrt{n}$, and why is this still too slow for RSA?** If $n = ab$ with $a \le b$, then $a \le \sqrt{n}$, so a composite always has a factor at most $\sqrt{n}$. But $\sqrt{n} = 2^{n_b/2}$ for an n_b-bit number, which is exponential in the input size (2^{512} trials for a 1024-bit number).
2. **Differentiate deterministic and probabilistic primality tests.** A deterministic test (divisibility, AKS) always answers correctly; a probabilistic test (Fermat, Miller–Rabin) is fast and its "composite" answer is always correct, but its "prime" answer may be wrong with a small probability that shrinks with every extra base tried ($\le 4^{-t}$ for Miller–Rabin).

3. **What is a Carmichael number, and why does 561 pass the Fermat test for every coprime base?** A composite n with $a^{n-1} \equiv 1$ for all a coprime to n. For $561 = 3 \cdot 11 \cdot 17$, each $p - 1 \in \{2, 10, 16\}$ divides 560, so $a^{560} \equiv 1$ modulo 3, 11 and 17, hence modulo 561 by the CRT.
4. **Apply Miller–Rabin with base 2 to 221.** $220 = 55 \cdot 2^2$; $T = 2^{55} \bmod 221 = 128$ (not ± 1); one squaring: $128^2 \bmod 221 = 30$ (not ± 1). The loop ends without a decision, so 221 is composite ($= 13 \times 17$).
5. **Why must the CRT moduli be pairwise coprime?** Step 3 needs $M_i^{-1} \bmod m_i$, which exists only if $\gcd(M_i, m_i) = 1$, and uniqueness needs lcm of the moduli to equal their product. Without it the system can be unsolvable: $x \equiv 1 \pmod 4$, $x \equiv 2 \pmod 6$ has no solution.
6. **Solve $x \equiv 1 \pmod 5$, $x \equiv 2 \pmod 7$, $x \equiv 3 \pmod 9$, $x \equiv 4 \pmod{11}$.** $M = 3465$; $M_i = 693, 495, 385, 315$; reduced $3, 5, 7, 7$; inverses $2, 3, 4, 8$; $x = (1386 + 2970 + 4620 + 10080) \bmod 3465 = 19056 \bmod 3465 = 1731$.
7. **How many modular multiplications does $a^{65537} \bmod n$ need, and why?** $65537 = 2^{16} + 1$ has 17 bits with two 1s, so 16 squarings and one non-trivial multiplication (17 in total) instead of 65536.
8. **Why does Fermat factorization break 8051 at once, and what does that teach about RSA?** $\lceil\sqrt{8051}\rceil = 90$ and $90^2 - 8051 = 49 = 7^2$, so $8051 = 97 \times 83$. The method ends after about $(p + q)/2 - \sqrt{n}$ steps, so p and q must not be close together.
9. **Solve $2^x \equiv 9 \pmod{11}$.** 2 is a primitive root of 11 (powers $2, 4, 8, 5, 10, 9, \ldots$), and $2^6 = 64 \equiv 9$, so $x = 6$ (unique modulo 10).
10. **Why is the discrete logarithm a good basis for public-key cryptography?** Computing $g^x \bmod p$ takes $O(\log p)$ multiplications by square-and-multiply, but recovering x needs exhaustive search over up to $p - 1$ exponents (or still-infeasible sub-exponential algorithms for 2048-bit p). This one-way property protects the Diffie–Hellman shared key and the ElGamal private key.

29 Algebraic Structures: Groups, Rings, Fields ★★

Every cipher so far has done arithmetic on a set of integers: the additive cipher adds mod 26, and the multiplicative cipher multiplies mod 26 and needs an inverse to decrypt. Modern ciphers such as AES work on bytes, and they need to add, multiply and divide bytes without ever leaving the set of bytes. A set together with the operations defined on it is called an **algebraic structure**. Three of them matter here: a **group** has one operation that can always be undone, a **ring** has two operations of which only the first can always be undone, and a **field** has two operations that can both be undone (except division by zero). This section builds up to the finite fields GF(p) and GF(2^n), because AES does all its byte arithmetic in GF(2^8), and the GF(2^8) inverse was asked in both past papers (8 marks in total).

29.1 Groups

A **group** $G = \langle S, \bullet \rangle$ is a set S with one binary operation $\bullet$ (a rule that combines any two elements into a third) satisfying four properties:

1. **Closure**: if $a, b \in G$, then $a \bullet b \in G$.
2. **Associativity**: $(a \bullet b) \bullet c = a \bullet (b \bullet c)$ for all a, b, c.
3. **Identity**: there is an element e with $e \bullet a = a \bullet e = a$ for every a.
4. **Inverse**: every a has an element a' with $a \bullet a' = a' \bullet a = e$.

If in addition $a \bullet b = b \bullet a$ for all a, b (**commutativity**), the group is called **abelian** or commutative.

The inverse property is the reason groups matter in cryptography: it gives a second operation for free. "Undo b" is just $a \bullet b'$. In a group under addition, b' is $-b$ and the derived operation is subtraction; in a group under multiplication, b' is b^{-1} and the derived operation is division. A group therefore supports one *pair* of operations, $(+, -)$ or $(\times, \div)$, which is exactly the shape of an encryption/decryption pair.

$\langle \mathbb{Z}_n, + \rangle$ is an abelian group. Closure holds because the result is reduced mod n and so lands back in $\mathbb{Z}_n = \{0, \ldots, n-1\}$; associativity and commutativity are inherited from ordinary addition; the identity is 0; and the inverse of a is $n - a$, since $a + (n - a) = n \equiv 0$. For example, in $\mathbb{Z}_{10}$ the inverse of 3 is 7, so $4 - 3$ is computed as $4 + 7 = 11 \equiv 1$.

$\langle \mathbb{Z}_n^*, \times \rangle$ is an abelian group. $\mathbb{Z}_n^*$ is the set of residues that are coprime to n, for example $\mathbb{Z}_{10}^* = \{1, 3, 7, 9\}$. Closure holds because a product of two numbers coprime to n is again coprime to n; the identity is 1; and every element has an inverse because $\gcd(a, n) = 1$ lets the extended Euclidean algorithm (Section 25) find s, t with $sn + ta = 1$, i.e. $ta \equiv 1 \pmod n$. Plain $\mathbb{Z}_n$ under $\times$ is *not* a group: 0 has no inverse, and neither does any a sharing a factor with n (in $\mathbb{Z}_{10}$, $2y$ is always even and so never $\equiv 1$). Removing exactly these elements is what the star in $\mathbb{Z}_n^*$ does.

Reading an operation table. For a small set, the operation can be given as a table whose entry in row x, column y is $x \bullet y$. Closure means every entry belongs to the set. The identity is the element whose row repeats the column header. The inverse of x is the column in which the identity appears in row x. Commutativity means the table is symmetric about its main diagonal. Associativity cannot be seen at a glance and has to be checked on triples.

•	a	b	c	d
a	a	b	c	d
b	b	c	d	a
c	c	d	a	b
d	d	a	b	c

In the slide's table on $\{a, b, c, d\}$, every entry is in the set, so closure holds. Row a copies the header, so a is the identity. The identity appears (shaded) in row b under d, in row c under c and in row d under b, giving the inverse pairs (a, a), (b, d), (c, c). The table is symmetric, so the group is abelian, and all 64 triples are associative (e.g. $(a \bullet b) \bullet c = a \bullet (b \bullet c) = d$). It is in fact $\langle \mathbb{Z}_4, + \rangle$ with $a, b, c, d = 0, 1, 2, 3$.

Permutation group. The set is all $3! = 6$ permutations of three positions, and the operation $\circ$ is **composition**: apply one permutation, then the other. Write $[a\ b\ c]$ for the permutation sending $1 \to a$, $2 \to b$, $3 \to c$. Composition is read right to left, so $[3\ 1\ 2] \circ [1\ 3\ 2]$ means "first $[1\ 3\ 2]$, then $[3\ 1\ 2]$": $1 \to 1 \to 3$, $2 \to 3 \to 2$, $3 \to 2 \to 1$, which gives $[3\ 2\ 1]$ (the slide's example). Closure holds because two rearrangements in a row are again a rearrangement; composition of functions is associative; the identity is $[1\ 2\ 3]$; and every permutation can be undone, so it has an inverse. The group is **not abelian**: $[1\ 3\ 2] \circ [2\ 1\ 3] = [3\ 1\ 2]$, but $[2\ 1\ 3] \circ [1\ 3\ 2] = [2\ 3\ 1]$.

Why a double transposition is no stronger. A keyed transposition cipher with block length k encrypts each block by applying a key permutation π. Applying a second transposition π_2 (same block length) after π_1 is, by closure, the same as applying the single permutation $\pi_2 \circ \pi_1$, which is again one of the $k!$ possible keys. The attacker never needs to find π_1 and π_2 separately: recovering the one equivalent permutation breaks the cipher, and the key space is still $k!$. The same closure argument shows that two additive (shift) ciphers in a row are just one shift cipher.

29.2 Order, subgroups and cyclic subgroups

A group is **finite** if its set has finitely many elements, and its **order** $|G|$ is that number of elements: $|\mathbb{Z}_6| = 6$, $|\mathbb{Z}_{10}^*| = 4$.

A **subgroup** H of G is a subset of G that is itself a group under the *same* operation as G. It follows that the two share the same identity, that $a \bullet b$ of two members of H stays in H, and that the inverse of a member of H is in H. The trivial group $\langle \{e\}, \bullet \rangle$ and G itself are always subgroups of G. "Same operation" is the condition that catches students: $\mathbb{Z}_{10}$ is a subset of $\mathbb{Z}_{12}$, but $\langle \mathbb{Z}_{10}, + \rangle$ is **not** a subgroup of $\langle \mathbb{Z}_{12}, + \rangle$, because one adds mod 10 and the other mod 12 (e.g. $7 + 5$ is 2 in one and 0 in the other).

Cyclic subgroups. In a group, the power a^k means $a \bullet a \bullet \ldots \bullet a$ (k times), with $a^0 = e$; in an additive group, a^k therefore means $k \cdot a \bmod n$. In a finite group, the powers $a^0, a^1, a^2, \ldots$ cannot all be different, so they eventually return to e and then repeat. The set of powers is closed under the operation and contains each member's inverse, so it is a subgroup, called the **cyclic subgroup generated by** a. If it is the whole of G, then G is a **cyclic group** and a is a **generator**.

	$\langle \mathbb{Z}_6, + \rangle$ (powers = $k \cdot a \bmod 6$)		$\langle \mathbb{Z}_{10}^*, \times \rangle$ (powers = $a^k \bmod 10$)		
a	generated subgroup	ord(a)	a	$a^0, a^1, a^2, \ldots$	ord(a)
0	$\{0\}$	1	1	1	1
1	$\{0, 1, 2, 3, 4, 5\} = G$	6	3	$1, 3, 9, 7\ (= G)$	4
2	$\{0, 2, 4\}$	3	7	$1, 7, 9, 3\ (= G)$	4
3	$\{0, 3\}$	2	9	$1, 9$	2
4	$\{0, 4, 2\}$	3			
5	$\{0, 5, 4, 3, 2, 1\} = G$	6			

So $\langle \mathbb{Z}_6, + \rangle$ has four cyclic subgroups, $\{0\}$, $\{0, 2, 4\}$, $\{0, 3\}$ and G, and is cyclic with generators 1 and 5. $\langle \mathbb{Z}_{10}^*, \times \rangle$ has three, $\{1\}$, $\{1, 9\}$ and G, and is cyclic with generators 3 and 7.

Lagrange's theorem and the order of an element. If H is a subgroup of a finite group G, then $|H|$ divides $|G|$. The table agrees: the subgroup sizes 1, 2, 3, 6 all divide 6, and 1, 2, 4 divide 4. The **order of an element**, ord(a), is the order of the cyclic subgroup it generates, which is the smallest $k \geq 1$ with $a^k = e$. Applying Lagrange to that subgroup shows ord(a) divides $|G|$, and therefore $a^{|G|} = e$ for every a. With $G = \mathbb{Z}_n^*$ and $|G| = \phi(n)$ this is exactly Euler's theorem $a^{\phi(n)} \equiv 1 \pmod{n}$ (Section 27). Lagrange also rules out orders: $\mathbb{Z}_7^*$ has 6 elements, so no element can have order 4 or 5 (the actual orders of $1, \ldots, 6$ are $1, 3, 6, 3, 6, 2$).

29.3 Rings and fields

A **ring** $R = \langle \{\ldots\}, +, \times \rangle$ has two operations. The first must satisfy all five properties (it is an abelian group), the second only closure and associativity, and the second must **distribute** over the first: $a \times (b + c) = (a \times b) + (a \times c)$ and $(a + b) \times c = (a \times c) + (b \times c)$. If the second operation is also commutative, the ring is a **commutative ring**. $\langle \mathbb{Z}, +, \times \rangle$ is a commutative ring: addition satisfies all five properties, multiplication satisfies closure, associativity and commutativity, and $5 \times (3 + 2) = (5 \times 3) + (5 \times 2) = 25$. A ring supports $(+, -)$ and $\times$, but not division: $12 \div 5 = 2.4$ is not in $\mathbb{Z}$, so most elements of $\mathbb{Z}$ have no multiplicative inverse.

A **field** $F = \langle \{\ldots\}, +, \times \rangle$ is a commutative ring in which the second operation also satisfies all five properties, except that the identity of the first operation (0) has no inverse under the second. A field therefore supports both pairs, $(+, -)$ and $(\times, \div)$, with the single exception of division by 0. The exception cannot be avoided: distributivity gives $0 \times a = (0 + 0) \times a = 0 \times a + 0 \times a$, so $0 \times a = 0$ for every a, and nothing multiplies 0 into 1.

★ MUST KNOW : which properties each structure needs

Structure	First operation (+)	Second operation (×)	Supports	Typical set
Group	closure, associativity, identity, inverse (+ commutativity: abelian)	–	(+, −) or (×, ÷)	$\mathbb{Z}_n$; $\mathbb{Z}_n^*$
Ring	all 5	closure, associativity (+ commutativity: commutative ring); distributes over +	(+, −) and ×	$\mathbb{Z}$
Field	all 5	all 5, except 0 has no inverse; distributes over +	(+, −) and (×, ÷)	$\mathbb{Z}_p$, GF(2^n)

29.4 Finite fields: why the size must be p^n

Zero divisors kill inverses. Suppose $a \times b = 0$ with $a \neq 0$ and $b \neq 0$. If a had an inverse a^{-1}, then $b = a^{-1} \times (a \times b) = a^{-1} \times 0 = 0$, a contradiction; so a has no inverse. This one argument explains every "not a field" result in this chapter. For a composite $n = ab$ with $1 < a, b < n$, we get $a \times b \equiv 0 \pmod{n}$, so $\mathbb{Z}_n$ is only a commutative ring. For a prime p no such factorisation exists, every non-zero element is coprime to p and has an inverse, and $\mathbb{Z}_p$ is a field.

Galois' result. A finite field must have p^n elements, p prime and $n \geq 1$; it is written GF(p^n) (Galois field). The reason is short. Add 1 to itself repeatedly: $1, 1+1, 1+1+1, \ldots$. The field is finite, so some sum $k \cdot 1$ must equal 0; let p be the smallest such k. This p must be prime, because $p = ab$ with $a, b < p$ would give $(a \cdot 1)(b \cdot 1) = 0$ with both factors non-zero, which the zero-divisor argument forbids. So the field contains a copy of $\mathbb{Z}_p$. Every element can then be written uniquely as $a_0 e_0 + a_1 e_1 + \cdots + a_{n-1} e_{n-1}$ with coefficients $a_i \in \mathbb{Z}_p$ for some fixed basis $e_0, \ldots, e_{n-1}$ (the field is an n-dimensional vector space over $\mathbb{Z}_p$). There are p choices for each of the n coefficients, giving p^n elements. For GF(2^n) the basis will be $1, x, \ldots, x^{n-1}$ and the coefficients will be bits, which is exactly the polynomial representation used below.

29.5 GF(p) arithmetic and division ★

★ In the papers: 2024–25 Q5(i): $5 \div 3$ in GF(7).

GF(p) is the set $\mathbb{Z}_p = \{0, 1, \ldots, p-1\}$ with addition and multiplication mod p. Subtraction is addition of the additive inverse, $a - b = a + (p - b)$, and division is multiplication by the multiplicative inverse, $a \div b = a \times b^{-1}$, which exists for every $b \neq 0$ because p is prime. The inverse is found with the extended Euclidean algorithm, by inspection for small p (look for $b \cdot y \equiv 1$), or by Fermat as $b^{-1} \equiv b^{p-2}$.

The smallest field, **GF(2)** = {0, 1}, is the one every bit lives in: addition and subtraction are both **XOR** ($1 + 1 = 0$, and $-1 = 1$), multiplication is **AND**, and 1 is its own inverse under both operations. In **GF(5)** the additive inverse pairs are $(1, 4), (2, 3)$ and the multiplicative ones $(1, 1), (2, 3), (4, 4)$; in **GF(7)** they are $(1, 6), (2, 5), (3, 4)$ and $(1, 1), (2, 4), (3, 5), (6, 6)$.

WORKED EXAMPLE : 2024–25 Q5(i), "In the field GF(7), find the result of $5 \div 3$."

Division in a field means multiplication by the inverse: $5 \div 3 = 5 \times 3^{-1} \bmod 7$.
Inverse of 3. Extended Euclid with $r_1 = 7$, $r_2 = 3$:

q	r_1	r_2	r	t_1	t_2	t
2	7	3	1	0	1	−2
3	3	1	0	1	−2	7
	1	0		**−2**	7	

gcd = 1, so the inverse exists: $3^{-1} \equiv -2 \equiv 5 \pmod 7$ (check $3 \times 5 = 15 = 2 \cdot 7 + 1$; Fermat agrees, $3^5 = 243 \equiv 5$).
Divide. $5 \div 3 = 5 \times 5 = 25 = 3 \cdot 7 + 4 \equiv$ **4**.
Check: $4 \times 3 = 12 \equiv 5 \pmod 7$ ✓.
A second one in the same style: in GF(13), $4 \div 9 = 4 \times 9^{-1} = 4 \times 3 =$ **12**, since $9 \times 3 = 27 \equiv 1$; check $12 \times 9 = 108 = 8 \cdot 13 + 4$ ✓.

Trap: "÷" is not integer division

$5 \div 3$ in GF(7) is not 1.67 and not 1; it is the element y with $3y \equiv 5$. The same past-paper question then asked for 15^{-1} mod 99, which does not exist because $\gcd(15, 99) = 3$: $\mathbb{Z}_{99}$ is not a field, so the inverse is only guaranteed in GF(p).

29.6 Why GF(2^n) instead of $\mathbb{Z}_{2^n}$

Computers store numbers as n-bit words ($n = 8, 16, 32, \ldots$), so the natural set is $\{0, \ldots, 2^n - 1\}$ with modulus 2^n. But 2^n is not prime, so $\mathbb{Z}_{2^n}$ has zero divisors and is not a field. In $\mathbb{Z}_{256}$, for example, $2 \times 128 = 256 \equiv 0$, and indeed no even byte has an inverse, because $2y$ stays even mod 256 and can never be 1. Half of all bytes could not be divided by.

There are two ways out. The first is to use GF(p) with p the largest prime below 2^n. This works but wastes values: for $n = 4$ the prime is 13, so 13, 14 and 15 are unusable, and for $n = 8$ it is 251, so 251–255 are unusable; an arbitrary byte of data could not even be represented. The second, which AES uses, is to keep all 2^n words and *redefine* addition and multiplication so that the field properties hold. Galois guarantees that a field of size 2^n exists ($p = 2$), and it is built from polynomials.

29.7 Polynomial representation and addition

An n-bit word $a_{n-1} \ldots a_1 a_0$ stands for the polynomial $a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$, whose coefficients are bits, i.e. elements of GF(2). The term x^i is the i-th term and the bit a_i is its coefficient, so the word records which powers of x are present. For example, `10011001` $= x^7 + x^4 + x^3 + 1$. In the other direction, $x^5 + x^2 + x$ as an 8-bit word is found by supplying the missing terms, $0x^7 + 0x^6 + 1x^5 + 0x^4 + 0x^3 + 1x^2 + 1x + 0$, which is `00100110`. There are therefore two levels of arithmetic: the coefficients are combined in GF(2), and the polynomials are combined modulo a fixed polynomial in GF(2^n).

Addition and subtraction add the coefficients of equal powers in GF(2), so $x^k + x^k = 0$. Because $-1 \equiv 1 \pmod 2$, we have $-B = B$, so $A - B = A + B$: addition and subtraction are the same operation. Adding two polynomials of degree at most $n - 1$ never raises the degree, so no reduction is needed. Since adding coefficients mod 2 bit by bit is exactly XOR, addition in GF(2^n) is the XOR of the two words, every element is its own additive inverse ($a \oplus a = 0$), and the identity is 0. Slide example: $(x^5 + x^2 + x) \oplus (x^3 + x^2 + 1) = x^5 + x^3 + x + 1$, because the two x^2 terms cancel; in bits, `00100110` $\oplus$ `00001101` $=$ `00101011`.

29.8 Irreducible polynomials: the modulus

Multiplying two polynomials of degree at most $n-1$ can produce degree up to $2n-2$, which no longer fits in n bits. So, as with integers mod p, the product is divided by a fixed polynomial $m(x)$ of degree n and only the remainder (degree $< n$) is kept. The modulus must be **irreducible**: it cannot be written as a product of two polynomials of lower degree (at least 1) with coefficients in GF(2). An irreducible polynomial plays the role of a prime. If instead $m = f \cdot g$, then $f \otimes g \equiv 0 \pmod m$ with $f, g \neq 0$, and the zero-divisor argument shows f has no inverse, so the structure would only be a ring.

For degree 2 or 3, a polynomial is reducible exactly when it has a factor of degree 1, i.e. a root in GF(2). It has the root 0 if it has no constant term, and it has the root 1 if it has an even number of terms. So $x^2 + 1 = (x + 1)^2$ is reducible (two terms), while $x^2 + x + 1$ (constant term, three terms) is irreducible. From degree 4 on, a product of two irreducible quadratics also has to be ruled out, e.g. $x^4 + x^2 + 1 = (x^2 + x + 1)^2$ has no root but is still reducible.

Degree	Irreducible polynomials over GF(2)
1	x, $x + 1$
2	$x^2 + x + 1$
3	$x^3 + x + 1$, $x^3 + x^2 + 1$
4	$x^4 + x + 1$, $x^4 + x^3 + 1$, $x^4 + x^3 + x^2 + x + 1$
5	$x^5 + x^2 + 1$, $x^5 + x^3 + 1$, $x^5 + x^3 + x^2 + x + 1$, $x^5 + x^4 + x^2 + x + 1$, $x^5 + x^4 + x^3 + x + 1$, $x^5 + x^4 + x^3 + x^2 + 1$

AES uses $m(x) = x^8 + x^4 + x^3 + x + 1$, the bit pattern `1 0001 1011` $=$ `0x11B`, one of the 30 irreducible polynomials of degree 8.

29.9 Polynomial long division over GF(2) ★

★ **In the papers:** Every row of the GF(2^8) inverse table (2024–25 Q4, 2025–26 Q2) is one such division, and so is the verification step. This is where marks are lost.

Long division of polynomials over GF(2) is ordinary long division in which every subtraction is an XOR. Terms of equal power cancel, there are no borrows and no negative coefficients, and no fractions appear because every leading coefficient is 1. The procedure is:

1. Write the dividend and the divisor in descending powers, and write the dividend in columns, one column per power, leaving gaps for missing powers.
2. Divide the leading term of the current dividend by the leading term of the divisor, $x^a / x^b = x^{a-b}$. This is the next term of the quotient.
3. Multiply the *whole* divisor by x^{a-b} (add $a - b$ to every exponent) and write it under the current line, each term

in the column of its power.

4. XOR the two lines: a power present in both lines cancels, and a power present in only one line drops down. The leading term always cancels.
5. Repeat on the new line while its degree is at least the degree of the divisor. The last line is the **remainder**, and the quotient is the sum of the terms found in step 2.

WORKED EXAMPLE : reduce $x^{12} + x^7 + x^2$ modulo $m(x) = x^8 + x^4 + x^3 + x + 1$ (Forouzan)

x^{12}				$+ x^7$								$+ x^2$				dividend
x^{12}	$+ x^8$	$+ x^7$		$+ x^5$	$+ x^4$											$\oplus\ x^4 \cdot m$
	x^8			$+ x^5$	$+ x^4$							$+ x^2$				
	x^8					$+ x^4$	$+ x^3$			$+ x$	$+ 1$					$\oplus\ 1 \cdot m$
				x^5			$+ x^3$	$+ x^2$	$+ x$	$+ 1$						remainder

Quotient $x^4 + 1$, **remainder** $x^5 + x^3 + x^2 + x + 1$. In the first step $x^{12}/x^8 = x^4$, so m is shifted up by 4; the x^{12} and x^7 terms cancel, and x^8, x^5, x^4 come down together with the untouched x^2. In the second step $x^8/x^8 = 1$.

The same division in bits (shift `100011011` under the leading 1 and XOR; a fast way to check): `1000010000100` ⊕ `1000110110000` = `100110100`; `100110100` ⊕ `100011011` = `000101111` = `0x2F`.

Trap: the usual slips in GF(2) division

Shifting only the leading term of the divisor instead of all of it; aligning terms by position on the page instead of by power; writing $x^k + x^k = 2x^k$ instead of 0; and stopping when the remainder is 0 rather than when its degree is below the divisor's.

29.10 Multiplication in GF(2^n) ★

▸ **Likely this year:** "Multiply {57} by {83}" or "find {02} · {87}" (Stallings/FIPS-197, needed for AES MixColumns), or Forouzan's $P_1 \otimes P_2$.

To multiply, expand the product term by term ($x^i \cdot x^j = x^{i+j}$, and coefficient products in GF(2) are AND), add all the partial products with XOR so that equal powers cancel in pairs, and then reduce modulo $m(x)$ by long division; the remainder is the answer.

WORKED EXAMPLE : $(x^5 + x^2 + x) \otimes (x^7 + x^4 + x^3 + x^2 + x)$ in GF(2^8), long method

Let $P_1 = x^5 + x^2 + x$ and $P_2 = x^7 + x^4 + x^3 + x^2 + x$. One partial product per term of P_1:

$$x^5 \cdot P_2 = x^{12} + x^9 + x^8 + x^7 + x^6$$
$$x^2 \cdot P_2 = x^9 + x^6 + x^5 + x^4 + x^3$$
$$x \cdot P_2 = x^8 + x^5 + x^4 + x^3 + x^2$$

XOR: $x^9, x^8, x^6, x^5, x^4, x^3$ each appear twice and cancel, leaving $x^{12} + x^7 + x^2$. Reducing this modulo $m(x)$ was the long division above, so $P_1 \otimes P_2 = \mathbf{x^5 + x^3 + x^2 + x + 1}$, i.e. {26} ⊗ {9E} = {2F}.

The shortcut: multiply by x repeatedly. Multiplying a polynomial by x raises every exponent by one, which is a left shift of the word by one bit. If the top bit (x^7) was 0, the result still has degree at most 7 and nothing else is needed. If the top bit was 1, the result contains x^8, which must be reduced. Since $m(x) = x^8 + (x^4 + x^3 + x + 1) \equiv 0$ and minus equals plus, $x^8 \equiv x^4 + x^3 + x + 1$, so the reduction is: drop the overflow bit and XOR with `00011011` = `0x1B`. This step is called **xtime**. Any product $a \otimes b$ is then the XOR of $a \otimes x^i$ over the 1-bits i of b, and $a \otimes x^i$ is obtained by applying xtime i times. Because every intermediate result is reduced straight away, no number ever exceeds 8 bits, which is why this is the version used in software and hardware.

WORKED EXAMPLE : the same product by the shortcut (slide table)

Build $x^i \otimes P_2$ for $i = 0, \ldots, 5$, each row being xtime of the row above:

Power	Operation	Result	Byte	Reduced?
$x^0 \otimes P_2$		$x^7 + x^4 + x^3 + x^2 + x$	9E	No
$x^1 \otimes P_2$	$x \otimes (x^7 + x^4 + x^3 + x^2 + x)$	$x^5 + x^2 + x + 1$	27	Yes
$x^2 \otimes P_2$	$x \otimes (x^5 + x^2 + x + 1)$	$x^6 + x^3 + x^2 + x$	4E	No
$x^3 \otimes P_2$	$x \otimes (x^6 + x^3 + x^2 + x)$	$x^7 + x^4 + x^3 + x^2$	9C	No
$x^4 \otimes P_2$	$x \otimes (x^7 + x^4 + x^3 + x^2)$	$x^5 + x + 1$	23	Yes
$x^5 \otimes P_2$	$x \otimes (x^5 + x + 1)$	$x^6 + x^2 + x$	46	No

Row 1 in bits: `10011110` shifted is 1 `00111100`, the top bit overflowed, so `00111100` ⊕ `00011011` = `00100111` (27). Row 4 likewise: `00111000` ⊕ `00011011` = `00100011` (23).
$P_1 = x^5 + x^2 + x$ uses rows 5, 2 and 1: $(x^6 + x^2 + x) \oplus (x^6 + x^3 + x^2 + x) \oplus (x^5 + x^2 + x + 1) = \mathbf{x^5 + x^3 + x^2 + x + 1}$, i.e. 46 ⊕ 4E ⊕ 27 = 2F ✓ (same as the long method).

WORKED EXAMPLE : {57} • {83} and {57} • {13} in GF(2^8) (Stallings / FIPS-197)

{57} = `01010111` = $x^6 + x^4 + x^2 + x + 1$. Repeated xtime (a 1 in the top bit means XOR 1B after the shift):

i	0	1	2	3	4	5	6	7
{57} • x^i	57	AE	47	8E	07	0E	1C	38
reduced?		no	yes	no	yes	no	no	no

(e.g. $i = 2$: AE = `10101110` → `01011100` ⊕ `00011011` = `01000111` = 47.)
{83} = `10000011` = $x^7 + x + 1$, so {57} • {83} = 38 ⊕ AE ⊕ 57 = **C1** = $x^7 + x^6 + 1$.
{13} = `00010011` = $x^4 + x + 1$, so {57} • {13} = 07 ⊕ AE ⊕ 57 = **FE**.
Check (long method for 57·83): the product is $x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1$; dividing by $m(x)$ gives quotient $x^5 + x^3$ and remainder $x^7 + x^6 + 1$ ✓.
MixColumns use. AES only multiplies by 02 and 03: {02} • a = xtime(a) and {03} • a = xtime(a) ⊕ a. For example, {02} • {87}: `10000111` → `00001110` ⊕ `00011011` = `00010101` = **15**; and {03} • {6E} = DC ⊕ 6E = **B2** (Section 32).

29.11 Multiplicative inverse in GF(2^n): extended Euclid on polynomials ★★

★ **In the papers:** 2024–25 Q4 (4 marks): inverse of x^5; 2025–26 Q2 (4 marks): inverse of x^7, both mod $x^8 + x^4 + x^3 + x + 1$.

Division in GF(2^n) needs the inverse a^{-1} with $a \otimes a^{-1} \equiv 1 \pmod m$. The idea is the same as for integers. The extended Euclidean algorithm on $m(x)$ and $a(x)$ produces polynomials s, t with $s \cdot m + t \cdot a = \gcd(m, a)$. Since m is irreducible and a is a non-zero polynomial of smaller degree, the gcd is 1, so $t \cdot a = 1 + s \cdot m \equiv 1 \pmod m$, and t is the inverse. As with integer inverses, only the t column is needed.

The table is Forouzan's integer table with two changes: the quotient q comes from a polynomial long division, and every subtraction is an XOR. Start with $r_1 = m$, $r_2 = a$, $t_1 = 0$, $t_2 = 1$, and in each row compute

$$q, r = \text{quotient and remainder of } r_1 \div r_2, \qquad t = t_1 - q \cdot t_2 = t_1 \oplus (q \cdot t_2),$$

then shift $(r_1, r_2) \leftarrow (r_2, r)$ and $(t_1, t_2) \leftarrow (t_2, t)$. Stop when $r_2 = 0$. The product $q \cdot t_2$ is an ordinary polynomial product (expand, XOR equal powers); it needs no reduction, because the t values never reach degree n until the very last row.

★ MUST KNOW : GF(2^n) inverse recipe and its three checks

$r_1 = m$, $r_2 = a$, $t_1 = 0$, $t_2 = 1$; each row: q, r from $r_1 \div r_2$; $t = t_1 \oplus q \cdot t_2$; shift; stop at $r_2 = 0$. **Answer = t_1 of the final row.**
Checks: (1) the final r_1 must be 1; (2) the last t computed always equals m itself (the last row says $t \cdot a$ is a multiple of m, and a shares no factor with m); (3) multiply $a \otimes t_1$ and reduce by m: the remainder must be 1.

Small example (slide): $(x^2 + 1)^{-1}$ in GF(2^4) modulo $x^4 + x + 1$. The divisions are $(x^4 + x + 1) \div (x^2 + 1)$ and $(x^2 + 1) \div x$:

$x^4 + x + 1$	dividend	
$x^4 + x^2$	$\oplus\, x^2 \cdot r_2$	
$x^2 + x + 1$		
$x^2 + 1$	$\oplus\, 1 \cdot r_2$	
x	remainder	

$x^2 + 1$	dividend
x^2	$\oplus\, x \cdot r_2$
1	remainder

so the quotients are $x^2 + 1$, x, x (the last row is $x \div 1$). The t values are $0 \oplus (x^2 + 1) \cdot 1 = x^2 + 1$, then $1 \oplus x(x^2 + 1) = x^3 + x + 1$, then $(x^2 + 1) \oplus x(x^3 + x + 1) = x^4 + x + 1 = m$ ✓.

q	r_1	r_2	r	t_1	t_2	t
x^2+1	x^4+x+1	x^2+1	x	0	1	x^2+1
x	x^2+1	x	1	1	x^2+1	x^3+x+1
x	x	1	0	x^2+1	x^3+x+1	x^4+x+1
	1	0		$\mathbf{x^3+x+1}$	x^4+x+1	

$(x^2+1)^{-1} = \mathbf{x^3+x+1}$. Check: $(x^2+1)(x^3+x+1) = x^5+x^2+x+1$, and $x^5+x^2+x+1 \oplus x\cdot m = 1$ ✓.

WORKED EXAMPLE : 2024–25 Q4, "In GF(2^8), find the inverse of (x^5) modulo ($x^8+x^4+x^3+x+1$)." (4 marks)

Setup. $r_1 = m = x^8+x^4+x^3+x+1$, $r_2 = x^5$, $t_1 = 0$, $t_2 = 1$.

Divisions (one per row of the table):

Row 1: $(x^8+x^4+x^3+x+1) \div x^5$

$x^8 + x^4 + x^3 + x + 1$	dividend
x^8	$\oplus\ x^3\cdot r_2$
$x^4 + x^3 + x + 1$	remainder

$q = x^3$, $r = x^4+x^3+x+1$.

Row 2: $x^5 \div (x^4+x^3+x+1)$

x^5	dividend
$x^5 + x^4 + x^2 + x$	$\oplus\ x\cdot r_2$
$x^4 + x^2 + x$	
$x^4 + x^3 + x + 1$	$\oplus\ 1\cdot r_2$
$x^3 + x^2 + 1$	remainder

$q = x+1$, $r = x^3+x^2+1$.

Row 3: $(x^4+x^3+x+1) \div (x^3+x^2+1)$

$x^4 + x^3 + x + 1$	dividend
$x^4 + x^3 + x$	$\oplus\ x\cdot r_2$
1	remainder

$q = x$, $r = 1$.

Row 4: $(x^3+x^2+1) \div 1$

Dividing by 1 gives the dividend back: $q = x^3+x^2+1$, $r = 0$.

t **values** ($t = t_1 \oplus q\cdot t_2$):

- Row 1: $0 \oplus x^3\cdot 1 = x^3$.
- Row 2: $1 \oplus (x+1)\cdot x^3 = 1 \oplus (x^4+x^3) = x^4+x^3+1$.
- Row 3: $x^3 \oplus x\cdot(x^4+x^3+1) = x^3 \oplus (x^5+x^4+x) = x^5+x^4+x^3+x$.
- Row 4: $(x^3+x^2+1)(x^5+x^4+x^3+x) = (x^8+x^7+x^6+x^4) \oplus (x^7+x^6+x^5+x^3) \oplus (x^5+x^4+x^3+x) = x^8+x$, so $t = (x^4+x^3+1) \oplus (x^8+x) = x^8+x^4+x^3+x+1 = m$ ✓.

Table.

q	r_1	r_2	r	t_1	t_2	t
x^3	$x^8+x^4+x^3+x+1$	x^5	x^4+x^3+x+1	0	1	x^3
$x+1$	x^5	x^4+x^3+x+1	x^3+x^2+1	1	x^3	x^4+x^3+1
x	x^4+x^3+x+1	x^3+x^2+1	1	x^3	x^4+x^3+1	$x^5+x^4+x^3+x$
x^3+x^2+1	x^3+x^2+1	1	0	x^4+x^3+1	$x^5+x^4+x^3+x$	m
	1	0		$\mathbf{x^5+x^4+x^3+x}$	m	

The final $r_1 = 1$, so the inverse exists and equals the final t_1: $\boxed{(x^5)^{-1} = x^5+x^4+x^3+x}$ (byte 3A).

Verification. $x^5 \otimes (x^5+x^4+x^3+x) = x^{10}+x^9+x^8+x^6$, reduced modulo m:

$x^{10} + x^9 + x^8 + x^6$	dividend
$x^{10} + x^6 + x^5 + x^3 + x^2$	$\oplus\ x^2\cdot m$
$x^9 + x^8 + x^5 + x^3 + x^2$	
$x^9 + x^5 + x^4 + x^2 + x$	$\oplus\ x\cdot m$
$x^8 + x^4 + x^3 + x$	
$x^8 + x^4 + x^3 + x + 1$	$\oplus\ 1\cdot m$
1	remainder

The remainder is 1, so the answer is correct ✓.

WORKED EXAMPLE : 2025–26 Q2, "Find multiplicative inverse of x^7 mod $(x^8 + x^4 + x^3 + x + 1)$." (4 marks)

Setup. $r_1 = m = x^8 + x^4 + x^3 + x + 1$, $r_2 = x^7$, $t_1 = 0$, $t_2 = 1$.

Divisions:

Row 1: $(x^8 + x^4 + x^3 + x + 1) \div x^7$

$x^8 + x^4 + x^3 + x + 1$	dividend
$\oplus\ x^8$	$\oplus\ x \cdot r_2$
$x^4 + x^3 + x + 1$	remainder

$q = x$, $r = x^4 + x^3 + x + 1$.

Row 2: $x^7 \div (x^4 + x^3 + x + 1)$

x^7	dividend
$\oplus\ x^7 + x^6 + x^4 + x^3$	$\oplus\ x^3 \cdot r_2$
$x^6 + x^4 + x^3$	
$\oplus\ x^6 + x^5 + x^3 + x^2$	$\oplus\ x^2 \cdot r_2$
$x^5 + x^4 + x^2$	
$\oplus\ x^5 + x^4 + x^2 + x$	$\oplus\ x \cdot r_2$
x	remainder

$q = x^3 + x^2 + x$, $r = x$.

Row 3: $(x^4 + x^3 + x + 1) \div x$

$x^4 + x^3 + x + 1$	dividend
$\oplus\ x^4$	$\oplus\ x^3 \cdot r_2$
$x^3 + x + 1$	
$\oplus\ x^3$	$\oplus\ x^2 \cdot r_2$
$x + 1$	
$\oplus\ x$	$\oplus\ 1 \cdot r_2$
1	remainder

$q = x^3 + x^2 + 1$, $r = 1$.

Row 4: $x \div 1$

$q = x$, $r = 0$.

t values ($t = t_1 \oplus q \cdot t_2$):

- Row 1: $0 \oplus x \cdot 1 = x$.
- Row 2: $1 \oplus (x^3 + x^2 + x) \cdot x = 1 \oplus (x^4 + x^3 + x^2) = x^4 + x^3 + x^2 + 1$.
- Row 3: $(x^3 + x^2 + 1)(x^4 + x^3 + x^2 + 1) = (x^7 + x^6 + x^5 + x^3) \oplus (x^6 + x^5 + x^4 + x^2) \oplus (x^4 + x^3 + x^2 + 1) = x^7 + 1$, so $t = x \oplus (x^7 + 1) = x^7 + x + 1$.
- Row 4: $(x^4 + x^3 + x^2 + 1) \oplus x(x^7 + x + 1) = (x^4 + x^3 + x^2 + 1) \oplus (x^8 + x^2 + x) = x^8 + x^4 + x^3 + x + 1 = m$ ✓.

Table.

q	r_1	r_2	r	t_1	t_2	t
x	$x^8+x^4+x^3+x+1$	x^7	x^4+x^3+x+1	0	1	x
x^3+x^2+x	x^7	x^4+x^3+x+1	x	1	x	$x^4+x^3+x^2+1$
x^3+x^2+1	x^4+x^3+x+1	x	1	x	$x^4+x^3+x^2+1$	x^7+x+1
x	x	1	0	$x^4+x^3+x^2+1$	x^7+x+1	m
	1	0		$\mathbf{x^7+x+1}$	m	

The final $r_1 = 1$, so $\boxed{(x^7)^{-1} = x^7 + x + 1}$ (byte 83, i.e. $\{80\}^{-1} = \{83\}$).

Verification. $x^7 \otimes (x^7 + x + 1) = x^{14} + x^8 + x^7$, reduced modulo m:

$x^{14} + x^8 + x^7$	dividend
$\oplus\ x^{14} + x^{10} + x^9 + x^7 + x^6$	$\oplus\ x^6 \cdot m$
$x^{10} + x^9 + x^8 + x^6$	
$\oplus\ x^{10} + x^6 + x^5 + x^3 + x^2$	$\oplus\ x^2 \cdot m$
$x^9 + x^8 + x^5 + x^3 + x^2$	
$\oplus\ x^9 + x^5 + x^4 + x^2 + x$	$\oplus\ x \cdot m$
$x^8 + x^4 + x^3 + x$	
$\oplus\ x^8 + x^4 + x^3 + x + 1$	$\oplus\ 1 \cdot m$
1	remainder

The remainder is 1 ✓. (A quicker check uses the reductions $x^8 \equiv x^4 + x^3 + x + 1$, $x^{14} \equiv x^7 + x^4 + x^3 + x$, from repeated xtime: $x^{14} + x^8 + x^7 \equiv (x^7 + x^4 + x^3 + x) + (x^4 + x^3 + x + 1) + x^7 = 1$.)

Trap: where the GF(2^8) inverse goes wrong

In row 1, $m \div x^5$ gives remainder $x^4 + x^3 + x + 1$ (only x^8 cancels), not 0 and not x^4. In the t column, $q \cdot t_2$ is a polynomial *product* that must be expanded before the XOR with t_1. The answer is t_1 in the last line, not the last t (which is m). And if a remainder ever has degree ≥ the divisor's, the division was stopped too early.

29.12 Small fields: GF(2^2) and GF(2^3)

GF(2^2) has the four 2-bit words `00`, `01`, `10`, `11` = $0, 1, x, x+1$. The only irreducible polynomial of degree 2 is $x^2 + x + 1$, so $x^2 \equiv x + 1$. Addition is XOR with identity `00`; multiplication has identity `01`:

⊕	00	01	10	11
00	00	01	10	11
01	01	00	11	10
10	10	11	00	01
11	11	10	01	00

⊗	00	01	10	11
00	00	00	00	00
01	00	01	10	11
10	00	10	11	01
11	00	11	01	10

For example $x \otimes x = x^2 \equiv x + 1$ (`11`), $x \otimes (x+1) = x^2 + x \equiv 1$, so x and $x+1$ are inverses, and $(x+1)^2 = x^2 + 1 \equiv x$.

GF(2^3) with the slide's modulus $x^3 + x^2 + 1$ (so $x^3 \equiv x^2 + 1$). Addition is again XOR (e.g. `011` ⊕ `101` = `110`). Multiplication, with the inverse pairs shaded:

⊗	000	001	010	011	100	101	110	111
000	000	000	000	000	000	000	000	000
001	000	001	010	011	100	101	110	111
010	000	010	100	110	101	111	001	011
011	000	011	110	101	001	010	111	100
100	000	100	101	001	111	011	010	110
101	000	101	111	010	011	110	100	001
110	000	110	001	111	010	100	011	101
111	000	111	011	100	110	001	101	010

Inverse pairs: $1 \leftrightarrow 1$, $x \leftrightarrow x^2 + x$, $x + 1 \leftrightarrow x^2$, $x^2 + 1 \leftrightarrow x^2 + x + 1$. Each non-zero row contains `001` exactly once, which is the field property "every non-zero element has exactly one inverse" made visible. One entry worked out: $x \otimes x^2 = x^3 \equiv x^2 + 1$, so `010` ⊗ `100` = `101`.

Corrections to the notes / slides

- Slide definition of a field: the second operation "satisfies all five properties … except that the identity of the first operation has no inverse". Read this as: every element except 0 has a multiplicative inverse.
- Irreducible-polynomial table (slide 35): degree 5 lists only five polynomials and misses $x^5 + x^3 + 1$; there are six (all listed above, checked by script).
- GF(2^3) multiplication table (slide 47): the binary entries are right, but two polynomial labels are wrong. $x \otimes x$ = `100` is x^2 (slide prints (x)), and $(x+1) \otimes (x^2 + x + 1)$ = `100` is x^2 (slide prints (x)).

Check your understanding

1. **Is $\langle \mathbb{Z}_n^*, \times \rangle$ a group? Is $\langle \mathbb{Z}_n, \times \rangle$?** $\mathbb{Z}_n^*$ is an abelian group: it is closed (a product of numbers coprime to n is coprime to n), associative, has identity 1, and every element has an inverse by extended Euclid because $\gcd(a, n) = 1$. $\mathbb{Z}_n$ under $\times$ is not a group, because 0 (and every a with $\gcd(a, n) > 1$) has no inverse.
2. **Is $\mathbb{Z}$ a field?** No. $\langle \mathbb{Z}, +, \times \rangle$ is a commutative ring: addition satisfies all five properties, but multiplication has no inverses except for ±1, so division leaves the set ($12 \div 5 = 2.4 \notin \mathbb{Z}$).
3. **Why is $\mathbb{Z}_{256}$ not a field, and what does AES use instead?** 256 is not prime, so there are zero divisors: $2 \times 128 \equiv 0$, which means 2 (and every even byte) has no inverse. AES uses GF(2^8), where bytes are polynomials, addition is XOR and multiplication is reduced modulo the irreducible $x^8 + x^4 + x^3 + x + 1$, so all 256 bytes are field elements.
4. **Differentiate a group, a ring and a field.** A group has one operation with closure, associativity, identity and inverses, and so supports one pair such as $(+, -)$; a ring adds a second operation that only needs closure and associativity and distributes over the first, so it supports $(+, -, \times)$; a field also requires inverses for the second operation (except for 0), so it supports $+, -, \times, \div$. Examples: $\langle \mathbb{Z}_n, + \rangle$, $\langle \mathbb{Z}, +, \times \rangle$, $\langle \mathbb{Z}_p, +, \times \rangle$.
5. **Why does applying two transposition ciphers one after another not make the cipher stronger?** Permutations of the same block length form a group under composition, so by closure the two permutations equal one permutation $\pi_2 \circ \pi_1$, and the attacker only has to find that one key from the same key space of $k!$.
6. **Is $\langle \mathbb{Z}_{10}, + \rangle$ a subgroup of $\langle \mathbb{Z}_{12}, + \rangle$?** No. Although $\mathbb{Z}_{10} \subset \mathbb{Z}_{12}$ as sets, a subgroup must use the same operation, and addition mod 10 is not addition mod 12.
7. **Can $\mathbb{Z}_{11}^*$ contain an element of order 4?** No. $|\mathbb{Z}_{11}^*| = 10$, and by Lagrange the order of an element divides the order of the group, so only the orders 1, 2, 5 and 10 are possible.

8. **Why must the modulus of GF(2^n) be irreducible? Can $x^4 + x^2 + 1$ be used for GF(2^4)?** If $m = f \cdot g$, then $f \otimes g \equiv 0$ with $f, g \neq 0$, so f has no inverse and the structure is not a field. $x^4 + x^2 + 1 = (x^2 + x + 1)^2$ is reducible even though it has no root, so it cannot be used.
9. **Why are addition and subtraction the same in GF(2^n)?** The coefficients are in GF(2), where $-1 \equiv 1$, so $-B = B$ and $A - B = A + B$; bit by bit, both are XOR.
10. **Find** $\{02\} \bullet \{C3\}$ **and** x^{-1} **in GF(2^8).** C3 = 11000011 has its top bit set, so shift to 10000110 and XOR 1B: **9D**. For x^{-1}: $x^8 \equiv x^4 + x^3 + x + 1$ means $x \cdot (x^7 + x^3 + x^2 + 1) \equiv 1$, so $x^{-1} = x^7 + x^3 + x^2 + 1$ (8D).
11. **In GF(11), find** $3 \div 7$**.** $7^{-1} = 8$ since $56 = 5 \cdot 11 + 1$, so $3 \div 7 = 3 \times 8 = 24 \equiv$ **2**; check $2 \times 7 = 14 \equiv 3$.

30 Modern Symmetric-Key Ciphers

The classical ciphers of the earlier sections work on letters of a 26-symbol alphabet. A computer, however, has to protect text, numbers, images, audio and video alike, and all of these are stored as bits. Modern ciphers are therefore **bit-oriented**: the plaintext, the ciphertext and the key are bit strings, and the operations are bit operations (permutation of bit positions, table lookup on small groups of bits, XOR). They come in two families, exactly as before: **block ciphers**, which encrypt a fixed-size block of n bits at a time, and **stream ciphers**, which encrypt a stream r bits at a time.

30.1 Modern block ciphers and padding

A **modern block cipher** encrypts an n-bit plaintext block with a k-bit key into an n-bit ciphertext block, and decryption must be the exact inverse, so the encryption mapping has to be a one-to-one function on the 2^n possible blocks. Since the cipher only accepts whole blocks, a message whose length is not a multiple of n is first **padded**. If $|M|$ is the message length and $|Pad|$ the padding length, we need $|M| + |Pad| \equiv 0 \pmod{n}$, which gives directly

$$|Pad| = (-|M|) \bmod n.$$

Only the last block carries padding; every other block is a full block of message bits.

WORKED EXAMPLE : padding (slide example)

Q. How many padding bits must be added to a message of 100 characters if 8-bit ASCII is used and the block cipher accepts 64-bit blocks?
A. The message is $100 \times 8 = 800$ bits. Then $|Pad| = (-800) \bmod 64$. Since $800 = 12 \times 64 + 32$, we get $-800 \equiv -32 \equiv 32 \pmod{64}$, so **32 bits** of padding (e.g. 0s) are added. The padded plaintext is 832 bits = **13** blocks of 64 bits, and the cipher is applied 13 times; only the 13th block contains padding.
Check: $832/64 = 13$ exactly.
Q (variant). 75 characters of 8-bit ASCII, 128-bit blocks (AES)? **A.** $|M| = 600$, $|Pad| = (-600) \bmod 128 = 640 - 600 =$ **40** bits, giving 640 bits = **5 blocks**.

Substitution or transposition at bit level. A block cipher can in principle be designed either as a transposition (rearrange the n bits) or as a substitution (replace the whole n-bit block by another). A transposition only moves bits, so it *preserves the number of 1s*: if the ciphertext has ten 1s, so does the plaintext. Eve then only has to try the $\binom{64}{10} = 151\,473\,214\,816 \approx 2^{37}$ blocks with ten 1s instead of all 2^{64} blocks. A substitution has no such invariant, which is why a modern block cipher is designed as a substitution cipher; transposition units appear only as components inside it.

Full-size key ciphers and why they are impractical. A **full-size key** cipher allows every possible mapping as a key. A full-size key transposition cipher on n bits has $n!$ keys, and a full-size key substitution cipher has $(2^n)!$ keys (every permutation of the 2^n blocks). For $n = 64$ the key would need $\lceil \log_2 64! \rceil = 296$ bits for transposition and $\log_2 (2^{64})! \approx 2^{70}$ bits for substitution, i.e. a key larger than any storage. Real ciphers are therefore **partial-size key** ciphers: DES uses a 56-bit key, so it realises only 2^{56} of the $(2^{64})!$ possible substitutions. The design task is to make those 2^{56} mappings look like randomly chosen ones, which is done by combining small components.

30.2 Components of a modern block cipher

A modern block cipher is not built as one huge substitution table. It is a combination of **transposition units (P-boxes)**, **substitution units (S-boxes)** and a few simple operations (XOR, circular shift, swap, split and combine), each small enough to be implemented and analysed.

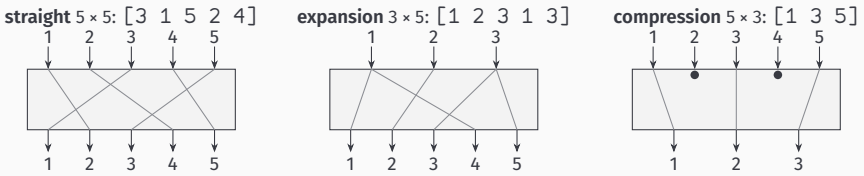
P-boxes ★

▸ **Likely this year:** "Differentiate straight, expansion and compression P-boxes; which are invertible and why?" or "Design an 8×8 permutation table that …".

A **P-box** (permutation box) is the bit-level analogue of a transposition cipher: it moves bits to new positions. It

is described by a **permutation table** with one entry per *output* position; entry j says which input bit is copied to output j. Three types occur.

- A **straight P-box** has n inputs and n outputs, and its table is a permutation of $1..n$. There are $n!$ straight P-boxes (all 6 for $n = 3$). The DES initial permutation is a 64×64 straight P-box (table starting `58 50 42 34 ...`).
- An **expansion P-box** has n inputs and $m > n$ outputs. Its table has m entries, of which $m-n$ are repeats, because some inputs are wired to two outputs. The slides' 12×16 example `01 09 10 11 12 01 02 03 03 04 05 06 07 08 09 12` repeats inputs 1, 3, 9 and 12.
- A **compression P-box** has n inputs and $m < n$ outputs. Its table has m entries drawn from $1..n$ with $n - m$ values missing: those inputs are **blocked** and never reach the output. The slides' 32×24 example blocks inputs 7, 8, 9, 15, 16, 23, 24 and 25.



Invertibility. A straight P-box is invertible: since every input appears exactly once in the table, each output position can be sent back to where it came from. The inverse table is found by writing, for each input position i, the output position that holds it. A compression P-box is *not* invertible, because a blocked input bit is simply lost and the inverse would have to invent its value. An expansion P-box is *not* invertible either: an input copied to two outputs would have two candidate sources in the reverse direction, and if the received bits disagree there is no definite answer (more precisely, its reverse is a compression, not an inverse). Only straight P-boxes are invertible. This is exactly why AES (all components invertible) uses only straight permutations, while DES (Feistel) can afford an expansion P-box inside its round function.

WORKED EXAMPLE : designing and inverting a straight P-box (slide example)

Q. Design an 8×8 permutation table for a straight P-box that moves the two middle bits (bits 4 and 5) of the input to the two ends (bits 1 and 8) of the output, without changing the relative positions of the other bits. Give the inverse table and apply the P-box to `10011100`.
A. Output bit 1 must take input bit 4 and output bit 8 must take input bit 5. The remaining outputs 2–7 take inputs 1, 2, 3, 6, 7, 8 in their original order. Hence the table is

`[4 1 2 3 6 7 8 5]`.

Inverse: input 1 went to output 2, input 2 to 3, input 3 to 4, input 4 to 1, input 5 to 8, input 6 to 5, input 7 to 6, input 8 to 7, so the inverse table is `[2 3 4 1 8 5 6 7]`.
Apply: input `10011100` has bits $b_1..b_8 = 1,0,0,1,1,1,0,0$. Output = $b_4 b_1 b_2 b_3 b_6 b_7 b_8 b_5$ = **11001001**.
Check: applying the inverse table to `11001001` returns `10011100`; the number of 1s (four) is unchanged, as it must be for any P-box.

S-boxes

An **S-box** (substitution box) is a miniature substitution cipher on a few bits: it takes an n-bit input and produces an m-bit output, where m and n need not be equal. It is normally given as a **lookup table**: some input bits select the row, the rest select the column, and the cell holds the output. In DES, for instance, a 6-bit input chooses one of 4 rows by its two outer bits and one of 16 columns by its four middle bits, and the cell is a 4-bit value.

Linear versus nonlinear. Write the inputs as $x_1..x_n$ and the outputs as $y_1..y_m$. The S-box is **linear** if every output is an XOR of inputs, $y_i = a_{i1}x_1 \oplus a_{i2}x_2 \oplus \cdots \oplus a_{in}x_n$ with constants $a_{ij} \in \{0,1\}$; then the whole box is a matrix product over GF(2). For example, $y_1 = x_1 \oplus x_2 \oplus x_3$, $y_2 = x_1$ is linear, with matrix $\left(\begin{smallmatrix}1&1&1\\1&0&0\end{smallmatrix}\right)$. If any output contains a product of inputs, such as $y_1 = x_1x_3 \oplus x_2$, the box is **nonlinear**. A cipher built only from linear boxes, XORs and P-boxes would be linear as a whole, so every ciphertext bit would be a fixed XOR of plaintext and key bits, and a handful of known plaintext–ciphertext pairs would give a solvable linear system for the key. The S-boxes are the only nonlinear part of a block cipher, and they must be nonlinear.

Invertibility. An S-box can be inverted only if it is a one-to-one map, which requires $m = n$ (a $6 \to 4$ box maps 64 inputs onto 16 outputs, so each output has 4 preimages). The slides' 3×3 example uses the leftmost bit as the row and the two rightmost bits as the column:

enc	00	01	10	11
0	011	101	111	100
1	000	010	001	110

dec	00	01	10	11
0	100	110	101	000
1	011	001	111	010

Input `001` (row 0, column 01) encrypts to `101`, and `101` (row 1, column 01) in the decryption table gives back `001`; a script confirms that the two tables undo each other for all 8 inputs.

XOR, circular shift, swap, split and combine

Exclusive-or is addition in GF(2^n), and five properties make it the standard way to mix in a key: **closure** (the XOR of two n-bit words is an n-bit word), **associativity**, **commutativity**, an **identity** $00\ldots0$, and an **inverse** for every word, namely the word itself, since $x \oplus x = 00\ldots0$. The last property is the important one: whatever is XORed in can be removed by XORing the same value again, $(x \oplus k) \oplus k = x$. A key mixed in this way is called a **whitener**.

A **circular shift** rotates an n-bit word left or right by k positions. It is invertible, since a left shift by k is undone by a right shift by k, and shifting by k is the same as shifting by k mod n. A **swap** is the special circular shift by $k = n/2$, exchanging the left and right halves; doing it twice restores the word, so it is **self-invertible**. A **split** cuts an n-bit word into two $n/2$-bit halves and a **combine** concatenates two halves; each is the inverse of the other.

30.3 Product ciphers: confusion, diffusion and rounds ★

▸ **Likely this year:** One-line theory: “Define confusion and diffusion; which component provides which?” or “What is the avalanche effect?”

A single small component is easy to analyse and therefore easy to break; a full-size substitution table is secure but impossible to store. Shannon’s answer was the **product cipher**: a cipher that alternates substitutions, permutations and key mixing, so that simple and cheap parts combine into a mapping that behaves like a random one. He named the two properties such a cipher must have.

Diffusion hides the relationship between the ciphertext and the plaintext: each ciphertext bit should depend on many or all plaintext bits, so that changing one plaintext bit changes many ciphertext bits. This frustrates an attacker who uses ciphertext statistics (letter or bit frequencies) to guess the plaintext, because any local pattern of the plaintext is spread over the whole block.

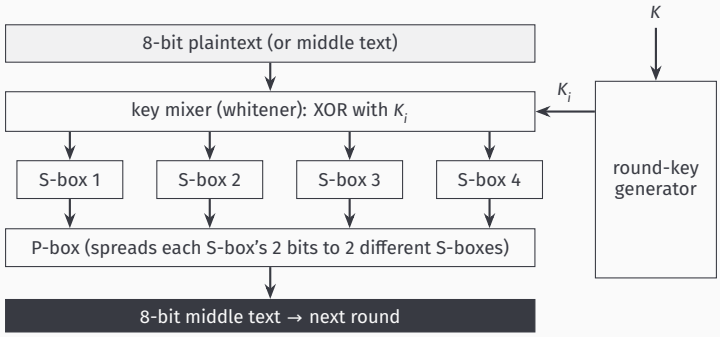
Confusion hides the relationship between the ciphertext and the key: each ciphertext bit should depend on the key in a complicated way, so that changing one key bit changes most ciphertext bits. This frustrates an attacker who uses known ciphertext to solve for the key.

Why S-boxes give confusion. The key enters the cipher only through XOR, and XOR and P-boxes are linear over GF(2). If the cipher had no S-boxes, every ciphertext bit would be a fixed XOR of plaintext bits and key bits, and a few plaintext–ciphertext pairs would give a linear system whose solution is the key. The nonlinear S-box placed after the key mixing breaks this: its output is a nonlinear function of (data $\oplus$ key), so after a few rounds the ciphertext is a high-degree nonlinear function of the key that cannot be solved for. That is precisely confusion.

Why P-boxes give diffusion. Suppose one input bit of a round changes. It enters one S-box, whose output may change in all its bits (say 2 bits for a 2×2 box). The P-box then sends those changed bits to *different* S-boxes of the next round, where each again changes up to 2 bits, so the change can grow from 1 bit to 2, 4, 8, … bits in successive rounds. Without the P-box the changed bits would stay inside the same S-box forever and the change would never spread; the permutation between rounds is what carries it across the block. That is diffusion.

Avalanche. The observable result of good confusion and diffusion is the **avalanche effect**: flipping one bit of the plaintext or of the key flips about half of the ciphertext bits. Section 31 shows DES going from 1 differing bit to 29 differing bits out of 64.

Rounds. One layer of S-boxes and P-boxes spreads a bit change only a little, so the layer is repeated. An **iterated product cipher** applies the same **round** N times; each round consists of a **key mixer** (whitener, XOR with a round key), a layer of S-boxes and a P-box. The text between two rounds is called the **middle text**, and a **key schedule** (round-key generator) derives the round keys $K_1, \ldots, K_N$ from the cipher key, so that each round mixes in a different key. Decryption runs N inverse rounds with the round keys in reverse order.



One round of the slides' 8-bit product cipher (the slide shows two such rounds).

30.4 Feistel ciphers ★

★ **In the papers:** 2024–25 Q6 (5 marks) opens with "what Feistel structure is prevalent in DES": the derivation below is that part of the answer.

Modern block ciphers fall into two classes. **Feistel ciphers** (DES) use invertible, self-invertible *and noninvertible* components; **non-Feistel ciphers** (AES) use only invertible components. The problem the Feistel design solves is this: noninvertible components such as an expansion P-box or a $6 \to 4$ S-box mix bits very effectively, but a cipher must be decryptable, and a noninvertible component cannot be run backwards. Feistel's idea is to collect all the noninvertible parts into one function f and never invert it. Its output is only ever *XORed* onto the data, and since XOR is self-inverse, the receiver can cancel it by computing the same f value again and XORing once more.

Step 1: the idea in its simplest form. Let f depend only on the key. Then

$$\text{encryption: } C = P \oplus f(K), \qquad \text{decryption: } C \oplus f(K) = P \oplus f(K) \oplus f(K) = P \oplus 00\ldots0 = P.$$

Nothing here requires f^{-1}; decryption simply computes $f(K)$ forwards.

WORKED EXAMPLE : the slides' first Feistel example

Q. Plaintext and ciphertext are 4 bits, the key is 3 bits. f takes the first and third bits of the key, reads them as a decimal number, squares it, and writes the result as 4 bits. Encrypt and decrypt $P = 0111$ with $K = 101$.
A. First and third key bits: $1, 1 \to (11)_2 = 3$; $3^2 = 9 = 1001$, so $f(101) = 1001$.
Encryption: $C = P \oplus f(K) = 0111 \oplus 1001 =$ **1110**.
Decryption: $C \oplus f(K) = 1110 \oplus 1001 = 0111 = P$ ✓.
f is noninvertible (e.g. keys 101 and 111 both give 1001, since bit 2 is ignored), yet decryption works because only f, never f^{-1}, is needed.

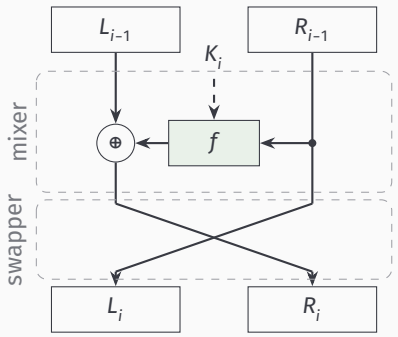
Step 2: let the data enter f. A function of the key alone adds nothing per block (it is a fixed XOR, i.e. a one-key Vernam). To make f depend on the data while keeping decryption possible, the block is **split** into a left half L and a right half R; R feeds f and L is XORed with the output. This unit is the **mixer**:

$$L_2 = L_1 \oplus f(R_1, K), \qquad R_2 = R_1.$$

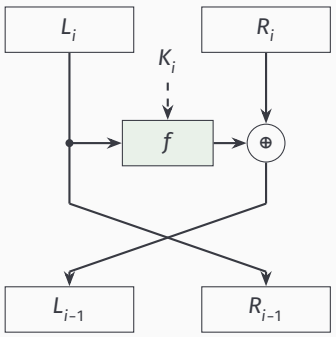
The receiver gets $R_2 = R_1$ unchanged, so it can recompute exactly the same $f(R_1, K)$ and cancel it: $L_2 \oplus f(R_2, K) = L_1 \oplus f(R_1, K) \oplus f(R_1, K) = L_1$. The mixer is therefore **self-invertible**: applying it twice with the same key is the identity.

Step 3: the swapper and more rounds. The mixer has one flaw: R is never changed, so half of the plaintext appears in clear in the ciphertext. The fix is to add a **swapper** after the mixer, so that the next round modifies the other half, and to repeat for several rounds. One Feistel round i is then

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i).$$



encryption round:
$L_i = R_{i-1}$, $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$



inverse round (same K_i), f only forwards:
$R_{i-1} = L_i$, $L_{i-1} = R_i \oplus f(L_i, K_i)$

★ MUST KNOW : why a Feistel round is invertible even if f is not

Given the round output (L_i, R_i), the first equation hands back $R_{i-1} = L_i$ directly. Knowing R_{i-1}, the receiver can *recompute* $f(R_{i-1}, K_i)$ forwards and XOR it away:

$$R_i \oplus f(L_i, K_i) = L_{i-1} \oplus f(R_{i-1}, K_i) \oplus f(R_{i-1}, K_i) = L_{i-1}.$$

f is only ever evaluated, never inverted, so it may be many-to-one (expansion P-box, $6 \to 4$ S-boxes). Since the mixer and the swapper are each self-inverse, decryption is the same structure run with the round keys in reverse order $(K_N, \ldots, K_1)$.

In the slides' two-round final design (slide L7-29), both rounds end with a swapper, so the decryption circuit applies the swapper first and then the mixer, round keys K_2 then K_1. DES removes the swapper from the *last* round; then encryption and decryption are literally the same algorithm and only the key order changes. The toy below uses that DES convention.

WORKED EXAMPLE : two-round Feistel cipher with a noninvertible f

Q. An 8-bit Feistel cipher has 4-bit halves and two rounds; round 1 has a swapper, round 2 does not (DES convention). The round function is $f(R, K) = (R \oplus K)^2 \bmod 16$. Encrypt $L_0R_0 = 1010\ 0111$ with $K_1 = 0100$, $K_2 = 1001$, then decrypt with the same algorithm and the keys reversed. Show that f is not invertible.
A. f is not invertible: $0011^2 = 9$ and $0101^2 = 25 \equiv 9 \pmod{16}$, so two inputs give 1001; in fact $x^2 \bmod 16$ only takes the values 0, 1, 4, 9.

Round	input L, R	$R \oplus K$	f	$L \oplus f$	output (after swap?)
Enc 1 ($K_1 = 0100$)	1010, 0111	0011	1001	0011	swap → 0111, 0011
Enc 2 ($K_2 = 1001$)	0111, 0011	1010	0100	0011	no swap → **0011, 0011**
Dec 1 ($K_2 = 1001$)	0011, 0011	1010	0100	0111	swap → 0011, 0111
Dec 2 ($K_1 = 0100$)	0011, 0111	0011	1001	1010	no swap → **1010, 0111**

Ciphertext = **0011 0011**; decryption returns 1010 0111 = P ✓. Each decryption round recomputes exactly the f value of the matching encryption round (1001 and 0100), because the half fed to f is passed through unchanged.

30.5 Non-Feistel ciphers

A **non-Feistel cipher** uses only invertible components, and every component of the encryption has a matching inverse component in the decryption. This constrains the parts: S-boxes must have as many output bits as input bits, and expansion and compression P-boxes are not allowed, because they cannot be inverted. Since there is no f whose output is XORed in, there is also no need to split the block into halves; each round transforms the whole block. The slides' 8-bit product cipher above is a non-Feistel cipher: XOR (self-invertible), 2×2 invertible S-boxes and a straight P-box, so each round is invertible and decryption runs the inverse rounds with keys K_2, K_1. AES is the real example.

	Feistel (DES)	**Non-Feistel** (AES)
Components	invertible, self-invertible and noninvertible	only invertible ($n \times n$ S-boxes, straight P-boxes)
Block handling	split into L, R; mixer $L \oplus f(R, K)$, then swapper	whole block in every round
Decryption	same structure, round keys reversed; f never inverted	inverse components in reverse order, round keys reversed

30.6 Attacks on modern block ciphers

Brute force on the key is normally infeasible because keys are long (but 2^{56} for DES is not). The serious attacks exploit the structure of the rounds.

- **Differential cryptanalysis** (Biham and Shamir) is a **chosen-plaintext** attack. Eve submits pairs of plaintexts with a chosen difference $P_1 \oplus P_2$ and studies the output difference $C_1 \oplus C_2$; a weak cipher propagates certain differences with high probability, which reveals key bits. The slides' extreme case is a cipher that is only an XOR with the key: $C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = P_1 \oplus P_2$, so the difference passes through untouched and is independent of the key. Nonlinear S-boxes are what destroy this predictability.
- **Linear cryptanalysis** (Matsui) is a **known-plaintext** attack. If an S-box is not fully nonlinear, some XOR of input and output bits equals an XOR of key bits with probability noticeably different from $1/2$; collecting many plaintext–ciphertext pairs and counting how often such an equation holds reveals those key bits.

30.7 Modern stream ciphers

A **modern stream cipher** encrypts and decrypts r bits at a time: the plaintext $P = p_n \dots p_2 p_1$, ciphertext $C = c_n \dots c_2 c_1$ and key stream $K = k_n \dots k_2 k_1$ consist of r-bit words, and $c_i = p_i \oplus k_i$, $p_i = c_i \oplus k_i$. Encryption and decryption are a single XOR, so the cipher is fast and needs no padding; all of its security lies in the **key stream**, which must look random and must never be reused. Stream ciphers are classified by how the key stream is produced.

Synchronous stream ciphers. In a **synchronous** stream cipher the key stream is *independent* of the plaintext and the ciphertext; it is generated from the key alone, and sender and receiver must stay in step (synchronised) with each other. The ideal case is the **one-time pad**: a truly random key stream as long as the message, used once, giving perfect secrecy, at the price that a key as long as all the traffic must be shared in advance. Practical synchronous ciphers replace the random pad by a pseudorandom generator such as a **feedback shift register** (FSR). An FSR has m one-bit cells $b_{m-1} \dots b_1 b_0$; at each step b_0 is output as a key bit, all cells shift right, and the new leftmost bit is computed by a **feedback function** of the old cells. In a **linear** FSR (LFSR) the feedback is an XOR of selected cells, so it costs one gate. With a well-chosen feedback an m-cell LFSR runs through all $2^m - 1$ nonzero states before repeating, the maximum period. Because it is linear, however, about $2m$ consecutive key-stream bits are enough to solve for the feedback taps and predict the rest, so real designs use a **nonlinear** FSR or combine several LFSRs nonlinearly.

WORKED EXAMPLE : LFSR key stream (Forouzan style)

Q. Create a 4-cell LFSR with feedback $b_4 = b_1 \oplus b_0$ (the new b_3). Show the output for 20 shifts if the seed $b_3 b_2 b_1 b_0$ is 0001, and find the period.
A. Each step outputs b_0, then the state becomes $(b_1 \oplus b_0)\, b_3\, b_2\, b_1$.

step	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
state	0001	1000	0100	0010	1001	1100	0110	1011	0101	1010	1101	1110	1111	0111	0011	**0001**
out b_0	1	0	0	0	1	0	0	1	1	0	1	0	1	1	1	1

At step 15 the state is the seed again, so the key stream repeats with period **15** = $2^4 - 1$ (maximum: every nonzero state appears once). The 20 output bits are **1000 1001 1010 1111 0001**.
Check: the bits 16–20 (1 0001) repeat bits 1–5 (1 0001), as the period predicts.

Nonsynchronous (self-synchronising) stream ciphers. In a **nonsynchronous** stream cipher each key-stream element depends on previous plaintext or ciphertext elements. The usual form is **self-synchronising**: the key stream is computed from the key and the last few ciphertext bits (as in CFB mode). If a ciphertext bit is corrupted or lost, the receiver's key stream is wrong only while the bad bit is inside its window, and decryption recovers by itself once the window has passed.

Error behaviour. In a synchronous cipher a flipped ciphertext bit spoils only the matching plaintext bit, but a lost or inserted bit shifts the two key streams against each other and garbles everything after it until the parties resynchronise. In a self-synchronising cipher a single error spoils a few following bits as well, but a lost bit is recovered from automatically.

Corrections to the notes / slides

- Feistel example (L7 slide 25): the problem says f uses the *first and third* key bits, but the solution "extracts the first and second bits to get 11". For $K = 101$ the first and second bits are $10 = 2$, giving $f = 0100$; only the first and third bits give $11 = 3$ and the stated $f = 1001$, $C = 1110$. Use first and third.
- Stream ciphers (L7 slide 39): the line reads "In a synchronous stream cipher the key is independent of the plaintext or ciphertext"; it is easily read as "asynchronous" (the old version of these notes did so). The key-independent class is **synchronous**. Some books (Stallings) call the *other* class, the self-synchronising ciphers, "asynchronous", so do not mix the words up.
- Nonlinear S-box (L7 slide 18): over GF(2), $x^3 = x$ and $x^2 = x$, so the slide's $y_1 = x_1^3 + x_2 = x_1 + x_2$ is actually linear; the box is nonlinear because of the product term $x_1 x_2$ in $y_2 = x_1^2 + x_1 x_2 + x_3$.

Check your understanding

1. **Why must a modern block cipher be a substitution and not a transposition cipher?** A transposition preserves the number of 1s in the block, so from a ciphertext with w ones Eve only has to try the $\binom{n}{w}$ blocks of weight w; a substitution has no such invariant and leaves all 2^n candidates.
2. **How many padding bits for 1000 bits of message with 64-bit blocks?** $(-1000) \bmod 64 = 1024 - 1000 = 24$ bits, giving 16 blocks.
3. **Why is a full-size key substitution cipher impractical for $n = 64$?** It needs $(2^{64})!$ keys, i.e. a key of about 2^{70} bits; real ciphers use partial-size keys (DES: 56 bits) and build a random-looking mapping from small components.
4. **Differentiate the three P-boxes and say which are invertible.** Straight $n \to n$ is a permutation and is invertible;

expansion $n \to m$ ($m > n$) repeats inputs and compression $n \to m$ ($m < n$) blocks inputs, and neither is invertible, because a blocked bit is lost and a repeated bit has no unique source.

5. **Which component gives confusion and which gives diffusion, and why?** S-boxes give confusion, because they are the only nonlinear part and make the ciphertext a nonlinear function of the key; P-boxes give diffusion, because they carry the changed output bits of one S-box to several S-boxes of the next round so that a change spreads over the block.
6. **Why can a Feistel cipher use a noninvertible round function?** Because f's output is only XORed onto the other half and f's input R_{i-1} is passed on unchanged as L_i; the receiver recomputes $f(L_i, K_i)$ forwards and XORs it away, so f^{-1} is never needed.
7. **What happens if a Feistel cipher has no swapper?** The right half is never modified, so half of the plaintext appears unchanged in the ciphertext.
8. **Can an S-box with 6 inputs and 4 outputs be invertible?** No: 64 inputs map onto 16 outputs, so each output has 4 preimages; only $n \times n$ S-boxes can be invertible.
9. **Differentiate synchronous and nonsynchronous stream ciphers.** In a synchronous cipher the key stream depends only on the key (OTP, LFSR), so an inserted or lost bit desynchronises the receiver; in a nonsynchronous cipher it also depends on earlier plaintext or ciphertext, and the self-synchronising form recovers automatically after an error.
10. **What is the maximum period of a 5-cell LFSR?** $2^5 - 1 = 31$, since the all-zero state is excluded (it would stay zero forever).

31 Data Encryption Standard (DES)

The **Data Encryption Standard** is the symmetric-key block cipher published by NIST (then NBS) in 1977, based on IBM's Lucifer. It encrypts a **64-bit block** with a **56-bit key** through **16 Feistel rounds**. The key is normally supplied as 64 bits, of which 8 are parity bits that are dropped, and a key schedule derives sixteen **48-bit round keys** $K_1, \ldots, K_{16}$ from it. Decryption is done by the **reverse cipher**, which is the same algorithm with the round keys used in the order $K_{16}, \ldots, K_1$. DES has since been replaced by AES because its key is too short, but its structure is the standard example of a Feistel cipher (Section 30).

31.1 DES structure ★

★ **In the papers:** 2024–25 Q6 (5 marks): Feistel structure, block diagrams of the DES algorithm and the round function, pseudocode for the S-box substitution and key generation.

Encryption consists of an **initial permutation** (IP) of the 64-bit plaintext, sixteen rounds, and a **final permutation** (FP). After IP the block is split into a left half L_0 and a right half R_0 of 32 bits each. Each round i takes (L_{i-1}, R_{i-1}) and produces (L_i, R_i) with a Feistel **mixer** followed by a **swapper**:

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i),$$

where f is the DES function described below. Round 16 has no swapper, and after it the halves are combined and passed through FP to give the 64-bit ciphertext. The block diagrams of the whole algorithm, of one round and of the key generator are drawn in the exam answer at the end of the key-generation subsection.

31.2 Initial and final permutations

IP and FP are keyless **straight P-boxes** (64×64) that are inverses of each other: $\text{FP} = \text{IP}^{-1}$. The IP table is read row by row; entry j is the input bit that goes to output position j.

Initial permutation

58	50	42	34	26	18	10	02
60	52	44	36	28	20	12	04
62	54	46	38	30	22	14	06
64	56	48	40	32	24	16	08
57	49	41	33	25	17	09	01
59	51	43	35	27	19	11	03
61	53	45	37	29	21	13	05
63	55	47	39	31	23	15	07

Final permutation

40	08	48	16	56	24	64	32
39	07	47	15	55	23	63	31
38	06	46	14	54	22	62	30
37	05	45	13	53	21	61	29
36	04	44	12	52	20	60	28
35	03	43	11	51	19	59	27
34	02	42	10	50	18	58	26
33	01	41	09	49	17	57	25

So output bit 1 of IP is input bit 58, and input bit 1 lands in output position 40; correspondingly the FP table has 40 in position 1 (FP puts it back).

Why they add no security. Both permutations are fixed, keyless and published. An attacker who knows a plaintext can apply IP to it himself, and can apply $\text{IP} = \text{FP}^{-1}$ to the ciphertext, which strips both permutations off and leaves exactly the 16 rounds to attack. IP and FP have no cryptographic significance; they are usually explained as a convenience for loading 8-bit bytes into 1970s hardware.

WORKED EXAMPLE : initial permutation (Forouzan style)

Q. Find the output of the initial permutation for the input `0x0002 0000 0000 0001` and show that FP restores it.
A. The input has only two 1-bits: hex digit 4 is 2 = 0010, i.e. bit 12 + 3 = **15**, and the last bit, **64**. In the IP table, 15 is entry 63 and 64 is entry 25, so the output has 1s exactly in positions 25 and 63:

$$\text{IP}(0002000000000001) = \texttt{0x0000 0080 0000 0002}.$$

(Bit 25 is the leading bit of hex digit 7, giving 8; bit 63 is the third bit of hex digit 16, giving 2.)
Check: in the FP table, entry 15 is 63 and entry 64 is 25, so FP moves bits 63 and 25 back to positions 15 and 64.

31.3 Rounds: mixer, swapper and the reverse cipher

Each round is a Feistel round (Section 30): the **mixer** replaces L_{i-1} by $L_{i-1} \oplus f(R_{i-1}, K_i)$ and leaves R_{i-1} alone, and the **swapper** exchanges the halves. The mixer is invertible because of the XOR: given the output, the receiver still has R_{i-1}, recomputes $f(R_{i-1}, K_i)$ and XORs it away. The mixer and the swapper are each **self-inverse**: applying either twice (the mixer with the same key) gives back the input.

Why decryption is the same algorithm with keys reversed. Write encryption as the sequence of operations

$$\text{IP},\ M_{K_1},\ S,\ M_{K_2},\ S,\ \ldots,\ M_{K_{15}},\ S,\ M_{K_{16}},\ \text{FP},$$

where M_K is the mixer with key K and S the swapper. The inverse of a sequence applies the inverses of its steps in reverse order, and here $\text{FP}^{-1} = \text{IP}$, $M_K^{-1} = M_K$, $S^{-1} = S$ and $\text{IP}^{-1} = \text{FP}$. Decryption is therefore

$$\text{IP},\ M_{K_{16}},\ S,\ M_{K_{15}},\ S,\ \ldots,\ S,\ M_{K_1},\ \text{FP}.$$

This is exactly the encryption algorithm with the round keys in the order $K_{16}, \ldots, K_1$. The match works only because round 16 has no swapper: with a swapper there, the inverse would start with S and would no longer have the same shape as the cipher. The last round has no swapper so that the reverse cipher is the cipher itself with reversed round keys. (Equivalently, Stallings lets all 16 rounds swap and adds a 32-bit swap before FP; the two descriptions give the same ciphertext.)

The slides' pseudocode for the cipher (Forouzan):

```
Cipher (plainBlock[64], RoundKeys[16, 48], cipherBlock[64])
{
    permute (64, 64, plainBlock, inBlock, InitialPermutationTable)
    split (64, 32, inBlock, leftBlock, rightBlock)
    for (round = 1 to 16)
    {
        mixer (leftBlock, rightBlock, RoundKeys[round])
        if (round ≠ 16) swapper (leftBlock, rightBlock)
    }
    combine (32, 64, leftBlock, rightBlock, outBlock)
    permute (64, 64, outBlock, cipherBlock, FinalPermutationTable)
}
```

Here mixer does $T \leftarrow f(\text{rightBlock}, K)$, leftBlock $\leftarrow$ leftBlock $\oplus\ T$, and swapper exchanges the two blocks through a temporary copy.

31.4 The DES function $f(R_{i-1}, K_i)$ ★

The DES function is the heart of the cipher and the only place where the key enters. It applies the 48-bit round key to the 32-bit right half and returns 32 bits, through four steps.

1. Expansion P-box (32 → 48). R_{i-1} has 32 bits but K_i has 48, and the S-boxes need 6-bit inputs, so R_{i-1} is expanded first. It is viewed as eight 4-bit sections; each section is widened to 6 bits by adding, on its left, the last bit of the previous section and, on its right, the first bit of the next section (cyclically, so section 1 borrows bit 32 and section 8 borrows bit 1):

32	01	02	03	04	05		04	05	06	07	08	09
08	09	10	11	12	13		12	13	14	15	16	17
16	17	18	19	20	21		20	21	22	23	24	25
24	25	26	27	28	29		28	29	30	31	32	01

The 16 bits that appear twice are the first and fourth bits of every section (1, 4, 5, 8, ..., 29, 32). This duplication is deliberate: a bit at the edge of a section reaches two neighbouring S-boxes, so changing it changes the input

of two S-boxes, which is what makes the avalanche grow quickly. Being an expansion P-box, this step is not invertible, which a Feistel cipher can afford.

2. Whitener. The 48-bit expanded half is XORed with the 48-bit round key K_i. This is the only point at which the key influences the round.

3. S-boxes (48 → 32). The 48 bits are cut into eight 6-bit groups, and group j enters S-box S_j, which outputs 4 bits. The S-boxes do the real mixing (confusion), being the only nonlinear part of DES. Each S-box is a table of 4 rows and 16 columns. For a 6-bit input $b_1b_2b_3b_4b_5b_6$:

$$\text{row} = (b_1b_6)_2 \in \{0..3\}, \qquad \text{column} = (b_2b_3b_4b_5)_2 \in \{0..15\},$$

and the cell is the 4-bit output. The outer bits pick one of four substitutions and the middle bits are substituted by it; every row of every S-box is a permutation of 0..15 (checked by script), so each output value has exactly 4 preimages and the box as a whole is not invertible. The eight S-boxes are different. S-box 1:

S1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

4. Straight P-box (32 → 32). The 32 S-box output bits are permuted by a straight P-box; entry j is the input bit sent to output j, so the seventh input bit becomes the second output bit:

16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25

Its job is diffusion: the 4 output bits of each S-box are sent to positions that feed different S-boxes in the next round.

WORKED EXAMPLE : S-box lookup

▸ **Likely this year:** "The input to S-box 1 is `100011`. What is the output?" (Forouzan's standard S-box question.)

Q. (a) The input to S-box 1 is `100011`. What is the output? (b) The input to S-box 8 is `000000`. What is the output?

A. (a) Outer bits b_1b_6 = 1 1, so row = $(11)_2$ = 3. Middle bits $b_2..b_5$ = 0001, so column = 1. In S1, row 3, column 1 holds 12, so the output is **1100**.

(b) Outer bits 0 0 ⇒ row 0; middle bits 0000 ⇒ column 0. S8 row 0 begins 13 2 8 4 ..., so the output is 13 = **1101**.

More S1 lookups for practice: `000000` → row 0, col 0 → 14 = 1110; `111111` → row 3, col 15 → 13 = 1101; `011011` → row 1, col 13 → 5 = 0101.

Trap: row and column bits

The row is the *first and last* bit (not the first two), and the column is the *middle four*. Numbering starts at 0 for both rows and columns. Writing the answer as a decimal (12) without converting it to 4 bits (`1100`) loses the mark that the output is a 4-bit word.

WORKED EXAMPLE : one full DES round (Forouzan's trace: P = `123456ABCD132536`, key `AABB09182736CCDD`)

Q. After the initial permutation, L_0 = `14A7D678` and R_0 = `18CA18AD`; the first round key is K_1 = `194CD072DE8C`. Compute $f(R_0, K_1)$, L_1 and R_1.

A. R_0 = 0001 1000 1100 1010 0001 1000 1010 1101. Expansion (each 4-bit section borrows its neighbours' edge bits) gives $E(R_0)$ = `8F16540F155A`; XOR with K_1 gives `965A847DCBD6`. The eight 6-bit groups go through the S-boxes:

S-box	1	2	3	4	5	6	7	8
$E(R_0)$ group	100011	110001	011001	010100	000011	110001	010101	011010
K_1 group	000110	010100	110011	010000	011100	101101	111010	001100
XOR (input)	100101	100101	101010	000100	011111	011100	101111	010110
row, col	3, 2	3, 2	2, 5	0, 2	1, 15	0, 14	3, 7	0, 11
output	8	10	15	14	6	5	7	14

S-box output = `8AFE657E`; the straight P-box gives $f(R_0, K_1)$ = `4EDF35EC`. Then

$$L_1 = R_0 = \texttt{18CA18AD}, \qquad R_1 = L_0 \oplus f = \texttt{14A7D678} \oplus \texttt{4EDF35EC} = \texttt{5A78E394}.$$

Check: this is the round-1 output in Forouzan's table; continuing all 16 rounds (round 16 output 19BA9212 CF26B472, no swap) and FP gives the ciphertext C0B7A8D05F3A829C, and running the same program with the keys reversed returns the plaintext.

31.5 Key generation ★

The round-key generator must turn one 56-bit cipher key into sixteen different 48-bit round keys, cheaply and so that every key bit influences many rounds. It works in four steps.

1. **Parity drop (64 → 56).** The key is supplied as 64 bits in which bits 8, 16, 24, ..., 64 are parity bits (each key byte has odd parity). A compression permutation (PC-1) discards these 8 bits and permutes the remaining 56.
2. **Split.** The 56 bits are split into two 28-bit halves C_0 and D_0.
3. **Shift.** In round i both halves are circularly shifted left, by **1 bit in rounds 1, 2, 9 and 16** and by 2 bits in all other rounds, giving C_i and D_i.
4. **Compression P-box (56 → 48).** $C_i \| D_i$ passes through a compression P-box (PC-2) that blocks 8 of the 56 bits and permutes the other 48; the result is K_i.

Why it is built this way. Because the halves are rotated before each compression, a different selection and arrangement of key bits reaches the S-boxes in every round; each key bit ends up in 12 to 15 of the 16 round keys (about 14 on average, counted by script). The shift amounts add up to $4 \times 1 + 12 \times 2 = 28$, a full rotation of a 28-bit half, so $C_{16} = C_0$ and $D_{16} = D_0$. This lets the decryption side generate $K_{16}, \ldots, K_1$ on the fly by *right* shifts starting from the original halves.

WORKED EXAMPLE : first round key (same key as the trace above)

Q. For the key AABB09182736CCDD, find C_0, D_0, C_1, D_1 and K_1.
A. Parity drop gives the 56-bit key C3C033A33F0CFA, so C_0 = C3C033A, D_0 = 33F0CFA. Round 1 shifts by 1 bit:

$$C_0 = 1100\,0011\,1100\,0000\,0011\,0011\,1010 \xrightarrow{\text{rotl 1}} C_1 = 1000\,0111\,1000\,0000\,0110\,0111\,0101 = 8780675$$
$$D_0 = 0011\,0011\,1111\,0000\,1100\,1111\,1010 \xrightarrow{\text{rotl 1}} D_1 = 0110\,0111\,1110\,0001\,1001\,1111\,0100 = 67E19F4$$

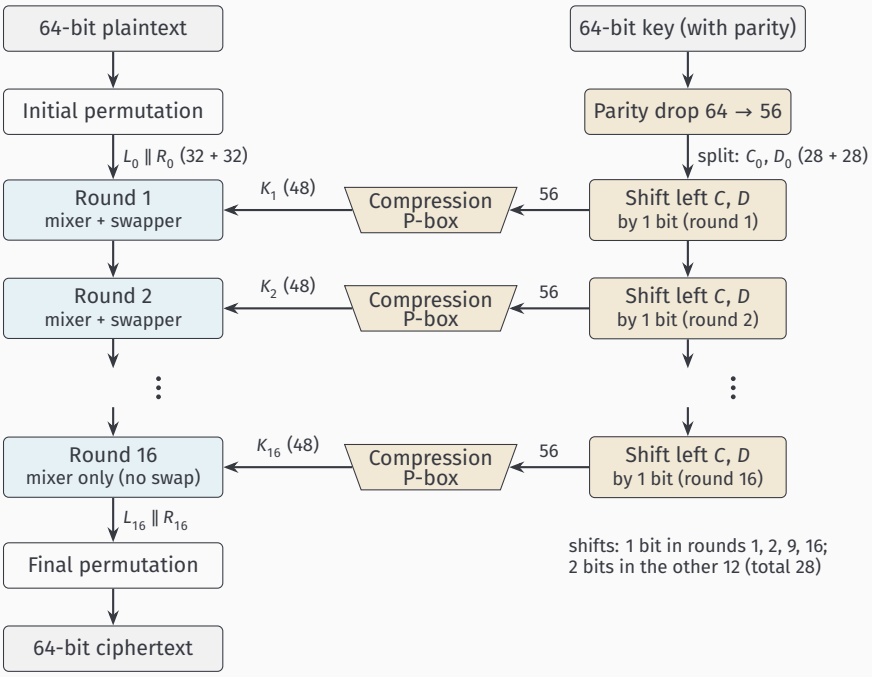
The compression P-box applied to $C_1 \| D_1$ gives K_1 = 194CD072DE8C, the key used in the round trace above. *Check:* in each half the leading bit moved to the end (e.g. C_0 starts with 1 and C_1 ends with 1).

WORKED EXAMPLE : 2024–25 Q6 (5 marks), answer as it should be written

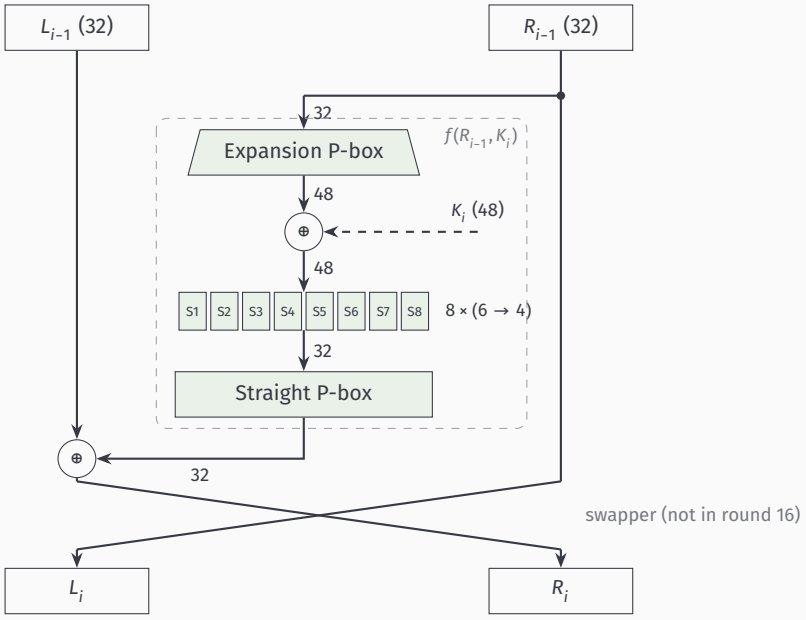
Q. In DES, what Feistel structure is prevalent? Use flow diagrams and key generation to define, with block diagrams, (1) the algorithm for DES and (2) the algorithm for the round function. How would you write the pseudocode for the substitution block and key generation?
A. Feistel structure. DES is a Feistel cipher: the 64-bit block (after IP) is split into 32-bit halves, and each of the 16 rounds applies a mixer and a swapper,

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i), \qquad i = 1..16 \text{ (no swap in round 16).}$$

All noninvertible parts (expansion P-box, 6 → 4 S-boxes) are inside f, whose output is only XORed in. Since $R_{i-1} = L_i$ is available to the receiver, $L_{i-1} = R_i \oplus f(L_i, K_i)$; so f never has to be inverted, and decryption is the same algorithm with keys $K_{16}, \ldots, K_1$.
(1) DES algorithm and key generation.



(2) Round i and the round function $f(R_{i-1}, K_i)$.



Inside f: the expansion P-box widens R_{i-1} from 32 to 48 bits, the whitener XORs in K_i, eight S-boxes reduce 48 bits to 32 (row = outer bits, column = middle 4 bits), and a straight P-box permutes the 32 bits.

Pseudocode (Forouzan style; bit arrays are indexed from 1).

```
substitute (inBlock[48], outBlock[32], SubstitutionTables[8, 4, 16])
{
   for (i = 1 to 8)
   {
      b ← (i − 1) × 6                          // start of group i
      row ← 2× inBlock[b+1] + inBlock[b+6]
      col ← 8× inBlock[b+2] + 4× inBlock[b+3]
            + 2× inBlock[b+4] + inBlock[b+5]
      value ← SubstitutionTables[i][row][col]
      o ← (i − 1) × 4
      outBlock[o+1] ← value / 8;  value ← value mod 8
      outBlock[o+2] ← value / 4;  value ← value mod 4
      outBlock[o+3] ← value / 2;  value ← value mod 2
      outBlock[o+4] ← value
   }
}
```

```
Key_Generator (keyWithParities[64], RoundKeys[16, 48], ShiftTable[16])
{
   permute (64, 56, keyWithParities, cipherKey, ParityDropTable)
   split (56, 28, cipherKey, leftKey, rightKey)
   for (round = 1 to 16)
   {
      shiftLeft (leftKey, ShiftTable[round])
      shiftLeft (rightKey, ShiftTable[round])
      combine (28, 56, leftKey, rightKey, preRoundKey)
      permute (56, 48, preRoundKey, RoundKeys[round], KeyCompressionTable)
   }
}
```

ShiftTable = [1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1]; shiftLeft is a circular left shift of a 28-bit half.

Check: the substitute routine reproduces the S-box output 8AFE657E of the round trace above (tested by script).

★ MUST KNOW : DES at a glance

64-bit block, 56-bit key (64 with parity), 16 Feistel rounds, 48-bit round keys. Round: $L_i = R_{i-1}$, $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$, no swap in round 16. f: expansion 32 → 48, XOR K_i, 8 S-boxes 6 → 4 (row = $b_1 b_6$, column = $b_2..b_5$), straight P-box 32 → 32. Keys: parity drop 64 → 56, split 28 + 28, rotate left 1 bit in rounds 1, 2, 9, 16 and 2 bits otherwise, compression 56 → 48. Decryption: same algorithm, keys $K_{16}..K_1$.

31.6 DES analysis: avalanche and completeness

Two properties are expected of a good block cipher. The **avalanche effect** means that a small change in the plaintext (or the key) should change many ciphertext bits, about half of them. **Completeness** means that each ciphertext bit should depend on many plaintext bits. In DES both come from the combination of S-boxes (confusion) and P-boxes, including the expansion that feeds each edge bit to two S-boxes (diffusion), and tests show DES is strong in both.

WORKED EXAMPLE : avalanche in DES (Forouzan's example)

Q. Encrypt the plaintexts 0000000000000000 and 0000000000000001 (differing in one bit) with the key 22234512987ABB23. How many ciphertext bits differ?
A. The ciphertexts are 4789FD476E82A5F1 and 0A4ED5C15A63FEA3; their XOR has **29** one-bits, so a 1-bit change of the plaintext changed 29 of the 64 ciphertext bits. The number of differing bits after each round (script) shows how fast the difference spreads:

round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
bits differing	1	6	20	29	30	33	32	29	32	39	33	28	30	31	30	29

After only 3 to 4 rounds the difference is already near the ideal of 32; the remaining rounds keep it there.

31.7 Weaknesses of DES

Design of the S-boxes and P-boxes. Critics point out a few irregularities, none of which has led to a practical attack:

- In S-box 4, the last three output bits can be derived in the same way as the first output bit by complementing some of the input bits.
- Two specially chosen inputs to the S-box array can produce the same output.
- The same output of a single round can be obtained by changing bits in only three neighbouring S-boxes.
- The initial and final permutations have no security benefit, and in the expansion P-box the first and fourth bits of every 4-bit section are repeated.

Key size. The real weakness is the 56-bit key. A brute-force attack on one ciphertext block must try up to $2^{56} \approx 7.2 \times 10^{16}$ keys. At 10^6 keys per second on each of 10^6 parallel chips, i.e. 10^{12} keys per second, the whole key space takes $2^{56}/10^{12} \approx 72\,058$ seconds ≈ **20 hours** (10 hours on average, since the key is found halfway on average). Networked computers simulate parallel hardware: the slides quote 3500 Internet computers finding an RSA Laboratories challenge key in 120 days, so 42 000 machines would need $3500 \times 120/42\,000 = 10$ days.

Weak keys. A **weak key** is one for which all 16 round keys are the same. This happens when, after the parity drop, each 28-bit half is all 0s or all 1s, because rotating such a half changes nothing. There are 4 weak keys (verified by script: each gives a single distinct round key):

Key before parity drop (64 bits)	Actual key: C_0, D_0 (hex)
0101 0101 0101 0101	0000000 0000000
1F1F 1F1F 0E0E 0E0E	0000000 FFFFFFF
E0E0 E0E0 F1F1 F1F1	FFFFFFF 0000000
FEFE FEFE FEFE FEFE	FFFFFFF FFFFFFF

With all round keys equal, the key order $K_{16}, \ldots, K_1$ is the same as $K_1, \ldots, K_{16}$, so the reverse cipher *is* the cipher: encrypting twice with a weak key returns the plaintext, $E_K(E_K(P)) = P$.

Semi-weak keys. A **semi-weak key** creates only 2 distinct round keys, each used 8 times (each half follows the alternating pattern 0101 … or 1010 …, or is constant, with at least one half alternating). They come in **6 pairs** (k_1, k_2) whose round-key sequences are each other's reverse, so $E_{k_2}(E_{k_1}(P)) = P$: one key of the pair decrypts what the other encrypts. The pairs (all checked by script) are:

First key	Second key	First key	Second key
01FE 01FE 01FE 01FE	FE01 FE01 FE01 FE01	1FFE 1FFE 0EFE 0EFE	FE1F FE1F FE0E FE0E
1FE0 1FE0 0EF1 0EF1	E01F E01F F10E F10E	011F 011F 010E 010E	1F01 1F01 0E01 0E01
01E0 01E0 01F1 01F1	E001 E001 F101 F101	E0FE E0FE F1FE F1FE	FEE0 FEE0 FEF1 FEF1

Possible weak keys. Forouzan lists **48 possible weak keys**, which create only 4 distinct round keys, each used 4 times (halves repeating with period 4, such as 0011 0011 ...). Since 2^{56} keys contain only $4+12+48 = 64$ such keys, a randomly chosen key is weak with probability 2^{-50}, so these keys matter only if they are chosen on purpose.

Key complement. Let $\bar{x}$ be the bitwise complement. Then $E_{\bar{K}}(\bar{P}) = \overline{E_K(P)}$. Proof: complementing K complements every round key, and complementing the data complements R_{i-1} and hence its expansion; then $E(\bar{R})\oplus\bar{K}_i = E(R)\oplus K_i$, so the S-box inputs and f are unchanged, and $\bar{L}\oplus f = \overline{L\oplus f}$, so every half stays complemented through all rounds (checked by script). As a result, a chosen-plaintext attacker who has $C_1 = E_K(P)$ and $C_2 = E_K(\bar{P})$ can test two keys with one encryption (K and $\bar{K}$), halving the brute-force work to 2^{55}.

31.8 Multiple DES

The obvious fix for a short key is to encrypt several times with different keys. This only helps because DES is **not a group**: if for every K_1, K_2 there were a K_3 with $E_{K_2}(E_{K_1}(P)) = E_{K_3}(P)$, double encryption would just be single DES with another key. It has been shown that no such K_3 exists in general, so cascading DES really produces new mappings.

Double DES. $C = E_{K_2}(E_{K_1}(P))$ and $P = D_{K_1}(D_{K_2}(C))$, with a 112-bit key (K_1, K_2). One would expect 2^{112} trials for a brute-force attack, but the **meet-in-the-middle attack** (a known-plaintext attack) needs only about 2^{57}. The derivation starts from the middle text M, the value between the two encryptions:

$$M = E_{K_1}(P) \quad \text{and also} \quad M = D_{K_2}(C).$$

Given one known pair (P, C), Eve encrypts P under all 2^{56} values of K_1 and stores the results M in a table sorted by M. She then decrypts C under all 2^{56} values of K_2 and looks up each $D_{K_2}(C)$ in the table. Every match gives a candidate pair (K_1, K_2), since it makes both equations hold. The cost is $2^{56}+2^{56} = 2^{57}$ DES operations (plus memory for 2^{56} entries), instead of $2^{56}\times 2^{56} = 2^{112}$, because the two keys are searched *one after the other* rather than in all combinations. Some matches are false: 2^{112} key pairs map to only 2^{64} middle values, so about $2^{112}/2^{64} = 2^{48}$ pairs fit one (P, C) by chance; testing the candidates on a second known pair (P', C') leaves about $2^{48}/2^{64} < 1$ false survivor, so the right key is identified. Double DES raises the work only from 2^{56} to 2^{57}, not to 2^{112}.

Triple DES with two keys. $C = E_{K_1}(D_{K_2}(E_{K_1}(P)))$ and $P = D_{K_1}(E_{K_2}(D_{K_1}(C)))$, the **EDE** (encrypt–decrypt–encrypt) form. The key is 112 bits, and a meet-in-the-middle attack now has to split three stages into one and two, which leaves a 2^{112} search on one side, so the attack no longer helps. The middle stage is a *decryption* for compatibility: with $K_1 = K_2$ the first two stages cancel and 3DES becomes single DES, so a 3DES device can talk to an old DES device (checked by script). Two-key 3DES is used in PEM and in the financial standard ANS X9.17 / ISO 8732.

Triple DES with three keys. $C = E_{K_3}(D_{K_2}(E_{K_1}(P)))$ with a 168-bit key. Meet-in-the-middle reduces its effective strength to about 2^{112}, which is still far beyond brute force; it is used in PGP and S/MIME.

31.9 Security of DES in one line each

- **Brute force:** 2^{56} keys (2^{55} with the complement property), feasible with special hardware or networked computers.
- **Differential cryptanalysis:** DES can be broken with 2^{47} chosen plaintexts; its S-boxes were evidently designed against this attack, which is why it is no better than brute force in practice.
- **Linear cryptanalysis:** DES can be broken with 2^{43} known plaintexts, a newer attack the designers did not anticipate.

Corrections to the notes / slides

- The slides' **substitute** pseudocode (L8 slide 16, as in Forouzan) declares inBlock[32], outBlock[48]; the sizes are swapped (48 bits in, 32 bits out). It also indexes inBlock[$i \times 6 + 1$] with $i = 1..8$, which reaches position 49 for $i = 8$; the group offset must be $(i - 1) \times 6$ (and $(i - 1) \times 4$ for outBlock), as in the corrected version in the Q6 answer (verified by script).
- Semi-weak key table (L8 slide 23), row 3: `01E0 01E1 01F1 01F1` should be `01E0 01E0 01F1 01F1`. `E1` differs from `E0` only in the parity bit, so it is the same 56-bit key, but it violates odd parity.
- Key-size slide (L8 slide 21): "In 1977 a team of researchers used 3500 computers ... RSA Laboratories" should read 1997 (RSA's DES Challenge); DES itself was only published in 1977.

Check your understanding

1. **Why do IP and FP add no security?** They are fixed, keyless and public, so an attacker can apply them himself and attack the 16 rounds directly.
2. **Why does round 16 have no swapper?** So that the sequence of operations of the reverse cipher is identical to the cipher; then decryption is the same algorithm with keys $K_{16}, \dots, K_1$.
3. **What is the output of S-box 1 for input `111111`?** Row $(11)_2 = 3$, column $(1111)_2 = 15$; S1[3][15] = 13 = `1101`.
4. **Why does DES need an expansion P-box?** To match the 48-bit round key and to give eight 6-bit S-box inputs; the duplicated edge bits also feed two neighbouring S-boxes, which speeds up the avalanche.
5. **How many bits are shifted in total, and why does it matter?** $4 \times 1 + 12 \times 2 = 28$, a full rotation, so $C_{16} = C_0$ and $D_{16} = D_0$ and decryption keys can be generated by right shifts from the original halves.
6. **Is the DES S-box invertible? Does it matter?** No, a $6 \to 4$ box has 4 preimages per output; it does not matter because it sits inside f, which a Feistel cipher never inverts.
7. **What is special about a weak key?** All 16 round keys are equal, so $E_K(E_K(P)) = P$; there are 4 weak keys, 6 pairs of semi-weak keys and 48 possible weak keys.
8. **Why is double DES not 112-bit secure?** Meet-in-the-middle: with one known (P, C), tabulate $E_{K_1}(P)$ for all 2^{56} keys and match against $D_{K_2}(C)$ for all 2^{56} keys, which costs 2^{57} operations instead of 2^{112}.
9. **Why is triple DES EDE and not EEE?** With $K_1 = K_2$ (or $K_2 = K_3$), EDE collapses to single DES, so new equipment stays compatible with single-DES equipment.
10. **State the complement property and its consequence.** $E_{\bar{K}}(\bar{P}) = \overline{E_K(P)}$, so a chosen-plaintext brute-force attack tests two keys per encryption and needs only 2^{55} trials.

32 Advanced Encryption Standard (AES)

Why DES had to be replaced. By the late 1990s the 56-bit DES key was within reach of brute force: a 2^{56} search is about 7.2×10^{16} trials, and a purpose-built machine (EFF, 1998) found a DES key in a few days. Triple DES fixes the key length but runs three DES encryptions per block, which is slow in software, and it keeps DES's 64-bit block, which is too small for encrypting large volumes of data under one key. NIST therefore ran an open competition, judged on three criteria (*security, cost, implementation*), and in 2000 chose **Rijndael** by Daemen and Rijmen. It was published as the **Advanced Encryption Standard** (FIPS 197) in December 2001.

What kind of cipher it is. AES is a symmetric-key, non-Feistel block cipher that encrypts 128-bit blocks with a 128-, 192- or 256-bit key in 10, 12 or 14 rounds. Non-Feistel means that, unlike DES, the round does not split the block into halves and hide a one-way function inside a swap. Instead every step of the round is itself invertible, and decryption runs the inverse steps in reverse order. Such a design is called a substitution–permutation network: substitution (SubBytes) gives confusion, and permutation plus mixing (ShiftRows, MixColumns) give diffusion.

32.1 Parameters and data units

The block is always 128 bits and every round key is always 128 bits, whatever the cipher-key length. Only the number of rounds N_r depends on the key size, because a longer key needs more rounds before every key bit has influenced every state bit. Since one round key is added before round 1 and one in each of the N_r rounds, the number of round keys is $N_r + 1$, and each round key is 4 words.

Version	Key (bits)	Rounds N_r	Round keys $N_r + 1$	Key words $4(N_r + 1)$	Brute force
AES-128	128	10	11	44 (w_0–w_{43})	2^{128}
AES-192	192	12	13	52 (w_0–w_{51})	2^{192}
AES-256	256	14	15	60 (w_0–w_{59})	2^{256}

Data units. AES names five units. A *bit* is a binary digit; a *byte* is 8 bits and is treated as an element of $\mathrm{GF}(2^8)$, written as two hex digits; a *word* is 4 bytes (32 bits), seen either as a row or as a column of 4 bytes; a *block* is 16 bytes (128 bits), $b_0 b_1 \dots b_{15}$; and the *state* is the block arranged as a 4×4 matrix of bytes $s_{r,c}$, which is what every transformation actually acts on. Each column of the state is one word, so the state is also four words $w_0 w_1 w_2 w_3$ side by side.

Block to state. The 16 bytes fill the state **column by column**: bytes $b_0..b_3$ form column 0 (top to bottom), $b_4..b_7$ column 1, and so on, so that

$$s_{r,c} = b_{r+4c}, \qquad \text{equivalently} \qquad b_i \to s_{i \bmod 4,\, \lfloor i/4 \rfloor}.$$

The reverse mapping reads the state column by column to get the output block. This column-major order matters because MixColumns and AddRoundKey work on columns, so four consecutive input bytes are processed together.

Trap: filling the state row-wise

Writing the 16 bytes *across* the rows is the most common slip. The first four bytes go *down* column 0. With the wrong order every later step (ShiftRows, MixColumns, the key words as columns) gives a wrong answer.

WORKED EXAMPLE : text block to state (slide Example 1, from Forouzan)

Q. Show the state for the text block "AES uses a matrix", letters coded A = 00 to Z = 25 in hex, padding with Z.

A. Removing spaces gives 14 letters, so two bogus Z's make 16: AESUSESAMATRIXZZ. Each letter's number is written in **hex** (for example M = 12 = 0C, R = 17 = 11, X = 23 = 17, Z = 25 = 19):

Text	A	E	S	U	S	E	S	A	M	A	T	R	I	X	Z	Z
Hex	00	04	12	14	12	04	12	00	0C	00	13	11	08	17	19	19

Filling column by column (first four bytes down column 0):

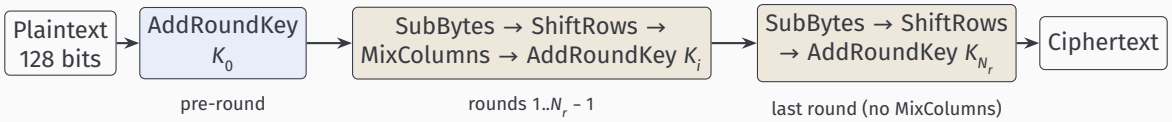
$$\text{State} = \begin{pmatrix} 00 & 12 & 0C & 08 \\ 04 & 04 & 00 & 17 \\ 12 & 12 & 13 & 19 \\ 14 & 00 & 11 & 19 \end{pmatrix}$$

Check: $s_{1,3} = b_{1+12} = b_{13} = 17$ (the X). The slide prints 23 here; see the corrections box.

32.2 Round structure

Encryption of AES-128 is one key addition followed by 10 rounds:

1. **Pre-round:** AddRoundKey with round key $K_0 = w_0..w_3$ (the cipher key itself).
2. **Rounds 1 to** $N_r - 1$**:** SubBytes, ShiftRows, MixColumns, AddRoundKey (with $K_i = w_{4i}..w_{4i+3}$).
3. **Last round** N_r**:** SubBytes, ShiftRows, AddRoundKey, **without MixColumns**.



Why a key addition comes first. Every step except AddRoundKey is public and keyless. If the cipher began with SubBytes, an attacker who knows the plaintext could compute that step himself, so it would add work but no secrecy. Adding the key first (key whitening) means that from the very first non-linear step onward, nothing is computable without the key.

Why the last round has no MixColumns. MixColumns is linear over GF(2^8), so $\text{MC}(Y) \oplus K = \text{MC}\big(Y \oplus \text{MC}^{-1}(K)\big)$. If the last round ended with MixColumns and then AddRoundKey, an attacker could apply InvMixColumns to the ciphertext and obtain $Y \oplus \text{MC}^{-1}(K_{N_r})$, which is just the same cipher without the final MixColumns under a different (equally unknown) key. The final MixColumns therefore adds no security. Leaving it out costs nothing and makes the decryption algorithm have the same shape as encryption, which allows the "equivalent inverse cipher" to reuse the same hardware.

★ MUST KNOW

AES: block 128, key 128/192/256, rounds 10/12/14, round keys $N_r + 1$ (always 128 bits each), $4(N_r + 1)$ words. Pre-round AddRoundKey; rounds 1..$N_r - 1$: **SubBytes, ShiftRows, MixColumns, AddRoundKey**; last round drops MixColumns. The state is filled **column by column**, $s_{r,c} = b_{r+4c}$.

32.3 SubBytes (substitution) ★

▸ **Likely this year:** "Find SubBytes of 0x53" or "apply SubBytes to the given state using the S-box"; or derive one S-box entry from the GF(2^8) inverse and the affine transform.

Problem and idea. A cipher built only from XORs, shifts and matrix multiplications is linear, and a linear cipher falls to solving a system of equations. SubBytes is the one non-linear step in AES. Each of the 16 bytes is replaced independently by looking it up in one fixed 16 × 16 table, the S-box. Because the same table is used for every byte, equal input bytes always give equal output bytes; SubBytes is an *intrabyte* transformation, since a byte's output depends on that byte alone.

Lookup rule. Write the byte as two hex digits *xy*. The **left digit** *x* **selects the row** and the **right digit** *y* **selects the column**; the entry there is the output. For example 53: row 5, column 3 gives ED, and 00: row 0, column 0 gives 63.

$x\backslash y$	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

AES S-box (FIPS-197), generated by script from the definition below and checked entry by entry.

How the S-box is built. The table is not arbitrary; each entry is computed in two steps from the byte a viewed as a polynomial in GF(2^8) with modulus $m(x) = x^8 + x^4 + x^3 + x + 1$ (11B).

1. **Multiplicative inverse:** $b = a^{-1}$ in GF(2^8), with 00 mapped to 00 (it has no inverse). The inverse map is highly non-linear, which is what resists linear and differential cryptanalysis. This is why AES works in GF(2^8) and not in $\mathbb{Z}_{256}$: in $\mathbb{Z}_{256}$ every even byte lacks an inverse, so the map would not be a permutation.
2. **Affine transformation over GF(2):** with $c = 63 = 0110\,0011$,

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i, \qquad i = 0..7.$$

   This step breaks the simple algebraic form $a \mapsto a^{-1}$ and removes its fixed points (00 $\mapsto$ 00 and 01 $\mapsto$ 01). As a result the AES S-box has no fixed point ($S(a) \neq a$ for all 256 bytes) and none with $S(a) = \bar{a}$.

InvSubBytes uses the inverse table: undo the affine map, then take the GF(2^8) inverse. Without that table, find the output in the S-box and read off its row and column (for example ED sits at row 5, column 3, so InvSubBytes(ED) = 53).

WORKED EXAMPLE : one S-box entry from the definition

Q. Show that SubBytes(53) = ED by computing it, not by looking it up.

A. *Step 1.* $53 = x^6 + x^4 + x + 1$. Its inverse modulo $m(x)$, by the extended Euclidean algorithm in GF(2^8), is $CA = x^7 + x^6 + x^3 + x$ (check: $53 \cdot CA = 01$). So $b = CA = 1100\,1010$, that is $b_7..b_0 = 1, 1, 0, 0, 1, 0, 1, 0$.
Step 2. Apply $b'_i = b_i \oplus b_{i+4} \oplus b_{i+5} \oplus b_{i+6} \oplus b_{i+7} \oplus c_i$ (indices mod 8, $c = 63$, so $c_0 = c_1 = c_5 = c_6 = 1$):

i	b_i	b_{i+4}	b_{i+5}	b_{i+6}	b_{i+7}	c_i	b'_i
0	0	0	0	1	1	1	**1**
1	1	0	1	1	0	1	**0**
2	0	1	1	0	1	0	**1**
3	1	1	0	1	0	0	**1**
4	0	0	1	0	1	0	**0**
5	0	1	0	1	0	1	**1**
6	1	0	1	0	0	1	**1**
7	1	1	0	0	1	0	**1**

Reading $b'_7..b'_0 = 1110\,1101$ gives **SubBytes(53) = ED**, which matches row 5, column 3 of the table.

WORKED EXAMPLE : SubBytes of a state (slide Example 2)

Q. Apply SubBytes to the state of Example 1.

A. Each byte is looked up independently (row = left digit, column = right digit): 00→63, 04→F2, 12→C9, 14→FA, 0C→FE, 13→7D, 11→82, 08→30, 17→F0, 19→D4.

$$\begin{pmatrix} 00 & 12 & 0C & 08 \\ 04 & 04 & 00 & 17 \\ 12 & 12 & 13 & 19 \\ 14 & 00 & 11 & 19 \end{pmatrix} \xrightarrow{\text{SubBytes}} \begin{pmatrix} 63 & C9 & FE & 30 \\ F2 & F2 & 63 & \mathbf{F0} \\ C9 & C9 & 7D & D4 \\ FA & 63 & 82 & D4 \end{pmatrix}$$

The two 04's in row 1 both become F2, showing that SubBytes alone does not hide repeated bytes; ShiftRows

and MixColumns are needed for that.

Single-bit changes spread: 5A and 5B differ in one bit, but $S(5A) = BE$ and $S(5B) = 39$ differ in four bits ($BE \oplus 39 = 87 = 1000\,0111$).

32.4 ShiftRows (permutation) ★

★ **In the papers:** 2025–26 Q4(a) (part of 6 marks): ShiftRows on a given state. Solved in full after AddRoundKey below.

Problem and idea. SubBytes changes each byte on its own, and MixColumns (next) mixes only the four bytes inside one column. Without a step that moves bytes *between* columns, the four columns would never interact, and AES would be four independent 32-bit ciphers running side by side, each attackable separately. ShiftRows is that step: it permutes bytes so that each column's four bytes end up in four different columns.

Mechanism. Row r of the state is rotated cyclically left by r bytes: row 0 is unchanged, row 1 moves left by 1, row 2 by 2, row 3 by 3. In symbols $s'_{r,c} = s_{r,(c+r) \bmod 4}$. The permutation is at the *byte* level: bytes change position but the bits inside a byte are untouched, unlike DES, whose P-boxes permute single bits. Since the four bytes of input column c lie in rows 0, 1, 2, 3, they are shifted by 0, 1, 2, 3 places and land in four different columns; after the next MixColumns every output column then depends on all four input columns. InvShiftRows rotates row r *right* by r bytes.

$$\begin{pmatrix} s_{00} & s_{01} & s_{02} & s_{03} \\ s_{10} & s_{11} & s_{12} & s_{13} \\ s_{20} & s_{21} & s_{22} & s_{23} \\ s_{30} & s_{31} & s_{32} & s_{33} \end{pmatrix} \xrightarrow{\text{ShiftRows}} \begin{pmatrix} s_{00} & s_{01} & s_{02} & s_{03} \\ s_{11} & s_{12} & s_{13} & s_{10} \\ s_{22} & s_{23} & s_{20} & s_{21} \\ s_{33} & s_{30} & s_{31} & s_{32} \end{pmatrix}$$

Applied to the SubBytes output of Example 2 (slide Example 3):

$$\begin{pmatrix} 63 & C9 & FE & 30 \\ F2 & F2 & 63 & F0 \\ C9 & C9 & 7D & D4 \\ FA & 63 & 82 & D4 \end{pmatrix} \xrightarrow{\text{ShiftRows}} \begin{pmatrix} 63 & C9 & FE & 30 \\ F2 & 63 & F0 & F2 \\ 7D & D4 & C9 & C9 \\ D4 & FA & 63 & 82 \end{pmatrix}$$

32.5 MixColumns (mixing) ★

▸ **Likely this year:** "Perform MixColumns on one column" (one or all four output bytes), or "show that the given column maps to …".

Problem and idea. SubBytes is intrabyte and ShiftRows only exchanges bytes, so neither makes a byte depend on its neighbours. MixColumns is the *interbyte* step: it replaces each column by a combination of all four of its bytes, so that changing one input byte changes all four output bytes. To guarantee that each new byte differs from the others even when all four inputs are equal, each input is first multiplied by a different constant and then the products are added. This is a matrix multiplication over GF(2^8), where "add" is XOR and "multiply" is polynomial multiplication modulo 11B:

$$\begin{pmatrix} s'_0 \\ s'_1 \\ s'_2 \\ s'_3 \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \end{pmatrix}, \qquad \text{InvMixColumns: } \begin{pmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{pmatrix}.$$

Each row of the matrix is the row above rotated right by one. Written out, $s'_0 = 02 \cdot s_0 \oplus 03 \cdot s_1 \oplus s_2 \oplus s_3$, $s'_1 = s_0 \oplus 02 \cdot s_1 \oplus 03 \cdot s_2 \oplus s_3$, $s'_2 = s_0 \oplus s_1 \oplus 02 \cdot s_2 \oplus 03 \cdot s_3$, $s'_3 = 03 \cdot s_0 \oplus s_1 \oplus s_2 \oplus 02 \cdot s_3$. The two matrices multiply to the identity in GF(2^8) (checked by script), which is why InvMixColumns undoes MixColumns.

Why the constants 01, 02, 03. They make encryption cheap, because multiplying by these needs only a shift and XORs. Multiplying by 02 is multiplying by x: shift the byte left by one bit, and if the bit shifted out (the old top bit b_7) was 1, the result has an x^8 term, which is reduced by XOR with 1B (since $x^8 \equiv x^4 + x^3 + x + 1$). This operation is called **xtime**. Multiplying by 03 $= x + 1$ is then $(02 \cdot b) \oplus b$. The matrix is also chosen so that the total number of non-zero bytes in an input column plus its output column is at least 5 (branch number 5), so one changed input byte always changes all four output bytes.

★ MUST KNOW : xtime

$02 \cdot b = (b \ll 1)$, then XOR 1B if the top bit of b was 1. $\quad 03 \cdot b = (02 \cdot b) \oplus b$. $\quad 01 \cdot b = b$.

WORKED EXAMPLE : one column (FIPS-197 test column)

Q. Apply MixColumns to the column $(DB, 13, 53, 45)^T$.

A. First the products by 02 and 03 of every input byte:

b	binary	shift left	top bit 1?	$02\cdot b$ / $03\cdot b = (02\cdot b)\oplus b$
DB	1101 1011	1011 0110 = B6	yes: B6 ⊕ 1B	AD / 76
13	0001 0011	0010 0110 = 26	no	26 / 35
53	0101 0011	1010 0110 = A6	no	A6 / F5
45	0100 0101	1000 1010 = 8A	no	8A / CF

Then each output byte is a row of the matrix times the column, XORed left to right:

$$s'_0 = 02{\cdot}DB \oplus 03{\cdot}13 \oplus 53 \oplus 45 = AD \oplus 35 \oplus 53 \oplus 45 = 98 \oplus 53 \oplus 45 = CB \oplus 45 = \mathbf{8E}$$
$$s'_1 = DB \oplus 02{\cdot}13 \oplus 03{\cdot}53 \oplus 45 = DB \oplus 26 \oplus F5 \oplus 45 = FD \oplus F5 \oplus 45 = 08 \oplus 45 = \mathbf{4D}$$
$$s'_2 = DB \oplus 13 \oplus 02{\cdot}53 \oplus 03{\cdot}45 = DB \oplus 13 \oplus A6 \oplus CF = C8 \oplus A6 \oplus CF = 6E \oplus CF = \mathbf{A1}$$
$$s'_3 = 03{\cdot}DB \oplus 13 \oplus 53 \oplus 02{\cdot}45 = 76 \oplus 13 \oplus 53 \oplus 8A = 65 \oplus 53 \oplus 8A = 36 \oplus 8A = \mathbf{BC}$$

Answer: $(DB, 13, 53, 45) \to (8E, 4D, A1, BC)$, the FIPS-197 test value. Check: every output byte changed, as the branch-number argument predicts.

A second check comes from the exam state below: column 0 after ShiftRows is $(87, 6E, 46, A6)$. With $02\cdot 87 = 0E \oplus 1B = 15$, $03\cdot 6E = DC \oplus 6E = B2$, the first output byte is $15 \oplus B2 \oplus 46 \oplus A6 = 47$, and the whole column becomes $(47, 37, 94, ED)$, which is exactly column 0 of the state given in part (b) of that question (Stallings' figures chain ShiftRows → MixColumns → AddRoundKey).

32.6 AddRoundKey (key adding) ★

★ **In the papers:** 2025–26 Q4(b) (part of 6 marks): AddRoundKey of a given state with a given key matrix.

Problem and idea. The three steps above are fixed and public, so on their own they would be a keyless scrambling that anyone can undo. AddRoundKey is the only step that uses the key: it XORs the 128-bit round key into the state. It proceeds column by column, like MixColumns, but where MixColumns multiplies each column by a constant matrix, AddRoundKey *adds* a key word to it: in round i, column c becomes $s_c \oplus w_{4i+c}$. Equivalently, write the round key as a 4×4 matrix whose columns are the four words and XOR the two matrices entry by entry.

Inverse. Addition and subtraction in GF(2^8) are both XOR, and $x \oplus k \oplus k = x$, so AddRoundKey is its own inverse; decryption uses the same operation with the same round key.

Doing the XOR by hand. XOR one hex digit at a time, through its 4-bit binary form: $4 \oplus A = 0100 \oplus 1010 = 1110 = E$. Useful shortcuts: $x \oplus x = 0$, $x \oplus 0 = x$, $x \oplus F$ flips all four bits.

WORKED EXAMPLE : 2025–26 Q4 (6 marks), Stallings' AES example

Q. In AES, the state is given below. (a) Show the ShiftRows transformation. (b) After MixColumns, assume the state and the round key are as given. Perform AddRoundKey.

A. (a) Row r is rotated left by r bytes: row 0 unchanged; row 1 EC 6E 4C 90 → 6E 4C 90 EC; row 2 4A C3 46 E7 → 46 E7 4A C3; row 3 8C D8 95 A6 → A6 8C D8 95.

$$\begin{pmatrix} 87 & F2 & 4D & 97 \\ EC & 6E & 4C & 90 \\ 4A & C3 & 46 & E7 \\ 8C & D8 & 95 & A6 \end{pmatrix} \xrightarrow{\text{ShiftRows}} \begin{pmatrix} 87 & F2 & 4D & 97 \\ \mathbf{6E} & \mathbf{4C} & \mathbf{90} & \mathbf{EC} \\ \mathbf{46} & \mathbf{E7} & \mathbf{4A} & \mathbf{C3} \\ \mathbf{A6} & \mathbf{8C} & \mathbf{D8} & \mathbf{95} \end{pmatrix}$$

(b) AddRoundKey is the byte-wise XOR of the state and the key matrix, $s'_{r,c} = s_{r,c} \oplus k_{r,c}$:

$$\begin{pmatrix} 47 & 40 & A3 & 4C \\ 37 & D4 & 70 & 9F \\ 94 & E4 & 3A & 42 \\ ED & A5 & A6 & BC \end{pmatrix} \oplus \begin{pmatrix} AC & 19 & 28 & 57 \\ 77 & FA & D1 & 5C \\ 66 & DC & 29 & 00 \\ F3 & 21 & 41 & 6A \end{pmatrix} = \begin{pmatrix} \mathbf{EB} & \mathbf{59} & \mathbf{8B} & \mathbf{1B} \\ \mathbf{40} & \mathbf{2E} & \mathbf{A1} & \mathbf{C3} \\ \mathbf{F2} & \mathbf{38} & \mathbf{13} & \mathbf{42} \\ \mathbf{1E} & \mathbf{84} & \mathbf{E7} & \mathbf{D6} \end{pmatrix}$$

Working, digit by digit (left digits, then right digits):

```
47⊕AC: 4⊕A=E, 7⊕C=B → EB   40⊕19: 5, 9 → 59   A3⊕28: 8, B → 8B   4C⊕57: 1, B → 1B
37⊕77: 4, 0 → 40            D4⊕FA: 2, E → 2E   70⊕D1: A, 1 → A1   9F⊕5C: C, 3 → C3
94⊕66: F, 2 → F2            E4⊕DC: 3, 8 → 38   3A⊕29: 1, 3 → 13   42⊕00: 4, 2 → 42
ED⊕F3: 1, E → 1E            A5⊕21: 8, 4 → 84   A6⊕41: E, 7 → E7   BC⊕6A: D, 6 → D6
```

For example $47 \oplus AC$: $0100\,0111 \oplus 1010\,1100 = 1110\,1011 = EB$. Check: XORing the answer with the key again returns the given state (for instance $EB \oplus AC = 47$), because AddRoundKey is self-inverse.

Trap: AddRoundKey

Do not add bytes as integers (47 + AC = F3 is wrong; the answer is 47 ⊕ AC = EB). And in the ShiftRows part, shift *left*, by the row index starting from 0; rotating right, or shifting row 1 by 2, is the usual slip.

32.7 Key expansion (AES-128) ★

▸ **Likely this year:** "Given the cipher key, find w_4–w_7 (the first round key)", Forouzan/Stallings style, using the S-box table.

Problem. AES-128 needs 11 round keys of 128 bits, that is 44 words $w_0..w_{43}$, but the user supplies only one 128-bit key. The expansion must be cheap, must spread every key bit into many round-key bits, and must not be linear; otherwise knowing part of one round key would give the others by simple algebra, and keys with patterns (like DES's weak keys) would produce repeating round keys.

Mechanism. The first four words are the cipher key itself, taken 4 bytes at a time: $w_0 = k_0k_1k_2k_3$, $w_1 = k_4..k_7$, and so on. Each later word is the word four positions back XORed with the previous word, except that every fourth word passes the previous word through a non-linear function first:

$$w_i = \begin{cases} w_{i-4} \oplus w_{i-1}, & i \bmod 4 \neq 0, \\ w_{i-4} \oplus t_i, \quad t_i = \mathrm{SubWord}\big(\mathrm{RotWord}(w_{i-1})\big) \oplus \mathrm{RCon}[i/4], & i \bmod 4 = 0. \end{cases}$$

- **RotWord** rotates the 4 bytes of a word left by one: $[a_0, a_1, a_2, a_3] \to [a_1, a_2, a_3, a_0]$. It is ShiftRows applied to a single row.
- **SubWord** applies the S-box to each of the 4 bytes. It is SubBytes on a single word and supplies the non-linearity.
- **RCon** (round constant) is a word whose rightmost three bytes are zero and whose left byte is $x^{j-1} \bmod m(x)$ for round j. Each value is 02 times the previous one in GF(2^8), so 01, 02, 04, 08, 10, 20, 40, 80, then $02 \cdot 80$ overflows and is reduced to $00 \oplus 1B = 1B$, and $02 \cdot 1B = 36$. RCon is different in every round, which breaks the symmetry between rounds, so that no key gives identical round keys.

Round j	1	2	3	4	5	6	7	8	9	10
RCon[j]	01	02	04	08	10	20	40	80	1B	36

(each followed by 00 00 00)

Round key K_j is the four words $w_{4j}, w_{4j+1}, w_{4j+2}, w_{4j+3}$, placed as the *columns* of a 4×4 matrix, so that AddRoundKey can XOR column c with w_{4j+c}. For AES-192 and AES-256 the rule is the same with 6 or 8 words per step instead of 4 (t is computed when $i \bmod 6 = 0$ or $i \bmod 8 = 0$); AES-256 additionally applies SubWord alone when $i \bmod 8 = 4$.

WORKED EXAMPLE : first round key (FIPS-197 / Stallings key)

Q. The AES-128 cipher key is 2B7E1516 28AED2A6 ABF71588 09CF4F3C. Find w_4, w_5, w_6, w_7.
A. w_0 = 2B7E1516, w_1 = 28AED2A6, w_2 = ABF71588, w_3 = 09CF4F3C.
$i = 4$ is a multiple of 4, so compute t_4 from w_3:

RotWord(w_3)	09 CF 4F 3C → CF 4F 3C 09
SubWord	S(CF) = 8A (row C, col F), S(4F) = 84, S(3C) = EB, S(09) = 01 ⇒ 8A84EB01
⊕ RCon[1]	8A84EB01 ⊕ 01000000 = t_4 = 8B84EB01

Then

$$w_4 = w_0 \oplus t_4 = \text{2B7E1516} \oplus \text{8B84EB01} = \mathbf{A0FAFE17}$$
$$w_5 = w_1 \oplus w_4 = \text{28AED2A6} \oplus \text{A0FAFE17} = \mathbf{88542CB1}$$
$$w_6 = w_2 \oplus w_5 = \text{ABF71588} \oplus \text{88542CB1} = \mathbf{23A33939}$$
$$w_7 = w_3 \oplus w_6 = \text{09CF4F3C} \oplus \text{23A33939} = \mathbf{2A6C7605}$$

As a matrix (words as columns) the round-1 key is

$$K_1 = \begin{pmatrix} \text{A0} & \text{88} & \text{23} & \text{2A} \\ \text{FA} & \text{54} & \text{A3} & \text{6C} \\ \text{FE} & \text{2C} & \text{39} & \text{76} \\ \text{17} & \text{B1} & \text{39} & \text{05} \end{pmatrix}.$$

Check: these are the FIPS-197 Appendix A values; continuing the same way gives t_8 = 52386BE5 (RotWord(2A6C7605) = 6C76052A, SubWord = 50386BE5, RCon = 02), w_8 = F2C295F2, and finally w_{43} = B6630CA6.

32.8 One complete round: FIPS-197 example

Putting the four steps together on the standard test vector (plaintext `3243F6A8 885A308D 313198A2 E0370734`, the key above) shows what a full round looks like; the mock paper's AES question (Q8) takes its matrices from this trace. The plaintext fills the state column by column, the pre-round XORs it with K_0 (the cipher key), and round 1 then gives:

$$\underbrace{\begin{pmatrix} 19 & A0 & 9A & E9 \\ 3D & F4 & C6 & F8 \\ E3 & E2 & 8D & 48 \\ BE & 2B & 2A & 08 \end{pmatrix}}_{\text{after } K_0} \xrightarrow{\text{Sub}} \begin{pmatrix} D4 & E0 & B8 & 1E \\ 27 & BF & B4 & 41 \\ 11 & 98 & 5D & 52 \\ AE & F1 & E5 & 30 \end{pmatrix} \xrightarrow{\text{Shift}} \begin{pmatrix} D4 & E0 & B8 & 1E \\ BF & B4 & 41 & 27 \\ 5D & 52 & 11 & 98 \\ 30 & AE & F1 & E5 \end{pmatrix} \xrightarrow{\text{Mix}} \begin{pmatrix} 04 & E0 & 48 & 28 \\ 66 & CB & F8 & 06 \\ 81 & 19 & D3 & 26 \\ E5 & 9A & 7A & 4C \end{pmatrix} \xrightarrow{\oplus K_1} \begin{pmatrix} A4 & 68 & 6B & 02 \\ 9C & 9F & 5B & 6A \\ 7F & 35 & EA & 50 \\ F2 & 2B & 43 & 49 \end{pmatrix}$$

After all ten rounds the ciphertext is `3925841D 02DC09FB DC118597 196A0B32`; a Python AES implementation reproduces this and decrypts it back.

32.9 Decryption

Since every step is invertible, decryption applies the inverse steps in reverse order with the round keys in reverse order (K_{N_r} first, K_0 last). In the original design (Forouzan's figure) this gives:

1. AddRoundKey with $K_{N_r} = w_{40}..w_{43}$.
2. Each of the first $N_r - 1$ decryption rounds: InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns.
3. Last decryption round: InvShiftRows, InvSubBytes, AddRoundKey with K_0 (no InvMixColumns).

InvShiftRows rotates right, InvSubBytes uses the inverse S-box, InvMixColumns uses the matrix `[0E 0B 0D 09]` and AddRoundKey is its own inverse. Because InvShiftRows and InvSubBytes commute (one moves bytes, the other changes each byte on its own) and InvMixColumns is linear, the steps can be reordered into the same pattern as encryption (InvSubBytes, InvShiftRows, InvMixColumns, AddRoundKey) if the middle round keys are passed through InvMixColumns; this is the *equivalent inverse cipher*. Unlike DES, AES decryption is a different algorithm from encryption, and InvMixColumns, with its larger constants, is slower than MixColumns.

32.10 Analysis of AES

Brute force. A 128-bit key needs up to 2^{128} trials against DES's 2^{56}, so if DES can be broken in t seconds, AES-128 needs $2^{128}/2^{56} \cdot t = 2^{72}\,t$ seconds. The 192- and 256-bit versions widen the gap further. The best published shortcut (biclique attack, 2011) lowers the AES-128 cost only to about 2^{126}, which is still far out of reach.

No weak keys. DES has keys whose 16 round keys are all equal, so encrypting twice decrypts. In AES, SubWord makes the key schedule non-linear and RCon makes each round's step different, so no such keys are known.

Diffusion and statistical attacks. The steps work together to spread a single change over the whole block in two rounds. In round 1 a changed byte stays one byte through SubBytes and ShiftRows, then MixColumns spreads it to all 4 bytes of its column. In round 2 ShiftRows sends those 4 bytes to 4 different columns, and MixColumns spreads each of them over its column, so all 16 bytes have changed (checked by script: 4 bytes after one round, 16 after two). With 10 or more rounds the ciphertext shows no frequency pattern of the plaintext, and statistical analysis has failed.

Differential and linear cryptanalysis. AES was designed after these attacks were known; the S-box (GF(2^8) inverse) has low differential and linear probabilities, and the branch number of MixColumns forces many active S-boxes over a few rounds, so no such attack works on full AES.

Simplicity and cost. The steps are byte operations, table lookups and XORs, so AES is fast on cheap 8-bit processors with little memory as well as on 32-bit machines (where SubBytes, ShiftRows and MixColumns can be merged into table lookups), and many processors have AES instructions.

32.11 AES versus DES

	DES	AES
Structure	Feistel (round function need not be invertible)	non-Feistel SPN (every step invertible)
Block size	64 bits	128 bits
Key size	56 bits (64 with parity)	128, 192 or 256 bits
Rounds	16	10, 12 or 14
Round key	48 bits, 16 keys	128 bits, $N_r + 1$ keys
Works on	bits (expansion, P-boxes)	bytes in GF(2^8) (state matrix)
Non-linearity	eight 6-to-4-bit S-boxes	one 8-to-8-bit S-box (inverse + affine)
Permutation	bit-level P-boxes	byte-level ShiftRows
Decryption	same algorithm, keys reversed	inverse steps, keys reversed
Brute force	2^{56} (broken in practice)	2^{128} or more
Weak keys	4 weak, 12 semi-weak	none known

Corrections to the notes / slides

- **Example 1, letter X.** The slide writes X as 23. X is letter 23 in *decimal*, which is 17 in hex (the same slide correctly converts M = 12 → 0C and R = 17 → 11). With 17, SubBytes gives F0; the slide's 26 (in Examples 2 and 3) is $S(23)$. If the exam reuses the slide's state, say which value you used.
- **S-box, row B, column 1.** The slide's table prints C8; the correct FIPS-197 entry is $S(B1) = C8$. C8 already appears at row 5, column 9 ($S(59) = C8$), and an S-box is a permutation, so it cannot appear twice. The inverse table on the next slide is right ($\text{InvS}(C8) = B1$). All other 255 entries of the slide match.
- **ShiftRows rows.** The slide numbers the rows 1 to 4 ("Row 1: no shifting, Row 2: 1-byte left shift, ..."), while the pseudocode and these notes number them 0 to 3. Either way the top row is not shifted and the bottom row moves left by 3.

Check your understanding

1. **Why does AES do its arithmetic in GF(2^8) rather than in $\mathbb{Z}_{256}$?** The S-box needs the inverse of every non-zero byte and MixColumns needs an invertible matrix. In $\mathbb{Z}_{256}$ the 128 even bytes have no inverse, while GF(2^8) is a field, so every non-zero byte has one, and addition is plain XOR.
2. **How many rounds, round keys and key words does AES-192 use?** 12 rounds, 12 + 1 = 13 round keys of 128 bits each, so 13 × 4 = 52 words.
3. **What would happen if ShiftRows were removed?** Every other step works within one column, so the four columns would never mix. AES would become four independent 32-bit ciphers, each of which could be attacked separately with a 2^{32}-sized codebook.
4. **Why is AddRoundKey its own inverse, and why is it the only keyed step?** XOR with k twice gives back the input. The other steps are fixed public maps, so all secrecy must enter through AddRoundKey, which is why it also comes before round 1.
5. **Compute $02 \cdot 87$ and $03 \cdot 87$.** 87 = 1000 0111; shifting gives 0000 1110 = 0E, and the top bit was 1, so $02 \cdot 87$ = 0E ⊕ 1B = 15. Then $03 \cdot 87 = 15 \oplus 87 = 92$.
6. **What is SubBytes(00), and why is it not 00?** 00 has no inverse and is mapped to 00, but the affine step then XORs in the constant 63, so $S(00) = 63$. The constant is there precisely so that the S-box has no fixed points.
7. **Why is MixColumns omitted in the last round?** It is linear and keyless, so a final MixColumns could be undone by anyone applying InvMixColumns to the ciphertext (moving it past AddRoundKey only changes the key). It adds no security, and omitting it lets decryption have the same structure as encryption.
8. **Given w_7 = 2A6C7605 and w_4 = A0FAFE17, find w_8.** 8 mod 4 = 0: RotWord gives 6C76052A, SubWord gives 503860E5, XOR RCon[2] = 02000000 gives t_8 = 523860E5, and $w_8 = w_4 \oplus t_8$ = F2C295F2.
9. **Is AES a Feistel cipher? Differentiate.** No. In a Feistel cipher (DES) half the block passes through a round function that need not be invertible and the halves are swapped, so decryption is the same algorithm. AES transforms the whole state each round with invertible steps, so decryption needs the inverse steps.
10. **After how many rounds does every output byte depend on every input byte?** Two: MixColumns in round 1 spreads one byte over its column, and ShiftRows plus MixColumns in round 2 spread those four bytes over all four columns.

33 Asymmetric-Key Cryptography and RSA ▸

▸ **Likely this year:** New this year, so expect one Forouzan-style numerical: "p, q, e given: find n, $\phi(n)$, d; encrypt P; decrypt C" (4–6 marks), plus a "why" question (proof of correctness, why $e = 2$ fails, one attack).

33.1 Why asymmetric-key cryptography exists ▸

▸ **Likely this year:** "Differentiate symmetric-key and asymmetric-key cryptography" / "Why is symmetric cryptography still needed?" (2–3 marks).

In symmetric-key cryptography both parties use the same secret key, so before Alice and Bob can talk privately they must already share a secret. This is the **key distribution problem**: the key has to travel over a secure channel that usually does not exist, and in a group of N users every pair needs its own key, so $N(N-1)/2$ secret keys must be created and stored (4950 for 100 users). Symmetric cryptography also cannot prove who sent a message, because anyone who can decrypt can also encrypt.

Asymmetric-key (public-key) cryptography removes the shared secret. Bob generates a **key pair**: a **public key** that he announces to everyone and a **private key** that only he knows. The public key locks; the private key unlocks. Anyone can encrypt for Bob with his public key, but only his private key decrypts, so Alice needs no earlier contact with Bob and N users need only N key pairs. Formally $C = f(K_{\text{public}}, P)$ and $P = g(K_{\text{private}}, C)$, where f is used only for encryption and g only for decryption, and knowing f must not reveal g. The secret is now carried by the receiver alone; for Bob to reply, he uses *Alice's* public key. Plaintext and ciphertext are treated as **numbers**, and encryption is a number-theoretic function rather than substitution and permutation of symbols.

Asymmetric cryptography does **not** replace symmetric cryptography, because it is far slower: one RSA operation is a modular exponentiation with 1024–2048-bit numbers, while AES needs a few table lookups and XORs per byte (RSA is roughly a thousand times slower). It is therefore used where symmetric cryptography cannot work: **exchanging a secret session key, authentication and digital signatures**. In practice the two are combined: a random AES key is sent encrypted under the receiver's RSA key, and the bulk data is encrypted with AES under that key.

	Symmetric-key	Asymmetric-key
Keys	one shared secret key	public/private key pair
Keys for N users	$N(N-1)/2$	N pairs
Secret held by	both parties	receiver only
Based on	substitution, permutation	number theory (factoring, discrete log)
Speed	fast (bulk data)	slow (keys, signatures)
Examples	DES, AES	RSA, ElGamal, ECC

33.2 Trapdoor one-way functions ▸

▸ **Likely this year:** "Define a trapdoor one-way function; give the RSA example (2 marks).

The design problem is to publish f while keeping f^{-1} hidden. A **one-way function** (OWF) is a function $y = f(x)$ that is easy to compute, while computing $x = f^{-1}(y)$ from y is computationally infeasible. A **trapdoor one-way function** (TOWF) has a third property: given *y and a secret* (the trapdoor), x is easy to compute. A public-key cipher needs exactly this: the public key lets everyone evaluate f, and the trapdoor is the private key.

Multiplication is one-way. For large primes p, q, computing $n = p \times q$ is easy, but recovering p and q from n is the **factorization problem**, for which no polynomial-time (classical) algorithm is known.

$y = x^k \bmod n$ **is a trapdoor one-way function.** Given x, k and n, fast exponentiation (§23) computes y in about $2\log_2 k$ multiplications; given only y, k and n, taking the k-th root modulo n is believed to be as hard as factoring n. The trapdoor is k' with $kk' \equiv 1 \pmod{\phi(n)}$: then $x = y^{k'} \bmod n$. Computing k' needs $\phi(n) = (p-1)(q-1)$, and computing $\phi(n)$ needs p and q, so the trapdoor exists only for whoever chose the primes. RSA is this function with $k = e$ and $k' = d$.

33.3 RSA: key generation, encryption, decryption ▸

▸ **Likely this year:** Write the three algorithms, then run them on small primes.

RSA (Rivest, Shamir, Adleman, 1977) is the most widely used public-key cryptosystem. Bob builds the trapdoor from two primes and publishes only their product; Alice encrypts by raising the plaintext to the public exponent, and Bob decrypts by raising the ciphertext to the private exponent.

★ MUST KNOW : RSA

Key generation (Bob)

1. Choose two large primes $p \neq q$; compute $n = p \times q$ and $\phi(n) = (p-1)(q-1)$.
2. Choose e with $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.
3. Compute $d = e^{-1} \bmod \phi(n)$ with the extended Euclidean algorithm, so $e \times d \equiv 1 \pmod{\phi(n)}$.
4. Public key (e, n), announced. Private key d, kept secret (with p, q, $\phi(n)$).

Encryption (Alice): $C = P^e \bmod n$, with $0 \le P < n$. **Decryption (Bob)**: $P = C^d \bmod n$.

Encryption and decryption are each one fast exponentiation, so both are polynomial in the size of n. Eve sees C, e and n and must compute $P = \sqrt[e]{C} \bmod n$, which is exponential for her unless she can factor n. The plaintext must be smaller than n, because the result is only known modulo n: two plaintexts that differ by n give the same ciphertext. A long message is cut into blocks, each a number less than n.

33.4 Why decryption works: proof of correctness ▸

▸ **Likely this year:** "Prove that RSA decryption recovers the plaintext" (2–3 marks).

Since d is the inverse of e modulo $\phi(n)$, $ed = 1 + k\,\phi(n)$ for some integer k. Decryption computes

$$C^d \equiv (P^e)^d = P^{ed} = P^{1+k\phi(n)} = P \cdot \left(P^{\phi(n)}\right)^k \pmod{n}.$$

Case 1: $\gcd(P, n) = 1$ (almost every P). Euler's theorem (§27) gives $P^{\phi(n)} \equiv 1 \pmod{n}$, so $C^d \equiv P \cdot 1^k = P \pmod{n}$. Since $0 \le P < n$, the reduced result is exactly P, not just something congruent to it.

Case 2: $\gcd(P, n) \neq 1$. Euler's theorem does not apply, but the result still holds. Since $0 < P < pq$, P is a multiple of exactly one prime, say $p \mid P$ and $q \nmid P$. Modulo p, $P \equiv 0$, so $P^{ed} \equiv 0 \equiv P$. Modulo q, Fermat's little theorem gives $P^{q-1} \equiv 1$, and $ed = 1 + k(p-1)(q-1)$, so $P^{ed} = P \cdot (P^{q-1})^{k(p-1)} \equiv P \pmod{q}$. Since $P^{ed} \equiv P$ modulo both p and q, and p, q

are coprime, the CRT (§28) gives $P^{ed} \equiv P \pmod{pq}$. (This is Euler's "second version", $P^{k\phi(n)+1} \equiv P \pmod{n}$ for $n = pq$ and every P.) Check: $n = 221 = 13 \cdot 17$, $e = 5$, $d = 77$, $P = 65 = 5 \cdot 13$: $65^5 \bmod 221 = 182$ and $182^{77} \bmod 221 = 65$ ✓. Such a P is no practical concern, because finding one would give $\gcd(P, n) = p$ and factor n.

The two algebraic structures. RSA works in two places (§29). Encryption and decryption use the **public ring** $R = \langle \mathbb{Z}_n, +, \times \rangle$: P and C can be any element of $\mathbb{Z}_n$ (Case 2 shows non-invertible elements also work), and exponentiation is repeated multiplication, which anyone can do since n is public. Key generation uses the **private group** $G = \langle \mathbb{Z}^*_{\phi(n)}, \times \rangle$: e must belong to this group, i.e. $\gcd(e, \phi(n)) = 1$, so that its inverse d exists in it. Only Bob can build G, because knowing $\phi(n)$ is as good as knowing the factors: $p+q = n-\phi(n)+1$, and p, q are the roots of $x^2-(p+q)x+n = 0$. For $n = 187$, $\phi = 160$: $p + q = 28$, $x^2 - 28x + 187 = 0$ gives $x = (28 \pm 6)/2 = 17, 11$.

33.5 The slide examples ▸

▸ **Likely this year:** Verify one row by hand, e.g. "with $n = 77$, $e = 13$, find d and encrypt 5.

The lecture uses the key $p = 7$, $q = 11$, $e = 13$: $n = 77$, $\phi(n) = 60$, and $d = 37$ because $13 \times 37 = 481 = 8 \cdot 60 + 1$. Alice encrypts 5 and John encrypts 63 with the same public key $(13, 77)$; Bob decrypts both with $d = 37$. The table also has the class homework key ($n = 187$) and a small textbook key ($n = 33$); all values were recomputed.

p, q	n	$\phi(n)$	e	d	Encrypt	Decrypt
7, 11	77	60	13	37	$5^{13} \bmod 77 = 26$	$26^{37} \bmod 77 = 5$
7, 11	77	60	13	37	$63^{13} \bmod 77 = 28$	$28^{37} \bmod 77 = 63$
17, 11	187	160	7	23	$88^7 \bmod 187 = 11$	$11^{23} \bmod 187 = 88$
3, 11	33	20	7	3	$5^7 \bmod 33 = 14$	$14^3 \bmod 33 = 5$

For the slide key, the d-table (§25) on $(60, 13)$ has quotients $4, 1, 1, 1, 1, 2$ and ends with $r_1 = 1$, $t_1 = -23$, so $d = -23 + 60 = 37$. A negative t_1 is normal; add $\phi(n)$, never n.

33.6 Exam-format RSA numericals ▸

▸ **Likely this year:** Exactly this layout: $\phi(n)$, gcd check, d-table, two square-and-multiply tables, a check line.

Every exponentiation below uses repeated squaring (§23): write the exponent as a sum of powers of 2, square repeatedly (reducing mod n each time), then multiply the needed powers, reducing after each product.

WORKED EXAMPLE : Stallings/Forouzan, the class homework key

Q. In RSA, $p = 17$, $q = 11$, $e = 7$. Find n, $\phi(n)$ and the private key d. Encrypt $P = 88$ and show that decryption recovers it.
Keys. $n = 17 \times 11 = 187$; $\phi(n) = 16 \times 10 = 160$. Since $160 = 2^5 \cdot 5$ and 7 is prime, $\gcd(7, 160) = 1$, so $e = 7$ is valid. $d = 7^{-1} \bmod 160$:

q	r_1	r_2	r	t_1	t_2	t
22	160	7	6	0	1	−22
1	7	6	1	1	−22	23
6	6	1	0	−22	23	−160
	1	0		**23**	−160	

$r_1 = 1$ (inverse exists), $d = t_1 = \mathbf{23}$. Check: $7 \times 23 = 161 = 160 + 1$ ✓. Public key $(7, 187)$, private key 23.
Encrypt $C = 88^7 \bmod 187$, with $7 = 4 + 2 + 1$:

Power	Computation	mod 187
88^1		88
88^2	$88^2 = 7744 = 41 \cdot 187 + 77$	77
88^4	$77^2 = 5929 = 31 \cdot 187 + 132$	132

$88^7 = 88^4 \cdot 88^2 \cdot 88^1$: $132 \cdot 77 = 10164 \equiv 66$; $66 \cdot 88 = 5808 \equiv 11$. $\Rightarrow C = \mathbf{11}$.
Decrypt $P = 11^{23} \bmod 187$, with $23 = 16 + 4 + 2 + 1$:

Power	Computation	mod 187
11^1		11
11^2	$11^2 = 121$	121
11^4	$121^2 = 14641 = 78 \cdot 187 + 55$	55
11^8	$55^2 = 3025 = 16 \cdot 187 + 33$	33
11^{16}	$33^2 = 1089 = 5 \cdot 187 + 154$	154

$154 \cdot 55 = 8470 \equiv 55$; $55 \cdot 121 = 6655 \equiv 110$; $110 \cdot 11 = 1210 \equiv 88$. $\Rightarrow P = \mathbf{88}$ ✓ (the original plaintext).

WORKED EXAMPLE : new, Forouzan exercise style (only n given)

Q. Bob's RSA public key is $(e, n) = (5, 221)$. (a) Find his private key. (b) Alice sends $P = 100$; find C. (c) Show Bob's decryption.

(a) Eve cannot factor a 1024-bit n, but 221 factors by trial division: $221 = 13 \times 17$. So $\phi(n) = 12 \times 16 = 192 = 2^6 \cdot 3$, and $\gcd(5, 192) = 1$. $d = 5^{-1} \bmod 192$:

q	r_1	r_2	r	t_1	t_2	t
38	192	5	2	0	1	−38
2	5	2	1	1	−38	77
2	2	1	0	−38	77	−192
	1	0		**77**	−192	

$d = \mathbf{77}$. Check: $5 \times 77 = 385 = 2 \cdot 192 + 1$ ✓.

(b) $5 = 4 + 1$: $100^2 = 10000 = 45 \cdot 221 + 55$; $55^2 = 3025 = 13 \cdot 221 + 152$, so $100^4 \equiv 152$. $C = 152 \cdot 100 = 15200 = 68 \cdot 221 + 172 \Rightarrow C = \mathbf{172}$.

(c) $P = 172^{77} \bmod 221$, with $77 = 64 + 8 + 4 + 1$:

Power	Computation	mod 221
172^1		172
172^2	$172^2 = 29584 = 133 \cdot 221 + 191$	191
172^4	$191^2 = 36481 = 165 \cdot 221 + 16$	16
172^8	$16^2 = 256 = 221 + 35$	35
172^{16}	$35^2 = 1225 = 5 \cdot 221 + 120$	120
172^{32}	$120^2 = 14400 = 65 \cdot 221 + 35$	35
172^{64}	$35^2 \equiv 120$ (as above)	120

$172^{64} \cdot 172^8 = 120 \cdot 35 = 4200 = 19 \cdot 221 + 1 \equiv 1$; $1 \cdot 16 = 16$; $16 \cdot 172 = 2752 = 12 \cdot 221 + 100$. $\Rightarrow P = \mathbf{100}$ ✓.

33.7 Choosing e; why $e = 2$ never works ▸

▸ **Likely this year:** "Why is $e = 2$ not allowed? or "is $e = 3$ valid for $p = 19$, $q = 23$? (1–2 marks).

Encryption cost depends on the number of bits and of 1-bits in e, so a small e with few 1-bits makes encryption cheap. The standard choice is $e = 2^{16} + 1 = 65537$: it is prime (so it is coprime to $\phi(n)$ unless a prime factor of $\phi(n)$ is 65537 itself), its binary form 1 0000 0000 0000 0001 needs only 16 squarings and one multiplication, and it is large enough to defeat the low-exponent attacks below. The private exponent d is then about as long as n, so decryption is the slow side (which the CRT speeds up).

$e = 2$ **(or any even e) is invalid.** Since p and q are odd primes, $p - 1$ and $q - 1$ are even, so $\phi(n)$ is even (in fact divisible by 4). Then $\gcd(2, \phi(n)) = 2 \neq 1$, e is not in $\mathbb{Z}^*_{\phi(n)}$, and no d exists. The cause is that squaring is not one-to-one: x and $n - x$ have the same square, so $C = P^2$ cannot be uniquely undone. The same check catches a Forouzan exercise: $p = 19$, $q = 23$, $e = 3$ gives $\phi(n) = 18 \cdot 22 = 396 = 3 \cdot 132$, so $\gcd(3, 396) = 3$ and $e = 3$ is not allowed for this n.

Trap: the usual RSA slips

d is the inverse of e modulo $\phi(n)$, never modulo n (for $(7, 187)$ the answer is 23; $7^{-1} \bmod 187 = 107$ is useless). Check $\gcd(e, \phi(n)) = 1$ before running the table. The plaintext must satisfy $P < n$. Encrypt with the *receiver's* public key. Reduce mod n after every multiplication, and turn a negative t_1 into $t_1 + \phi(n)$.

33.8 Faster decryption with the CRT ▸

▸ **Likely this year:** "Decrypt using the Chinese remainder theorem" on a small key, or "why is CRT decryption about 4 times faster".

Decryption is slow because d and n are both full size. Bob, however, knows p and q, so he can work modulo each prime separately and combine the answers with the CRT (§28). Modulo p the exponent can be reduced: since $C^{p-1} \equiv 1 \pmod p$ (Fermat), $C^d \equiv C^{d \bmod (p-1)} \pmod p$. Bob computes

$$d_p = d \bmod (p - 1), \quad d_q = d \bmod (q - 1), \qquad P_p = C^{d_p} \bmod p, \quad P_q = C^{d_q} \bmod q,$$

and then the unique $P < n$ with $P \equiv P_p \pmod p$ and $P \equiv P_q \pmod q$:

$$P = \left(P_p \cdot q \cdot (q^{-1} \bmod p) + P_q \cdot p \cdot (p^{-1} \bmod q)\right) \bmod n.$$

Each half uses a modulus of half the length (a multiplication costs about 1/4) and an exponent of half the

length (half as many steps), so each half costs about 1/8 of the full exponentiation and both together about 1/4: decryption is roughly **4 times faster**. Only the key owner can use it, since it needs p and q.

WORKED EXAMPLE : CRT decryption of the previous ciphertext

Q. With $p = 13$, $q = 17$, $d = 77$, decrypt $C = 172$ using the CRT.
$d_p = 77 \bmod 12 = 5$, $d_q = 77 \bmod 16 = 13$; $C_p = 172 \bmod 13 = 3$, $C_q = 172 \bmod 17 = 2$.
$P_p = 3^5 \bmod 13 = 243 \bmod 13 = 9$. $P_q = 2^{13} \bmod 17$: $2^4 = 16 \equiv -1$, $2^8 \equiv 1$, so $2^{13} = 2^8 \cdot 2^4 \cdot 2 \equiv -2 \equiv 15$.
$q^{-1} \bmod p = 17^{-1} \bmod 13 = 4^{-1} \bmod 13 = 10$ ($4 \cdot 10 = 40 = 3 \cdot 13 + 1$); $p^{-1} \bmod q = 13^{-1} \bmod 17 = 4$ ($13 \cdot 4 = 52 = 3 \cdot 17 + 1$).
$P = 9 \cdot 17 \cdot 10 + 15 \cdot 13 \cdot 4 = 1530 + 780 = 2310 = 10 \cdot 221 + 100 \Rightarrow P =$ **100** ✓, the same as $172^{77} \bmod 221$, with exponents 5 and 13 instead of 77.

33.9 Attacks on RSA ▸

▸ **Likely this year:** "Explain any two attacks on RSA" or "why must p and q be large / not close / why OAEP".

No devastating attack on correctly used RSA is known; every known attack exploits a weak choice of parameters or a careless use of textbook RSA. The slide's list, each with the reason it works:

- **Factorization.** If Eve factors n she computes $\phi(n)$ and then $d = e^{-1} \bmod \phi(n)$, exactly as Bob did. The whole security rests on n being too large to factor.
- **Chosen-ciphertext.** RSA is multiplicative: $(P_1P_2)^e = P_1^e P_2^e$. To decrypt C, Eve picks X, asks Bob to decrypt the innocent-looking $Y = C \cdot X^e \bmod n$, receives $Z = Y^d = P \cdot X$, and computes $P = Z \cdot X^{-1}$. With $(7, 187)$, $C = 11$, $X = 2$: $Y = 11 \cdot 128 \bmod 187 = 99$, $Z = 99^{23} \bmod 187 = 176$, $P = 176 \cdot 94 \bmod 187 = 88$ ✓.
- **Low encryption exponent.** If e is small and $P^e < n$, no reduction happens and $P = \sqrt[e]{C}$ over the integers ($e = 3$, $n = 187$, $P = 5$: $C = 125$, $\sqrt[3]{125} = 5$). **Coppersmith's** theorem extends this to finding small unknown parts of P. **Broadcast**: if the same P goes to e receivers with moduli $n_1, \ldots, n_e$, the CRT gives $P^e \bmod n_1 \cdots n_e$, which equals P^e because $P^e < n_1 \cdots n_e$, so an integer root reveals P (with $e = 3$, $n = 55, 391, 1189$ and $P = 42$: ciphertexts $3, 189, 370$, CRT gives $74088 = 42^3$). **Related messages** (Franklin–Reiter) and **short pad** attacks similarly recover messages that are linearly related or padded with too few random bits.
- **Low decryption exponent.** A small d speeds decryption but Wiener's attack recovers d from e/n by continued fractions when $d < \frac{1}{3}n^{1/4}$. A **revealed** d is worse than it looks: n with one pair (e, d) lets anyone factor n, so all keys on that modulus are dead.
- **Plaintext attacks.** *Short message*: if P comes from a small set (a PIN), Eve encrypts every candidate with the public key and compares, since textbook RSA is deterministic. *Cyclic*: C is a permutation of $\mathbb{Z}_n$, so re-encrypting $C, C^e, C^{e^2}, \ldots$ eventually returns to C, and the value just before C is P; with $(7, 33)$ and $C = 14$: $14 \to 20 \to 26 \to 5 \to 14$, so $P = 5$. *Unconcealed*: some P satisfy $P^e \equiv P$ (always 0, 1, $n - 1$); their number is $(1 + \gcd(e - 1, p - 1))(1 + \gcd(e - 1, q - 1))$, which for the slide key $(13, 77)$ is $7 \cdot 3 = 21$ messages (e.g. $10^{13} \equiv 10$), because $e - 1 = 12$ is a multiple of $p - 1 = 6$.
- **Common modulus.** If two users share n with exponents e_1, e_2, $\gcd(e_1, e_2) = 1$, and the same P is sent to both, Eve finds $se_1 + te_2 = 1$ and gets $C_1^s C_2^t = P^{se_1 + te_2} = P$. With $n = 187$, $e_1 = 3$, $e_2 = 7$, $P = 5$: $C_1 = 125$, $C_2 = 146$, $s = -2$, $t = 1$, $P = (125^{-1})^2 \cdot 146 = 3^2 \cdot 146 = 1314 \equiv 5$ ✓. Also, any user of the shared n can factor it from his own (e, d).
- **Implementation (timing, power).** Square-and-multiply does an extra multiplication for every 1-bit of d, so the decryption time or power trace leaks the bits of d. Defences: constant-time exponentiation, random delays, or *blinding* (decrypt $C \cdot r^e$, then multiply by r^{-1}).

33.10 Recommendations

Each recommendation blocks one of the attacks above.

- n at least **1024 bits** (about 309 decimal digits; 2048 today), so each of p, q at least **512 bits** (about 154 digits). This defeats factorization.
- p and q must not be close to each other, otherwise Fermat factoring ($n = a^2 - b^2$ with $a \approx \sqrt{n}$) finds them quickly. Both $p - 1$ and $q - 1$ should have a large prime factor (against Pollard's $p - 1$ method), and p/q should not be close to a fraction with small numerator and denominator.
- The modulus must not be shared between users (common-modulus attack).
- $e = 2^{16} + 1$ or an integer close to it (against low-exponent attacks, while keeping encryption fast).
- If d leaks, Bob must change n, e and d, since one (e, d) pair reveals the factors of n.
- Messages must be padded with **OAEP** (Optimal Asymmetric Encryption Padding). OAEP mixes the message with a fresh random value r through two hash-based masks ($M_1 = P \oplus G(r)$, $M_2 = r \oplus H(M_1)$) and encrypts $M_1 \| M_2$. This makes RSA probabilistic (same P, different C each time), fills the block so $P^e > n$, and destroys the multiplicative structure, blocking the short-message, broadcast, cyclic-style and chosen-ciphertext tricks.

Corrections to the notes / slides

- Slide on trapdoor functions: "there is not a polynomial time solution" to factoring should read *no polynomial-time classical algorithm is known*; it is not proven impossible (Shor's quantum algorithm is polynomial).
- Slide on RSA size: a 512-bit prime has 154 or 155 decimal digits (2^{511} has 154, 2^{512} has 155); 1024 bits is 309 digits, as stated.

Check your understanding

1. **Differentiate symmetric and asymmetric key cryptography.** Symmetric uses one shared secret key for both directions, needs $N(N-1)/2$ keys and a secure key exchange, and is fast. Asymmetric uses a public/private pair per user, needs only N pairs and no prior secret, rests on number-theoretic problems, and is slow, so it is used for key exchange and signatures.
2. **Why is $e = 2$ invalid?** $\phi(n) = (p-1)(q-1)$ is even, so $\gcd(2, \phi(n)) \neq 1$ and 2 has no inverse modulo $\phi(n)$; squaring maps x and $n-x$ to the same value, so it cannot be inverted uniquely.
3. **Why is d computed modulo $\phi(n)$ and not n?** Exponents of an element of $\mathbb{Z}_n^*$ repeat with period dividing $\phi(n)$ (Euler), so $P^{ed} = P$ needs $ed \equiv 1 \pmod{\phi(n)}$; a congruence modulo n says nothing about exponents.
4. **Why must $P < n$?** Results are only defined modulo n, so P and $P+n$ encrypt to the same C and decryption returns $P \bmod n$.
5. **Given $n = 187$ and $e = 17$, find d.** $187 = 11 \cdot 17$, $\phi = 160$, $17^{-1} \bmod 160 = 113$ ($17 \cdot 113 = 1921 = 12 \cdot 160 + 1$).
6. **Given $n = 3937$ and $e = 17$, find d.** $3937 = 31 \cdot 127$, $\phi = 30 \cdot 126 = 3780$, $d = 3113$ ($17 \cdot 3113 = 52921 = 14 \cdot 3780 + 1$).
7. **Why does knowing $\phi(n)$ break RSA?** It gives d directly, and also $p+q = n - \phi(n) + 1$, so p, q are roots of $x^2 - (p+q)x + n$.
8. **Why is RSA not used to encrypt bulk data?** It is about a thousand times slower than AES and textbook RSA is deterministic; it encrypts a session key instead (hybrid encryption).
9. **What does OAEP add?** Randomness and structure: the same message encrypts differently each time and malleability is destroyed, which stops short-message, broadcast and chosen-ciphertext attacks.
10. **Why is CRT decryption only for Bob?** It needs p and q, which only the key owner knows; the gain is about a factor of 4.

34 ElGamal and Elliptic Curve Cryptosystems ▸

▸ **Likely this year:** New this year: an ElGamal encrypt/decrypt with a small prime, "find all points on $E_p(a, b)$", one point addition or doubling, and "why must r be fresh / why ECC has smaller keys".

34.1 ElGamal cryptosystem ▸

▸ **Likely this year:** "Explain ElGamal key generation, encryption and decryption; prove decryption is correct; encrypt P with p, e_1, d, r given" (4–5 marks).

RSA rests on factoring. ElGamal rests on a different hard problem, the **discrete logarithm problem** (DLP, §28). Let p be a large prime and e_1 a **primitive root** of p, i.e. an element whose powers $e_1^1, e_1^2, \ldots, e_1^{p-1}$ run through all of $\mathbb{Z}_p^* = \{1, \ldots, p-1\}$ (§27). Computing $e_2 = e_1^d \bmod p$ is easy by square-and-multiply, but given e_1, e_2 and p, finding $d = \log_{e_1} e_2 \bmod p$ is infeasible for large p. So $d \mapsto e_1^d$ is a one-way function, and Bob's private key is the exponent d hidden inside the public e_2.

The idea of encryption is to multiply P by a **one-time mask** e_2^r that only Bob can rebuild. Alice picks a fresh random r and sends two numbers: $C_1 = e_1^r$, a "hint" that reveals nothing about r (DLP again), and $C_2 = P \cdot e_2^r$, the masked plaintext. Bob rebuilds the mask from the hint with his secret: $C_1^d = e_1^{rd} = (e_1^d)^r = e_2^r$. Eve would need either d or r, and both are discrete logarithms. This is the Diffie–Hellman idea turned into a cipher: $e_2^r = e_1^{dr}$ is a shared value that Alice computes from (r, e_2) and Bob from (d, C_1).

★ MUST KNOW : ElGamal

Key generation (Bob): choose a large prime p and a primitive root e_1 of p; choose d with $1 \le d \le p-2$; compute $e_2 = e_1^d \bmod p$. Public key (e_1, e_2, p), private key d.
Encryption (Alice): choose a **fresh** random $r \in \mathbb{Z}_p^*$; $C_1 = e_1^r \bmod p$, $C_2 = (P \times e_2^r) \bmod p$. Send (C_1, C_2).
Decryption (Bob): $P = \left[C_2 \times (C_1^d)^{-1}\right] \bmod p$, and $(C_1^d)^{-1} = C_1^{p-1-d} \bmod p$.

Proof of correctness. $C_2 \times (C_1^d)^{-1} = (P \cdot e_2^r)(e_1^{rd})^{-1} = P \cdot e_2^r (e_2^r)^{-1} = P \pmod p$, because $e_1^{rd} = (e_1^d)^r = e_2^r$. The inverse exists since $C_1^d \in \mathbb{Z}_p^*$.

The inverse shortcut. By Fermat, $C_1^{p-1} \equiv 1$, so $C_1^d \cdot C_1^{p-1-d} \equiv 1$, i.e. $(C_1^d)^{-1} = C_1^{p-1-d}$. One exponentiation replaces an exponentiation followed by an extended-Euclid inverse.

Properties. ElGamal is **probabilistic**: the same P encrypted twice gives different ciphertexts because r changes,

so Eve cannot test guesses by re-encrypting them (unlike textbook RSA). The price is **ciphertext expansion**: the ciphertext is the pair (C_1, C_2), each as long as p, so it is **twice the size** of the plaintext. Like RSA, it is slow and is used for short data such as session keys.

WORKED EXAMPLE : the slide example

Q. Bob chooses $p = 11$, $e_1 = 2$, $d = 3$. Find his public key. Alice encrypts $P = 7$ with $r = 4$. Find the ciphertext and show Bob's decryption.
Keys. 2 is a primitive root of 11 (its powers are $2, 4, 8, 5, 10, 9, 7, 3, 6, 1$). $e_2 = 2^3 \bmod 11 = 8$. Public key $(2, 8, 11)$, private key 3.
Encrypt. $C_1 = 2^4 \bmod 11 = 16 \bmod 11 = 5$. $e_2^r = 8^4 = 4096 = 372 \cdot 11 + 4$, so $C_2 = 7 \cdot 4 = 28 \bmod 11 = 6$. Ciphertext $(\mathbf{5}, \mathbf{6})$.
Decrypt. $C_1^d = 5^3 = 125 = 11 \cdot 11 + 4 \equiv 4$; $4^{-1} \bmod 11 = 3$ $(4 \cdot 3 = 12)$. $P = 6 \cdot 3 = 18 \bmod 11 = \mathbf{7}$ ✓.
Shortcut. $(C_1^d)^{-1} = 5^{11-1-3} = 5^7$: $5^2 \equiv 3$, $5^4 \equiv 9$, $5^7 = 9 \cdot 3 \cdot 5 = 135 = 12 \cdot 11 + 3 \equiv 3$ ✓.

WORKED EXAMPLE : new exam-style example

Q. In ElGamal, $p = 23$, $e_1 = 5$, Bob's private key $d = 6$. (a) Find e_2. (b) Encrypt $P = 20$ with $r = 3$. (c) Decrypt, and verify with the shortcut.
(a) $5^2 = 25 \equiv 2$, $5^4 \equiv 4$, $e_2 = 5^6 = 5^4 \cdot 5^2 \equiv 4 \cdot 2 = \mathbf{8}$. Public key $(5, 8, 23)$. (5 is a primitive root of 23: its order is 22.)
(b) $C_1 = 5^3 = 125 = 5 \cdot 23 + 10 \Rightarrow C_1 = 10$. $e_2^r = 8^3 = 512 = 22 \cdot 23 + 6 \equiv 6$. $C_2 = 20 \cdot 6 = 120 = 5 \cdot 23 + 5 \Rightarrow C_2 = 5$. Ciphertext $(\mathbf{10}, \mathbf{5})$.
(c) $C_1^d = 10^6$: $10^2 = 100 \equiv 8$, $10^4 \equiv 64 \equiv 18$, $10^6 = 18 \cdot 8 = 144 \equiv 6$ (this is e_2^r again, as the proof says). $6^{-1} \bmod 23 = 4$ $(6 \cdot 4 = 24)$. $P = 5 \cdot 4 = \mathbf{20}$ ✓.
Shortcut: $10^{23-1-6} = 10^{16}$: $10^8 = 18^2 = 324 = 14 \cdot 23 + 2 \equiv 2$, $10^{16} \equiv 4$ ✓ (the same inverse).

34.2 Security of ElGamal ▸

▸ **Likely this year:** "Why must Alice use a fresh r for every message?" with the algebra.

Low-modulus attack. If p is small, Eve solves the discrete logarithm $d = \log_{e_1} e_2$ once (by search or a sub-exponential algorithm), stores d, and decrypts every message sent to Bob for as long as he keeps the key. With the slide key she tries $2^1 = 2$, $2^2 = 4$, $2^3 = 8 = e_2$ and has $d = 3$. Hence p must be at least 1024 bits (about 309 digits).

Known-plaintext attack when r is reused. The mask e_2^r depends only on r, so two messages encrypted with the same r share the same mask. If Eve learns one plaintext P and sees both ciphertexts $C_2 = P \cdot e_2^r$ and $C_2' = P' \cdot e_2^r$, she removes the mask without knowing d or r:

$$e_2^r = C_2 \times P^{-1} \bmod p, \qquad P' = C_2' \times (e_2^r)^{-1} \bmod p.$$

In the example above ($p = 23$, $e_2 = 8$, $r = 3$), suppose Alice also sends $P' = 17$ with the same r: $C_2' = 17 \cdot 6 = 102 \equiv 10$. Eve, knowing $P = 20$ and $C_2 = 5$: $20^{-1} \bmod 23 = 15$, so $e_2^r = 5 \cdot 15 = 75 \equiv 6$; then $P' = 10 \cdot 6^{-1} = 10 \cdot 4 = 40 \equiv 17$ ✓. The repeated C_1 even tells Eve that r was reused. Alice must use a fresh random r for every message.

	RSA	ElGamal
Hard problem	factoring $n = pq$	discrete logarithm mod p
Public / private key	(e, n) / d	(e_1, e_2, p) / d
Encryption	deterministic, $C = P^e$	probabilistic, fresh r each time
Ciphertext size	same as plaintext	two numbers: twice the plaintext
Key generation	two primes, $d = e^{-1} \bmod \phi(n)$	one prime, $e_2 = e_1^d$
Main misuse	small e, no padding, shared n	reused r, small p

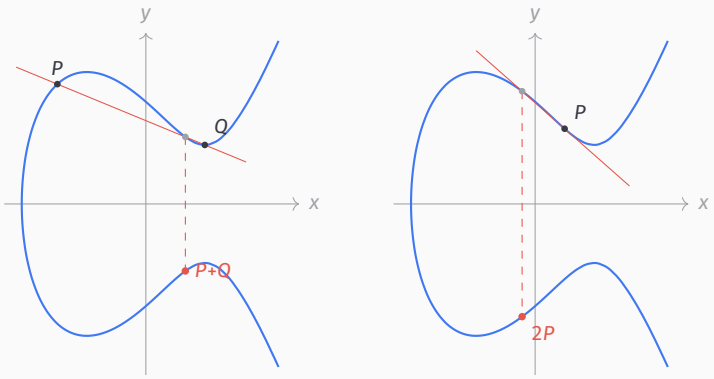
34.3 Elliptic curves over the real numbers

RSA and ElGamal are secure, but their security costs large keys (1024 bits and more). **Elliptic curve cryptography** (ECC) gets the same security with much smaller keys by replacing the group $\mathbb{Z}_p^*$ with the group of points on an elliptic curve, where the discrete logarithm is harder.

An **elliptic curve** is the set of points (x, y) satisfying the cubic $y^2 = x^3 + ax + b$. It is symmetric about the x-axis, since $(x, -y)$ satisfies the equation whenever (x, y) does. The curve must be **non-singular**, meaning $x^3 + ax + b$ has three distinct roots, which is equivalent to $4a^3 + 27b^2 \neq 0$. A singular curve has a cusp or a self-crossing where the tangent is undefined, and the group law below breaks there; for example $a = -3$, $b = 2$ gives $4(-27) + 27 \cdot 4 = 0$ and $x^3 - 3x + 2 = (x - 1)^2(x + 2)$ has a double root. The slide's curve $y^2 = x^3 - 3x + 3$ has $4a^3 + 27b^2 = -108 + 243 = 135 \neq 0$ and one real root, so it is a single connected curve.

Adding points (chord-and-tangent rule). To add $P \neq Q$, draw the line through P and Q; a line meets a cubic in three points, so it cuts the curve at a third point, and $R = P + Q$ is the **reflection** of that third point in the x-axis. To

double P, use the **tangent** at P (the line through P and a point infinitely close to it), take its second intersection and reflect it. If the line is vertical ($Q = -P$, the reflection of P), there is no third point; we say it meets the curve at the **point at infinity** O, which is the identity: $P + (-P) = O$, $P + O = P$. With this rule the points form an abelian group (§29).



addition: chord doubling: tangent

Deriving the formulas. Let the line through $P = (x_1, y_1)$ have slope s: $y = s(x - x_1) + y_1$. Substituting into $y^2 = x^3 + ax + b$ gives a cubic in x whose expanded form starts $x^3 - s^2x^2 + \cdots = 0$. Its three roots are the x-coordinates of the three intersection points, and the sum of the roots of a monic cubic is minus the x^2 coefficient, so $x_1 + x_2 + x_3 = s^2$, giving $x_3 = s^2 - x_1 - x_2$. The third point is $(x_3,\ s(x_3 - x_1) + y_1)$; reflecting it gives $y_3 = s(x_1 - x_3) - y_1$. For addition, s is the chord slope $(y_2 - y_1)/(x_2 - x_1)$. For doubling, s is the tangent slope: differentiating $y^2 = x^3 + ax + b$ implicitly gives $2y\,y' = 3x^2 + a$, so $s = (3x_1^2 + a)/(2y_1)$, and $x_2 = x_1$. The constant b vanishes on differentiation, which is why the doubling slope contains a and never b.

34.4 Elliptic curves over GF(p) ▸

▸ **Likely this year:** Find all the points on $E_{13}(1, 1)$ / $E_{11}(1, 6)$" and find the inverse of a point" (3–4 marks).

Cryptography cannot use real numbers (rounding errors, unbounded size), so the same equation is taken modulo a prime $p > 3$. The curve $E_p(a, b)$ is the set of all pairs $(x, y) \in \mathbb{Z}_p \times \mathbb{Z}_p$ with

$$y^2 \equiv x^3 + ax + b \pmod{p}, \qquad a, b \in \mathbb{Z}_p, \quad 4a^3 + 27b^2 \not\equiv 0 \pmod{p},$$

together with the point at infinity O. The curve" is now a finite scatter of points, but the addition formulas derived above still define a group, with every division done as multiplication by an inverse modulo p.

Finding the points. For each $x = 0, 1, \ldots, p - 1$, compute $w = (x^3 + ax + b) \bmod p$, which must equal y^2. If $w = 0$ there is one point $(x, 0)$. If w is a nonzero **quadratic residue** (a perfect square mod p), there are two points $(x, \sqrt{w})$ and $(x, p - \sqrt{w})$. If w is not a square, no point has that x. Make the table of squares y^2 mod p first; since $(p - y)^2 \equiv y^2$, only $y = 0, \ldots, (p - 1)/2$ are needed.

Negatives. The inverse of $P = (x, y)$ is $-P = (x, -y) = (x, p - y)$: the same x, only y flipped (the reflection in the x-axis). A point with $y = 0$ is its own inverse. Points therefore come in vertical pairs $(x, y), (x, p - y)$.

Hasse's theorem. The number of points $\#E$ (including O) satisfies $p + 1 - 2\sqrt{p} \le \#E \le p + 1 + 2\sqrt{p}$: about half the x values give a square and those give two points each, so #E is close to $p + 1$. It is a quick check on a point count.

WORKED EXAMPLE : Forouzan/slide, all points of $E_{13}(1, 1)$

Q. Find all points on $E_{13}(1, 1)$: $y^2 = x^3 + x + 1 \pmod{13}$. List the inverse pairs.
Valid curve: $4 + 27 = 31 \equiv 5 \not\equiv 0 \pmod{13}$. **Squares mod 13:** $y = 0..6$ give $y^2 = 0, 1, 4, 9, 3, 12, 10$, so the nonzero squares are $\{1, 3, 4, 9, 10, 12\}$ with roots $1 \to \pm1$, $3 \to \pm4$, $4 \to \pm2$, $9 \to \pm3$, $10 \to \pm6$, $12 \to \pm5$.

x	x^3+x+1	w	square?	points
0	1	1	yes	(0, 1), (0, 12)
1	3	3	yes	(1, 4), (1, 9)
2	11	11	no	–
3	31	5	no	–
4	69	4	yes	(4, 2), (4, 11)
5	131	1	yes	(5, 1), (5, 12)
6	223	2	no	–
7	351	0	$y=0$	(7, 0)
8	521	1	yes	(8, 1), (8, 12)
9	739	11	no	–
10	1011	10	yes	(10, 6), (10, 7)
11	1343	4	yes	(11, 2), (11, 11)
12	1741	12	yes	(12, 5), (12, 8)

17 points + O = 18. Each row's two points are inverses (e.g. $4+9=13$, so $(1,9)=-(1,4)$); $(7,0)$ is its own inverse. $x=2,3,6,9$ give non-squares. Hasse: $14\pm2\sqrt{13}=[6.8,\ 21.2]$ contains 18 ✓.

WORKED EXAMPLE : new, all points of $E_{11}(1,6)$

Q. Find all points on $y^2=x^3+x+6 \pmod{11}$ and verify Hasse's bound.
Valid: $4+27\cdot36=976=88\cdot11+8\not\equiv0$. **Squares mod 11:** $y=0..5$ give $0,1,4,9,5,3$; nonzero squares $\{1,3,4,5,9\}$ with roots $1\to\pm1$, $3\to\pm5$, $4\to\pm2$, $5\to\pm4$, $9\to\pm3$.

x	x^3+x+6	w	sq?	points	x	x^3+x+6	w	sq?	points
0	6	6	no	–	6	228	8	no	–
1	8	8	no	–	7	356	4	yes	(7, 2), (7, 9)
2	16	5	yes	(2, 4), (2, 7)	8	526	9	yes	(8, 3), (8, 8)
3	36	3	yes	(3, 5), (3, 6)	9	744	7	no	–
4	74	8	no	–	10	1016	4	yes	(10, 2), (10, 9)
5	136	4	yes	(5, 2), (5, 9)					

12 points + O = 13. Hasse: $12\pm2\sqrt{11}=[5.4,\ 18.6]$ contains 13 ✓. Since 13 is prime, every point except O generates the whole group (Lagrange).

34.5 Point addition and doubling over GF(p) ▸

▸ **Likely this year:** "Add P and Q / find $2P$ on $E_p(a,b)$" (2–3 marks), always with modular inverses.

★ MUST KNOW : EC group law over GF(p)

For $P=(x_1,y_1)$, $Q=(x_2,y_2)$, $R=P+Q=(x_3,y_3)$ with

$$x_3=s^2-x_1-x_2 \bmod p, \qquad y_3=s(x_1-x_3)-y_1 \bmod p,$$

$$s=\frac{y_2-y_1}{x_2-x_1} \bmod p \ \ (P\neq Q,\ \text{addition}), \qquad s=\frac{3x_1^2+a}{2y_1} \bmod p \ \ (P=Q,\ \text{doubling}).$$

Division means multiplying by the inverse mod p. Special cases: $P+O=P$; if $x_1=x_2$ and $y_2=-y_1$ then $P+Q=O$; doubling a point with $y_1=0$ gives O.

WORKED EXAMPLE : one addition and two doublings

Q1 (Forouzan style). On $E_{13}(1,1)$, add $P=(4,2)$ and $Q=(10,6)$.
$s=(6-2)(10-4)^{-1}=4\cdot6^{-1}=4\cdot11=44\equiv5$ $(6\cdot11=66=5\cdot13+1)$.
$x_3=25-4-10=11$; $y_3=5(4-11)-2=-37\equiv-37+39=2$. **P + Q = (11, 2)**, which is in the point list ✓.
Q2. On $E_{13}(1,1)$, find $2P$ for $P=(1,4)$.
$s=(3\cdot1^2+1)(2\cdot4)^{-1}=4\cdot8^{-1}=4\cdot5=20\equiv7$ $(8\cdot5=40=3\cdot13+1)$.
$x_3=49-1-1=47\equiv8$; $y_3=7(1-8)-4=-53\equiv-53+65=12$. **2P = (8, 12)** ✓.
Q3 (slide). On $E_{17}(2,2)$: $y^2=x^3+2x+2$, find $2P$ for $P=(5,1)$.
$s=(3\cdot5^2+2)(2\cdot1)^{-1}=77\cdot2^{-1}\equiv9\cdot9=81\equiv13$ $(77=4\cdot17+9,\ 2^{-1}=9)$.
$x_3=169-5-5=159=9\cdot17+6\equiv6$; $y_3=13(5-6)-1=-14\equiv3$. **2P = (6, 3)**.
Check: $3^2=9$ and $6^3+2\cdot6+2=230=13\cdot17+9$ ✓. (This curve has 19 points and (5, 1) generates all of them.)

Trap: point arithmetic slips

The doubling slope uses a (from $3x^2 + a$), never b. Compute s as an integer mod p with a modular inverse, never as a fraction or decimal. $-P = (x, p - y)$ flips only y. Use the addition formula only for $P \neq Q$ (it would divide by $x_2 - x_1 = 0$). Reduce negative results by adding multiples of p, and check the answer lies on the curve.

34.6 Scalar multiplication and the ECDLP ▸

▸ **Likely this year:** "Compute kP for small k and "state the ECDLP / Hasse's theorem.

Multiplying a point by an integer k means adding it to itself: $kP = P + P + \cdots + P$ (k times). It is computed by **double-and-add**, the analogue of square-and-multiply: scan the bits of k, double at every bit and add P at every 1-bit, so kP costs about $\log_2 k$ doublings. The slides' two examples on $E_{13}(1,1)$, recomputed:

- $4 \cdot (1,4) = 2 \cdot (2 \cdot (1,4)) = 2 \cdot (8,12)$: $s = (3 \cdot 64 + 1) \cdot 24^{-1} \equiv 11 \cdot 11^{-1} = 1$, $x_3 = 1 - 16 \equiv 11$, $y_3 = 1 \cdot (8 - 11) - 12 = -15 \equiv 11$, so $4 \cdot (1,4) = \mathbf{(11,11)}$ ✓.
- $3 \cdot (8,1) = 2 \cdot (8,1) + (8,1)$: doubling gives $s = 193 \cdot 2^{-1} \equiv 11 \cdot 7 \equiv 12$, $(11,2)$; then $(11,2) + (8,1)$: $s = (-1)(-3)^{-1} \equiv 12 \cdot 10^{-1} = 12 \cdot 4 \equiv 9$, $x_3 = 81 - 19 \equiv 10$, $y_3 = 9(11 - 10) - 2 = 7$, so $3 \cdot (8,1) = \mathbf{(10,7)}$ ✓.

$(1,4)$ has order 18, so it generates all of $E_{13}(1,1)$; its multiples (useful for checking the examples below) are:

k	1	2	3	4	5	6	7	8	9
$k(1,4)$	(1,4)	(8,12)	(0,12)	(11,11)	(5,1)	(10,6)	(12,8)	(4,2)	(7,0)
k	17	16	15	14	13	12	11	10	18
$k(1,4)$	(1,9)	(8,1)	(0,1)	(11,2)	(5,12)	(10,7)	(12,5)	(4,11)	O

The second row is the negative of the first ($(18 - k)P = -kP$), and $9P = (7,0)$ has $y = 0$, so $18P = 2 \cdot 9P = O$.

Elliptic curve discrete logarithm problem (ECDLP). Given a point P (ideally a generator) and $T = dP$, find d with $1 \le d \le \#E$. Computing T from d takes about $\log_2 d$ doublings, but no shortcut is known for the reverse: on a well-chosen curve the best attack is a generic one such as **Pollard's rho**, which needs about $\sqrt{\#E}$ steps. On the toy curve the table answers it at once ($T = (4,2) \Rightarrow d = 8$); for a 160-bit curve, $\sqrt{\#E} \approx 2^{80}$ steps is infeasible.

34.7 ECC simulating ElGamal ▸

▸ **Likely this year:** "Explain ECC encryption/decryption; verify the given example" (the slide's $E_{67}(2,3)$ example).

ElGamal works in any group where the discrete log is hard, so it carries over by a dictionary: multiplication mod p becomes point addition, exponentiation becomes scalar multiplication, and the inverse becomes the negative point. The plaintext must be encoded as a point P on the curve.

★ MUST KNOW : EC ElGamal

Keys (Bob): choose $E_p(a,b)$, a point e_1 on it, and an integer d; compute $e_2 = d \times e_1$. Public $(E_p(a,b), e_1, e_2)$, private d.
Encrypt (Alice): fresh random r; $C_1 = r \times e_1$, $C_2 = P + r \times e_2$.
Decrypt (Bob): $P = C_2 - (d \times C_1)$, where subtracting means adding the negative point.
Proof: $C_2 - dC_1 = P + r(de_1) - d(re_1) = P + (rd)e_1 - (dr)e_1 = P + O = P$.

WORKED EXAMPLE : the slide example on $E_{67}(2,3)$, every step

Q. Bob selects $E_{67}(2,3)$, $e_1 = (2,22)$, $d = 4$. Alice sends $P = (24,26)$ with $r = 2$. Find e_2, C_1, C_2, and verify decryption.

Step	Slope s (mod 67)	x_3	y_3	Result
$2e_1 = 2(2,22)$	$14 \cdot 44^{-1} = 14 \cdot 32 \equiv 46$	$2112 \equiv 35$	$-1540 \equiv 1$	$(35,1)$
$4e_1 = 2(35,1)$	$3677 \cdot 2^{-1} \equiv 59 \cdot 34 \equiv 63$	$3899 \equiv 13$	$1385 \equiv 45$	$(13,45)$
$2e_2 = 2(13,45)$	$509 \cdot 90^{-1} \equiv 40 \cdot 35 \equiv 60$	$3574 \equiv 23$	$-645 \equiv 25$	$(23,25)$
$P + 2e_2$	$(-1)(-1)^{-1} = 1$	$-46 \equiv 21$	$-23 \equiv 44$	$(21,44)$
$C_2 + (23,42)$	$(-2) \cdot 2^{-1} \equiv 66$	$4312 \equiv 24$	$-242 \equiv 26$	$(24,26)$

Keys: $e_2 = 4e_1 = (13,45)$. **Encrypt**: $C_1 = 2e_1 = (35,1)$, $re_2 = (23,25)$, $C_2 = (24,26) + (23,25) = (21,44)$.
Decrypt: $d \times C_1 = 4 \cdot (35,1) = 8e_1 = 2e_2 = (23,25)$ (two doublings, rows 2–3). Its negative is $(23, 67 - 25) = (23,42)$, and $(21,44) + (23,42) = \mathbf{(24,26)} = P$ ✓.
Inverses used: $44 \cdot 32 = 1408 = 21 \cdot 67 + 1$; $2 \cdot 34 = 68$; $23 \cdot 35 = 805 = 12 \cdot 67 + 1$ ($90 \equiv 23$).

WORKED EXAMPLE : new exam-style example on $E_{13}(1,1)$

Q. Bob uses $E_{13}(1,1)$ with $e_1 = (1,4)$ and private key $d = 3$. Alice encrypts the point $P = (4,2)$ with $r = 2$. Find Bob's public key, the ciphertext, and verify decryption.

Step	Slope s (mod 13)	x_3	y_3	Result
$2e_1 = 2(1,4)$	$4 \cdot 8^{-1} = 4 \cdot 5 \equiv 7$	$47 \equiv 8$	$-53 \equiv 12$	$(8,12)$
$e_2 = (8,12) + (1,4)$	$(-8)(-7)^{-1} = 5 \cdot 6^{-1} = 5 \cdot 11 \equiv 3$	0	12	$(0,12)$
$2e_2 = 2(0,12)$	$1 \cdot 24^{-1} = 11^{-1} = 6$	$36 \equiv 10$	$-72 \equiv 6$	$(10,6)$
$C_2 = (4,2) + (10,6)$	$4 \cdot 6^{-1} \equiv 5$	11	$-37 \equiv 2$	$(11,2)$
$2C_1 = 2(8,12)$	$193 \cdot 24^{-1} \equiv 11 \cdot 6 \equiv 1$	$-15 \equiv 11$	$-15 \equiv 11$	$(11,11)$
$3C_1 = (11,11) + (8,12)$	$1 \cdot (-3)^{-1} = 10^{-1} = 4$	$-3 \equiv 10$	$-7 \equiv 6$	$(10,6)$
$C_2 + (10,7)$	$5 \cdot (-1)^{-1} = 5 \cdot 12 \equiv 8$	$43 \equiv 4$	$54 \equiv 2$	$(4,2)$

Public key $(E_{13}(1,1),(1,4),(0,12))$. **Ciphertext** $C_1 = 2e_1 = (\mathbf{8,12})$, $C_2 = (\mathbf{11,2})$.
Decrypt: $d \times C_1 = 3 \cdot (8,12) = (10,6)$, the same point as $r \times e_2$; negative $(10,7)$; $(11,2) + (10,7) = (\mathbf{4,2})$ ✓. In multiples of e_1 (table above): $e_2 = 3e_1$, $C_1 = 2e_1$, $P = 8e_1$, $C_2 = (8+6)e_1 = 14e_1$, and $14 - 6 = 8$ ✓.

34.8 Security of ECC and key size ▸

▸ **Likely this year:** "Why does ECC need smaller keys than RSA for the same security? (2 marks).

To decrypt, Eve needs r (then $P = C_2 - r \times e_2$) or d (then $P = C_2 - d \times C_1$). Finding r from $C_1 = r \times e_1$, or d from $e_2 = d \times e_1$, is the ECDLP, which for large p (or large n in GF(2^n)) is infeasible by Pollard's rho. As in ElGamal, r must be fresh for every message.

Why ECC keys are smaller. For factoring and for the discrete log in $\mathbb{Z}_p^*$ there are **sub-exponential** algorithms (the number field sieve, index calculus) that exploit the arithmetic structure of integers, so RSA and ElGamal need 1024-bit and larger moduli to stay ahead. On a well-chosen elliptic curve no such structure is known, and only generic attacks costing about $\sqrt{\#E}$ work, i.e. $2^{k/2}$ for a k-bit curve. A 160-bit curve therefore costs about 2^{80} steps, the same effort as factoring a 1024-bit RSA modulus: ECC over GF(2^n) or GF(p) with a 160-bit modulus gives the same security as RSA with a 1024-bit modulus (and 256-bit ECC ≈ 3072-bit RSA). Smaller keys mean less computation, bandwidth and storage, which suits smart cards and mobile devices.

Check your understanding

1. **Differentiate RSA and ElGamal.** RSA rests on factoring $n = pq$, is deterministic, and its ciphertext is one number the size of n. ElGamal rests on the discrete logarithm modulo a prime, is probabilistic because of the fresh r, and its ciphertext is a pair (C_1, C_2), twice the plaintext size.
2. **Why must r be fresh for every ElGamal message?** The mask e_2^r depends only on r; if two messages share r, one known plaintext P gives $e_2^r = C_2 P^{-1}$, and then every other message encrypted with that r is $P' = C_2'(e_2^r)^{-1}$.
3. **Why is $(C_1^d)^{-1} = C_1^{p-1-d}$?** $C_1^d \cdot C_1^{p-1-d} = C_1^{p-1} \equiv 1 \pmod p$ by Fermat's little theorem.
4. **ElGamal:** $p = 11$, $e_1 = 2$, $d = 3$, $r = 4$, $P = 7$. $(C_1, C_2) = (5,6)$; decryption $6 \cdot (5^3)^{-1} = 6 \cdot 3 \equiv 7$.
5. **Why must $4a^3 + 27b^2 \neq 0$?** Otherwise the cubic has a repeated root, the curve is singular (a cusp or crossing), the tangent there is undefined, and the points do not form a group.
6. **What is $-(5,1)$ on $E_{13}(1,1)$ and what is $(7,0) + (7,0)$?** $-(5,1) = (5,12)$ (only y flips); $(7,0) + (7,0) = O$ because the tangent at a point with $y = 0$ is vertical.
7. **Why does the doubling slope contain a but not b?** It is the derivative dy/dx from $2y\,y' = 3x^2 + a$, and the constant b differentiates to 0.
8. **Is $(3,5)$ on $E_{11}(1,6)$? Find $2 \cdot (2,7)$.** Yes: $27 + 3 + 6 = 36 \equiv 3 \equiv 5^2$. For $2(2,7)$: $s = 13 \cdot 14^{-1} \equiv 2 \cdot 3^{-1} = 2 \cdot 4 = 8$, $x_3 = 64 - 4 = 60 \equiv 5$, $y_3 = 8(2-5) - 7 = -31 \equiv 2$, so $(5,2)$.
9. **Why does 160-bit ECC match 1024-bit RSA?** Factoring has sub-exponential algorithms, while the ECDLP on a good curve only has generic $\sqrt{n}$-time attacks, so a 160-bit curve already needs about 2^{80} work.
10. **Why does the ElGamal ciphertext double in size?** Two values C_1 and C_2, each in $\mathbb{Z}_p$, are sent for one plaintext block in $\mathbb{Z}_p$.

Corrections to the notes / slides

- ElGamal block diagram: the encryption box labels both lines C_1; the second line, $(e_2^r \cdot P) \bmod p$, is C_2.
- $E_{67}(2,3)$ example, step 8: "He uses $2 \times C_1$" should read $d \times C_1 = 4 \times C_1 = (23,25)$. With $C_1 = 2e_1$, $2 \times C_1 = 4e_1 = e_2 = (13,45)$, not $(23,25)$ (recomputed). The same misprint is in Forouzan.
- ElGamal security slide: 1024 bits is about **309** decimal digits (as on the RSA slide), not 300.

35 Message Integrity and Hash Functions

35.1 Integrity versus secrecy; message and digest ▸

▸ **Likely this year:** "Why is a hash, not encryption, used for integrity?" and "how is integrity checked with a digest?" (2–3 marks).

Every scheme so far gave *secrecy*: Eve cannot read the message. Many documents need the opposite guarantee. Alice's will, for example, need not be encrypted, because after her death anyone may read it; what must be preserved is its *integrity*, meaning that nobody can change its contents without the change being noticed. On paper this is achieved by putting Alice's fingerprint at the bottom of the document: Eve can read the will, but she cannot alter it or write a false one, because she cannot forge the fingerprint.

The electronic equivalent of the document–fingerprint pair is the **message–digest pair**. The message is passed through a **cryptographic hash function** h, which maps a message of any length to a short fixed-length string $h(M)$, called the **message digest**. The digest is a compressed image of the message that plays the role of the fingerprint. To check integrity later, the receiver runs the same hash function on the message he now holds and compares the new digest with the stored one: if the two digests are equal, the message has not been changed; if they differ, the message was changed and is discarded.

Why a hash and not encryption. Encryption hides the message but does not stop Eve from altering the ciphertext; flipping a bit of a one-time-pad or stream-cipher ciphertext flips the same bit of the plaintext, and Bob has no way to tell a changed plaintext from a genuine one. A hash works the other way round: it hides nothing (the message can be public), but since any change to the message produces a different digest, any change is detected. The two services are therefore independent, and a message can need one, the other, or both.

The digest itself must be safe. The check only works if the stored or received digest is genuine. If Eve can replace both M and $h(M)$, she simply sends M' with $h(M')$ and the comparison passes. Hence the digest is kept in a safe place or sent over an authenticated channel; binding the digest to a secret key gives the MAC of the next lecture.

35.2 The three security criteria ▸

▸ **Likely this year:** "Define and differentiate preimage, second preimage and collision resistance" (3–4 marks), possibly with a figure of what Eve is given and must find.

A cryptographic hash function must make three kinds of forgery infeasible. They differ in how much Eve is given at the start, and therefore in how much freedom she has.

Preimage resistance. Given a hash function h and a digest $y = h(M)$, it must be extremely difficult for Eve to find *any* message M' with $h(M') = y$. If h were not preimage resistant, Eve could intercept just the digest $h(M)$, construct a message M' that hashes to it, and send M' to Bob as if it were M. In other words, the hash must be one-way.

Second preimage resistance. Given a specific message M *and* its digest $h(M)$, it must be extremely difficult to find another message $M' \neq M$ with $h(M') = h(M)$. This is the criterion that stops forgery of a particular message: if Alice sends M and $h(M)$ to Bob, Eve cannot create a different message with the same digest and substitute it. Eve here has more information than in the preimage case (she also knows one valid preimage), but she still has to hit one fixed target value.

Collision resistance. It must be extremely difficult to find *any* two messages $M \neq M'$ with $h(M) = h(M')$. Here Eve is given nothing and is free to choose both messages from scratch; such a pair is called a **collision**. This is the danger when Eve prepares two documents herself (say a fair contract and an unfair one with the same digest), gets Alice to approve the fair one, and later swaps in the other.

Criterion	Eve is given	Eve must find	Attack it stops
Preimage resistance	h and a digest $y = h(M)$	any M' with $h(M') = y$	preimage
Second preimage resistance	h, a message M and $h(M)$	$M' \neq M$ with $h(M') = h(M)$	second preimage
Collision resistance	only h	any pair $M \neq M'$ with $h(M) = h(M')$	collision

Collision resistance implies second preimage resistance. Suppose Eve had a method that, given any M, produces a second preimage M'. She could then pick any M she likes, run the method, and output the pair (M, M'), which is a collision. So a function that is *not* second-preimage resistant cannot be collision resistant; equivalently, a collision-resistant function is automatically second-preimage resistant. The converse fails, because finding a collision is the easier task (Eve chooses both sides), and the birthday analysis below shows by how much: a collision costs about $2^{n/2}$ hash evaluations, a second preimage about 2^n.

Why a compressor or a checksum is not a cryptographic hash. A lossless compression method such as ZIP or StuffIt also shrinks the message, but it is reversible: anyone can decompress the output and get the original, so it is not even a function to a fixed-length output, and it fails preimage resistance trivially. A checksum (adding up the bytes, a CRC) is fixed-length but linear and easy to invert: given a target checksum, Eve can compute what to

change in any message to reach it, so it is neither preimage nor second-preimage resistant. Checksums detect accidental errors, not deliberate ones.

★ MUST KNOW : the three criteria

Preimage: given y, find any M' with $h(M') = y$. **Second preimage**: given M, find $M' \neq M$ with $h(M') = h(M)$. **Collision**: find any $M \neq M'$ with $h(M) = h(M')$.
Cost for an ideal n-bit hash: preimage and second preimage $\approx 2^n$, collision $\approx 2^{n/2}$. Collision resistance $\Rightarrow$ second preimage resistance.

35.3 The random oracle model ▸

▸ **Likely this year:** "Explain the random oracle model; why can the oracle not use a formula?" (2–3 marks).

To analyse attacks we need a model of a perfect hash function, against which real designs are compared. The **random oracle model** (Bellare and Rogaway, 1993) describes such an ideal function as an oracle that keeps a table of the messages it has already hashed and obeys three rules:

1. When a *new* message of any length is given, the oracle creates a fixed-length digest that is a truly random string of 0s and 1s, gives it out, and records the message and digest in its table.
2. When a message is given for which a digest already exists, the oracle simply returns the digest in the table. This makes the function deterministic: the same message always gets the same digest.
3. The digest for a new message must be chosen independently of all previous digests. This implies that the oracle cannot use a formula or an algorithm to calculate the digest.

Example (slides 20–23). An oracle with 16-bit digests already holds three entries. It is asked to hash AB1234CD8765BDAD, which is not in its table, so it flips a fair coin 16 times and gets HHTHHHTTHTHHTTTH. Reading H as 1 and T as 0 gives 1101 1100 1011 0001 = DCB1, which it returns and records. When it is then asked for 4523AB1352CDEF45126, which is already in the table, it does not flip again but returns the stored digest 13AB.

Message	Digest
4523AB1352CDEF45126	13AB
723BAE38F2AB3457AC	02CA
AB1234CD8765BDAD	DCB1 (new)
AB45CD1048765412AAAB6662BE	A38B

Why a formula violates rule 3. Suppose the oracle used $h(M) = M \bmod n$ and had already issued $h(M_1)$ and $h(M_2)$. For the new message $M_3 = M_1 + M_2$ anyone can predict the digest without asking, since

$$h(M_3) = (M_1 + M_2) \bmod n = [\, h(M_1) + h(M_2) \,] \bmod n.$$

The digest of M_3 is therefore not independent of the earlier digests, which breaks rule 3. Real hash functions are algorithms, so they can only approximate the oracle; the model tells us the best security an n-bit design can hope for.

What the model gives an attacker. With an n-bit digest there are $N = 2^n$ possible digests, and each new message receives one of them uniformly at random. Some values may be chosen several times and some never. Eve knows the algorithm and n, so every attack reduces to drawing random digests and waiting for a match, which is exactly a birthday problem.

35.4 The four birthday problems ▸

▸ **Likely this year:** Derive the probability formula for one problem and get k for $P = 1/2$; or "how many people for a shared birthday" (23).

All four problems use a random variable that is uniform over N values (0 to $N - 1$; $N = 365$ for birthdays) and ask for the minimum number of samples k that makes an event *likely*, meaning probability $P \geq 1/2$. Every derivation uses the approximation $1 - x \approx e^{-x}$ for small x.

Problem 1: a sample equals a predefined value. What is the minimum number of students such that it is likely that at least one has a *predefined* birthday (say 1 January)? A single sample misses the target with probability $1 - 1/N$, and the k samples are independent, so all of them miss with probability $(1 - 1/N)^k \approx e^{-k/N}$. Hence

$$P_1 \approx 1 - e^{-k/N}, \qquad k \approx \ln \frac{1}{1-P} \times N \xrightarrow{P=1/2} k \approx \ln 2 \times N = 0.69N.$$

For $N = 365$: $0.693 \times 365 = 252.99$, so $k = 253$.

Problem 2: a sample equals one chosen from the set. What is the minimum number such that it is likely that at least one student has the same birthday as a student *selected by the professor*? The selected student is one of

the k, so only the other $k-1$ can match, and the problem is Problem 1 with $k-1$ samples:

$$P_2 \approx 1 - e^{-(k-1)/N}, \qquad k \approx \ln 2 \times N + 1 = 0.69N + 1; \qquad N = 365 : \; k = 254.$$

Problem 3: any two samples are equal (the birthday paradox). What is the minimum number such that it is likely that *at least two* students share a birthday? It is easier to count the complement, that all k values are different. The first value is free, the second must avoid 1 value, the i-th must avoid $i-1$ values, so

$$\Pr[\text{all different}] = \prod_{i=1}^{k-1}\left(1 - \frac{i}{N}\right) \approx \prod_{i=1}^{k-1} e^{-i/N} = e^{-(1+2+\cdots+(k-1))/N} = e^{-k(k-1)/2N}.$$

Therefore

$$P_3 \approx 1 - e^{-k(k-1)/2N}, \qquad k \approx \left\{2 \ln \frac{1}{1-P}\right\}^{1/2} N^{1/2} \xrightarrow{P=1/2} k \approx \sqrt{2 \ln 2}\,\sqrt{N} = 1.18\sqrt{N},$$

using $k(k-1) \approx k^2$. For $N = 365$: $1.18 \times 19.1 = 22.5$, so $k = 23$ (exactly, 23 people give $P = 0.507$ and 22 give 0.476). The answer is a "paradox" only because intuition compares each person with *one* date, whereas k people form $k(k-1)/2 = 253$ pairs, each a chance to match.

Problem 4: a sample of one set equals a sample of another. Two classes each have k students. What is the minimum k such that it is likely that some student of the first class shares a birthday with some student of the second? Treat the k values of set 1 as (nearly) distinct. Each value of set 2 then misses all of them with probability $1 - k/N$, and the k values of set 2 all miss with probability $(1-k/N)^k \approx e^{-k^2/N}$. Hence

$$P_4 \approx 1 - e^{-k^2/N}, \qquad k \approx \left\{\ln \frac{1}{1-P}\right\}^{1/2} N^{1/2} \xrightarrow{P=1/2} k \approx \sqrt{\ln 2}\,\sqrt{N} = 0.83\sqrt{N}; \qquad N = 365 : \; k = 16.$$

(Exactly, $k = 16$ gives $P = 0.504$.)

Problem	Probability P	General k	k at $P = \frac{1}{2}$	$N = 365$	Attack
1	$1 - e^{-k/N}$	$\ln \frac{1}{1-P} \cdot N$	$0.69N$	253	preimage
2	$1 - e^{-(k-1)/N}$	$\ln \frac{1}{1-P} \cdot N + 1$	$0.69N + 1$	254	second preimage
3	$1 - e^{-k(k-1)/2N}$	$\{2 \ln \frac{1}{1-P}\}^{1/2} N^{1/2}$	$1.18\sqrt{N}$	23	collision
4	$1 - e^{-k^2/N}$	$\{\ln \frac{1}{1-P}\}^{1/2} N^{1/2}$	$0.83\sqrt{N}$	16	alternate collision

The constants are $0.69 = \ln 2$, $1.18 = \sqrt{2 \ln 2}$ and $0.83 = \sqrt{\ln 2}$. The key comparison is that k **grows like N in Problems 1 and 2 but only like $\sqrt{N}$ in Problems 3 and 4.** Problems 1 and 2 model the preimage and second preimage attacks, Problems 3 and 4 the collision attacks, so a collision attack is far cheaper than a preimage attack on the same hash.

Trap: rounding the constant

With 0.69 exactly, $0.69 \times 365 = 251.85$, which would suggest 252; the table value 253 comes from $\ln 2 = 0.6931$ ($0.6931 \times 365 = 252.99$). Use 0.693 when the answer must be an exact head count. Also do not mix up Problems 1 and 3: "someone has *my* birthday" needs 253 people, "*some two* share a birthday" needs only 23.

WORKED EXAMPLE : birthday numbers for a 16-bit digest

Q. A hash function produces 16-bit digests. Using the birthday results, how many random messages must Eve hash for a probability of at least 1/2 of (a) matching a given digest, (b) finding any two messages with the same digest, (c) a match between two lists of k messages each?
A. $N = 2^{16} = 65\,536$ and $\sqrt{N} = 2^8 = 256$.
(a) Problem 1: $k \approx 0.69 \times 65\,536 \approx 45\,220$ messages (order 2^{16}).
(b) Problem 3: $k \approx 1.18 \times 256 \approx$ **302** messages (order 2^8).
(c) Problem 4: $k \approx 0.83 \times 256 \approx$ **213** messages in each list.
Check: (b) is about 150 times cheaper than (a), as $2^{16}/2^8 = 256$ predicts up to the constants.

35.5 Attacks on an n-bit digest ▸

▸ **Likely this year:** "A hash uses a 64-bit digest; how many digests must Eve create to find a collision (or the original message) with $P > 0.5$, and how long does it take?" (3–4 marks).

Assume the hash behaves like a random oracle with n-bit digests, so its outputs are uniform over $N = 2^n$ values, and that Eve knows the algorithm and n. Each attack below is a loop that creates messages, hashes them, and compares digests; its success probability is the matching birthday problem.

Preimage attack. Eve has intercepted a digest $D = h(M)$ and wants any M' with $h(M') = D$. She creates k messages $M[1], \ldots, M[k]$, computes $T = h(M[i])$ for each, and returns $M[i]$ as soon as $T = D$. The target D is a predefined value, so this is Problem 1: $P = 1 - e^{-k/N}$, and success with $P \geq 1/2$ needs $k = 0.69 \times 2^n$ digests, which is of order 2^n.

Second preimage attack. Eve has intercepted both M and $D = h(M)$ and wants $M' \neq M$ with $h(M') = D$. She creates $k - 1$ further messages and compares each digest with D. The target is one of the k samples (the given M), so this is Problem 2: $P = 1 - e^{-(k-1)/N}$ and $k = 0.69 \times 2^n + 1$, again of order 2^n.

Collision attack. Eve wants any $M \neq M'$ with $h(M) = h(M')$. She creates k messages, stores the digests $D[1], \ldots, D[k]$, and compares each new digest with all earlier ones. Any two equal digests suffice, so this is Problem 3: $P = 1 - e^{-k(k-1)/2N}$ and $k = 1.18 \times 2^{n/2}$, of order only $2^{n/2}$. The square root comes from the number of pairs: k digests give about $k^2/2$ pairs, and once $k^2/2$ is comparable to 2^n a coincidence is likely.

Alternate collision attack. A random collision is usually useless, because the two colliding messages are meaningless strings. A real forger needs a collision between a *genuine* message M (which Alice will approve) and a *bogus* message M' (which Eve will later substitute), both meaningful. Eve therefore makes k variants of each without changing their meaning (an extra space after a word, a synonym, "will not" versus "won't", a redundant phrase): $M_1, \ldots, M_k$ and $M'_1, \ldots, M'_k$. She hashes both lists and looks for $h(M_i) = h(M'_j)$. A match must be *across* the two lists, so this is Problem 4: $P = 1 - e^{-k^2/N}$ and $k = 0.83 \times 2^{n/2}$ variants of each, still of order $2^{n/2}$. Variants are cheap to produce: if a letter has $n/2$ independent places where one of two equivalent forms can be used, it has $2^{n/2}$ variants.

Attack	Birthday problem	k for $P = 1/2$	Order
Preimage	1	0.69×2^n	2^n
Second preimage	2	$0.69 \times 2^n + 1$	2^n
Collision	3	$1.18 \times 2^{n/2}$	$2^{n/2}$
Alternate collision	4	$0.83 \times 2^{n/2}$	$2^{n/2}$

WORKED EXAMPLE : a 64-bit digest (slides 27 and 32)

Q. A cryptographic hash function uses a 64-bit digest. (a) How many digests must Eve create to find the original message with probability more than 0.5, and how long does it take at 2^{30} (about one billion) digests per second? (b) How many digests must she create to find two messages with the same digest with probability more than 0.5, and how long does it take at 2^{20} (about one million) digests per second? Is a 64-bit digest secure?

A. $n = 64$, $N = 2^{64}$.

(a) Preimage attack (Problem 1): $k = 0.69 \times 2^{64} \approx 1.27 \times 10^{19}$ digests.

Time $= 0.69 \times 2^{64}/2^{30} = 0.69 \times 2^{34}$ s $= 1.185 \times 10^{10}$ s $\div (3.156 \times 10^7$ s/year$) \approx$ **376 years**. Secure against the preimage attack.

(b) Collision attack (Problem 3): $k = 1.18 \times 2^{64/2} = 1.18 \times 2^{32} \approx 5.07 \times 10^9$ digests.

Time $= 1.18 \times 2^{32}/2^{20} = 1.18 \times 2^{12}$ s $= 4833$ s $\approx$ **1.34 hours**. **Not** secure against the collision attack, even with a computer a thousand times slower.

Check: the collision attack is cheaper by about 2^{32} in digests, exactly the gap between 2^n and $2^{n/2}$.

What digest length is needed. Since the cheapest generic attack costs about $2^{n/2}$ hash evaluations, an n-bit hash offers only $n/2$ bits of security against collisions. To demand at least 2^{80} work, the digest must have $n/2 \geq 80$, i.e. $n \geq 160$ bits, which is why SHA-1 has 160 bits and the SHA-2 family 224–512 bits. **MD5's 128-bit digest** gives only 2^{64} collision security: $1.18 \times 2^{64} \approx 2.2 \times 10^{19}$ digests, about 640 years on one machine at 2^{30} per second, but well within reach of a large cluster. In addition, structural attacks (Wang et al., 2004) now find MD5 collisions in minutes on a PC, so MD5 must not be used where collisions matter.

WORKED EXAMPLE : new exam-style numerical

Q. A hash function produces 40-bit digests and Eve can compute 2^{20} digests per second. (a) How long does a preimage attack with success probability 1/2 take? (b) How long does a collision attack take? (c) If Eve hashes 2^{20} messages, what is the probability that two of them collide?

A. $N = 2^{40}$.

(a) $k = 0.69 \times 2^{40}$; time $= 0.69 \times 2^{20}$ s $= 723\,517$ s $\approx$ **8.4 days**.

(b) $k = 1.18 \times 2^{20} \approx 1.24 \times 10^6$; time $= 1.18$ s, **about one second**.

(c) $P = 1 - e^{-k(k-1)/2N} \approx 1 - e^{-2^{40}/2^{41}} = 1 - e^{-0.5} =$ **0.39**.

Check: (b) is (a) divided by roughly 2^{20}, consistent with the orders 2^{40} and 2^{20}.

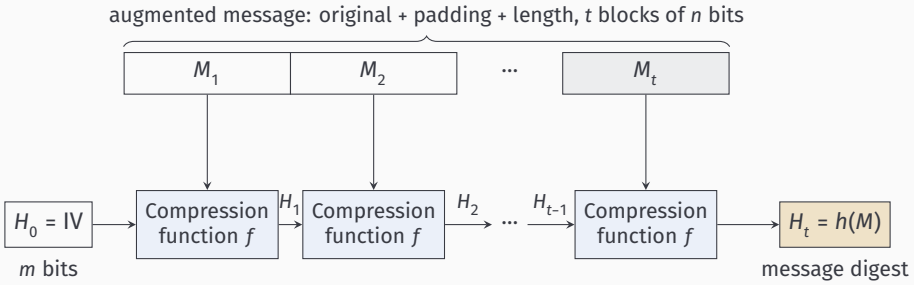
WORKED EXAMPLE : how many variants for an alternate collision

Q. Eve attacks a 64-bit hash with the alternate collision attack. How many variants of the genuine and the bogus letter does she need for success probability about 1/2, and how many two-way choice points must each letter contain? What is her success probability with 2^{32} variants of each?
A. Problem 4: $k = 0.83 \times 2^{64/2} = 0.83 \times 2^{32} \approx 3.56 \times 10^9$ variants of each letter. With m independent places where one of two equivalent forms can be chosen, a letter has 2^m variants, so $m = 32$ choice points per letter are enough ($2^{32} > 0.83 \times 2^{32}$).
With $k = 2^{32}$: $P = 1 - e^{-k^2/N} = 1 - e^{-2^{64}/2^{64}} = 1 - e^{-1} = \mathbf{0.63}$.

35.6 Iterated hash functions: the Merkle–Damgård scheme ▸

▸ **Likely this year:** "Explain the Merkle–Damgård scheme with a diagram" (3–4 marks).

A hash function must accept messages of any length and still produce a fixed-length digest, and designing one function for arbitrary input lengths directly is hard. The standard solution is **iteration**: design a **compression function** f with *fixed-size* input, which compresses an $(n + m)$-bit input (an n-bit message block and an m-bit chaining value) to m bits, and apply it as many times as the message requires. A hash built this way is an **iterated cryptographic hash function**, and the **Merkle–Damgård scheme** is the standard way of doing it.



The scheme works in five steps.

1. The message length and padding are appended to the message to create an **augmented message** whose length is a multiple of n, the block size of the compression function.
2. The augmented message is split into t blocks $M_1, M_2, \ldots, M_t$ of n bits each.
3. Before the iteration starts, the chaining value H_0 is set to a fixed public value called the **IV** (initial value).
4. Iteration i combines the previous digest with the next block: $H_i = f(H_{i-1}, M_i)$ for $i = 1, \ldots, t$.
5. The last chaining value H_t is the hash of the original message, $h(M) = H_t$.

Why it is collision resistant if f is. Suppose two different messages $M \neq M'$ gave the same final digest $H_t = H'_t$. Then the last compression steps $f(H_{t-1}, M_t)$ and $f(H'_{t-1}, M'_t)$ have equal outputs. If their inputs differ, that is a collision of f. If their inputs are equal, then $H_{t-1} = H'_{t-1}$ and the same argument applies one step earlier, and so on backwards. Since $M \neq M'$, the inputs must differ at some step, so a collision of h always yields a collision of f. The length field is essential to this argument: it puts the message length in the last block, so two messages of different lengths already differ in their final inputs and cannot be lined up to share a suffix. As a result, the designer only has to make one fixed-size function collision resistant.

35.7 Compression functions: from scratch or from a block cipher ▸

▸ **Likely this year:** "Differentiate Rabin, Davies–Meyer and Miyaguchi–Preneel; why is Rabin weak?" (3 marks).

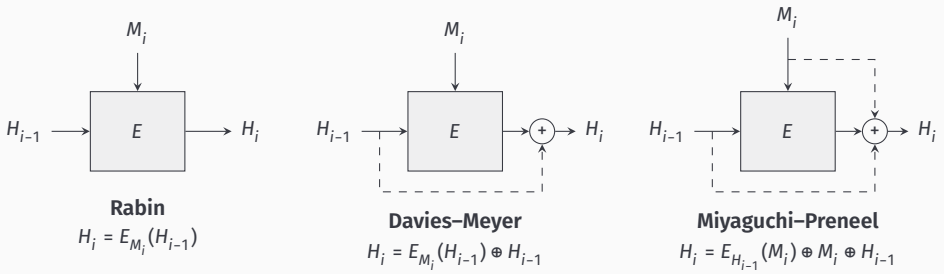
There are two ways to obtain the compression function f. In the first, f is **designed from scratch** specifically for hashing; in the second, an existing **symmetric-key block cipher** is used as f, so that the confidence gained in the cipher carries over.

Made from scratch: the MD and SHA families. The **Message Digest** family by Rivest has several versions (MD2, MD4, MD5). MD5 is a strengthened MD4 that divides the message into 512-bit blocks and creates a 128-bit digest; this digest turned out to be too short to resist collision attacks. The **Secure Hash Algorithm** family, the NIST standard also called the Secure Hash Standard (SHS), started with SHA (later SHA-1) and added SHA-224, SHA-256, SHA-384 and SHA-512 (together called SHA-2). All of them are Merkle–Damgård constructions.

Characteristic	MD5	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Maximum message size (bits)	any*	$2^{64} - 1$	$2^{64} - 1$	$2^{64} - 1$	$2^{128} - 1$	$2^{128} - 1$
Block size (bits)	512	512	512	512	1024	1024
Digest size (bits)	128	160	224	256	384	512
Number of rounds / steps	64	80	64	64	80	80
Word size (bits)	32	32	32	32	64	64

The SHA columns are the slide's table (slide 41); the MD5 column is added for comparison (*MD5 records the length modulo 2^{64}, so it accepts any length). The pattern to remember is that the 32-bit-word members use 512-bit blocks and the 64-bit-word members (SHA-384, SHA-512) use 1024-bit blocks; SHA-384 is SHA-512 with a different IV and a truncated digest.

Made from a block cipher. Let $E_K(P)$ be a block cipher with key K and plaintext P. One input of f is the message block M_i and the other is the chaining value H_{i-1}; the schemes differ in which goes into the key, which into the plaintext, and what is XORed onto the output.



In the Rabin and Davies–Meyer pictures the block M_i enters as the *key* (from above) and H_{i-1} as the plaintext; in Miyaguchi–Preneel it is the other way round: M_i is the plaintext and H_{i-1} the key (slide 45). Dashed lines are the feed-forward XORs.

Rabin scheme. Rabin plugs a block cipher directly into Merkle–Damgård: the message block is used as the key, the previous digest as the plaintext, and the ciphertext is the new digest, $H_i = E_{M_i}(H_{i-1})$. The digest size equals the cipher's block size, so with DES the digest is only 64 bits. Although the scheme is very simple, it is subject to a **meet-in-the-middle attack**. Because Eve knows every key (the message blocks are her choice), each step can be run *backwards*: $H_{i-1} = D_{M_i}(H_i)$. To forge a message with a given digest H_t, she encrypts forward from H_0 with about $2^{m/2}$ random choices of the first half of the blocks, stores the intermediate values, then decrypts backward from H_t with about $2^{m/2}$ random choices of the second half, and looks for a common intermediate value. A match gives a full message with digest H_t after only about $2 \times 2^{m/2}$ operations instead of 2^m; for DES that is about 2^{32}, which is feasible.

Davies–Meyer scheme. Davies–Meyer is the Rabin scheme plus a **feed-forward**: the previous digest is XORed onto the cipher output, $H_i = E_{M_i}(H_{i-1}) \oplus H_{i-1}$. Since H_{i-1} now appears both inside the encryption and in the XOR, knowing H_i and M_i no longer lets Eve compute H_{i-1} by decryption, so the backward half of the meet-in-the-middle attack is blocked. This is the protection the slide refers to.

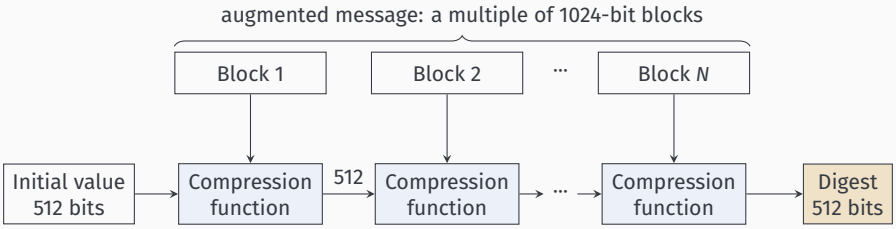
Matyas–Meyer–Oseas scheme (Forouzan, not on the slides). This is the dual of Davies–Meyer: the previous digest is the key (passed through a function g that adapts it to the key size) and the message block is the plaintext, with the block fed forward, $H_i = E_{g(H_{i-1})}(M_i) \oplus M_i$. It is useful when the key and block sizes of the cipher are equal.

Miyaguchi–Preneel scheme. To make the algorithm stronger, the plaintext, the cipher key and the ciphertext are all XORed together to create the new digest: $H_i = E_{H_{i-1}}(M_i) \oplus M_i \oplus H_{i-1}$. It is Matyas–Meyer–Oseas with the chaining value also fed forward. Whirlpool (below) is built on this scheme.

35.8 SHA-512: overview and parameters ▸

▸ **Likely this year:** "Explain SHA-512 with a block diagram: padding, word expansion, round function" (4–5 marks).

SHA-512 is the member of the SHA-2 family with the longest digest. It follows the Merkle–Damgård scheme exactly: the message is padded and given a length field, split into 1024-bit blocks, and each block is fed with the previous 512-bit digest into a compression function; the output of the last block is the 512-bit digest.



SHA-512 is **word oriented**: a word is 64 bits. Each 1024-bit message block is 16 words, and the 512-bit digest is 8 words, named A, B, C, D, E, F, G, H. The parameters are: maximum message length $2^{128} - 1$ bits, block 1024 bits, digest 512 bits, 80 rounds per block, 64-bit words, 80 round constants $K_0, \ldots, K_{79}$, and 80 expanded words $W_0, \ldots, W_{79}$ per block.

35.9 SHA-512: padding and length field ▸

▸ **Likely this year:** "Compute the number of padding bits for a message of length L; is padding needed if the length is already a multiple of 1024?" (2–3 marks).

Before processing, Merkle–Damgård requires the message to be a whole number of blocks, with its length recorded at the end. SHA-512 therefore appends two things to the original message M: **padding**, which is a single 1 bit followed by as many 0 bits as needed, and a **128-bit length field** holding $|M|$ as an unsigned integer (this is why the maximum message length is $2^{128} - 1$). The total $|M| + |P| + 128$ must be a multiple of 1024, so

$$|M| + |P| + 128 \equiv 0 \pmod{1024} \implies \boxed{|P| = (-|M| - 128) \bmod 1024}, \qquad 0 \le |P| \le 1023.$$

Since the mod gives a value from 0 to 1023, the padding is always less than one block.

WORKED EXAMPLE : padding (Forouzan Example 12.1)

Q. What is the number of padding bits if the length of the original message is 2590 bits?
A. $|P| = (-2590 - 128) \bmod 1024 = (-2718) \bmod 1024$. Since $-2718 + 3 \times 1024 = 354$, $|P|$ = **354** bits: a 1 followed by 353 zeros.
Check: $2590 + 354 + 128 = 3072 = 3 \times 1024$, so the augmented message is exactly 3 blocks.

WORKED EXAMPLE : padding, new example

Q. A message of 5000 bits is hashed with SHA-512. Find the number of padding bits, the number of blocks, and the content of the length field.
A. $|P| = (-5000 - 128) \bmod 1024 = (-5128) \bmod 1024 = -5128 + 6 \times 1024$ = **1016** bits (a 1 followed by 1015 zeros). Total = $5000 + 1016 + 128 = 6144$ = **6 blocks**. The 128-bit length field holds $5000 = 1388$ in hex, i.e. 115 zero bits followed by `1001110001000`.
Check: $6144/1024 = 6$ exactly.

Edge cases (Forouzan Examples 12.2–12.3). A message whose length is *already* a multiple of 1024 still needs padding, because the length field must be added: for $|M| = 1024$, $|P| = (-1152) \bmod 1024 = 896$ and the result is 2 blocks. The formula gives the **minimum** padding $|P| = 0$ when $|M| \equiv 896 \pmod{1024}$ (the message plus the length field already fill the block) and the **maximum** $|P| = 1023$ when $|M| \equiv 897 \pmod{1024}$. In the actual standard (FIPS 180) the single 1 bit is always appended, so the padding is at least 1 bit; for $|M| \equiv 896$ this forces a full extra block of 1024 padding bits. A short check of the real rule: the 24-bit message "abc" gets $(-24 - 128) \bmod 1024 = 872$ padding bits and fits in one block.

Trap: padding arithmetic

Take the mod of a negative number correctly: add multiples of 1024 until the value lies in 0–1023. Do not forget the 128-bit length field (SHA-1 and SHA-256 use a 64-bit field and 512-bit blocks, giving $|P| = (-|M| - 64) \bmod 512$).

35.10 SHA-512: word expansion and the compression function ▸

Word expansion. Each round uses one 64-bit word, but a block has only 16 words, so the block is expanded to 80 words $W_0, \ldots, W_{79}$. The first 16 are the block itself; each later word mixes four earlier ones:

$$W_i = W_{i-16} \oplus \text{RotShift}_{1\text{-}8\text{-}7}(W_{i-15}) \oplus W_{i-7} \oplus \text{RotShift}_{19\text{-}61\text{-}6}(W_{i-2}), \qquad 16 \le i \le 79,$$

$$\text{RotShift}_{l\text{-}m\text{-}n}(x) = \text{RotR}_l(x) \oplus \text{RotR}_m(x) \oplus \text{ShL}_n(x),$$

where $\text{RotR}_i(x)$ rotates x right by i bits (bits leaving on the right re-enter on the left) and $\text{ShL}_i(x)$ shifts x left by i bits, filling with 0s (bits leaving are lost). This is the slides' (Forouzan's) form; see the correction box for the standard's exact operations. Since every later word depends on several earlier ones, a change in one message bit spreads into many of the 80 words.

Initial values and constants. The eight words of H_0 are the first 64 bits of the fractional parts of the square roots of the first eight primes (2 to 19); for example A_0 = `6A09E667F3BCC908` from $\sqrt{2}$. The 80 round constants $K_0, \ldots, K_{79}$ are the first 64 bits of the fractional parts of the cube roots of the first 80 primes (2 to 409); for example K_0 = `428A2F98D728AE22` from $\sqrt[3]{2}$. Taking them from irrational numbers shows that the designers hid no trapdoor in the constants ("nothing up my sleeve" numbers).

Compression function. Processing one block takes 80 rounds. At the start, the eight digest words from the previous block (or the IV) are saved in eight temporary variables. Round i mixes the current $A, \ldots, H$ with the word W_i and the constant K_i and produces a new set of eight words. After round 79, the saved values are added to the outputs word by word modulo 2^{64}; this **final adding** produces the digest that is passed to the next block (or

is the final digest). The final adding is a feed-forward, exactly like Davies–Meyer, and it makes the compression function impossible to run backwards.

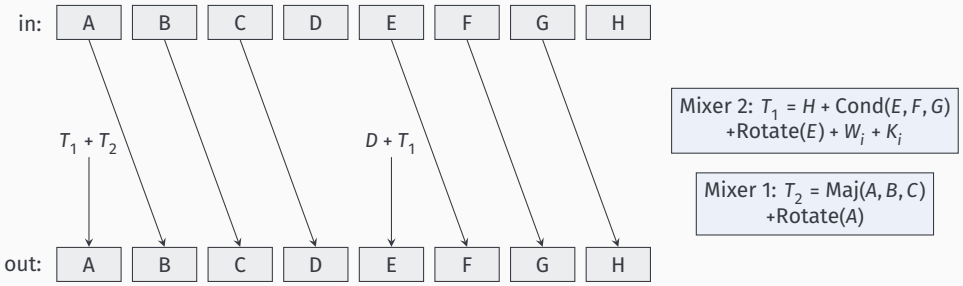
Structure of each round. Six of the eight words are simply moved one place right, and only two new words are computed, using two mixers (every + here is addition modulo 2^{64}):

$$T_1 = H + \text{Conditional}(E,F,G) + \text{Rotate}(E) + W_i + K_i, \qquad T_2 = \text{Majority}(A,B,C) + \text{Rotate}(A),$$

$$H \leftarrow G,\ G \leftarrow F,\ F \leftarrow E,\ E \leftarrow D + T_1,\ D \leftarrow C,\ C \leftarrow B,\ B \leftarrow A,\ A \leftarrow T_1 + T_2.$$

The three functions are bitwise (except Rotate, which moves bits within the word):

- Majority$(x,y,z) = (x \wedge y) \oplus (y \wedge z) \oplus (z \wedge x)$: each output bit is the majority value of the three input bits.
- Conditional$(x,y,z) = (x \wedge y) \oplus (\overline{x} \wedge z)$: each bit of x acts as a selector, taking the bit of y where $x = 1$ and the bit of z where $x = 0$ ("if x then y else z").
- Rotate$(x) = \text{RotR}_{28}(x) \oplus \text{RotR}_{34}(x) \oplus \text{RotR}_{39}(x)$.



In Forouzan's notation the new A is called X and the new E is Y. Because A and E change every round and the other six words are shifted, every input word affects all eight words within a few rounds.

★ MUST KNOW : SHA-512 in numbers

Block 1024 bits = 16 words of 64 bits; digest 512 bits = 8 words A–H; 80 rounds; 80 words W_0–W_{79}; 80 constants K_i; length field 128 bits; $|P| = (-|M| - 128) \bmod 1024$; max message $2^{128} - 1$ bits; final adding modulo 2^{64}.

35.11 Exercises on the SHA-512 operations ▸

▸ **Likely this year:** One of the slide exercises (RotR, ShL, Conditional, Majority, Rotate) on given hex words.

The slide exercises use a **128-bit** x (real SHA-512 words are 64 bits) and work in hex: a rotation or shift by 12 bits moves the string by 3 hex digits. All results below were recomputed.

$$x = 1234\ 5678\ ABCD\ 2345\ 3456\ 5678\ ABCD\ 2468$$

WORKED EXAMPLE : RotR, ShL, Conditional, Majority, Rotate (slides 52–56)

$\text{RotR}_{12}(x)$. Rotate right by 12 bits = 3 hex digits: the last three digits 468 move to the front.
4681 2345 678A BCD2 3453 4565 678A BCD2
$\text{ShL}_{12}(x)$. Shift left by 3 hex digits: the first three digits 123 are lost and three 0 digits enter on the right.
4567 8ABC D234 5345 6567 8ABC D246 8000
Conditional(x,y,z) with y = 2234 5678 …, z = 3234 5678 … (the three words differ only in the first digit). For the first digit, $x = 0001$ selects y where $x = 1$ and z where $x = 0$: $(0001 \wedge 0010) \oplus (1110 \wedge 0011) = 0000 \oplus 0010 = 0010 = 2$. For every other digit the three inputs are equal, and Conditional of three equal digits is that digit.
Result: 2234 5678 ABCD 2345 3456 5678 ABCD 2468
Majority(x,y,z), same x,y,z. First digit: $(0001 \wedge 0010) \oplus (0010 \wedge 0011) \oplus (0011 \wedge 0001) = 0000 \oplus 0010 \oplus 0001 = 0011 = 3$. The other digits are unchanged.
Result: 3234 5678 ABCD 2345 3456 5678 ABCD 2468
Rotate$(x) = \text{RotR}_{28}(x) \oplus \text{RotR}_{34}(x) \oplus \text{RotR}_{39}(x)$. First the three rotations, then the XOR:

$\text{RotR}_{28}(x)$	BCD2	4681	2345	678A	BCD2	3453	4565	678A
$\text{RotR}_{34}(x)$	2AF3	491A	048D	159E	2AF3	48D1	4D15	959E
$\text{RotR}_{39}(x)$	F157	9A48	D024	68AC	F157	9A46	8A68	ACAC
Rotate(x)	6776	95D3	F7EC	1AB8	6776	E6C4	8218	5EB8

RotR_{28} is a rotation by 7 hex digits, so it can be done digit-wise; RotR_{34} and RotR_{39} are not multiples of 4 and must be done in binary.

WORKED EXAMPLE : new exercise on 16-bit and 64-bit words

Q. (a) For the 16-bit words x = F0F0, y = AAAA, z = 5555, find Conditional(x, y, z) and Majority(x, y, z). (b) For the 64-bit word w = 0123 4567 89AB CDEF, find $\text{RotR}_8(w)$ and $\text{ShL}_8(w)$.

A. (a) Conditional takes y where x is 1 and z where x is 0. $x \wedge y$ = A0A0, $\overline{x} \wedge z$ = 0F0F $\wedge$ 5555 = 0505; Conditional = A0A0 $\oplus$ 0505 = **A5A5**.
Majority: $x \wedge y$ = A0A0, $y \wedge z$ = 0000 (since $z = \overline{y}$), $z \wedge x$ = 5050; XOR = **F0F0**. This equals x, as it must: y and z disagree in every bit, so x always casts the deciding vote.
(b) 8 bits = 2 hex digits. $\text{RotR}_8(w)$ = EF01 2345 6789 ABCD (EF wraps to the front); $\text{ShL}_8(w)$ = 2345 6789 ABCD EF00 (01 is lost, 00 enters).

35.12 Whirlpool

Whirlpool, designed by Rijmen and Barreto and selected by the NESSIE project (New European Schemes for Signatures, Integrity, and Encryption), is an iterated hash function based on the **Miyaguchi–Preneel** scheme, in which the compression function is a block cipher: $H_i = E_{H_{i-1}}(M_i) \oplus M_i \oplus H_{i-1}$. The cipher, called W, is a **modified AES** tailored for this purpose.

Parameters. Block size 512 bits, digest 512 bits, cipher key 512 bits (the previous digest), initial value H_0 all 0s, and maximum message length $2^{256} - 1$ bits. The message is padded (a 1 followed by 0s) to an *odd* multiple of 256 bits and then a **256-bit length field** is appended; an odd multiple of 256 plus 256 is an even multiple, i.e. a multiple of 512. For example, $|M| = 1000$ needs $|P| = (-1000 - 256) \bmod 512 = 280$ padding bits, since $1000 + 280 = 1280 = 5 \times 256$ (odd) and $1280 + 256 = 1536 = 3 \times 512$.

How the Whirlpool cipher differs from AES. It uses the same four kinds of transformation, but on a larger state and with all rounds alike:

- The block and key are 512 bits, held as an 8×8 matrix of bytes (AES: 128 bits as 4×4).
- There are 10 rounds, each with SubBytes, **ShiftColumns** (column j is rotated down by j positions, instead of AES's ShiftRows), **MixRows** (each row is multiplied by a fixed 8×8 circulant matrix, instead of AES's MixColumns) and AddRoundKey.
- Arithmetic is in $\text{GF}(2^8)$ with modulus $x^8 + x^4 + x^3 + x^2 + 1$ (AES uses $x^8 + x^4 + x^3 + x + 1$), and the S-box is built from smaller 4-bit boxes.
- The round keys are generated by running the same round function on the key, with round constants taken from the S-box in place of a key, instead of AES's separate key-expansion routine.

Corrections to the notes / slides

- Slide 27: "0.69×2^{34} seconds or more than 500 years". In fact 0.69×2^{34} s $\approx$ 376 years (2^{34} s alone is $\approx$ 544 years). The conclusion (secure against preimage attacks) stands.
- Slides 54–55 swap the binary for the first digit: Conditional(0001, 0010, 0011) = 0010 = 2 and Majority(0001, 0010, 0011) = 0011 = 3; the slides print 0011 for Conditional and 0010 for Majority, though the hex results 2 and 3 are right. Both slides also label the third word x instead of z.
- The exercise x (slides 52–56) is 128 bits, though real SHA-512 words are 64 bits; the rotations on the slides are done on 128 bits, and the results above follow the slides.
- Slide 11 prints Problem 4 as $P \approx 1 - e^{-k^2/2N}$; that formula would give $k = 1.18\sqrt{N}$. The correct one is $P \approx 1 - e^{-k^2/N}$ (as on slide 35), which gives $k = 0.83\sqrt{N} = 16$.
- Slide 40 lists SHA under the "block cipher" approach. Forouzan classifies MD and SHA as compression functions *made from scratch* and Rabin, Davies–Meyer, Matyas–Meyer–Oseas and Miyaguchi–Preneel as block-cipher based; write it that way in the exam.
- Slides 48 and 51 (from Forouzan) simplify the standard: in FIPS 180 the word expansion uses addition modulo 2^{64} and a *right* shift (SHR), not XOR and ShL, and Rotate(E) uses $\text{RotR}_{14} \oplus \text{RotR}_{18} \oplus \text{RotR}_{41}$, not 28-34-39. The slide's "ShL...padding the left by 0s" is self-contradictory (a left shift fills on the right). For the exam, use the slide formulas.

Check your understanding

1. **Differentiate preimage, second preimage and collision resistance.** Preimage resistance: given only a digest y, Eve cannot find any message hashing to y. Second preimage resistance: given a message M and its digest, she cannot find a different message with the same digest. Collision resistance: she cannot find any two different messages with the same digest, choosing both herself. For an n-bit ideal hash the first two cost about 2^n and the third about $2^{n/2}$.
2. **Why does collision resistance imply second preimage resistance?** If Eve could find a second preimage for a given M, she could choose any M, find M', and output (M, M') as a collision. So breaking second preimage resistance breaks collision resistance, and a collision-resistant function must be second-preimage resistant.
3. **Why can a checksum not serve as a cryptographic hash?** A checksum is linear and easy to invert, so given

a target value Eve can adjust any message to reach it; it is therefore not preimage resistant. It only detects accidental errors.

4. **Why can the random oracle not use the formula $h(M) = M \bmod n$?** Because then $h(M_1+M_2) = [h(M_1)+h(M_2)] \bmod n$ can be predicted from earlier digests, violating the rule that each new digest is chosen independently at random.
5. **Derive the probability that among k random n-bit digests two are equal.** All k differ with probability $\prod_{i=1}^{k-1}(1 - i/N) \approx e^{-k(k-1)/2N}$, $N = 2^n$, so $P \approx 1 - e^{-k(k-1)/2N}$; setting $P = 1/2$ gives $k \approx 1.18\sqrt{N} = 1.18 \times 2^{n/2}$.
6. **How many digests must Eve create to find a collision in a 64-bit hash with $P > 0.5$?** $k = 1.18 \times 2^{32} \approx 5.07 \times 10^9$; at 2^{20} per second this takes $1.18 \times 2^{12} \approx 4833$ s, about 1.3 hours, so 64 bits is insecure against collisions (a preimage would take about 376 years at 2^{30} per second).
7. **What digest length gives 2^{128} security against collisions?** The collision attack costs about $2^{n/2}$, so $n/2 = 128$ and $n = 256$ bits (SHA-256).
8. **Compute the SHA-512 padding for $|M| = 3000$ bits.** $|P| = (-3000 - 128) \bmod 1024 = -3128 + 4 \times 1024 = 968$ bits; total $3000 + 968 + 128 = 4096 = 4$ blocks.
9. **Why is the Rabin scheme weak, and how does Davies–Meyer fix it?** In Rabin, $H_i = E_{M_i}(H_{i-1})$ and Eve knows every key, so she can decrypt backwards from a target digest and meet a forward computation in the middle with about $2^{m/2}$ work each way. Davies–Meyer XORs H_{i-1} onto the output, so H_{i-1} can no longer be recovered from H_i and M_i, which blocks the backward half.
10. **Why is the length field appended in Merkle–Damgård?** It makes the last block depend on the message length, which lets a collision of the whole hash be traced back to a collision of the compression function; without it, messages of different lengths could share a suffix of chaining values.

36 Previous Year Papers (MNNIT, CSN15403), Fully Solved

What these papers tell you

- **Format:** 90 minutes, 25 marks, 5–7 questions, "precise answer with proper justification". Show every step: tables, matrices, divisions.
- **Sources:** questions come almost word for word from the two standard textbooks: **Forouzan** (Eve/Alice, $\mathbb{Z}_{100}$, keyed transposition, multiplicative/affine/auto-key, GF(2^8) inverse) and **Stallings** ("sendmoremoney" one-time pad, the AES state values).
- **Repeat topics:** Playfair (both years), GF(2^8) inverse (both years), extended Euclid (inverse / Diophantine, both years), DES/AES (both years).

Topic	2024–25	2025–26	In these notes?
Playfair encryption	3	4	Section 12
One-time pad / Vigenère	–	8	Sections 14, 15
Cipher properties, attack types, keyed transposition	2 + 3	–	Sections 6, 18
Modular arithmetic, exponentiation, inverses	3	–	Sections 22–27
Extended Euclid / Diophantine	5	3	Section 25
GF(2^8) polynomial inverse	4	4	Section 29
DES (Feistel, round function, key schedule)	5	–	Section 31
AES (ShiftRows, AddRoundKey)	–	6	Section 32

Every topic in these papers is now covered in its own section; the boxes below are one-page summaries for quick revision.

36.1 Quick-revision method boxes

Extended Euclidean algorithm (Forouzan table): inverse of b mod n

Columns $q, r_1, r_2, r, t_1, t_2, t$. Start with $r_1 = n$, $r_2 = b$, $t_1 = 0$, $t_2 = 1$. Each row:

$$q = \lfloor r_1 / r_2 \rfloor, \quad r = r_1 - q\,r_2, \quad t = t_1 - q\,t_2; \qquad \text{then shift: } r_1 \leftarrow r_2,\ r_2 \leftarrow r,\ t_1 \leftarrow t_2,\ t_2 \leftarrow t.$$

Stop when $r_2 = 0$. If $r_1 = 1$, then $b^{-1} = t_1 \bmod n$. If $r_1 \neq 1$, there is **no inverse** (gcd $\neq 1$). **Pseudocode:**

```
r1=n; r2=b; t1=0; t2=1; while r2>0 { q=r1/r2; r=r1-q*r2; r1=r2; r2=r; t=t1-q*t2; t1=t2; t2=t }; if r1==1 return t1 mod n
```

Linear Diophantine equation $ax + by = c$

Let $d = \gcd(a, b)$. If $d \nmid c$: **no solution**. Otherwise divide by d, find s, t with $as + bt = d$ (extended Euclid), and multiply by c/d to get a particular solution (x_0, y_0). **General solution:** $x = x_0 + \frac{b}{d}k$, $y = y_0 - \frac{a}{d}k$ $(k \in \mathbb{Z})$.

GF(p) and GF(2^8)

GF(p) = $\{0, \ldots, p-1\}$ with + and × mod a prime p. Every non-zero element has an inverse, and "$a \div b$" means $a \times b^{-1}$. **GF**(2^8) elements are polynomials of degree < 8 with coefficients 0/1 (one byte). **Add = XOR** (so + and − are the same). Multiply polynomials, then reduce mod $m(x) = x^8 + x^4 + x^3 + x + 1$. **Inverse:** the same extended-Euclid table with polynomial division, where every subtraction is an XOR.

Two classical ciphers and the attack types

Multiplicative: $C = k_1 P \bmod 26$, with $\gcd(k_1, 26) = 1$. **Affine:** $C = (k_1 P + k_2) \bmod 26$, decrypted by $P = k_1^{-1}(C - k_2)$. **Attacks:** *ciphertext-only* (Eve has only ciphertext), *known-plaintext* (has some plaintext–ciphertext pairs), *chosen-plaintext* (can encrypt plaintext of her choice), *chosen-ciphertext* (can decrypt ciphertext of her choice).

36.2 Mid-Sem 2024–25, solved

Q1 (2). *Max characters changed in the ciphertext if one plaintext character changes: (a) multiplicative (b) affine (c) Vigenère (d) auto-key.*
(a) **1**, (b) **1**, (c) **1**: each encrypts one character on its own (Vigenère uses a different shift per position, but still one character in, one out). (d) **2**. In auto-key the plaintext is also the key: $k_{i+1} = P_i$ (Forouzan; or $k_{i+m} = P_i$ with an m-letter keyword, as in our lecture). So changing P_i changes C_i and the one later character it acts as key for.

Q2 (3). *Playfair, key "GUIDANCE": "The key is hidden under the door pad".*

G	U	I/J	D	A
N	C	E	B	F
H	K	L	M	O
P	Q	R	S	T
V	W	X	Y	Z

Digrams (dd → dx; oo → ox; odd length, so the final d becomes dx):
`th ek ey is hi dx de nu nd er th ed ox or pa dx`
`PO CL BX DR LG IY IB CG BG LX PO BI LZ LT TG IY`
(er→LX is a column case; all the others are rectangles.)

Ciphertext: **POCLBXDRLGIYIBCGBGLXPOBILZLTTGIY**.

Q3 (3). *Eve types "abcdefghij" on Alice's machine and sees "CABDEHFGIJ". Keyed transposition.*
(a) **Chosen-plaintext attack**: Eve chooses the plaintext and sees its encryption. (b) The pattern repeats every 5: `abcde`→`CABDE` and `fghij`→`HFGIJ` (positions 3, 1, 2, 4, 5 both times). So the **key size is 5**. Encryption key: output takes input positions **(3 1 2 4 5)**. Decryption key = inverse permutation **(2 3 1 4 5)**.

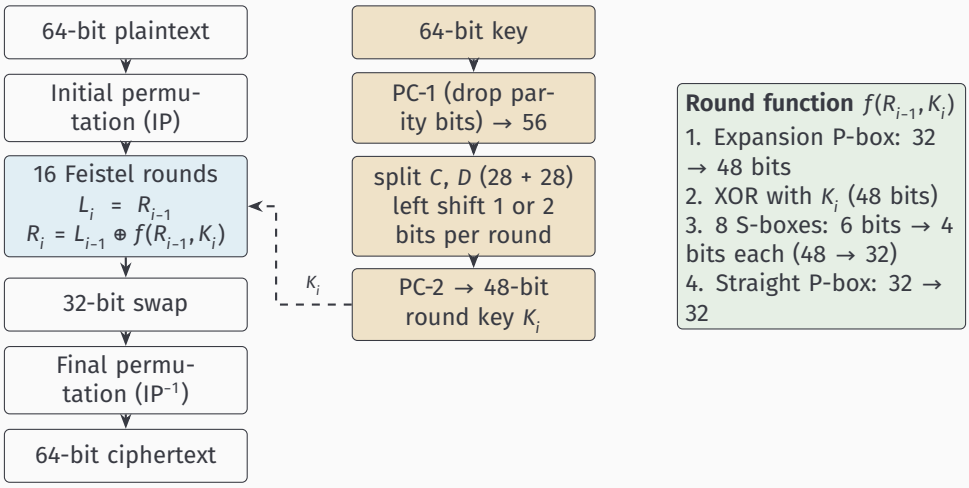
Q4 (4). *Inverse of x^5 in GF(2^8) mod $m = x^8 + x^4 + x^3 + x + 1$.*

q	r_1	r_2	r	t_1	t_2	t
x^3	$x^8+x^4+x^3+x+1$	x^5	x^4+x^3+x+1	0	1	x^3
$x+1$	x^5	x^4+x^3+x+1	x^3+x^2+1	1	x^3	x^4+x^3+1
x	x^4+x^3+x+1	x^3+x^2+1	1	x^3	x^4+x^3+1	$x^5+x^4+x^3+x$
x^3+x^2+1	x^3+x^2+1	1	0	x^4+x^3+1	$x^5+x^4+x^3+x$	m
	1	0		$\mathbf{x^5+x^4+x^3+x}$		

$(x^5)^{-1} = \mathbf{x^5 + x^4 + x^3 + x}$ (byte 3A). Check: using $x^8 = x^4+x^3+x+1$, $x^9 = x^5+x^4+x^2+x$, $x^{10} = x^6+x^5+x^3+x^2$: $x^5(x^5+x^4+x^3+x) = x^{10}+x^9+x^8+x^6$, and every term cancels in pairs except 1 ✓.

Q5 (3). (i) GF(7): $5 \div 3 = 5 \times 3^{-1} = 5 \times 5 = 25 \equiv \mathbf{4}$ (since $3 \cdot 5 = 15 \equiv 1$). (ii) $100^{95} \bmod 9$: $100 \equiv 1$, so the answer is **1**. $658^{17} \bmod 17$: $658 = 38 \cdot 17 + 12$, and by Fermat $a^{17} \equiv a \pmod{17}$, so the answer is **12**. $145^{102} \bmod 101$: $145 \equiv 44$; Fermat gives $44^{100} \equiv 1$, so $44^{102} \equiv 44^2 = 1936 = 19 \cdot 101 + 17$, i.e. **17**. (iii) $7^{-1} \bmod 17 = \mathbf{5}$ ($35 = 2 \cdot 17 + 1$). $15^{-1} \bmod 99$: $\gcd(15, 99) = 3 \neq 1$, so it **does not exist**. (That's the trap.)

Q6 (5). DES: Feistel structure, round function, key generation.



- **Feistel structure:** each round changes only one half and then swaps the halves. Decryption is the *same* algorithm with the round keys used in reverse ($K_{16}, \ldots, K_1$). f need not be invertible.
- **Shifts:** 1 bit in rounds 1, 2, 9, 16; 2 bits in all the others (28 bits in total).

S-box substitution (pseudocode)

```
for i = 1 to 8:
  b = in[6i-5 .. 6i]  // 6 bits
  row = 2*b1 + b6  // outer bits
  col = b2b3b4b5  // inner 4 bits
  out[4i-3 .. 4i] = S[i][row][col]
return out  // 32 bits
```

Key generation (pseudocode)

```
K56 = PC1(K64)
C = K56[1..28]; D = K56[29..56]
for i = 1 to 16:
  s = (i in {1,2,9,16}) ? 1 : 2
  C = rotl(C, s); D = rotl(D, s)
  K[i] = PC2(C || D)  // 48 bits
return K[1..16]
```

Q7 (5). *Pseudocode for finding an inverse, and finding 23^{-1} in $\mathbb{Z}_{100}$.* This is the **extended Euclidean algorithm** (pseudocode in the method box). Mr X's worry is real: an inverse exists only when gcd = 1, and the same algorithm tells him so when it doesn't. Here $\gcd(100, 23) = 1$:

Iter.	q	r_1	r_2	r	t_1	t_2	t
1	4	100	23	8	0	1	−4
2	2	23	8	**7**	1	−4	9
3	1	8	7	1	−4	9	**−13**
4	7	7	1	0	9	−13	**100**
		1	0		**−13**	100	

1. **4 iterations**.
2. Remainder in iteration 2: r = **7**.
3. Coefficient in iteration 4: t = **100** (always $\pm n$ in the last step; a useful self-check).
4. Inverse value in iteration 3: t = −13, which is the final answer: $23^{-1} \equiv -13 \equiv$ **87** (mod 100). Check: $23 \times 87 = 2001 = 20 \cdot 100 + 1$ ✓.

36.3 Mid-Sem 2025–26, solved

Q1 (8). *One-time-pad Vigenère; key stream 9 0 1 7 23 15 21 14 11 11 2 8 9.*

Plain	s	e	n	d	m	o	r	e	m	o	n	e	y
P	18	4	13	3	12	14	17	4	12	14	13	4	24
Key	9	0	1	7	23	15	21	14	11	11	2	8	9
P+K	**27**	4	14	10	**35**	**29**	**38**	18	23	25	15	12	**33**
mod 26	1	4	14	10	9	3	12	18	23	25	15	12	7
Cipher	**B**	**E**	**O**	**K**	**J**	**D**	**M**	**S**	**X**	**Z**	**P**	**M**	**H**

(a) **BEOKJDMSXZPMH**. (b) We need $K = (C - P) \bmod 26$ with P = `cashnotneeded`:

C	B 1	E 4	O 14	K 10	J 9	D 3	M 12	S 18	X 23	Z 25	P 15	M 12	H 7
P	c 2	a 0	s 18	h 7	n 13	o 14	t 19	n 13	e 4	e 4	d 3	e 4	d 3
$C - P$	−1	4	−4	3	−4	−11	−7	5	19	21	12	8	4
Key	**25**	**4**	**22**	**3**	**22**	**15**	**19**	**5**	**19**	**21**	**12**	**8**	**4**

Key stream: **25 4 22 3 22 15 19 5 19 21 12 8 4**. This is the point of the question: for *any* ciphertext there is a key giving *any* plaintext of the same length, which is why a one-time pad can't be broken (Section 13).

Q2 (4). *Inverse of x^7 mod $x^8 + x^4 + x^3 + x + 1$.*

q	r_1	r_2	r	t_1	t_2	t
x	$x^8+x^4+x^3+x+1$	x^7	x^4+x^3+x+1	0	1	x
x^3+x^2+x	x^7	x^4+x^3+x+1	x	1	x	$x^4+x^3+x^2+1$
x^3+x^2+1	x^4+x^3+x+1	x	1	x	$x^4+x^3+x^2+1$	x^7+x+1
x	x	1	0	$x^4+x^3+x^2+1$	x^7+x+1	m
	1	0		$\mathbf{x^7+x+1}$		

$(x^7)^{-1} = \mathbf{x^7 + x + 1}$ (byte 83). (Row 2 detail: $x^7 \div (x^4+x^3+x+1)$ gives quotient x^3+x^2+x and remainder x, all with XOR subtraction.)

Q3 (4). *Playfair with the given key matrix: "Insta Reels Pe Time Waste Na Kar" (filler X, I/J together).*

L	A	R	K	S
P	U	B	C	D
E	F	G	H	I/J
M	N	O	Q	T
V	W	X	Y	Z

Digrams (`ee` in "reels" → `ex`, then re-pair; odd length, so `r` becomes `rx`):
`in st ar ex el sp et im ew as te na ka rx`
`FT DZ RK GV MP LD IM ET FV RL MI WU SR BR`
(rows: `ar`, `as`, `ka`; columns: `st`, `el`, `na`, `rx`; the rest are rectangles)

Ciphertext: **FTDZRKGVMPLDIMETFVRLMIWUSRBR**. *Trap:* in "reels" the `ee` lands inside one pair (`r e | e l` would be wrong). Pairing from the start gives `in st ar ee`, so the filler goes in there.

Q4 (6). *AES: (a) ShiftRows, (b) AddRoundKey.*
(a) Row r is rotated **left by r bytes** (row 0 unchanged):

$$\begin{pmatrix} 87 & F2 & 4D & 97 \\ EC & 6E & 4C & 90 \\ 4A & C3 & 46 & E7 \\ 8C & D8 & 95 & A6 \end{pmatrix} \xrightarrow{\text{ShiftRows}} \begin{pmatrix} 87 & F2 & 4D & 97 \\ 6E & 4C & 90 & EC \\ 46 & E7 & 4A & C3 \\ A6 & 8C & D8 & 95 \end{pmatrix}$$

(b) AddRoundKey = **byte-wise XOR** of state and key. E.g. $47 \oplus AC$: $0100\,0111 \oplus 1010\,1100 = 1110\,1011 = EB$.

$$\begin{pmatrix} 47 & 40 & A3 & 4C \\ 37 & D4 & 70 & 9F \\ 94 & E4 & 3A & 42 \\ ED & A5 & A6 & BC \end{pmatrix} \oplus \begin{pmatrix} AC & 19 & 28 & 57 \\ 77 & FA & D1 & 5C \\ 66 & DC & 29 & 00 \\ F3 & 21 & 41 & 6A \end{pmatrix} = \begin{pmatrix} \mathbf{EB} & \mathbf{59} & \mathbf{8B} & \mathbf{1B} \\ \mathbf{40} & \mathbf{2E} & \mathbf{A1} & \mathbf{C3} \\ \mathbf{F2} & \mathbf{38} & \mathbf{13} & \mathbf{42} \\ \mathbf{1E} & \mathbf{84} & \mathbf{E7} & \mathbf{D6} \end{pmatrix}$$

Tip: XOR one hex digit at a time ($4 \oplus A = 0100 \oplus 1010 = 1110 = E$, $7 \oplus C = 0111 \oplus 1100 = 1011 = B$).

Q5 (3). *Solve* $19x + 13y = 20$. $\gcd(19, 13) = 1$, and $1 \mid 20$, so solutions exist. Euclid: $19 = 1 \cdot 13 + 6$, $13 = 2 \cdot 6 + 1$. Back-substitute: $1 = 13 - 2 \cdot 6 = 13 - 2(19 - 13) = 3 \cdot 13 - 2 \cdot 19$, so $s = -2$, $t = 3$. Times 20: $x_0 = -40$, $y_0 = 60$. General solution: $x = -40 + 13k$, $y = 60 - 19k$. Taking $k = 3$: $x = -1$, $y = 3$, so in general $x = -1 + 13k$, $y = 3 - 19k$. Check: $19(-1) + 13(3) = -19 + 39 = 20$ ✓.

36.4 Mock Mid-Sem paper (same style), 90 minutes, 25 marks

Precise answer with proper justification will be considered to get complete marks. Attempt all questions.

Q		Marks
1	What is the maximum number of characters that change in the *ciphertext* if a single plaintext character is changed, for: (a) Hill (2×2) (b) Playfair (c) row–column transposition? (d) In the auto-key cipher (keyword length m), if *one ciphertext character* is corrupted in transit, how many plaintext characters come out wrong after decryption?	2
2	Using the Playfair cipher with the key "MNNIT ALLAHABAD" (I/J in one cell, filler X), encrypt "Jazz night at the hall tomorrow".	4
3	Eve types "abcdefghijkl" on Alice's computer and the screen shows "BDACFHEGJLIK". Alice uses a keyed transposition cipher. (a) What type of attack is this? (b) What is the key size? (c) Give the encryption and decryption keys. (d) Decrypt "ETMEETMAONNO".	3
4	In GF(2^8) find the inverse of ($x^6 + x^4 + x + 1$) modulo ($x^8 + x^4 + x^3 + x + 1$). Verify your answer.	4
5	(i) In GF(11) find $7 \div 9$. (ii) Find 2^{90} mod 13 and 1000^{999} mod 7. (iii) Find, if they exist, 11^{-1} mod 26, 26^{-1} mod 39 and 17^{-1} mod 101.	3
6	Solve the linear Diophantine equations, or show there is no solution: (i) $24x + 138y = 18$ (ii) $15x + 21y = 7$.	3
7	Using the extended Euclidean algorithm, find 47^{-1} in $\mathbb{Z}_{120}$. State (a) the number of iterations, (b) the remainder in iteration 2, (c) t in iteration 3. (d) Does 46 have an inverse in $\mathbb{Z}_{120}$? Justify.	3
8	AES. State $S = \begin{pmatrix} 19 & A0 & 9A & E9 \\ 3D & F4 & C6 & F8 \\ E3 & E2 & 8D & 48 \\ BE & 2B & 2A & 08 \end{pmatrix}$, round key $K = \begin{pmatrix} A0 & 88 & 23 & 2A \\ FA & 54 & A3 & 6C \\ FE & 2C & 39 & 76 \\ 17 & B1 & 39 & 05 \end{pmatrix}$. (a) Apply ShiftRows to S. (b) Apply AddRoundKey with K to your result from (a) (skip MixColumns here).	3

36.5 Mock paper: model answers

M1. (a) **2**: one block of 2 letters; both can change. (b) Usually 2 (one digram), but the **maximum is every remaining character**: changing a letter can create or remove a doubled pair (e.g. "balk" → "ball"), which adds or removes a filler and shifts every later digram. (c) **1**: transposition only moves letters, so exactly one ciphertext letter (somewhere else) changes. (d) The wrong P_i is then used as the key for P_{i+m}, which is used for P_{i+2m}, and so on: the error **propagates to every m-th character until the end**.

M2. Key without repeats: M N I T A L H B D, then the rest.

M	N	I/J	T	A
L	H	B	D	C
E	F	G	K	O
P	Q	R	S	U
V	W	X	Y	Z

`jazznightatthehalltomorrow`, J → I. Only `zz` lands in one pair: `zx`. The `tt`, `ll` and `rr` are each split across pairs (traps!). Odd length, so `w` becomes `wx`:
`ia zx zn ig ht at th eh al lt om or ro wx`
`TM VY WA BR DN MA ND FL MC DM EA GU UG XY`

Ciphertext: **TMVYWABRDNMANDFLMCDMEAGUUGXY**. (Note `or`→GU and `ro`→UG: reversed pairs give reversed output.)

M3. (a) **Chosen-plaintext**. (b) The pattern repeats every 4 (`abcd`→BDAC, `efgh`→FHEG), so the **key size is 4**. (c) Encryption key **(2 4 1 3)**: output position j takes input position key[j]. Decryption key = inverse **(3 1 4 2)**. (d) Blocks ETME | ETMA | ONNO, each rearranged as positions 3, 1, 4, 2: **meet me at noon**.

M4. Table ($m = x^8+x^4+x^3+x+1$, $b = x^6+x^4+x+1$):

q	r_1	r_2	r	t_1	t_2	t
x^2+1	m	x^6+x^4+x+1	x^2	0	1	x^2+1
x^4+x^2	x^6+x^4+x+1	x^2	$x+1$	1	x^2+1	x^6+x^2+1
$x+1$	x^2	$x+1$	1	x^2+1	x^6+x^2+1	$x^7+x^6+x^3+x$
$x+1$	$x+1$	1	0	x^6+x^2+1	$x^7+x^6+x^3+x$	m
	1	0		$\mathbf{x^7+x^6+x^3+x}$		

Inverse = $\mathbf{x^7 + x^6 + x^3 + x}$ (byte CA; i.e. $\{53\}^{-1} = \{CA\}$, the standard AES example).
Verified: $(x^6+x^4+x+1)(x^7+x^6+x^3+x) \bmod m = 1$.

M5. (i) $9^{-1} \bmod 11 = 5$ ($45 = 4 \cdot 11 + 1$), so $7 \div 9 = 7 \cdot 5 = 35 \equiv$ **2**. (ii) Fermat: $2^{12} \equiv 1 \pmod{13}$ and $90 = 7 \cdot 12 + 6$, so $2^{90} \equiv 2^6 = 64 \equiv$ **12**. $1000 = 142 \cdot 7 + 6 \equiv -1$, so $(-1)^{999} = -1 \equiv$ **6**. (iii) $11^{-1} \bmod 26 =$ **19** ($209 = 8 \cdot 26 + 1$). $26^{-1} \bmod 39$: $\gcd(26, 39) = 13$, so it **does not exist**. $17^{-1} \bmod 101 =$ **6** ($102 = 101 + 1$).

M6. (i) $\gcd(24, 138) = 6$ and $6 \mid 18$, so solutions exist. Divide by 6: $4x + 23y = 3$. Since $4 \cdot 6 + 23 \cdot (-1) = 1$, times 3 gives $x_0 = 18$, $y_0 = -3$. General: $x = 18 + 23k$, $y = -3 - 4k$; with $k = -1$: $x = -5 + 23k$, $y = 1 - 4k$. Check: $24(-5) + 138(1) = 18$ ✓. (ii) $\gcd(15, 21) = 3$, and $3 \nmid 7$: **no solution**. (The left side is always a multiple of 3.)

M7.

Iter.	q	r_1	r_2	r	t_1	t_2	t
1	2	120	47	26	0	1	−2
2	1	47	26	**21**	1	−2	3
3	1	26	21	5	−2	3	**−5**
4	4	21	5	1	3	−5	23
5	5	5	1	0	−5	23	−120
		1	0		**23**		

(a) **5 iterations**; (b) **21**; (c) **−5**. $47^{-1} =$ **23** (check: $47 \times 23 = 1081 = 9 \cdot 120 + 1$ ✓). (d) **No**: $\gcd(46, 120) = 2 \neq 1$ (both even), so $46x$ is always even mod 120 and can never be 1.

M8. (a) $\begin{pmatrix} 19 & A0 & 9A & E9 \\ F4 & C6 & F8 & 3D \\ 8D & 48 & E3 & E2 \\ 08 & BE & 2B & 2A \end{pmatrix}$ (b) XOR with K: $\begin{pmatrix} \mathbf{B9} & \mathbf{28} & \mathbf{B9} & \mathbf{C3} \\ \mathbf{0E} & \mathbf{92} & \mathbf{5B} & \mathbf{51} \\ \mathbf{73} & \mathbf{64} & \mathbf{DA} & \mathbf{94} \\ \mathbf{1F} & \mathbf{0F} & \mathbf{12} & \mathbf{2F} \end{pmatrix}$ (e.g. 19 ⊕ A0 = 0001 1001 ⊕ 1010 0000 = 1011 1001 = B9).

37 Exam-Pattern Drill Bank

Every question type that appeared in the 2024–25 and 2025–26 papers, several times over, with new numbers. Do each group until the method is automatic. Marks follow the papers. Answers are in Section 37.1.

Type 1: cipher behaviour and attack types (2–3 marks)

E1. *[2]* What is the maximum number of characters that change in the ciphertext if a single plaintext character is changed? (a) one-time pad (b) Hill cipher with a 3×3 key (c) keyed transposition (d) auto-key ($k_1 = k$, $k_i = P_{i-1}$).

E2. *[2]* A single *ciphertext* character is corrupted in transit. How many plaintext characters are wrong after decryption? (a) Vigenère (b) Hill (2×2) (c) auto-key ($k_i = P_{i-1}$) (d) Playfair.

E3. *[2]* Name the attack: (a) Eve only listens to the channel. (b) Eve finds an old printout with a plaintext and its ciphertext. (c) Eve borrows Alice's encryption device for an hour. (d) Eve tricks Bob into decrypting a ciphertext she made up.

E4. *[3]* Eve uses Alice's computer, types "abcdefghijkl" and sees "DFACEBJLGIKH". Alice uses a keyed transposition cipher. (a) What type of attack is this? (b) What is the key size? (c) Find the encryption and decryption keys. (d) Decrypt "AKATCTANADWT".

E5. *[2]* Encrypt "enemy attacks tonight" with the keyed transposition key (3 1 4 5 2), padding with "z".

Type 2: Playfair (3–4 marks)

E6. *[4]* Use the Playfair cipher to encipher "We meet at the bridge at midnight Jack". The key is made by filling the first rows with "CRYPTOGRAPHY" and the rest with the remaining alphabet (I/J together, filler X).

E7. *[3]* Encrypt "Balloon pops at noon" with this key matrix:

K	E	Y	W	O
R	D	A	B	C
F	G	H	I/J	L
M	N	P	Q	S
T	U	V	X	Z

E8. *[3]* The Playfair key is "SECURITY". Decrypt DAFSAFCFPMHSLGCSAFRTSBFHUV.

Type 3: GF(2^8) arithmetic (4 marks)

For all of these, $m(x) = x^8 + x^4 + x^3 + x + 1$.

E9. *[4]* Find the multiplicative inverse of x^2 in GF(2^8) mod $m(x)$.

E10. *[4]* Find the multiplicative inverse of $(x^3 + 1)$ in GF(2^8) mod $m(x)$.

E11. *[4]* Find the multiplicative inverse of x^4 in GF(2^8) mod $m(x)$.

E12. *[4]* Find the multiplicative inverse of x^6 in GF(2^8) mod $m(x)$.

E13. *[3]* In GF(2^8): (a) add $(x^6 + x^4 + x^2 + x + 1)$ and $(x^7 + x + 1)$; (b) multiply them mod $m(x)$.

Type 4: GF(p), modular exponentiation, inverses (3 marks)

E14. *[3]* (i) In GF(13) find $4 \div 7$, and in GF(17) find $3 \div 5$. (ii) Find 999^{999} mod 10, 7^{1000} mod 13 and 1234^{4321} mod 5. (iii) Find, if they exist, 13^{-1} mod 47, 12^{-1} mod 30 and 5^{-1} mod 28.

E15. *[3]* Find 777^{333} mod 7, 2^{1000} mod 7, 11^{500} mod 12 and 46^{47} mod 47.

Type 5: extended Euclidean algorithm (5 marks)

E16. *[5]* Find 17^{-1} in $\mathbb{Z}_{43}$ with the extended Euclidean algorithm. State the number of iterations, the remainder in iteration 3, and t in iteration 2.

E17. *[5]* Mr X wants 38^{-1} in $\mathbb{Z}_{180}$. Run the extended Euclidean algorithm and explain what the final row tells him. Which elements of $\{36, 37, 38, 39\}$ *do* have inverses in $\mathbb{Z}_{180}$?

E18. *[5]* Find 550^{-1} mod 1769. How many iterations are needed? Comment on the answer.

Type 6: linear Diophantine equations (3 marks)

E19. *[3]* Solve, or show there is no solution: (i) $21x + 14y = 35$ (ii) $40x + 15y = 110$ (iii) $12x + 18y = 25$.

E20. *[3]* A shop sells pens for Rs. 19 and notebooks for Rs. 13. A student spends exactly Rs. 220. How many of each did they buy? Is the answer unique?

Type 7: one-time pad / Vigenère with a key stream (8 marks)

E21. *[8]* One-time-pad Vigenère: each plaintext letter is shifted by the next number of the key stream. (a) Encrypt "meetmeatnoon" with the key stream 3 19 5 0 24 11 7 2 14 9 25 8. (b) Using your ciphertext, find a key stream that makes it decrypt to "holdposition". (c) Why does (b) show that Eve can't break this cipher by brute force?

E22. *[3]* Encrypt "polyalphabetic" with the Vigenère keyword "LEMON".

Type 8: AES state operations (6 marks)

E23. *[6]* AES state (rows) $S = \begin{pmatrix} 32 & 88 & 31 & E0 \\ 43 & 5A & 31 & 37 \\ F6 & 30 & 98 & 07 \\ A8 & 8D & A2 & 34 \end{pmatrix}$, key $K = \begin{pmatrix} 2B & 28 & AB & 09 \\ 7E & AE & F7 & CF \\ 15 & D2 & 15 & 4F \\ 16 & A6 & 88 & 3C \end{pmatrix}$. (a) Apply ShiftRows to S. (b) Apply AddRoundKey to S (the original) with K.

E24. *[3]* After ShiftRows the state is $\begin{pmatrix} D4 & E0 & B8 & 1E \\ 27 & BF & B4 & 41 \\ 11 & 98 & 5D & 52 \\ AE & F1 & E5 & 30 \end{pmatrix}$. What was it before ShiftRows?

E25. *[3]* ★ (MixColumns, one byte.) A state column is $(87, 6E, 46, A6)$. Compute the first byte of its MixColumns output, $02 \cdot 87 \oplus 03 \cdot 6E \oplus 46 \oplus A6$, in GF($2^8$).

Type 9: DES (5 marks)

E26. *[2]* DES S-box S_1 (below). Find the output for the 6-bit inputs (a) 100011 (b) 011011 (c) 110110.

S_1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

E27. *[3]* DES key schedule. (a) By how many bits in total have C and D been rotated after rounds 2, 9 and 16? (b) Why is the total after round 16 important? (c) How many bits does PC-1 discard, and why?

E28. *[5]* Explain with a diagram why a Feistel cipher can decrypt with the *same* round function even if f is not invertible. Given one round's output (L_1, R_1) and K_1, write L_0 and R_0.

Type 10: Fermat, Euler, primitive roots (3–4 marks)

E29. *[3]* Using Fermat's little theorem, find (i) $6^{10} \bmod 11$ (ii) $3^{12} \bmod 11$ (iii) $2^{-1} \bmod 13$ (iv) $7^{-1} \bmod 19$.

E30. *[4]* Using Euler's theorem, find the multiplicative inverses: (i) $3^{-1} \bmod 20$ (ii) $7^{-1} \bmod 15$ (iii) $8^{-1} \bmod 77$ (iv) $60^{-1} \bmod 187$ (v) $71^{-1} \bmod 100$.

E31. *[3]* Find (i) $7^{222} \bmod 11$ (ii) $4^{532} \bmod 11$ (iii) $11^{2024} \bmod 35$.

E32. *[4]* (i) Show that 3 is a primitive root of 17. (ii) Find all primitive roots of 11. (iii) How many primitive roots does 23 have? Given that 5 is one, find another without building a table.

E33. *[3]* Find the order of every element of $\mathbb{Z}_7^*$ and identify its primitive roots. Does $\mathbb{Z}_8^*$ have a primitive root? Justify.

E34. *[3]* Evaluate, stating whether Fermat's theorem applies: (i) $12^{16} \bmod 17$ (ii) $17^{16} \bmod 17$ (iii) $2^{14} \bmod 15$: what does this tell you about 15? (iv) $2^{340} \bmod 341$ turns out to be 1. Is 341 prime?

37.1 Drill bank: model answers

E1. (a) **1** (b) **3**: the whole block can change (c) **1**: letters only move, so one letter (at a new position) changes (d) **2**: C_i, and C_{i+1} because P_i is the key for the next letter.

E2. (a) **1** (b) **2**: the whole block decrypts wrongly (c) **every character from there on**: wrong P_i becomes the key for P_{i+1}, which is then wrong, and so on (d) **2**: one digram.

E3. (a) ciphertext-only (b) known-plaintext (c) chosen-plaintext (d) chosen-ciphertext.

E4. (a) **Chosen-plaintext**. (b) `abcdef`→DFACEB and `ghijkl`→JLGIKH: the same pattern, so the **key size is 6**. (It can't be 3: the first output letter, `d`, comes from outside `abc`.) (c) Encryption key **(4 6 1 3 5 2)**: output *j* takes input key[*j*]. Decryption key = inverse **(3 6 4 1 5 2)**. (d) Blocks AKATCT | ANADWT, taking positions 3, 6, 4, 1, 5, 2 of each: **attack at dawn**.

E5. 19 letters, padded to 20 with one `z`, in blocks of 5: `enemy`, `attac`, `kston`, `ightz`. Each block is output as positions 3, 1, 4, 5, 2: `enemy`→EEMYN, `attac`→TAACT, `kston`→TKONS, `ightz`→HITZG. Answer: **EEMYNTAACTTKONSHITZG** (Forouzan's own example).

E6. Matrix: C R Y P T / O G A H B / D E F I K / L M N Q S / U V W X Z. Text: `we meet at the bridge at midnight iack` (J→I). The `ee` and `tt` split across pairs, so no filler is needed; the length is odd, so pad with `x`: `we me et at th eb ri dg ea tm id ni gh ti ac kx`
→ VF VM KR BY PB KG PE EO FG RS KE QF AB PK OY IZ (`me`: column; `id`, `gh`: rows).
Answer: **VFVMKRBYPBKGPEEOFGRSKEQFABPKOYIZ**.

E7. `balloonpopsatnoon`: `ba`, then `ll` is a double, so `lx` and re-pair: `lo on po ps at no on`. Even length now, so no padding. `ba lx lo on po ps at no on` → **CB IZ SC ES SY QM RV SE ES**. *Trap:* after `lx`, the `oo` falls across `lo` | `on`, so no second filler is needed.

E8. Matrix: S E C U R / I T Y A B / D F G H K / L M N O P / Q V W X Z. Decrypt (row: left; column: up; rectangle: same rule): `hi de th eg ol du nd er th eb ri dg ex` → remove the final filler: **"hide the gold under the bridge"**.

E9. q, r_1, r_2, r, t_1, t_2, t:

x^6+x^2+x	m	x^2	$x+1$	0	1	x^6+x^2+x
$x+1$	x^2	$x+1$	1	1	x^6+x^2+x	$x^7+x^6+x^3+x+1$
$x+1$	$x+1$	1	0	x^6+x^2+x	$x^7+x^6+x^3+x+1$	m

$(x^2)^{-1}$ = $\mathbf{x^7 + x^6 + x^3 + x + 1}$ (CB). Row 1: $m \div x^2$ gives $q = x^6+x^2+x$, $r = x+1$.

E10.

x^5+x^2+x+1	m	x^3+1	x^2	0	1	x^5+x^2+x+1
x	x^3+1	x^2	1	1	x^5+x^2+x+1	$x^6+x^3+x^2+x+1$
x^2	x^2	1	0	x^5+x^2+x+1	$x^6+x^3+x^2+x+1$	m

$(x^3+1)^{-1}$ = $\mathbf{x^6 + x^3 + x^2 + x + 1}$ (4F).

E11.

x^4+1	m	x^4	x^3+x+1	0	1	x^4+1
x	x^4	x^3+x+1	x^2+x	1	x^4+1	x^5+x+1
$x+1$	x^3+x+1	x^2+x	1	x^4+1	x^5+x+1	$x^6+x^5+x^4+x^2$
x^2+x	x^2+x	1	0	x^5+x+1	$x^6+x^5+x^4+x^2$	m

$(x^4)^{-1}$ = $\mathbf{x^6 + x^5 + x^4 + x^2}$ (74).

E12.

x^2	m	x^6	x^4+x^3+x+1	0	1	x^2
x^2+x+1	x^6	x^4+x^3+x+1	1	1	x^2	$x^4+x^3+x^2+1$
x^4+x^3+x+1	x^4+x^3+x+1	1	0	x^2	$x^4+x^3+x^2+1$	m

$(x^6)^{-1}$ = $\mathbf{x^4 + x^3 + x^2 + 1}$ (1D). *Pattern:* in the first row, $x^k \cdot q$ is always the top part of m; the remainder is the low part x^4+x^3+x+1 whenever $k \geq 5$.

E13. (a) Addition is XOR, so equal terms cancel: $(x^6+x^4+x^2+x+1) + (x^7+x+1) = \mathbf{x^7 + x^6 + x^4 + x^2}$ (57 ⊕ 83 = D4). (b) The product is $x^{13}+x^{11}+x^9+x^8+x^6+x^5+x^4+x^3+1$. Reduce with $x^8 = x^4+x^3+x+1$ (and $x^9, \ldots, x^{13}$ likewise) to get $\mathbf{x^7 + x^6 + 1}$ (C1). This is Stallings' worked example {57} · {83} = {C1}.

E14. (i) $7^{-1} \bmod 13 = 2$ ($14 \equiv 1$), so $4 \div 7 = 8$. $5^{-1} \bmod 17 = 7$ ($35 \equiv 1$), so $3 \div 5 = 21 \equiv 4$. Answers: **8, 4**. (ii) $999 \equiv -1 \pmod{10}$, odd power: **9**. $7^{12} \equiv 1 \pmod{13}$ and $1000 = 83 \cdot 12 + 4$: $7^4 = 2401 \equiv$ **9**. $1234 \equiv -1 \pmod 5$, odd power: **4**. (iii) $13^{-1} \bmod 47 =$ **29** ($377 = 8 \cdot 47 + 1$). $12^{-1} \bmod 30$: gcd = 6, **does not exist**. $5^{-1} \bmod 28 =$ **17** ($85 = 3 \cdot 28 + 1$).

E15. $777 = 111 \cdot 7 \equiv 0$, so the answer is **0**. (*Trap:* Fermat needs $\gcd(a, p) = 1$; you can't conclude "≡ 1".) $2^3 \equiv 1 \pmod 7$ and $1000 = 333 \cdot 3 + 1$, so the answer is **2**. $11 \equiv -1 \pmod{12}$, even power: **1**. $46 \equiv -1 \pmod{47}$, odd power: **46** (or by Fermat, $a^p \equiv a$).

E16.

Iter.	q	r_1	r_2	r	t_1	t_2	t
1	2	43	17	9	0	1	−2
2	1	17	9	8	1	−2	**3**
3	1	9	8	**1**	−2	3	−5
4	8	8	1	0	3	−5	43
		1	0		−5		

4 iterations; remainder in iteration 3 = **1**; t in iteration 2 = **3**. $17^{-1} = -5 \equiv$ **38** (check: $17 \cdot 38 = 646 = 15 \cdot 43 + 1$ ✓).

E17. Table: $q = 4, 1, 2, 1, 4$; remainders 28, 10, 8, 2, 0. The final row has $r_1 =$ **2**, not 1, so $\gcd(180, 38) = 2$ and **38 has no inverse** in $\mathbb{Z}_{180}$. The algorithm itself detects the failure. $180 = 2^2 \cdot 3^2 \cdot 5$, so only numbers co-prime to 2, 3 and 5 have inverses: 36 (no), **37 (yes)**, 38 (no), 39 (no, divisible by 3).

E18. Quotients 3, 4, 1, 1, 1, 1, 4, 3, so **9 iterations**. The final $t_1 =$ **550**: $550^{-1} \equiv 550 \pmod{1769}$. *550 is its own inverse*: $550^2 = 302500 = 171 \cdot 1769 + 1$. (Stallings' example; a surprising answer worth checking rather than doubting.)

E19. (i) $\gcd(21, 14) = 7$ and $7 \mid 35$. Divide by 7: $3x + 2y = 5$. Here $x = 1, y = 1$ works, so $x = 1 + 2k$, $y = 1 - 3k$. (ii) $\gcd(40, 15) = 5$ and $5 \mid 110$. Divide: $8x + 3y = 22$; $x = 2, y = 2$ works, so $x = 2 + 3k$, $y = 2 - 8k$. (iii) $\gcd(12, 18) = 6$ and $6 \nmid 25$: **no solution**.

E20. $19x + 13y = 220$, gcd = 1. From Euclid, $19(-2) + 13(3) = 1$, so $x_0 = -440$, $y_0 = 660$, and in general $x = -440 + 13k$, $y = 660 - 19k$. Both must be ≥ 0: $k \geq 33.9$ and $k \leq 34.7$, so $k = 34$: **2 pens and 14 notebooks** ($38 + 182 = 220$). The non-negative answer is **unique**.

E21. (a) $P + K$: 12+3, 4+19, 4+5, 19+0, 12+24, 4+11, 0+7, 19+2, 13+14, 14+9, 14+25, 13+8 = 15, 23, 9, 19, 36, 15, 7, 21, 27, 23, 39, 21. Mod 26: **PXJTKPHVBXNV**. (b) $K = C - P \pmod{26}$ with $P =$ holdposition: **8 9 24 16 21 1 15 13 8 15 25 8**. (c) Every 12-letter message is reachable from this ciphertext with some key, and every key is equally likely, so the ciphertext gives no information (perfect secrecy).

E22. Key LEMONLEMONLEMO: **ASXMNWTTOOPXUQ**.

E23. (a) Rotate row r left by r: $\begin{pmatrix} 32 & 88 & 31 & E0 \\ 5A & 31 & 37 & 43 \\ 98 & 07 & F6 & 30 \\ 34 & A8 & 8D & A2 \end{pmatrix}$. (b) XOR with K: $\begin{pmatrix} 19 & A0 & 9A & E9 \\ 3D & F4 & C6 & F8 \\ E3 & E2 & 8D & 48 \\ BE & 2B & 2A & 08 \end{pmatrix}$ (e.g. 32⊕2B = 0011 0010⊕ 0010 1011 = 0001 1001 = 19). These are the official AES (FIPS-197) test values: the initial AddRoundKey of round 0.

E24. InvShiftRows rotates row r **right** by r: $\begin{pmatrix} D4 & E0 & B8 & 1E \\ BF & B4 & 41 & 27 \\ 5D & 52 & 11 & 98 \\ 30 & AE & F1 & E5 \end{pmatrix}$.

E25. $02 \cdot b$ ("xtime"): shift left 1; if a 1 falls off the top, XOR with 1B. $03 \cdot b = (02 \cdot b) \oplus b$. $02 \cdot 87$: 1000 0111 → 0000 1110 ⊕ 0001 1011 = 15. $02 \cdot 6E = DC$ (no overflow), so $03 \cdot 6E = DC \oplus 6E = B2$. $15 \oplus B2 \oplus 46 \oplus A6 = A7 \oplus 46 \oplus A6 = E1 \oplus A6 =$ **47**. That is exactly the 47 in the 2025–26 paper's "after MixColumns" state.

E26. Row = (first bit, last bit), column = the middle 4 bits. (a) 1 0001 1: row 3, column 1 → 12 = **1100**. (b) 0 1101 1: row 1, column 13 → 5 = **0101**. (c) 1 1011 0: row 2, column 11 → 7 = **0111**.

E27. (a) The shifts are 1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1. Running totals: after round 2 = **2**, after round 9 = **15**, after round 16 = **28**. (b) 28 is the length of each half, so after round 16 C and D are back where they started. Decryption can then generate the keys in reverse with right shifts. (c) PC-1 drops **8 parity bits** (every 8th bit): $64 \to 56$. They are for error checking, not secrecy, so the real key size is 56 bits.

E28. One round: $L_1 = R_0$, $R_1 = L_0 \oplus f(R_0, K_1)$. To go back:

$$R_0 = L_1, \qquad L_0 = R_1 \oplus f(L_1, K_1) \quad (\text{since } R_1 \oplus f(R_0, K_1) = L_0 \oplus f \oplus f = L_0).$$

We only ever *recompute* f (on $L_1 = R_0$, which we know) and XOR it away; we never invert f. So f can be any function (DES's S-boxes aren't invertible). Diagram: two halves, f box fed by the right half and K_i, XOR into the left half, then a swap (see the 2024–25 Q6 answer above).

E29. (i) $p = 11$, $11 \nmid 6$: $6^{10} \equiv$ **1** directly. (ii) $3^{12} = 3^{10} \cdot 3^2 \equiv 1 \cdot 9 =$ **9**. (iii) $2^{-1} \equiv 2^{13-2} = 2^{11} = 2048 = 157 \cdot 13 + 7$, so **7** (check $2 \cdot 7 = 14 \equiv 1$). (iv) $7^{-1} \equiv 7^{17} \bmod 19 =$ **11** (check $7 \cdot 11 = 77 = 4 \cdot 19 + 1$).

E30. Use $a^{-1} \equiv a^{\Phi(n)-1}$. (i) $\Phi(20) = 8$: 3^7; since $3^4 = 81 \equiv 1$, $3^7 \equiv 3^3 = 27 \equiv$ **7**. (ii) $\Phi(15) = 8$: 7^7; powers 7, 4, 13, 1, so $7^4 \equiv 1$ and $7^7 \equiv 7^3 \equiv$ **13**. (iii) $\Phi(77) = 60$: 8^{59}, with $59 = 32 + 16 + 8 + 2 + 1$. Table mod 77: $8^1 = 8$, $8^2 = 64$, $8^4 = 15$, $8^8 = 71$, $8^{16} = 36$, $8^{32} = 64$. Product: $64 \cdot 36 \to 71$, $\times 71 \to 36$, $\times 64 \to 71$, $\times 8 \to$ **29** (check $8 \cdot 29 = 232 = 3 \cdot 77 + 1$). (iv) $\Phi(187) = 160$: $60^{159} \equiv$ **53** (check $60 \cdot 53 = 3180 = 17 \cdot 187 + 1$). (v) $\Phi(100) = 40$: $71^{39} \equiv$ **31** (check $71 \cdot 31 = 2201$). (These are Forouzan's examples. In the exam, extended Euclid is usually faster; Euler is the method they ask for when they name it.)

E31. (i) $222 \bmod 10 = 2$, so $7^2 = 49 \equiv$ **5**. (ii) $532 \bmod 10 = 2$, so $4^2 = 16 \equiv$ **5**. (iii) $\gcd(11, 35) = 1$, $\Phi(35) = 24$, $2024 \bmod 24 = 8$, so 11^8: $11^2 = 121 \equiv 16$, $11^4 \equiv 256 \equiv 11$, $11^8 \equiv 11^2 \equiv$ **16**. (In fact $11^3 \equiv 1$, and $2024 = 3 \cdot 674 + 2$.)

E32. (i) Chain mod 17: 3, 9, 10, 13, 5, 15, 11, 16, 14, 8, 7, 4, 12, 2, 6, 1. All 16 distinct, so **yes**. (Fast test: $16 = 2^4$, so only check $3^8 \equiv 16 \neq 1$.) (ii) $10 = 2 \cdot 5$: a is a primitive root iff $a^5 \not\equiv 1$ and $a^2 \not\equiv 1$. That gives **2, 6, 7, 8** ($\Phi(10) = 4$ of them). (iii) $\Phi(22) =$ **10**. Every primitive root is 5^k with $\gcd(k, 22) = 1$: e.g. $5^3 = 125 = 5 \cdot 23 + 10$, so **10**. ($5^2 = 2$ is *not* one, since $\gcd(2, 22) \neq 1$.)

E33. Orders in $\mathbb{Z}_7^*$: ord(1) = 1, ord(2) = 3, ord(3) = 6, ord(4) = 3, ord(5) = 6, ord(6) = 2. Primitive roots (order $6 = \Phi(7)$): **3 and 5**. $\mathbb{Z}_8^* = \{1, 3, 5, 7\}$: $3^2 = 9 \equiv 1$, $5^2 = 25 \equiv 1$, $7^2 = 49 \equiv 1$. Every order is $\leq 2 < \Phi(8) = 4$, so there is **no primitive root**.

E34. (i) 17 is prime and $17 \nmid 12$: **1**. (ii) $17 \mid 17$, so Fermat does **not** apply: $17^{16} \equiv$ **0**. (iii) $2^4 = 16 \equiv 1 \pmod{15}$, so $2^{14} = 2^{12} \cdot 2^2 \equiv 4 \neq 1$. If 15 were prime, Fermat would force 1, so **15 is composite** (Fermat test). (iv) **No**: $341 = 11 \cdot 31$. Passing the Fermat test doesn't prove primality; 341 is a *pseudoprime* to base 2.

38 Practice Problems

These go beyond the lecture examples, and many contain a trap. Show your working as you would in an exam. Answers start on a new page (Section 39); don't peek.

A. Security definitions and shift ciphers

Q1. A message is worth $500 and is useful for only 1 hour. Breaking the cipher needs $100 of computing time and takes 2 hours. (a) Is the scheme computationally secure? (b) What if breaking it took 30 minutes instead?

Q2. (a) Alice encrypts with a shift cipher, key 7, then encrypts the result again with key 21. Which single key gives the same ciphertext? Did encrypting twice make it stronger? (b) Bob says he used a shift cipher with key 29. Which cipher did he really use? (c) An attacker brute-forcing a shift cipher got readable text by moving every letter *forward* by 16. What was the encryption key?

Q3. This was made with a shift cipher; the key is unknown. Decrypt it, then say which principle from the course the message states.

DROOXOWIUXYGCDROCICDOW

B. Monoalphabetic

Q4. Keyword cipher: write the keyword **SECURITY** (no repeats) under a b c …, then continue with the unused letters in alphabetical order. (a) Encrypt "meet me at noon". (b) Decrypt IMRLPRHCXSHSFXNANVAHN. (c) Which plaintext letter is not disguised at all by this key? Why is this a typical weakness of keyword ciphers?

Q5. (a) The lecture says a monoalphabetic cipher has 26! keys. How many are actually useful, and why? (b) Is the Caesar cipher a monoalphabetic cipher? (c) In a 20-letter ciphertext, Q is the most common letter (4 times). Can you conclude Q = e?

C. Playfair

Q6. Keyword **PLAYFAIR EXAMPLE**. Encrypt "hide the gold in the tree stump".

Q7. Keyword **MONARCHY**. Split "committee" into digrams, then encrypt it.

Q8. Keyword **NETWORK**. Decrypt GCUYOSODNUZY and write the final, readable message.

Q9. (a) Under some Playfair key, AB encrypts to XY. What does BA encrypt to? Does it depend on which of the three rules applied? (b) Can a Playfair plaintext letter ever encrypt to itself? (c) An intercepted message contains the pair QQ (as one digram), and elsewhere the letter J. Could it be Playfair ciphertext?

D. Hill

Q10. Which of these are valid Hill keys (mod 26)? Give the reason for each.

$$K_1 = \begin{pmatrix} 2 & 3 \\ 1 & 4 \end{pmatrix} \quad K_2 = \begin{pmatrix} 4 & 2 \\ 3 & 1 \end{pmatrix} \quad K_3 = \begin{pmatrix} 13 & 1 \\ 2 & 1 \end{pmatrix} \quad K_4 = \begin{pmatrix} 5 & 4 \\ 3 & 5 \end{pmatrix} \quad K_5 = \begin{pmatrix} 1 & 4 \\ 3 & 1 \end{pmatrix}$$

Q11. $K = \begin{pmatrix} 3 & 3 \\ 2 & 5 \end{pmatrix}$. (a) Encrypt "help" with $C = PK$ (the lecture's convention). (b) A textbook using $C = KP$ gets a different answer. What is it? (c) Why do they differ?

Q12. $K = \begin{pmatrix} 5 & 8 \\ 17 & 3 \end{pmatrix}$. (a) Find K^{-1} mod 26 and check it. (b) Decrypt EGXJ.

Q13. Known-plaintext attack. A 2 × 2 Hill cipher ($C = PK$) encrypts "help" to PQEX. Find the key K. Then decrypt CKJM, which was sent with the same key.

E. Polyalphabetic

Q14. Vigenère, key **NESO**. Decrypt SVWEHIFQLEFOYCKWFJSWYWZSEI.

Q15. (a) What does a Vigenère cipher with key AAAA do? And with key DDDD? (b) In a long Vigenère ciphertext, the sequence KWX appears at positions 10, 34 and 70. What can you say about the key length? (c) Suppose you know the key length is 5. How would you break it?

Q16. Autokey, keyword **KEY**. (a) Encrypt "meet at noon". (b) Encrypt it with ordinary Vigenère (key KEY) as well. Where do the two ciphertexts first differ, and why? (c) Why must an autokey receiver decrypt strictly left to right?

F. Vernam and one-time pad

Q17. (a) P = 10110011, K = 01101010. Find C. (b) Compute $C \oplus P$. What do you get, and why? (c) Why doesn't (b) weaken a one-time pad?

Q18. Two ciphertexts were made with the **same** one-time pad key: C_1 = JUVUZEXJGN, C_2 = YHZOFXYIWY. You know C_1 is "meet at noon". Recover the key and the second message.

Q19. The ciphertext XMCKL came from a one-time pad. (a) Find a key that decrypts it to "hello" and another that decrypts it to "later". (b) What does this show?

Q20. Which of these give **perfect secrecy**? (a) A one-time pad whose random key is used for two messages. (b) Vigenère with a truly random key as long as the message, used once. (c) Vernam with a random 1,000-bit tape, looped to encrypt a 10,000-bit message. (d) A key stream produced from a random 128-bit seed by a fixed formula. (e) A Caesar shift with a randomly chosen key, used once on a one-letter message.

G. Transposition

Q21. Rail fence, depth 3. Decrypt WECRLTEERDSOEEFEAOCAIVDEN.

Q22. Row–column transposition, 4 columns, key **3 1 4 2**, padded with x. Decrypt EEEETTFTAXMMTHREARPY.

Q23. (a) A ciphertext has normal English letter frequencies (lots of E, T, A) but is unreadable. Substitution or transposition? (b) Show that a rail fence of depth 2 is really a row–column transposition. Which rectangle and key? (c) Encrypt "retreat now" with Caesar (k = 3) and then rail fence (depth 2). Now do it in the other order. Compare the results and explain.

H. Steganography, key counting, primes

Q24. A captured note reads: "*Don't wait, we'll discuss food now.*" Analysts believe a message is hidden in it and that a Caesar cipher was also used. Recover the message. Which approach from the lecture did the sender use?

Q25. LSB steganography, 2 LSBs per channel, bits hidden in the order R, G, B. (a) Hide the bits 101101 in the pixel $(200, 13, 87)$. What is the new pixel? (b) What is the largest amount any channel value can change? (c) How many bytes can a 1024×768 image hide at most? (d) Why must the stego image not be saved as a JPEG?

Q26. A company of 100 employees uses symmetric keys so that every pair can talk privately. (a) How many keys are needed? (b) One more employee joins. How many *new* keys are needed? (c) How many keys would public-key cryptography need for the 101 employees?

Q27. Columnar transposition with keyword **LETTER** (repeated letters numbered left to right), padded with x. Decrypt:

TOELXCPNOXAPNIMKOTAXTSDTXATUWX

Q28. Compute: (a) $-19 \bmod 3$ (b) $3 \bmod 19$ (c) $2^{10} \bmod 11$. For (c), what do you notice? Which upcoming theorem does it preview?

Q29. (a) Classify each as prime, composite or neither: 1, 2, 51, 91, 97, 221. (b) To test whether 221 is prime by trial division, what is the largest divisor you need to try? Why? (c) What is wrong with the sentence "RSA is secure because factoring large prime numbers is hard"?

Q30. (a) $100 = 2^2 \cdot 5^2$ has 9 divisors. Find a rule that gets the number of divisors straight from the prime factorisation, and use it for $5040 = 2^4 \cdot 3^2 \cdot 5 \cdot 7$. (b) Which numbers have *exactly three* divisors? Give three examples.

I. Modular arithmetic

Q31. Valid or invalid? (a) $73 \equiv 13 \pmod{12}$ (b) $-17 \equiv 3 \pmod{5}$ (c) $100 \equiv 1 \pmod{9}$ (d) $-7 \equiv 7 \pmod{7}$ (e) $14 \equiv -14 \pmod{6}$ (f) $0 \equiv 26 \pmod{13}$ (g) $38 \bmod 12 = 14$

Q32. Without a calculator: (a) $(1234 \times 5678) \bmod 9$ (b) $7^{100} \bmod 10$ (c) $2^{50} \bmod 7$ (d) $-123 \bmod 10$

Q33. Work in $\mathbb{Z}_{10}$. (a) Give the additive inverses of 3, 0 and 5. (b) Give the multiplicative inverses of 3, 7, 9, 5 and 4, where they exist. (c) Explain why 4 can never have one.

Q34. Find *all* solutions in $\mathbb{Z}_{12}$: (a) $4x \equiv 8 \pmod{12}$ (b) $3x \equiv 8 \pmod{12}$ (c) $5x \equiv 3 \pmod{12}$

Q35. (a) What time does a 12-hour clock show 100 hours after 9 o'clock? (b) 50 hours before 3 o'clock? (c) Use additive inverses to explain why decrypting a shift cipher with key k is the same as *encrypting* with key $26 - k$. What is the "encryption key" that decrypts Caesar?

Q36. Rail fence, depth 4. Decrypt TIRTHYSETAEEUDHMKNE. (Careful: the rows are not all the same length.)

J. Modular exponentiation

Q37. (Lecture homework 1) Compute $11^{23} \bmod 187$ by repeated squaring. How is the answer related to the lecture's $88^7 \bmod 187$?

Q38. (Lecture homework 2) Find the last three digits of 175^{209}.

Q39. Compute $2^{33} \bmod 30$ (the example on the definition slide). Do it two ways: repeated squaring, and by spotting a cycle in the powers of 2.

Q40. (a) Find the last two digits of 7^{222}. (b) Find its last digit.

Q41. A student computes $5^{117} \bmod 19$ like this: "$117 \bmod 19 = 3$, so the answer is $5^3 = 125 \equiv 11$." (a) What is wrong? (b) Find the correct answer.

Q42. (a) $100^{101} \bmod 101$ (b) $100^{100} \bmod 101$ (c) $99^{99} \bmod 100$ (d) How many multiplications does repeated squaring need for a^{100}, compared with multiplying a by itself over and over? And for a^{1024}?

K. GCD and the Euclidean algorithm

Q43. (Lecture homework) Find $\gcd(1005, 105)$ with the table method and $\gcd(529, 123)$ with the recursive method. Are 529 and 123 relatively prime? Is either of them prime?

Q44. True or false? Give a reason or a counterexample for each. (a) If $\gcd(a, b) = 1$ then a and b are both prime. (b) Two even numbers can be relatively prime. (c) Any two consecutive integers n and $n + 1$ are relatively prime. (d) $\gcd(a, b) = \gcd(a, a + b)$ for all positive a, b. (e) In the table method you can stop as soon as $R = 0$ and read the answer from column B.

Q45. (a) Find $\gcd(89, 55)$ and count the divisions. (b) Why does it take so many steps for such small numbers?

Q46. A Hill cipher works mod 26. (a) Which values from 1 to 25 can the determinant of a valid key take? How many are there? (b) Use the Euclidean algorithm to show that $\gcd(13, 26) \neq 1$ but $\gcd(15, 26) = 1$.

Q47. (a) $\gcd(0, 17)$ (b) $\gcd(17, 17)$ (c) $\gcd(1, 10^{100})$ (d) Someone starts the table method for $\gcd(105, 1005)$ with $A = 105$, $B = 1005$ (the wrong way round). What happens in the first row? Do they still get the right answer?

L. Relatively prime numbers and Euler's totient

Q48. (Lecture homework) (a) Use Euclid to decide whether 790 and 121 are relatively prime. (b) Find $\Phi(369)$ and $\Phi(372)$.

Q49. Spot the mistakes. (a) A student writes $\Phi(25) = (5-1)(5-1) = 16$. (b) Another writes $\Phi(1000) = 1000\,(1-\frac{1}{2})^3(1-\frac{1}{5})^3$ "because $1000 = 2^3 \cdot 5^3$". (c) A third says $\Phi(12) = \Phi(2) \cdot \Phi(6) = 1 \cdot 2 = 2$. Give the correct values, and also find $\Phi(27)$, $\Phi(1024)$ and $\Phi(97)$.

Q50. These are totient questions in disguise. Say why, then answer. (a) How many of the fractions $\frac{1}{36}, \frac{2}{36}, \ldots, \frac{35}{36}$ are already in lowest terms? (b) How many integers from 1 to 1000 are divisible by neither 2 nor 5? (c) A "multiplicative cipher" encrypts $C = k \cdot P \bmod 26$. How many values of k give a usable cipher?

Q51. (a) Find *all* n with $\Phi(n) = 4$. (Hint: if a prime p divides n, then $p - 1$ divides $\Phi(n)$.) (b) Show that $\Phi(n)$ is even for every $n \geq 3$.

Q52. ★ Prove that **no** integer n has $\Phi(n) = 14$.

Q53. ★ RSA with $n = 187$. (a) Given $187 = 11 \cdot 17$, find $\Phi(187)$. (b) Check that $e = 7$ and $d = 23$ satisfy $e \cdot d \equiv 1 \pmod{\Phi(n)}$. (c) Eve doesn't know 11 and 17, but she steals $\Phi(n) = 160$. Show that she can now factor n. (d) What does (c) tell you about keeping $\Phi(n)$ secret?

M. ★ Challenge set (mixed topics)

Each of these hides a trap or needs ideas from two different sections.

Q54. The **multiplicative cipher** encrypts $C = k \cdot P \bmod 26$ (a = 0, ..., z = 25). (a) Why can't $k = 13$ be used? Find two plaintext letters that collide. (b) Which two letters are *never* disguised, whatever valid k is chosen? (c) Adding a shift, $C = (kP + b) \bmod 26$, gives the *affine* cipher. How many keys does it have?

Q55. Alice and Bob use a perfect one-time pad (fresh random key, used once). Alice's answer is either "YES" or "NO". Eve intercepts a 3-letter ciphertext. What does Eve learn? Does this contradict perfect secrecy? How should Alice fix it?

Q56. (a) A message is Vigenère-encrypted with key AB, then the result is encrypted again with key BCD. Show that this is a single Vigenère cipher and find its key. (b) If the two keys have lengths 4 and 6, how long is the combined key? (c) Is double Vigenère much stronger?

Q57. Known plaintext: Vigenère turns `thequickbrownfox` into VYCFNKTIQKQNLUHZ. Find the key and its length.

Q58. Find (a) $3^{(3^3)} \bmod 10$ and (b) $(3^3)^3 \bmod 10$. Why are they different?

Q59. Without calculating the huge numbers: (a) show that $10^{101} + 1$ is divisible by 11; (b) find $(10^{100} + 1) \bmod 101$.

Q60. Find $\gcd(2^{30} - 1,\ 2^{20} - 1)$ without multiplying anything out. (Hint: $2^{30} - 1 = 2^{10}(2^{20} - 1) + (2^{10} - 1)$.)

Q61. A rail fence of depth 2 is applied to `abcdefghijklmnop` (16 letters), then again to the result, and so on. (a) After how many encryptions does the original text come back? (b) What if the message had 20 letters? (c) What does this say about "encrypt twice for extra security"?

Q62. (a) Split "taxxi" into Playfair digrams using the lecture's rules. What goes wrong, and how do you fix it? (b) A Playfair ciphertext contains RS and, later, SR. What can an attacker deduce?

Q63. Hill key $K = \begin{pmatrix} 1 & 2 \\ 0 & 25 \end{pmatrix}$ ($C = PK$). (a) Show $K^2 \equiv I \pmod{26}$. (b) Encrypt "hi", then encrypt the result again. (c) Look at the first letter of every block. Why is this key a bad choice despite being valid?

Q64. Brute-force the shift-cipher ciphertext FKHHU. How many readable answers are there? What does this mean for the lecture's claim that brute force reveals the one meaningful plaintext?

N. ★ Fermat, Euler, primitive roots

Q65. (a) Show $2^{340} \equiv 1 \pmod{341}$ in two lines. (Hint: look at 2^{10}.) (b) Show that base 3 exposes 341 as composite, *without* computing $3^{340} \bmod 341$. (Hint: work mod 31.)

Q66. Find the last two digits of (a) 3^{2024} and (b) 2^{2024}. Careful: can Euler's theorem be used in (b)?

Q67. 6 is a primitive root of 41. (a) How many primitive roots does 41 have? (b) Find two more without building any table. (c) Explain why 6^2 is *not* a primitive root.

Q68. (a) Is 2 a primitive root of 101? Decide with at most two modular powers. (b) Is 2 a primitive root of 17?

Q69. Toy Diffie–Hellman with $p = 23$ and $g = 5$. Alice's secret is 6 and Bob's is 15. (a) Find what each sends and the shared key. (b) Why is g chosen to be a primitive root? (c) What would go wrong with $g = 22$?

Q70. (a) A student computes $4^{-1} \bmod 9$ as $4^{9-2} \bmod 9$. What is wrong, and what is the right answer? (b) Applying Euler's formula blindly to $6^{-1} \bmod 9$ gives $6^{\Phi(9)-1} = 6^5 \bmod 9$. What do you get, and what does it show?

O. ★ Attacks and Lecture 5 extras

Q71. Classify (attack, goal threatened, passive/active), and say whether encryption alone stops it: (a) Eve can't read the CEO's encrypted mail but notices it goes to a law firm every night at 2 a.m.; (b) a buyer receives goods, then claims she never ordered them; (c) Eve resends a captured *encrypted* "pay Rs. 500 to Eve" message; (d) a botnet floods a web server.

Q72. (a) Why is $(k_1, k_2) = (13, 5)$ an illegal affine key? Show two letters that collide. (b) Known plaintext: an affine cipher maps e→R and n→Y. Find the key and decrypt KTF.

Q73. A Kasiski test finds repeated trigrams at distances 30, 42, 66 and 18. Which key lengths are possible, and which do you try first? Why does this attack fail completely against the one-time pad?

Q74. Keyed transposition, blocks of 5, encryption key 4 2 5 1 3. (a) Find the decryption key. (b) Encrypt `attackatdawn` (pad with z). (c) The key happens to be its own inverse. Is that a weakness?

Q75. Hill 2×2 (row vectors, $C = PK$). Eve knows `help` encrypts to DPLE. Recover K.

Q76. Lecture 6 says composing two permutations gives a single permutation, so double transposition adds no strength. Lecture 5 says double transposition defeats the pattern attack. Reconcile the two.

P. ★ Extended Euclid, congruences, Diophantine

Q77. Full table: $\gcd(1820, 231)$ and s, t with $1820s + 231t = \gcd$.

Q78. Find $97^{-1} \bmod 1000$ with the table. Without any table, why does $98^{-1} \bmod 1000$ not exist?

Q79. Solve $35x \equiv 10 \pmod{50}$ (all solutions in $\mathbb{Z}_{50}$). Then $35x \equiv 11 \pmod{50}$.

Q80. Using $(84)(-19) + (320)(5) = 4$: solve $84x + 320y = 8$ (particular and general) and explain why $84x + 320y = 10$ has no solution.

Q. ★ Primes, primality, CRT, discrete logs

Q81. 341 passes the Fermat test with base 2 ($2^{340} \equiv 1 \pmod{341}$). Run Miller–Rabin with base 2. Then run it on $2047 = 23 \times 89$ with base 2 and with base 3. What do the results teach you?

Q82. (a) Solve $x \equiv 1\ (3),\ 2\ (4),\ 3\ (5)$. (b) Trap: solve $x \equiv 3 \pmod 6$, $x \equiv 5 \pmod 8$, and then $x \equiv 2 \pmod 6$, $x \equiv 5 \pmod 8$. Can the four-step CRT be used?

Q83. Factor 8051 by Fermat's method. Why is it so fast here?

Q84. Given that 2 is a primitive root of 11, list all primitive roots of 11 without testing each number. How many should there be?

Q85. Solve (a) $2^x \equiv 9 \pmod{11}$; (b) $3^x \equiv 4 \pmod{11}$; (c) $3^x \equiv 2 \pmod{11}$.

Q86. $\operatorname{ord}_{17}(2) = 8$. Can any element of $\mathbb{Z}_{17}^*$ have order 6? Order 16? Which $a \in \{2, 3, 4\}$ is a primitive root?

Q87. Compute $5^{117} \bmod 19$ with the least work, then explain what the answer says about $\operatorname{ord}_{19}(5)$.

R. ★ Algebraic structures and GF(2^n)

Q88. $\langle \mathbb{Z}_{10}^*, \times \rangle$ is cyclic. Is $\langle \mathbb{Z}_{12}^*, \times \rangle$? Justify with orders.

Q89. Group, ring or field (and why): $\langle \mathbb{Z}_{11}, +, \times \rangle$; $\langle \mathbb{Z}_{12}, +, \times \rangle$; GF($2^4$); $\langle \mathbb{Z}, +, \times \rangle$; $\langle \mathbb{Z}_9^*, \times \rangle$.

Q90. In GF(2^4) with $x^4 + x + 1$: compute $(x^3 + x + 1)(x^2 + 1)$, and find $(x^3 + x^2)^{-1}$ and x^{-1}.

Q91. $|\mathbb{Z}_{15}^*| = 8$. Can it contain an element of order 3? Order 8? Is it cyclic?

Q92. AES arithmetic: compute $57 \cdot 83$ and $57 \cdot 13$ in GF(2^8).

Q93. $x^4 + x^2 + 1$ has no root in GF(2) (it equals 1 at both $x = 0$ and $x = 1$). Is it irreducible?

S. ★ Modern block ciphers, DES, AES

Q94. A 1000-byte file is encrypted with AES. How many padding bits, and how many blocks? With DES?

Q95. Which of these can appear in a non-Feistel cipher: a 32×48 expansion P-box, a 6×4 S-box, a 64×64 straight P-box, a 4×4 S-box?

Q96. DES S-box 1: find the outputs for `101010`, `110011`, `010101`.

Q97. DES key schedule: how far has each half rotated in total after round 3, after round 9, after round 16? Why must the total be 28?

Q98. One Feistel round: $L_0 = 1010$, $R_0 = 0110$, $f(R_0, K_1) = 1100$. Find L_1, R_1, then decrypt them back.

Q99. Double DES has a 112-bit key. Explain why its strength is only about 2^{57}, and what the attack needs in memory.

Q100. AES-128 key expansion: for how many of the words $w_4..w_{43}$ is SubWord applied? How many round keys does AES-256 use?

Q101. Compute MixColumns of the columns `[01 01 01 01]`, `[C6 C6 C6 C6]` and `[01 00 00 00]`. What does this show?

Q102. SubBytes maps `00` to `63`, not `00`, even though 00^{-1} is defined as `00`. Why?

T. ★ RSA

Q103. $p = 13$, $q = 17$, $e = 5$: find d, encrypt $P = 65$, decrypt.

Q104. $p = 7$, $q = 13$, $e = 3$. What goes wrong?

Q105. Two users publish $n_1 = 143$ and $n_2 = 187$. Break both.

Q106. You learn $n = 3233$ and $\phi(n) = 3120$. Find p, q and, for $e = 17$, d.

Q107. $e = 3$, n is 2048 bits, Alice sends $P = 2$. Eve sees $C = 8$. What's wrong?

U. ★ ElGamal and ECC

Q108. ElGamal: $p = 23$, $e_1 = 5$, $d = 6$. Encrypt $P = 10$ with $r = 3$, and decrypt.

Q109. Alice reuses $r = 3$ (same keys as above) and sends a second ciphertext $(10, 10)$. Eve knows the first plaintext was 10. Find the second plaintext.

Q110. $E_{11}(1, 6)$: list all points, count them, and compute $2 \cdot (2, 7)$.

Q111. Is $y^2 = x^3$ over $\mathbb{Z}_p$ allowed as an ECC curve? What does Hasse say about $\#E$ for $p = 101$?

Q112. ECC-ElGamal on $E_{13}(1, 1)$: $e_1 = (1, 4)$, $d = 3$. Find e_2, encrypt $P = (5, 1)$ with $r = 2$, and decrypt.

V. ★ Message integrity

Q113. For a 128-bit digest give the work for a preimage and for a collision attack (50%). Which number is the real security level, and why is MD5 broken while a 128-bit AES key isn't?

Q114. A 40-bit digest: about how many messages give a 50% chance of a collision?

Q115. Match each to an attack and a birthday problem: (a) Eve has the stored hash of a password and wants *any* matching password; (b) Eve has a signed contract and its digest and wants a different contract with the same digest; (c) Eve prepares a fair and an unfair contract, each in many reworded versions, looking for a matching pair.

Q116. SHA-512: padding for a 3000-bit message, number of blocks, and number of words in the expanded schedule per block.

Q117. Rabin's scheme with DES produces a 64-bit digest. Why is it insecure, and what does Davies–Meyer change?

Q118. Compute Majority and Conditional for the nibbles $x = 1010$, $y = 1100$, $z = 0110$.

Q119. 30 students: probability that two share a birthday (approximation and exact).

39 Answer Key

A. Security definitions and shift ciphers

A1. (a) **Yes.** The cost criterion fails ($100 < $500), but the time criterion holds (2 h > 1 h). Meeting *either* criterion is enough. (b) **No.** Now it's cheap *and* fast (30 min < 1 h), so neither criterion holds. *Trap:* thinking both conditions must hold.

A2. (a) $7 + 21 = 28 \equiv \mathbf{2}$. Two shifts combine into one shift, so it is **no stronger**: still only 25 keys to try. (b) $29 \bmod 26 = 3$: the **Caesar cipher**. (c) Moving forward 16 is the same as moving back 10 ($16 \equiv -10$). The encryption key is **10**, not 16 (compare the GERMANY example).

A3. O appears 5 times out of 22, so guess O = e: key = 14 − 4 = **10**. Shift every letter back 10: **"the enemy knows the system"**. This is **Kerckhoffs's principle** (Shannon's maxim): assume the attacker knows the algorithm; only the key is secret.

B. Monoalphabetic

A4. Cipher line:

Plain	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Cipher	S	E	C	U	R	I	T	Y	A	B	D	F	G	H	J	K	L	M	N	O	P	Q	V	W	X	**Z**

(a) **GRROGRSOHJJH**. (b) Look each letter up in the *bottom* row: **"frequency analysis wins"**. (c) **z → Z**: it maps to itself. The keyword uses no late letters, so the end of the alphabet barely moves (w→V, x→W, y→X, z→Z). A short keyword leaves much of the alphabet in near-normal order.

A5. (a) 26! − 1. The identity permutation (a→A, b→B, …) leaves the message unchanged, just like shift key 0. (b) **Yes.** Every shift is one particular permutation of the alphabet; Caesar is one of the 26!. (c) **No.** Frequency analysis needs a long ciphertext. In 20 letters the counts are too small to trust (the lecture's example had 120 letters).

C. Playfair

A6. "PLAYFAIR EXAMPLE" without repeats is P L A Y F I R E X M; then B C D G H K N O Q S T U V W Z.

P	L	A	Y	F
I	R	E	X	M
B	C	D	G	H
K	N	O	Q	S
T	U	V	W	Z

Digrams: `hi de th eg ol di nt he tr ex es tu mp`. *Trap:* in "tree", `t r e e` gives `tr` then `ee`, a double in one pair, so it becomes `ex` and the second `e` moves on (`es`).
Answer: **BMODZBXDNABEKUDMUIXMMOUVIF** (`ex`→XM and `tu`→UV are row cases; `de`→OD is a column case; the rest are rectangles.)

A7. `c o m m i t t e e` → `co`, `mm`→`mx`, then `mi`, `tt`→`tx`, then `te`, then `e` alone → `ex`. Digrams: **`co mx mi tx te ex`**. *Trap:* `ee` does *not* get a filler: after re-pairing, the two e's are in different pairs. All six are rectangles: **HM AU AE SZ LK IU** ⇒ **HMAUAESZLKIU**.

A8. Matrix, then decrypt (row → left, column → up, rectangle as usual):

N	E	T	W	O
R	K	A	B	C
D	F	G	H	I
L	M	P	Q	S
U	V	X	Y	Z

`GC`→`ia`, `UY`→`zx`, `OS`→`zi`, `OD`→`ni`, `NU`→`ul`, `ZY`→`yx` ⇒ `iazxziniulyx`.
Remove the fillers (`zx z` was a split `zz`; the final `x` is padding): `iazzinjuly`. Decide i/j from context: **"jazz in july"**.

A9. (a) **YX**, always. In every rule, each letter's replacement depends only on its own cell and the other letter's cell, not on the order. (Lecture: `at`→RS, `ta`→SR; also `mo`→ON, `om`→NO.) (b) **Never.** Right neighbour, cell below and other-column corner are all different cells. (c) **No, for two reasons.** Plaintext pairs never contain two equal letters, and two different cells always map to two different cells, so a ciphertext pair is never doubled. And J never appears in Playfair ciphertext (the shared cell is written I).

D. Hill

A10. Valid $\iff$ $\gcd(\det K, 26) = 1$, i.e. det (mod 26) is odd and not 13.

	det	mod 26	Valid?	Why
K_1	$8-3=5$	5	**yes**	odd, not 13
K_2	$4-6=-2$	24	**no**	even (shares factor 2)
K_3	$13-2=11$	11	**yes**	an entry of 13 is fine; only the det matters
K_4	$25-12=13$	13	**no**	shares factor 13
K_5	$1-12=-11$	15	**yes**	negative is fine: $-11+26=15$

A11. (a) he = $(7,4)$: $(7\cdot3+4\cdot2,\ 7\cdot3+4\cdot5) = (29,41) \equiv (3,15) =$ DP. lp = $(11,15)$: $(33+30,\ 33+75) = (63,108) \equiv (11,4) =$ LE. Answer: **DPLE**.
(b) $C = KP$ with column vectors: $(3\cdot7+3\cdot4,\ 2\cdot7+5\cdot4) = (33,34) \equiv (7,8) =$ HI; $(3\cdot11+3\cdot15,\ 2\cdot11+5\cdot15) = (78,97) \equiv (0,19) =$ AT. Answer: **HIAT**.
(c) KP with K is the same as PK with the **transpose** K^T. Same key numbers, different matrix, so a different cipher. Always state the convention.

A12. (a) $\det K = 15 - 136 = -121 \equiv -121 + 130 = 9$, and $9^{-1} \equiv 3$ (since $27 = 26 + 1$). For 2×2 there is a shortcut: swap the diagonal, negate the other two.

$$\operatorname{adj}\begin{pmatrix} a & b \\ c & d \end{pmatrix} = \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \quad\Rightarrow\quad \operatorname{adj} K = \begin{pmatrix} 3 & -8 \\ -17 & 5 \end{pmatrix} \equiv \begin{pmatrix} 3 & 18 \\ 9 & 5 \end{pmatrix}$$

$$K^{-1} = 3\begin{pmatrix} 3 & 18 \\ 9 & 5 \end{pmatrix} = \begin{pmatrix} 9 & 54 \\ 27 & 15 \end{pmatrix} \equiv \boxed{\begin{pmatrix} 9 & 2 \\ 1 & 15 \end{pmatrix}} \qquad \text{Check: } KK^{-1} = \begin{pmatrix} 53 & 130 \\ 156 & 79 \end{pmatrix} \equiv I \checkmark$$

(b) EG = $(4,6)$: $(4\cdot9+6\cdot1,\ 4\cdot2+6\cdot15) = (42,98) \equiv (16,20) =$ qu. XJ = $(23,9)$: $(207+9,\ 46+135) = (216,181) \equiv (8,25) =$ iz. Answer: **quiz**.

A13. Stack the blocks as rows: $P = \begin{pmatrix} 7 & 4 \\ 11 & 15 \end{pmatrix}$ (he, lp), $C = \begin{pmatrix} 15 & 16 \\ 4 & 23 \end{pmatrix}$ (PQ, EX). Since $C = PK$, we get $K = P^{-1}C$.

$\det P = 105 - 44 = 61 \equiv 9$, and $9^{-1} = 3$. So $P^{-1} = 3\begin{pmatrix} 15 & -4 \\ -11 & 7 \end{pmatrix} \equiv \begin{pmatrix} 19 & 14 \\ 19 & 21 \end{pmatrix}$.

$$K = \begin{pmatrix} 19 & 14 \\ 19 & 21 \end{pmatrix}\begin{pmatrix} 15 & 16 \\ 4 & 23 \end{pmatrix} = \begin{pmatrix} 341 & 626 \\ 369 & 787 \end{pmatrix} \equiv \boxed{\begin{pmatrix} 3 & 2 \\ 5 & 7 \end{pmatrix}}.$$

To decrypt: $\det K = 11$, $11^{-1} = 19$ $(209 = 8\cdot26+1)$, $K^{-1} = 19\begin{pmatrix} 7 & -2 \\ -5 & 3 \end{pmatrix} \equiv \begin{pmatrix} 3 & 14 \\ 9 & 5 \end{pmatrix}$. CK = $(2,10) \to (96,78) \equiv (18,0) =$ sa; JM = $(9,12) \to (135,186) \equiv (5,4) =$ fe. Answer: **safe**.
Lesson: Hill is linear, so a few known plaintext–ciphertext pairs reveal the whole key. This only works when P is invertible (here $\det P = 9$).

E. Polyalphabetic

A14. $P = (C - K) \bmod 26$ with key NESONESO…. The first letters, with the negatives fixed:

C	S 18	V 21	W 22	E 4	H 7	I 8	F 5	Q 16
K	N 13	E 4	S 18	O 14	N 13	E 4	S 18	O 14
$C-K$	5	17	4	−10	−6	4	−13	2
mod 26	5 f	17 r	4 e	16 q	20 u	4 e	13 n	2 c

Answer: **“frequency analysis fails here”**.

A15. (a) AAAA: shift 0 everywhere, so **nothing is encrypted**. DDDD: shift 3 everywhere, which is just the **Caesar cipher**. A key whose letters are all equal is a shift cipher. (b) The distances are $34-10=24$ and $70-34=36$ (and 60). Repeats happen when the same plaintext meets the same part of the key, so the distances are multiples of the key length. The key length divides $\gcd(24,36) = 12$: it is **1, 2, 3, 4, 6 or 12**, most likely 6 or 12. (This is the Kasiski idea mentioned in the notes.) (c) Split the ciphertext into 5 groups: letters 1, 6, 11, …; 2, 7, 12, …; and so on. Each group was shifted by **one** key letter, so it is a **Caesar cipher**. Break each with frequency analysis (its most common letter is probably e).

A16. (a) Key stream KEY + meetatn = keymeetatn. Ciphertext: **WICFEXGOHA**. (b) Vigenère: WICDERXSMX. They agree on WIC (both use the keyword KEY) and **differ from letter 4**: Vigenère repeats KEY, while autokey switches to the plaintext m, e, e, … (c) From letter 4 on, the key *is* the plaintext. The receiver learns each key letter only by decrypting an earlier letter first.

F. Vernam and one-time pad

A17. (a) C = 11011001. (b) $C \oplus P$ = 01101010 = K, because $P \oplus K \oplus P = K$. Knowing a plaintext and its ciphertext reveals the key. (c) That key is used for **one message only** and then destroyed. Knowing it tells you nothing about any other message.

A18. $K = C_1 - P_1$: J–m = 9 – 12 = –3 → 23 = x, U–e = 20 – 4 = 16 = q, … ⇒ K = **xqrbzlkvsa**. $P_2 = C_2 - K$: Y–x = 24 – 23 = 1 = b, H–q = 7 – 16 = –9 → 17 = r, … ⇒ **"bring money"**. *Lesson:* reusing a pad key is fatal. One known message reveals the other.

A19. (a) $K = C - P$. For "hello": **qirzx**. For "later": **mmjgu**. (b) Every 5-letter message is possible with some key, so the ciphertext reveals nothing. This is **perfect secrecy**, and it is why brute force is useless against an OTP.

A20. (a) **No**: key reuse (see A18). (b) **Yes**: that *is* a one-time pad; Vigenère's formula with a perfect key. (c) **No**: the key repeats 10 times. (d) **No**: only 2^{128} possible key streams, so it is at best computationally secure (a stream cipher). (e) **Yes**: a random shift used once on one letter is a one-time pad of length 1. Every plaintext letter is equally likely. *Trap:* "Caesar is weak" is only true when the key is reused across many letters.

G. Transposition

A21. 25 letters; the zig-zag repeats every 2(3 – 1) = 4 letters. Row 1 holds positions 1, 5, 9, …, 25 (7 letters); row 3 holds 3, 7, …, 23 (6 letters); row 2 the rest (12). Cut: WECRLTE | ERDSOEEFEAOC | AIVDEN, then read along the zig-zag: **"we are discovered flee at once"**.

A22. 20 letters, 4 columns ⇒ 5 rows, 5 letters per column. *Trap:* key **3 1 4 2** means column 1 is read *third*, column 2 *first*, column 3 *fourth*, column 4 *second*. So the read order is col 2, col 4, col 1, col 3: col 2 = EEEET, col 4 = TFTAX, col 1 = MMTHR, col 3 = EARPY.

m	e	e	t
m	e	a	f
t	e	r	t
h	e	p	a
r	t	y	*x*

Read rows: **"meet me after the party"** (+ filler x).

(Notice the ciphertext starts EEEE: transposition keeps letters, so the e's stay visible.)

A23. (a) **Transposition**: it moves letters but never changes them, so the frequencies stay English. (b) Write the text row by row into a rectangle with **2 columns** (letters 1, 2 on row 1; 3, 4 on row 2; …), then read column 1, then column 2 (key **1 2**). Column 1 holds the odd-position letters, which is exactly rail fence row 1. For "neso academy is the best" both give NSAAEYSHBSEOCDMITEET. (c) Both orders give **UWHWRHUDQZ**. Caesar changes each letter whatever its position; transposition moves positions whatever the letter. So they commute, and the order doesn't matter.

H. Steganography, key counting, primes

A24. First letters: **D**on't **w**ait, **w**e'll **d**iscuss **f**ood **n**ow ⇒ DWWDFN. That isn't a word, so undo Caesar (shift back 3): **ATTACK**. The sender **encrypted first, then hid** the ciphertext. Even someone who spots the hidden letters only gets ciphertext.

A25. (a) Bits 10 | 11 | 01: R 200 = 11001**00** → 11001**10** = 202; G 13 = 00001**01** → 00001**11** = 15; B 87 = 01010**11** → 01010**01** = 85. New pixel: (**202**, **15**, **85**). (b) **3**: the last two bits can go from 00 to 11 (0 → 3) or back. *Trap:* answering 2 because "2 bits". (c) 1024 · 768 · 6 = 4,718,592 bits = **589,824 bytes** (576 KB). (d) JPEG is **lossy**: compression changes pixel values slightly, which destroys the LSBs. Use a lossless format like PNG (as OpenStego did).

A26. (a) $\frac{100 \cdot 99}{2}$ = **4950**. (b) The newcomer needs a key with each of the 100 existing people: **100** new keys. (Check: $\frac{101 \cdot 100}{2}$ – 4950 = 5050 – 4950 = 100.) (c) One key pair each: **101 pairs = 202 keys**. It grows linearly instead of roughly $N^2/2$.

A27. Keyword L E T T E R → E(col 2) = 1, E(col 5) = 2, L = 3, R = 4, T(col 3) = 5, T(col 4) = 6. Key: **3 1 5 6 2 4**. 30 letters, 6 columns ⇒ 5 letters per column. Cut the ciphertext into fives and put them back in read order (col 2, 5, 1, 6, 3, 4): col 2 = TOELX, col 5 = CPNOX, col 1 = APNIM, col 6 = KOTAX, col 3 = TSDTX, col 4 = ATUWX.

a	t	t	a	c	k
p	o	s	t	p	o
n	e	d	u	n	t
i	l	t	w	o	a
m	*x*	*x*	*x*	*x*	*x*

Rows: **"attack postponed until two am"**

A28. (a) $-19 = (-7)\cdot 3 + \mathbf{2}$, so the answer is **2**. *Trap:* not −1; the remainder must be in $0, 1, 2$. (b) $3 = 0 \cdot 19 + 3$, so the answer is **3**. A smaller number mod a bigger one is itself. (c) $2^{10} = 1024 = 93 \cdot 11 + 1$, so the answer is **1**. Here 11 is prime and the exponent is 11 − 1, which previews **Fermat's theorem**: $a^{p-1} \equiv 1 \pmod{p}$.

A29. (a) 1: **neither** (only one divisor). 2: **prime**. $51 = 3{\cdot}17$: composite. $91 = 7{\cdot}13$: composite. 97: **prime**. $221 = 13{\cdot}17$: composite. *Traps:* 51, 91 and 221 all end in 1 and "look" prime. (b) Up to $\sqrt{221} \approx 14.9$, so the primes **2, 3, 5, 7, 11, 13**. If $n = ab$ with both $a, b > \sqrt{n}$, then $ab > n$, which is impossible, so one factor must be $\le \sqrt{n}$. Here 13 works: $221 = 13 \cdot 17$. (c) Primes can't be factored. The hard problem is **factoring the product** $n = p \cdot q$ of two large primes.

A30. (a) A divisor of $p^a q^b \cdots$ picks each exponent from 0 to a, 0 to b, ..., so the count is $(a + 1)(b + 1)\cdots$ Check: $100 = 2^2 5^2 \to 3 \cdot 3 = 9$ ✓. $5040 \to (4 + 1)(2 + 1)(1 + 1)(1 + 1) = \mathbf{60}$. (b) Exactly 3 divisors means $(a + 1)\cdots = 3$, so the number is p^2 for a prime p: **4, 9, 25, 49, ...** (e.g. 9 has 1, 3, 9, as in the lecture).

I. Modular arithmetic

A31. Test whether $m \mid (a - b)$: (a) $60 = 5 \cdot 12$: **valid**. (b) $-20 = -4 \cdot 5$: **valid**. (c) $99 = 11 \cdot 9$: **valid**. (d) $-14 = -2 \cdot 7$: **valid**. *Trap:* −7 and 7 look "opposite", but both are $\equiv 0 \pmod 7$. (e) 28 is not a multiple of 6: **invalid**. (f) $-26 = -2 \cdot 13$: **valid**. (g) **False**. The mod operator gives the least remainder: $38 \bmod 12 = 2$. (The congruence $38 \equiv 14 \pmod{12}$ is true, but that's a different statement.)

A32. (a) A number is $\equiv$ its digit sum $\pmod 9$: $1234 \equiv 1 + 2 + 3 + 4 = 10 \equiv 1$, and $5678 \equiv 26 \equiv 8$. So the product $\equiv 1 \cdot 8 = \mathbf{8}$. (b) $7^2 = 49 \equiv 9$, $7^4 \equiv 81 \equiv 1$, so $7^{100} = (7^4)^{25} \equiv \mathbf{1}$. (c) $2^3 = 8 \equiv 1 \pmod 7$, so $2^{50} = (2^3)^{16} \cdot 2^2 \equiv 1 \cdot 4 = \mathbf{4}$. (d) $-123 = -13 \cdot 10 + 7$, so the answer is **7**. *Trap:* not −3.

A33. (a) $-3 \equiv \mathbf{7}$, $-0 = \mathbf{0}$, $-5 \equiv \mathbf{5}$ (5 is its own additive inverse: $5 + 5 = 10 \equiv 0$). (b) $3^{-1} = \mathbf{7}$ ($21 \equiv 1$), $7^{-1} = \mathbf{3}$, $9^{-1} = \mathbf{9}$ ($81 \equiv 1$), and **5 and 4 have none** ($\gcd(5, 10) = 5$, $\gcd(4, 10) = 2$). (c) $4x$ is always even, and reducing an even number mod 10 (also even) leaves it even. It can never equal 1.

A34. (a) *Trap:* "divide by 4, so $x = 2$" finds only one answer. You can't divide, because $\gcd(4, 12) = 4$. Check every x: $4x \bmod 12$ is 8 for $x = \mathbf{2}, \mathbf{5}, \mathbf{8}, \mathbf{11}$ (8, 20, 32, 44). Four solutions. (b) $3x \bmod 12$ is always one of $0, 3, 6, 9$. **No solution.** (c) $\gcd(5, 12) = 1$, so 5^{-1} exists: $5 \cdot 5 = 25 \equiv 1$, so $5^{-1} = 5$. Then $x \equiv 5 \cdot 3 = 15 \equiv \mathbf{3}$. Exactly one solution (check: $15 \equiv 3$ ✓). *Rule:* $ax \equiv b \pmod n$ has exactly one solution when $\gcd(a, n) = 1$. Otherwise it has none or several.

A35. (a) $(9 + 100) \bmod 12 = 109 \bmod 12 = 1$: **1 o'clock**. (b) $(3 - 50) \bmod 12 = -47 \bmod 12 = -47 + 48 = 1$: **1 o'clock**. (c) $C - k \equiv C + (-k) \equiv C + (26 - k) \pmod{26}$, because $26 - k$ is the additive inverse of k. So decrypting shift k is encrypting with $26 - k$. For Caesar ($k = 3$) that's **23**.

A36. 19 letters; the pattern repeats every $2(4 - 1) = 6$: rows 1 2 3 4 3 2 | 1 2 3 4 3 2 | 1 2 3 4 3 2 | 1. Counts: row 1 = 4 (positions 1, 7, 13, 19), row 2 = 6, row 3 = 6, row 4 = 3 (positions 4, 10, 16). Cut: `TIRT | HYSETA | EEUDHM | KNE`. Put the pieces back and read positions 1–19: **"the key is under the mat"**. *Trap:* splitting 19 into roughly equal rows gives nonsense. The end rows (1 and 4) come up only once per cycle, the middle rows twice.

J. Modular exponentiation

A37. $23 = 16 + 4 + 2 + 1$ (binary `10111`). Table mod 187: $11^1 = 11$; $11^2 = 121$; $11^4 = 121^2 = 14641 = 78 \cdot 187 + 55 \to 55$; $11^8 = 55^2 = 3025 \to 33$; $11^{16} = 33^2 = 1089 \to 154 \equiv -33$. Multiply, reducing each time: $(-33)(55) = -1815 \equiv 55$; $55 \cdot 121 = 6655 \equiv 110$; $110 \cdot 11 = 1210 \equiv \mathbf{88}$. It **undoes** the lecture's example: $88^7 \equiv 11$ and $11^{23} \equiv 88 \pmod{187}$. That's RSA with $n = 187$, $e = 7$, $d = 23$: encrypting 88 gives 11, and decrypting 11 gives 88 back.

A38. Last 3 digits means mod 1000. $175^2 = 30625 \to \mathbf{625}$, and $625^2 = 390625 \to \mathbf{625}$ again! So $175^{2^k} \equiv 625$ for every $k \ge 1$. $209 = 128 + 64 + 16 + 1$, so $175^{209} \equiv 625 \cdot 625 \cdot 625 \cdot 175 \equiv 625 \cdot 175 = 109375 \to \mathbf{375}$. *Trick:* spotting that $625^2 \equiv 625$ saves all the work.

A39. Repeated squaring: $2^2 = 4$, $2^4 = 16$, $2^8 = 256 \equiv 16$, $2^{16} \equiv 16$, $2^{32} \equiv 16$. With $33 = 32 + 1$: $16 \cdot 2 = 32 \equiv \mathbf{2}$. Cycle: $2, 4, 8, 16, 32 \equiv 2, 4, \ldots$ repeats every 4 from 2^1. Since $33 = 1 + 8 \cdot 4$, $2^{33} \equiv 2^1 = \mathbf{2}$. (Fermat can't be used: 30 isn't prime.)

A40. (a) $7^2 = 49$, $7^4 = 2401 \equiv \mathbf{1} \pmod{100}$. So $7^{222} = (7^4)^{55} \cdot 7^2 \equiv 49$: the last two digits are **49**. (b) The last digit is the last of those two: **9**. (Directly: $7^4 \equiv 1 \pmod{10}$ too, so $7^{222} \equiv 7^2 = 49 \equiv 9$.)

A41. (a) You may reduce the **base** mod 19, but **not the exponent**. The exponent counts how many times you multiply; it isn't a residue mod 19. (For prime p, Fermat lets you reduce the exponent mod $p - 1$, not mod p.) (b) $117 = 64 + 32 + 16 + 4 + 1$. Mod 19: $5^1 = 5$, $5^2 = 25 \to 6$, $5^4 = 36 \to 17$, $5^8 = 289 \to 4$, $5^{16} = 16$, $5^{32} = 256 \to 9$, $5^{64} = 81 \to 5$. Product: $5 \cdot 9 \cdot 16 \cdot 17 \cdot 5$: $45 \to 7$; $7 \cdot 16 = 112 \to 17$; $17 \cdot 17 = 289 \to 4$; $4 \cdot 5 = 20 \to \mathbf{1}$. The student's 11 is wrong. (Shortcut: $5^9 \equiv 1 \pmod{19}$, and $117 = 13 \cdot 9$.)

A42. (a) $100 \equiv -1 \pmod{101}$ and 101 is odd: $(-1)^{101} = -1 \equiv$ **100**. (b) 100 is even: $(-1)^{100} =$ **1**. (c) $99 \equiv -1 \pmod{100}$ and 99 is odd: $-1 \equiv$ **99**. (d) a^{100}: 6 squarings (up to a^{64}) + 2 multiplications ($a^{64} \cdot a^{32} \cdot a^4$) = **8**, versus **99** naively. $a^{1024} = a^{2^{10}}$: just **10** squarings, versus 1023.

K. GCD and the Euclidean algorithm

A43. Table for gcd(1005, 105):

Q	A	B	R
9	1005	105	60
1	105	60	45
1	60	45	15
3	45	15	0
X	**15**	0	X

$\gcd(529, 123) = \gcd(123, 37)$ $(529 = 4 \cdot 123 + 37)$
$= \gcd(37, 12)$ $(123 = 3 \cdot 37 + 12)$
$= \gcd(12, 1)$ $(37 = 3 \cdot 12 + 1)$
$= \gcd(1, 0) =$ **1** $(12 = 12 \cdot 1 + 0)$

$\gcd(1005, 105) =$ **15**. And $\gcd(529, 123) = 1$, so they **are** relatively prime, *but neither is prime*: $529 = 23^2$ and $123 = 3 \cdot 41$. (Trap: coprime doesn't mean prime.)

A44. (a) **False**: $\gcd(8, 9) = 1$, and neither is prime. (b) **False**: both are divisible by 2, so their GCD is at least 2. (c) **True**: any common divisor of n and $n + 1$ divides their difference, which is 1. (d) **True**: any common divisor of a and b divides $a + b$; any common divisor of a and $a + b$ divides $(a + b) - a = b$. Same common divisors, same GCD. (e) **True** (same answer, different place to read it). When $R = 0$, the B in that row is the last non-zero remainder, which moves into A in the next row anyway. Just don't read column A of the $R = 0$ row, which is the lecture's warning.

A45. (a) $89 = 1 \cdot 55 + 34$, $55 = 1 \cdot 34 + 21$, $34 = 1 \cdot 21 + 13$, $21 = 1 \cdot 13 + 8$, $13 = 1 \cdot 8 + 5$, $8 = 1 \cdot 5 + 3$, $5 = 1 \cdot 3 + 2$, $3 = 1 \cdot 2 + 1$, $2 = 2 \cdot 1 + 0$. gcd = **1**, after **9 divisions**. (b) These are consecutive **Fibonacci numbers** (each is the sum of the two before it). Every quotient is 1, so the numbers shrink as slowly as possible. This is the worst case for Euclid, and even that is only about 5 steps per decimal digit.

A46. (a) Exactly the numbers coprime to $26 = 2 \cdot 13$, i.e. the **odd numbers except 13**: 1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25. That's **12 values**. (b) $\gcd(26, 13)$: $26 = 2 \cdot 13 + 0 \Rightarrow \gcd = 13 \neq 1$ (invalid). $\gcd(26, 15)$: $26 = 1 \cdot 15 + 11$, $15 = 1 \cdot 11 + 4$, $11 = 2 \cdot 4 + 3$, $4 = 1 \cdot 3 + 1$, $3 = 3 \cdot 1 + 0 \Rightarrow \gcd = 1$ (valid).

A47. (a) $\gcd(17, 0) =$ **17**: every number divides 0. (b) **17**. (c) **1**: 1 is coprime to everything. (d) Row 1: $105 =$ **0** $\cdot 1005 + 105$, so $Q = 0$ and $R = 105$. The shift then gives $A = 1005$, $B = 105$: the numbers swap themselves. They still get **15**, just one row later.

L. Relatively prime numbers and Euler's totient

A48. (a) $790 = 6 \cdot 121 + 64$; $121 = 1 \cdot 64 + 57$; $64 = 1 \cdot 57 + 7$; $57 = 8 \cdot 7 + 1$; $7 = 7 \cdot 1 + 0$. So gcd = 1: **relatively prime**, though both are composite ($790 = 2 \cdot 5 \cdot 79$, $121 = 11^2$). (b) $369 = 3^2 \cdot 41$: $369 \cdot \frac{2}{3} \cdot \frac{40}{41} =$ **240**. $372 = 2^2 \cdot 3 \cdot 31$: $372 \cdot \frac{1}{2} \cdot \frac{2}{3} \cdot \frac{30}{31} =$ **120**.

A49. (a) $(p - 1)(q - 1)$ needs **distinct** primes. Correct: $25(1 - \frac{1}{5}) =$ **20**. (b) The exponents must **not** appear. Their version gives $1000 \cdot \frac{1}{8} \cdot \frac{64}{125} = 64$. Correct: $1000 \cdot \frac{1}{2} \cdot \frac{4}{5} =$ **400**. (c) Multiplying only works when the factors are **co-prime**, and $\gcd(2, 6) = 2$. Correct: $12 = 4 \cdot 3$, so $\Phi(4)\Phi(3) = 2 \cdot 2 =$ **4** (the numbers 1, 5, 7, 11). $\Phi(27) = 27 - 9 =$ **18**; $\Phi(1024) = \Phi(2^{10}) = 1024 - 512 =$ **512**; $\Phi(97) =$ **96** (97 is prime).

A50. (a) $\frac{k}{36}$ is in lowest terms exactly when $\gcd(k, 36) = 1$. That's $\Phi(36) = 36 \cdot \frac{1}{2} \cdot \frac{2}{3} =$ **12**. (b) "Divisible by neither 2 nor 5" means co-prime to 10, and also to $1000 = 2^3 5^3$. 1000 itself doesn't qualify, so the count is $\Phi(1000) =$ **400**. (c) k needs an inverse mod 26 (to decrypt), so $\gcd(k, 26) = 1$: $\Phi(26) = 26 \cdot \frac{1}{2} \cdot \frac{12}{13} =$ **12**.

A51. (a) A prime $p \mid n$ needs $(p - 1) \mid 4$, so $p \in \{2, 3, 5\}$. Also $p^2 \mid n$ would put p into $\Phi(n)$, so 3 and 5 can appear at most once. Checking $n = 2^a 3^b 5^c$ gives exactly **n** = **5, 8, 10, 12** (e.g. $\Phi(10) = 1 \cdot 4$, $\Phi(12) = 2 \cdot 2$). *Trap:* assuming $\Phi(n) = 4$ has one answer. Φ is not one-to-one. (b) If an odd prime $p \mid n$, then $(p - 1)$, which is even, divides $\Phi(n)$. Otherwise $n = 2^a$ with $a \geq 2$, and $\Phi = 2^{a-1}$ is even. (Another way: k and $n - k$ are both co-prime to n or both not, so the co-prime numbers come in pairs. They can only coincide when $k = n/2$, but $\gcd(n/2, n) = n/2 > 1$ for $n \geq 3$.)

A52. The factor 7 in $14 = 2 \cdot 7$ must come from somewhere. A factor of $\Phi(n)$ comes either from $q - 1$ for a prime $q \mid n$, or from p itself when $p^2 \mid n$.

- $q - 1$ a multiple of 7 that divides 14: $q - 1 \in \{7, 14\}$, so $q \in \{8, 15\}$. Neither is prime.
- $7^2 \mid n$: then $\Phi(49) = 42$ divides $\Phi(n)$, and $42 > 14$. Impossible.

So $\Phi(n) = 14$ has **no solution** (a script check of all n up to 500 agrees).

A53. (a) $\Phi(187) = 10 \cdot 16 =$ **160**. (b) $7 \cdot 23 = 161 = 160 + 1 \equiv$ **1** $\pmod{160}$ ✓. That's why $88^7 \to 11$ and $11^{23} \to 88$ undo each other. (c) $n - \Phi(n) + 1 = pq - (p-1)(q-1) + 1 = p + q$, so $p + q = 187 - 160 + 1 = 28$ and $pq = 187$. Then p, q are the roots of $x^2 - 28x + 187 = 0$: $x = \frac{28 \pm \sqrt{784 - 748}}{2} = \frac{28 \pm 6}{2} =$ **17**, **11**. (d) Knowing $\Phi(n)$ is as good as knowing the factors. It must stay as secret as p and q; otherwise d can be computed immediately.

M. Challenge set

A54. (a) $\gcd(13, 26) = 13$, so there's no inverse and decryption is impossible. Collision: $13 \cdot 0 = 0$ and $13 \cdot 2 = 26 \equiv 0$, so **a and c both encrypt to A**. (In fact every even letter goes to A, and every odd one to N.) (b) **a** ($k \cdot 0 = 0$) and **n**: $13k \bmod 26 = 13$ for every odd k, and every valid k is odd. (c) k has $\Phi(26) = 12$ choices and b has 26, so $12 \cdot 26 =$ **312** keys (311 if you exclude the identity $k = 1, b = 0$). Still easy to brute-force.

A55. Eve learns the answer is **YES**: it's the only 3-letter option. Perfect secrecy says the ciphertext reveals nothing *among messages of the same length*. The OTP hides content, **not length**. Fix: **pad** every message to the same length (e.g. send "NOX" or use fixed-length codes).

A56. (a) Adding the keys position by position: the shifts repeat every $\operatorname{lcm}(2, 3) = 6$ letters. Combined key = (A+B, B+C, A+D, B+B, A+C, B+D) = **BDDCCE**. Check: "hello" gives IHONQ either way. (b) $\operatorname{lcm}(4, 6) =$ **12**. *Trap:* not $4 + 6$ or 4×6. (c) Only a little. It is still a Vigenère cipher (with period 12), and the same key-length attack breaks it.

A57. $K = C - P$, letter by letter: `cryptcryptcryptc`. The key is **CRYPT**, length **5**. (One known plaintext is enough to break Vigenère completely.)

A58. (a) Exponents stack **from the top**: $3^{(3^3)} = 3^{27}$. Powers of 3 mod 10 cycle 3, 9, 7, 1 (period 4), and $27 = 6 \cdot 4 + 3$, so the answer is **7**. (b) $(3^3)^3 = 27^3 \equiv 7^3 = 343$, so the answer is **3** (it equals 3^9, and $9 \equiv 1 \pmod 4$ gives 3). *Trap:* a^{b^c} means $a^{(b^c)}$, not $(a^b)^c = a^{bc}$.

A59. (a) $10 \equiv -1 \pmod{11}$, so $10^{101} + 1 \equiv (-1)^{101} + 1 = 0$. Divisible ✓. (b) $10^2 = 100 \equiv -1 \pmod{101}$, so $10^{100} = (10^2)^{50} \equiv (-1)^{50} = 1$. The answer is $1 + 1 =$ **2**.

A60. Euclid step 1: $2^{30} - 1 = 2^{10}(2^{20} - 1) + (2^{10} - 1)$, so the remainder is $2^{10} - 1$. Step 2: $2^{20} - 1 = (2^{10} + 1)(2^{10} - 1) + 0$. So gcd = $2^{10} - 1 =$ **1023**. The pattern: $\gcd(2^a - 1, 2^b - 1) = 2^{\gcd(a,b)} - 1$, because Euclid on the numbers mirrors Euclid on the exponents ($\gcd(30, 20) = 10$).

A61. (a) **4** times (`acegikmobdfhjlnp` after the first). (b) **18** times! The 16-letter and 20-letter cases are completely different. *Why:* number positions $0, \ldots, n-1$. Depth 2 sends position i to $i/2$ (even i) or $n/2 + (i-1)/2$ (odd i), which is $i \cdot 2^{-1} \bmod (n-1)$. The last letter stays put. So after t rounds a letter has moved to $i \cdot 2^{-t}$, and the text returns when $2^t \equiv 1 \pmod{n-1}$. That happens first at $t = 4$ for $n - 1 = 15$ ($2^4 = 16$), and only at $t = 18$ for $n - 1 = 19$. (c) Repeating the same transposition just gives another transposition, and eventually the original. No real security is added.

A62. (a) `t a x x i` gives `ta`, then `xx`. Inserting the filler `x` just produces `xx` again, forever. Fix: use a **different filler** for x's, e.g. `q`: `ta xq xi`. (b) A reversed ciphertext pair means a **reversed plaintext pair** (practice A9a): if RS came from `ab`, then SR came from `ba`. Words like "re…er" or "de…ed" leak this way. It's a classic foothold for breaking Playfair.

A63. (a) $K^2 = \begin{pmatrix} 1 & 2+50 \\ 0 & 625 \end{pmatrix} = \begin{pmatrix} 1 & 52 \\ 0 & 625 \end{pmatrix} \equiv \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ ($52 = 2 \cdot 26$, $625 = 24 \cdot 26 + 1$) ✓. K is its own inverse. (b) `hi` $= (7, 8) \to (7 \cdot 1 + 8 \cdot 0,\ 7 \cdot 2 + 8 \cdot 25) = (7, 214) \equiv (7, 6) =$ HG. Encrypting HG gives HI back. (c) $C_1 = P_1 \cdot 1 + P_2 \cdot 0 = P_1$: the **first letter of every pair is not encrypted at all**. Half the message shows through, even though $\det K = 25$ is perfectly valid. Validity (invertibility) is not the same as security.

A64. Shift back by 3: **cheer**. Shift back by 22 (i.e. forward 4): **jolly**. Two meaningful answers, because "cheer" and "jolly" are themselves shifts of each other by 7. With very short ciphertexts, brute force can produce several valid plaintexts, and you need context (or more ciphertext) to decide. The lecture's "only one answer makes sense" only holds for long enough messages.

N. Fermat, Euler, primitive roots

A65. (a) $2^{10} = 1024 = 3 \cdot 341 + 1$, so $2^{10} \equiv 1$ and $2^{340} = (2^{10})^{34} \equiv 1$. (b) If $3^{340} \equiv 1 \pmod{341}$, it would also be $\equiv 1 \pmod{31}$ (since $31 \mid 341$). Mod 31, Fermat gives $3^{30} \equiv 1$; $340 \bmod 30 = 10$, so $3^{340} \equiv 3^{10}$. Now $3^5 = 243 \equiv -5$, so $3^{10} \equiv 25 \neq 1$. So $3^{340} \not\equiv 1 \pmod{341}$, and **341 is composite**. (Base 2 lied; base 3 doesn't. That's why real primality tests try many bases. Worse still, numbers like $561 = 3 \cdot 11 \cdot 17$ fool *every* co-prime base: Carmichael numbers.)

A66. (a) $\gcd(3,100) = 1$, $\Phi(100) = 40$, $2024 \bmod 40 = 24$, so 3^{24}. In fact $3^{20} \equiv 1 \pmod{100}$, so $3^{24} \equiv 3^4 = 81$: the last two digits are **81**. (b) **No**: $\gcd(2,100) = 2$, so Euler doesn't apply. Split $100 = 4 \times 25$ instead. Mod 4: $2^{2024} \equiv 0$. Mod 25: $\Phi(25) = 20$ and $2024 \bmod 20 = 4$, so $2^4 = 16$. The number below 100 that is $\equiv 0 \pmod 4$ and $\equiv 16 \pmod{25}$ is **16**. (This is the Chinese remainder theorem, coming soon.)

A67. (a) $\Phi(40) =$ **16**. (b) The primitive roots are exactly 6^k with $\gcd(k,40) = 1$. $k = 3$: $6^3 = 216 = 5 \cdot 41 + 11$, so **11**. $k = 7$: $6^7 \equiv$ **29**. (c) $6^2 = 36$ has order $40/\gcd(2,40) = 20 < 40$: its powers repeat after 20 steps.

A68. (a) $100 = 2^2 \cdot 5^2$, so check $2^{100/2} = 2^{50}$ and $2^{100/5} = 2^{20} \bmod 101$: $2^{50} \equiv 100 \neq 1$ and $2^{20} \equiv 95 \neq 1$. So **yes**. (Two powers instead of a 100-row table.) (b) $16 = 2^4$; check $2^8 = 256 = 15 \cdot 17 + 1 \equiv 1$. **No**: 2 has order 8, not 16.

A69. (a) Alice sends $A = 5^6 \bmod 23 =$ **8**; Bob sends $B = 5^{15} \bmod 23 =$ **19**. Key: Alice computes $B^6 = 19^6 \bmod 23 =$ **2**; Bob computes $A^{15} = 8^{15} \bmod 23 =$ **2**. They match, because both equal 5^{90}. (b) A primitive root's powers cover all of $1, \ldots, 22$, so the shared key can be any of 22 values. That makes guessing as hard as possible. (c) $22 \equiv -1$: its powers are only ± 1 (22, 1, 22, 1, ...). Every public value and every key is 1 or 22, so Eve can guess the key.

A70. (a) a^{p-2} needs p **prime**, and 9 isn't. The student gets $4^7 \equiv 4 \pmod 9$, but $4 \cdot 4 = 16 \equiv 7 \neq 1$. Correct (Euler, $\Phi(9) = 6$): $4^{-1} \equiv 4^5 = 1024 \equiv$ **7** (check $4 \cdot 7 = 28 = 3 \cdot 9 + 1$). (b) $6^5 = 7776 \equiv$ **0** $\pmod 9$, and $6 \cdot 0 \neq 1$. The formula silently fails because $\gcd(6,9) = 3$, so no inverse exists. Always check the gcd first.

O. Attacks and Lecture 5 extras

A71. (a) **Traffic analysis**, confidentiality, passive; encryption does **not** stop it (metadata leaks). (b) **Repudiation**, integrity, active, by a party; needs a digital signature. (c) **Replay**, integrity, active; encryption doesn't help (the ciphertext is valid); needs timestamps/nonces. (d) **DoS**, availability, active. Encryption only stops snooping.

A72. (a) $\gcd(13,26) = 13 \neq 1$: no inverse. $a = 0 \to 5$ and $c = 2 \to 31 \equiv 5$: both become F, so decryption is ambiguous. (b) $4k_1 + k_2 \equiv 17$, $13k_1 + k_2 \equiv 24 \Rightarrow 9k_1 \equiv 7$, $9^{-1} = 3$, $k_1 = 21$, $k_2 = 17 - 84 \equiv 11$. Decrypt with $21^{-1} = 5$: KTF → **vow**.

A73. $\gcd(30,42,66,18) = 6$: $m \in \{2,3,6\}$; try 6 first (2 and 3 are also consistent but give shorter repeats). OTP keys never repeat, so repeated ciphertext segments are coincidences with no common distance.

A74. (a) `DecKey[EncKey[i]] = i` gives `4 2 5 1 3`: the same key. (b) `attac katda wnzzz` → ATCAT DAAKT ZNZWZ. (c) No: an involution key just means encryption = decryption; security depends on the key being secret, not on it differing from its inverse.

A75. $P = \begin{pmatrix} 7 & 4 \\ 11 & 15 \end{pmatrix}$ (he, lp), $C = \begin{pmatrix} 3 & 15 \\ 11 & 4 \end{pmatrix}$. $\det P = 61 \equiv 9$, $9^{-1} = 3$, $P^{-1} = \begin{pmatrix} 19 & 14 \\ 19 & 21 \end{pmatrix}$, $K = P^{-1}C = \begin{pmatrix} 3 & 3 \\ 2 & 5 \end{pmatrix}$.

A76. Both are true. The double transposition is still one permutation of positions, so it is not a new kind of cipher. But it is not a *columnar* permutation with the same regular structure: the position pattern (steps of the row count) that the pattern attack exploits disappears. It hides structure; it doesn't square the key space.

P. Extended Euclid, congruences, Diophantine

A77. Rows $(q; r_1, r_2, r; s; t)$: $(7; 1820, 231, 203; 1; -7)$, $(1; 231, 203, 28; -1; 8)$, $(7; 203, 28, 7; 8; -63)$, $(4; 28, 7, 0; \ldots)$. gcd = 7, $s = 8$, $t = -63$: $8 \cdot 1820 - 63 \cdot 231 = 14560 - 14553 = 7$ ✓.

A78. t column: $0, 1, -10, 31, -134,$ **433** with quotients 10, 3, 4, 3, 2; final $r_1 = 1 \Rightarrow 97^{-1} = 433$ ($97 \cdot 433 = 42001$). $\gcd(98,1000) = 2$ (both even), so no inverse.

A79. $d = \gcd(35,50) = 5 \mid 10$: 5 solutions. $7x \equiv 2 \pmod{10}$, $7^{-1} = 3$, $x_0 = 6$: **6, 16, 26, 36, 46**. $35x \equiv 11$: $5 \nmid 11$, **none**.

A80. Multiply by $8/4 = 2$: particular $(-38, 10)$; general $x = -38 + 80k$, $y = 10 - 21k$. For 10: gcd = $4 \nmid 10$.

Q. Primes, primality, CRT, discrete logs

A81. $340 = 85 \cdot 2^2$: $T = 2^{85} \bmod 341 = 32$, then $32^2 \bmod 341 =$ **1** ⇒ composite (a nontrivial square root of 1). 2047: $2046 = 1023 \cdot 2$, $T = 2^{1023} \bmod 2047 = 1$ ⇒ "prime" (wrong: a strong pseudoprime to base 2); base 3 gives $T = 1565 \neq \pm 1$ with no loop ⇒ composite. Lesson: MR catches Fermat liars like 341 but still needs several bases.

A82. (a) $M = 60$; $M_i = 20, 15, 12$; inverses 2, 3, 3; $x = (40 + 90 + 108) \bmod 60 =$ **58**. (b) 6 and 8 aren't coprime, so the four-step formula doesn't apply. A solution exists iff $\gcd(6,8) = 2$ divides the difference: $5 - 3 = 2$ ✓ ⇒ $x =$ **21** $\pmod{24}$; $5 - 2 = 3$ is odd ⇒ **no solution**.

A83. $\lceil \sqrt{8051} \rceil = 90$, $90^2 - 8051 = 49 = 7^2 \Rightarrow 8051 = 97 \times 83$, in one step, because the factors are close to $\sqrt{n}$ (the reason RSA wants p, q far apart).

A84. The primitive roots are 2^k with $\gcd(k, 10) = 1$, $k = 1, 3, 7, 9$: $2, 8, 7, 6$. Count = $\phi(\phi(11)) = \phi(10) = 4$ ✓.

A85. (a) $2^6 = 64 \equiv 9$: $x = 6$. (b) Powers of 3: $3, 9, 5, 4, 1$: $x = 4$ (3 has order 5, not a primitive root). (c) **No solution**: 2 isn't in $\{1, 3, 4, 5, 9\}$, the subgroup generated by 3.

A86. Orders divide 16, so **no** element of order 6. Order 16 exists (a primitive root): ord(3) = 16; ord(2) = 8, ord(4) = 4. So 3.

A87. Fermat: 117 mod 18 = 9, so $5^{117} \equiv 5^9 \equiv \mathbf{1} \pmod{19}$. Hence $\text{ord}_{19}(5)$ divides 9 (it is 9): 5 is **not** a primitive root.

R. Algebraic structures and GF(2^n)

A88. $\mathbb{Z}_{12}^* = \{1, 5, 7, 11\}$ and $5^2, 7^2, 11^2 \equiv 1$: all orders ≤ 2 < 4, so **not cyclic** (no generator). Same order 4 as $\mathbb{Z}_{10}^*$, different structure.

A89. $\mathbb{Z}_{11}$: **field** (11 prime). $\mathbb{Z}_{12}$: commutative **ring**, not a field (2, 3, 4, 6, … have no inverse). GF(2^4): **field**. $\mathbb{Z}$: commutative **ring** (no division). $\langle \mathbb{Z}_9^*, \times \rangle$: abelian **group** (one operation), in fact cyclic (generator 2).

A90. $(x^3 + x + 1)(x^2 + 1) \equiv \mathbf{1}$ (they are inverses: compare the worked example in Section 29). $(x^3 + x^2)^{-1} = x^3 + x$; $x^{-1} = x^3 + 1$ (check: $x^4 + x \equiv (x + 1) + x = 1$).

A91. Orders divide 8, so no order 3. All orders in $\mathbb{Z}_{15}^*$ are 1, 2 or 4: no element of order 8 ⇒ **not cyclic** (consistent: primitive roots exist only for $2, 4, p^t, 2p^t$).

A92. $57 \cdot 83 = C1$; $57 \cdot 13 = FE$ (FIPS-197's examples).

A93. No: $x^4 + x^2 + 1 = (x^2 + x + 1)^2$. Having no roots only rules out linear factors; a degree-4 polynomial can split into two irreducible quadratics.

S. Modern block ciphers, DES, AES

A94. 8000 bits: AES (128): $-8000 \bmod 128 = 64$ bits, 63 blocks. DES (64): $8000 = 125 \times 64$, **0** bits, 125 blocks.

A95. Only the invertible ones: the 64×64 straight P-box and the 4×4 S-box (if it's a bijection). Expansion P-boxes and 6×4 S-boxes are noninvertible: Feistel only.

A96. `101010`: row 2, col 5 → 6 = `0110`. `110011`: row 3, col 9 → 11 = `1011`. `010101`: row 1, col 10 → 12 = `1100`.

A97. 1 + 1 + 2 = 4; after round 9: 15; after 16: **28**. Each half is 28 bits, so a total of 28 brings it back to the original, which the decryption key schedule relies on.

A98. $L_1 = R_0 = 0110$, $R_1 = L_0 \oplus f = 1010 \oplus 1100 = 0110$. Decrypt: $R_0 = L_1 = 0110$, $L_0 = R_1 \oplus f(L_1, K_1) = 0110 \oplus 1100 = 1010$ ✓ (same f, no inverse of f needed).

A99. Meet-in-the-middle (known plaintext P, C): encrypt P under all 2^{56} keys, store the M values (memory ≈ 2^{56} blocks); decrypt C under all 2^{56} keys and look up matches: $2 \times 2^{56} = 2^{57}$ operations, then check candidates on another pair.

A100. When $i \bmod 4 = 0$: $i = 4, 8, \ldots, 40$: **10** times. AES-256: 14 rounds, **15** round keys (60 words).

A101. Each column of equal bytes b maps to itself, since $02 \oplus 03 \oplus 01 \oplus 01 = 01$: `[01 01 01 01]`, `[C6 C6 C6 C6]` unchanged. `[01 00 00 00]` → `[02 01 01 03]` (the first column of the matrix). MixColumns alone gives no diffusion on uniform columns; ShiftRows first breaks that up.

A102. The affine step adds the constant 63, so the S-box has no fixed point at 0 (and no "opposite" fixed points); otherwise SubBytes would be the pure (algebraic, more attackable) inverse map.

T. RSA

A103. $n = 221$, $\phi = 192$, $d = 5^{-1} \bmod 192 = \mathbf{77}$ ($5 \cdot 77 = 385 = 2 \cdot 192 + 1$). $C = 65^5 \bmod 221 = \mathbf{182}$; $182^{77} \bmod 221 = 65$ ✓.

A104. $\phi = 72$ and $\gcd(3, 72) = 3$: e has no inverse mod ϕ; choose another e (e.g. 5).

A105. $\gcd(143, 187) = 11$ ⇒ $143 = 11 \times 13$, $187 = 11 \times 17$: both factored, so both private keys follow. Never let moduli share a prime (or a modulus).

A106. $p + q = n - \phi + 1 = 114$, $pq = 3233$: roots of $z^2 - 114z + 3233$: **61, 53**. $d = 17^{-1} \bmod 3120 =$ **2753**. (Knowing $\phi(n)$ is as good as factoring.)

A107. $P^e = 8 < n$, so no reduction happened: $P = \sqrt[3]{8} = 2$ over the integers. Small e with short unpadded messages: use OAEP padding.

U. ElGamal and ECC

A108. $e_2 = 5^6 \bmod 23 = 8$. $C_1 = 5^3 = 125 \equiv$ **10**, $C_2 = 10 \cdot 8^3 = 5120 \equiv$ **14**. Decrypt: $C_1^d = 10^6 \equiv 6$, $6^{-1} = 4$, $14 \cdot 4 = 56 \equiv$ **10** ✓.

A109. $e_2^r = C_2 \cdot P^{-1} = 14 \cdot 10^{-1} = 14 \cdot 7 \equiv 6$. Then $P' = 10 \cdot 6^{-1} = 10 \cdot 4 = 40 \equiv$ **17**. (Same $C_1 = 10$ reveals that r was reused.)

A110. 12 points: `(2,4) (2,7) (3,5) (3,6) (5,2) (5,9) (7,2) (7,9) (8,3) (8,8) (10,2) (10,9)`, plus O: $\#E = 13$. $2(2,7)$: $s = (3 \cdot 4 + 1)(14)^{-1} = 13 \cdot 3^{-1} \equiv 2 \cdot 4 = 8$; $x_3 = 64 - 4 = 60 \equiv 5$, $y_3 = 8(2 - 5) - 7 = -31 \equiv 2$: **(5, 2)**.

A111. No: $a = b = 0$ gives $4a^3 + 27b^2 = 0$, a singular curve (cusp), which the definition excludes. Hasse: $102 - 2\sqrt{101} \le \#E \le 102 + 2\sqrt{101}$, i.e. $82 \le \#E \le 122$.

A112. $e_2 = 3(1, 4) = (0, 12)$. $C_1 = 2(1, 4) = (8, 12)$; $2e_2 = (10, 6)$; $C_2 = (5, 1) + (10, 6) = (12, 5)$. Decrypt: $3C_1 = (10, 6)$, inverse $(10, 7)$, $C_2 + (10, 7) =$ **(5, 1)** ✓.

V. Message integrity

A113. Preimage 0.69×2^{128}; collision 1.18×2^{64}. Security level = 2^{64} (the weakest attack). A 128-bit *key* faces only brute force (2^{128}); a 128-bit *digest* faces the birthday bound (2^{64}), which is reachable, so MD5 is broken.

A114. $1.18 \times 2^{20} \approx 1.24$ million messages.

A115. (a) Preimage, problem 1 ($\sim 2^n$). (b) Second preimage, problem 2 ($\sim 2^n$). (c) Alternate collision, problem 4 ($\sim 2^{n/2}$): the only one at birthday cost.

A116. $|P| = (-3000 - 128) \bmod 1024 = 968$; $3000 + 968 + 128 = 4096 =$ **4 blocks**; 80 words per block.

A117. A 64-bit digest falls to a birthday collision in $\sim 2^{32}$ hashes, and the scheme is open to meet-in-the-middle. Davies–Meyer XORs the previous digest into the output ($H_i = E_{M_i}(H_{i-1}) \oplus H_{i-1}$), which blocks the meet-in-the-middle trick (the digest is still only as long as the cipher block).

A118. Majority $= (x \wedge y) \oplus (y \wedge z) \oplus (z \wedge x) = 1000 \oplus 0100 \oplus 0010 =$ **1110** (bitwise majority). Conditional $= (x \wedge y) \oplus (\bar{x} \wedge z) = 1000 \oplus 0100 =$ **1100** (take y where $x = 1$, z where $x = 0$).

A119. $1 - e^{-30 \cdot 29/730} \approx 0.696$; exact $1 - \prod_{i=0}^{29} \frac{365-i}{365} \approx 0.706$.